Serie Banco ZK

3 Volúmenes · 1000+ páginas · 20 Capítulos

Volumen I

Fundamentos Teóricos

Capítulos 1–5: Criptografía, Blockchain, Zero-Knowledge

Contenido del Volumen 1:

Parte I: Fundamentos Criptográficos
Grupos • Curvas Elípticas • Hash • Firmas Digitales

Parte II: Blockchain y Escalabilidad
Consenso • L1/L2/L3 • Rollups • Sidechains

Parte III: Zero-Knowledge Proofs
Definición • Propiedades • Protocolos Interactivos • SNARKs

Parte IV: Circuitos ZK
Circuitos Aritméticos • R1CS • QAP • Groth16

Walter Milstein

2025

Banco ZK: Criptografía y Privacidad en Blockchain
Volumen 1: Fundamentos Teóricos

© 2025 Walter Milstein
Todos los derechos reservados.

Primera Edición: 2025

Serie completa:
Volumen 1: Fundamentos Teóricos (Caps. 1–5)
Volumen 2: Aplicaciones y Privacidad (Caps. 6–13)
Volumen 3: Herramientas Avanzadas y Ecosistema (Caps. 14–20)

El código incluido es de dominio público y puede ser utilizado libremente.
Este libro es para uso personal y educativo.

Los 20 Bloques Temáticos

Bloque	Tema
1	**Fundamentos Criptográficos**
2	**Blockchain Base (L1)**
3	**Escalabilidad (L2/L3)**
4	**Zero-Knowledge Proofs (Core)**
5	**Circuitos ZK**
6	Compromisos y Estructuras
7	Infraestructura ZK
8	Privacidad Avanzada
9	DeFi y Aplicaciones
10	Avanzado/Investigación
11	Protocolos de Anonimización
12	Monedas Privadas (Privacy Coins)
13	Técnicas de Unlinkability
14	Identidad y Credenciales Anónimas
15	Compliance vs Privacidad
16	Metadata y Network Privacy
17	Private Smart Contracts
18	Criptografía Avanzada para Privacidad
19	Ataques y Defensa
20	Herramientas y Ecosistema

En negrita: capítulos incluidos en este volumen

Contents

1 Fundamentos Criptograficos **1**

1.1 Hash Functions . . . 1

1.1.1 SHA-256 . . . 1

1.2 Álgebra para Cripto (Campos y Polinomios) . . . 2

1.2.1 Campos Finitos $\mathbb{F}_p$ . . . 2

1.2.2 Extensiones de Campo $\mathbb{F}_{p^k}$ . . . 4

1.2.3 Polinomios: Interpolación y Evaluación . . . 6

1.2.4 FFT/NTT: Transformada Rápida . . . 8

1.2.5 Grupos Cíclicos y Supuestos de Seguridad . . . 10

1.2.6 Keccak (SHA-3) . . . 23

1.2.7 Poseidon . . . 33

1.3 Curvas Elipticas . . . 44

1.3.1 Definición y Ley de Grupo . . . 44

1.3.2 secp256k1 . . . 54

1.3.3 BN254 (alt_bn128) . . . 66

1.3.4 BLS12-381 . . . 73

1.3.5 Pasta (Pallas y Vesta) . . . 80

1.4 Firmas Digitales . . . 92

1.4.1 ECDSA . . . 92

1.4.2 Schnorr . . . 108

1.4.3 EdDSA (Ed25519) . . . 124

1.4.4 BLS . . . 132

1.5 Merkle Trees . . . 143

1.5.1 Contexto Histórico . . . 143

1.5.2 Construcción Básica . . . 144

1.5.3 Merkle Proof (Prueba de Inclusión) . . . 145

1.5.4 Propiedades y Complejidad . . . 146

1.5.5 Árboles con Número Impar de Hojas . . . 147

1.5.6 Merkle Trees en Bitcoin . . . 147

1.5.7 Merkle Patricia Trie en Ethereum . . . 148

1.5.8 Sparse Merkle Trees . . . 149

1.5.9 Indexed Merkle Trees . . . 150

1.5.10 Append-Only Merkle Trees . . . 150

1.5.11 Hash Functions para Merkle Trees . . . 151
1.5.12 Merkle Trees en Sistemas ZK . . . 151
1.5.13 Optimizaciones . . . 152
1.5.14 Ataques y Consideraciones de Seguridad . . . 153
1.5.15 Implementación . . . 153
1.5.16 Aplicación en Banco ZK . . . 156
1.5.17 Resumen . . . 159
1.6 Merkle Patricia Tries . . . 159
1.6.1 Contexto: ¿Por Qué No un Merkle Tree Simple? . . . 160
1.6.2 Fundamentos: El Trie Básico . . . 160
1.6.3 Patricia Trie: Compresión de Caminos . . . 161
1.6.4 Merkle Patricia Trie: Añadiendo Hashes . . . 162
1.6.5 Tipos de Nodos en Detalle . . . 163
1.6.6 Hex-Prefix Encoding . . . 163
1.6.7 RLP Encoding . . . 164
1.6.8 Ejemplo Completo de MPT . . . 165
1.6.9 Operaciones . . . 166
1.6.10 Pruebas de Inclusión y Exclusión . . . 167
1.6.11 Ethereum: Los Cuatro Tries . . . 168
1.6.12 Tamaño del Estado de Ethereum . . . 169
1.6.13 Problemas y Limitaciones . . . 169
1.6.14 Verkle Trees: El Futuro . . . 170
1.6.15 Implementación . . . 170
1.6.16 Aplicación en Banco ZK . . . 173
1.6.17 Resumen . . . 175
1.7 Commitment Schemes . . . 175
1.7.1 Qué son y para qué sirven . . . 175
1.7.2 Pedersen Commitments . . . 179
1.7.3 KZG Commitments . . . 184
1.7.4 Comparación de Commitment Schemes . . . 190
1.7.5 Aplicación en Banco ZK . . . 190
1.7.6 Resumen . . . 192
1.7.7 Polynomial Commitments . . . 192

2 Blockchain Base (L1) 205
2.1 Qué es L1 . . . 205
2.1.1 Contexto Histórico . . . 205
2.1.2 Las Cuatro Funciones Fundamentales . . . 206
2.1.3 Comparación de L1s Principales . . . 207
2.2 Consenso . . . 208
2.2.1 Proof of Work (PoW) . . . 208
2.2.2 Proof of Stake (PoS) . . . 215
2.2.3 Proof of Authority (PoA) . . . 219

2.2.4 Otros Mecanismos de Consenso . . . 220
2.3 Bloques y Transacciones . . . 221
2.3.1 Estructura de Bloque . . . 222
2.3.2 Transacciones . . . 224
2.4 Gas y Fees . . . 228
2.4.1 Costos de Gas por Operación . . . 228
2.4.2 EIP-1559: El Nuevo Modelo de Fees . . . 230
2.4.3 Gas Limit y Gas Used . . . 232
2.4.4 EIP-4844: Blob Gas . . . 232
2.4.5 Optimización de Gas . . . 233
2.5 EVM (Ethereum Virtual Machine) . . . 235
2.5.1 Arquitectura de la EVM . . . 235
2.5.2 Opcodes de la EVM . . . 236
2.5.3 Ejemplo de Ejecución EVM . . . 239
2.5.4 Contexto de Ejecución . . . 240
2.6 Estado Global (State) . . . 241
2.6.1 Tipos de Cuentas . . . 242
2.6.2 Storage Layout . . . 243
2.6.3 State Trie . . . 244
2.7 Nodos . . . 245
2.7.1 Full Node . . . 245
2.7.2 Light Node . . . 246
2.7.3 Archive Node . . . 246
2.7.4 Validator Node (Post-Merge) . . . 247
2.7.5 RPC Nodes y Providers . . . 247
2.8 Mempool . . . 249
2.8.1 Cómo Funciona la Mempool . . . 249
2.8.2 Validación en Mempool . . . 249
2.8.3 Políticas de Mempool . . . 250
2.8.4 Propagación P2P . . . 250
2.8.5 Mempool Privadas y PBS . . . 251
2.9 MEV (Maximal Extractable Value) . . . 252
2.9.1 Tipos de MEV . . . 252
2.9.2 Anatomía de un Sandwich Attack . . . 253
2.9.3 Ecosistema MEV . . . 254
2.9.4 Flashbots . . . 255
2.9.5 MEV por los Números . . . 256
2.9.6 Impacto Negativo del MEV . . . 256
2.9.7 Soluciones y Mitigaciones . . . 257
2.9.8 MEV-Share . . . 258
2.9.9 Aplicación en Banco ZK . . . 258
2.10 Resumen del Bloque 2 . . . 260

3 Escalabilidad (L2/L3) 263
3.1 El Problema de Escalabilidad (Trilema) . . . 263
3.1.1 Historia del Trilema . . . 263
3.1.2 Análisis de las Tres Propiedades . . . 264
3.1.3 Enfoques para Escalar . . . 266
3.1.4 El Throughput en Perspectiva . . . 267
3.2 Qué es L2 . . . 267
3.2.1 La Definición Precisa de L2 . . . 268
3.2.2 Tipos de Soluciones L2 . . . 269
3.2.3 Comparación de Tipos de L2 . . . 271
3.3 Sidechains vs Rollups . . . 271
3.3.1 Qué es una Sidechain . . . 272
3.3.2 Análisis de Seguridad . . . 272
3.3.3 Bridges de Rollups vs Sidechains . . . 273
3.3.4 ¿Cuándo Usar Cada Uno? . . . 274
3.4 Optimistic Rollups . . . 275
3.4.1 Cómo Funcionan . . . 276
3.4.2 Fraud Proofs . . . 277
3.4.3 Arquitectura de Arbitrum . . . 279
3.4.4 El Problema de los 7 Días . . . 281
3.4.5 Compresión de Datos . . . 281
3.4.6 Seguridad y Trust Assumptions . . . 282
3.4.7 Ejemplos: Arbitrum, Optimism, Base . . . 283
3.4.8 Comparación con ZK Rollups . . . 284
3.5 ZK Rollups . . . 285
3.5.1 Cómo Funcionan . . . 285
3.5.2 Validity Proofs . . . 286
3.5.3 Tipos de ZK Rollups . . . 287
3.5.4 zkEVM: Probando la EVM . . . 288
3.5.5 Ejemplos: zkSync, StarkNet, Scroll, Polygon zkEVM . . . 289
3.5.6 Prover: El Cuello de Botella . . . 290
3.5.7 Data Availability en ZK Rollups . . . 291
3.5.8 Ventajas y Desventajas de ZK Rollups . . . 292
3.5.9 ZK Rollups con Privacidad . . . 292
3.6 Sequencers . . . 294
3.6.1 Rol del Sequencer . . . 294
3.6.2 Centralización vs Descentralización . . . 295
3.6.3 Soluciones para Descentralización . . . 296
3.7 Data Availability (DA) . . . 298
3.7.1 Por Qué Importa . . . 298
3.7.2 Opciones de DA . . . 299
3.8 Blobs (EIP-4844) . . . 302

3.8.1 Contexto y Motivación . . . 302
3.8.2 Anatomía de un Blob . . . 303
3.8.3 Blob Transactions . . . 304
3.8.4 Proto-Danksharding vs Full Danksharding . . . 305
3.8.5 Impacto de EIP-4844 . . . 305
3.9 L3 y App-chains . . . 306
3.9.1 ¿Por Qué L3? . . . 307
3.9.2 App-chains . . . 308
3.9.3 Interoperabilidad entre L2s/L3s . . . 309
3.10 Bridges . . . 310
3.10.1 Cómo Funcionan . . . 310
3.10.2 Tipos de Bridges . . . 310
3.10.3 Mecanismos de Verificación . . . 311
3.10.4 Riesgos de Bridges . . . 312
3.10.5 ZK Bridges . . . 313
3.11 Resumen del Bloque 3 . . . 316

4 Zero-Knowledge Proofs (Core) 319
4.1 Qué es una ZK Proof . . . 319
4.1.1 Historia y Contexto . . . 319
4.1.2 La Intuición: Cueva de Ali Baba . . . 320
4.1.3 Definición Formal . . . 321
4.2 Propiedades: Completeness, Soundness, Zero-Knowledge . . . 322
4.2.1 Completeness (Completitud) . . . 322
4.2.2 Soundness (Solidez) . . . 323
4.2.3 Zero-Knowledge . . . 324
4.2.4 Trade-offs entre Propiedades . . . 326
4.3 Interactive vs Non-interactive Proofs . . . 327
4.3.1 Pruebas Interactivas . . . 327
4.3.2 NIZK (Non-Interactive ZK) . . . 329
4.3.3 Transformación Fiat-Shamir . . . 330
4.3.4 Comparación Interactive vs NIZK . . . 332
4.4 Witness (Testigo) . . . 332
4.4.1 Lenguajes NP y Relaciones . . . 333
4.4.2 Witness en Circuitos ZK . . . 334
4.4.3 Witness Generation . . . 334
4.5 Prover vs Verifier . . . 335
4.5.1 El Rol del Prover . . . 335
4.5.2 El Rol del Verifier . . . 336
4.5.3 Asimetría Prover-Verifier . . . 337
4.6 Trusted Setup vs Transparent Setup . . . 338
4.6.1 Trusted Setup . . . 338
4.6.2 Transparent Setup . . . 342

4.7 CRS (Common Reference String) . . . 344
4.7.1 Estructura del CRS . . . 345
4.7.2 Proving Key vs Verification Key . . . 347
4.7.3 Derivación de CRS Específico de Circuito . . . 347
4.8 Sistemas de Prueba . . . 349
4.8.1 SNARKs . . . 349
4.8.2 STARKs . . . 350
4.8.3 Bulletproofs . . . 352
4.8.4 PLONK . . . 354
4.8.5 Groth16 . . . 355
4.8.6 Halo2 . . . 357
4.8.7 Plonky2/Plonky3 . . . 358
4.8.8 Tabla Comparativa de Sistemas . . . 360
4.9 Resumen del Bloque 4 . . . 362

5 Circuitos ZK **365**
5.1 Qué es un Circuito Aritmético . . . 365
5.1.1 Contexto e Intuición . . . 365
5.1.2 Estructura de un Circuito . . . 366
5.1.3 De Programas a Circuitos . . . 368
5.2 R1CS (Rank-1 Constraint System) . . . 369
5.2.1 Entendiendo R1CS . . . 369
5.2.2 Ejemplo Completo de R1CS . . . 370
5.2.3 Matrices R1CS . . . 371
5.3 Constraints . . . 372
5.3.1 Costos de Operaciones . . . 372
5.3.2 Constraints Comunes en Detalle . . . 372
5.3.3 Optimización de Constraints . . . 374
5.4 Señales (Signals) . . . 375
5.4.1 Anatomía de las Señales . . . 376
5.4.2 Señales en Diferentes Frameworks . . . 377
5.5 Witness Generation . . . 377
5.5.1 El Proceso de Witness Generation . . . 378
5.5.2 Witness vs Constraints . . . 378
5.5.3 Formato del Witness . . . 379
5.6 Circom . . . 380
5.6.1 Historia y Contexto . . . 380
5.6.2 Sintaxis Básica . . . 380
5.6.3 Operadores . . . 381
5.6.4 Templates . . . 382
5.6.5 Components . . . 384
5.6.6 Arrays y Loops . . . 385
5.6.7 Circomlib: Biblioteca Estándar . . . 385

5.7 snarkjs . . . 388
5.7.1 Visión General del Flujo . . . 389
5.7.2 Setup . . . 389
5.7.3 Prove . . . 392
5.7.4 Verify . . . 394
5.7.5 Exportar a Solidity . . . 395
5.7.6 Generar Calldata . . . 396
5.7.7 Script Completo de Ejemplo . . . 396
5.7.8 Otros Backends: PLONK y FFLONK . . . 398
5.8 Halo2 . . . 399
5.8.1 Historia y Contexto . . . 399
5.8.2 PLONKish Arithmetization . . . 399
5.8.3 Custom Gates . . . 401
5.8.4 Lookup Tables . . . 402
5.8.5 API de Halo2 (Rust) . . . 404
5.8.6 Chips y Regiones . . . 406
5.8.7 Halo2 vs Circom . . . 407
5.9 Arkworks . . . 408
5.9.1 Arquitectura del Ecosistema . . . 409
5.9.2 ark-ff: Campos Finitos . . . 409
5.9.3 ark-relations: R1CS . . . 410
5.9.4 ark-groth16: Generando Pruebas . . . 411
5.9.5 ark-r1cs-std: Gadgets . . . 413
5.9.6 Cuándo Usar Arkworks . . . 413
5.10 Noir . . . 414
5.10.1 Historia y Filosofía . . . 414
5.10.2 Sintaxis Básica . . . 415
5.10.3 Tipos de Datos . . . 416
5.10.4 Control de Flujo . . . 416
5.10.5 Biblioteca Estándar . . . 417
5.10.6 Toolchain de Noir . . . 418
5.10.7 ACIR: Representación Intermedia . . . 419
5.10.8 Comparación de Lenguajes ZK . . . 420
5.11 Resumen del Bloque 5 . . . 420

Apéndice: Notación **423**

Chapter 1

Fundamentos Criptograficos

Bloque 1 de 20

1.1 Hash Functions

1.1.1 SHA-256

Definición 1.1 (SHA-256)**.** Función hash de la familia SHA-2 que produce salida de 256 bits.

Características:

- Construcción Merkle-Damgård
- Bloques de 512 bits
- 64 rondas por bloque
- Operaciones: rotaciones, shifts, XOR, sumas mod 2^{32}

Usado en: Bitcoin (PoW, merkle roots), certificados SSL, integridad de archivos.

Contexto Histórico

Intuición

SHA significa *Secure Hash Algorithm*. La familia SHA fue desarrollada por la NSA y publicada por NIST. SHA-256 es parte de SHA-2, publicado en 2001 como sucesor de SHA-1 (que hoy se considera inseguro tras el ataque de colisión de Google en 2017).

SHA-256 se convirtió en el estándar de facto para criptomonedas cuando Satoshi Nakamoto lo eligió para Bitcoin en 2008. Esta elección no fue accidental: SHA-256 ofrecía el mejor balance entre seguridad probada, eficiencia en hardware, y resistencia a ataques conocidos.

256 bits de salida significa 2^{256} posibles hashes. Este número es astronómicamente grande:

$$2^{256} \approx 1.16 \times 10^{77}$$

Para contexto: hay aproximadamente 10^{80} átomos en el universo observable. Un ataque de fuerza bruta es físicamente imposible.

1.2 Álgebra para Cripto (Campos y Polinomios)

La criptografía moderna, especialmente Zero-Knowledge Proofs, se construye sobre estructuras algebraicas precisas. Esta sección establece los fundamentos matemáticos que aparecerán constantemente en SNARKs, STARKs, KZG, y sistemas como PLONK.

Intuición

¿Por qué álgebra abstracta en criptografía? Porque necesitamos operaciones que:

1. Sean **eficientes** de calcular
2. Tengan **estructura predecible** (asociatividad, inversos)
3. Presenten **problemas difíciles** para atacantes

Los campos finitos y grupos cíclicos satisfacen perfectamente estos requisitos.

1.2.1 Campos Finitos $\mathbb{F}_p$

Definición 1.2 (Campo Finito Primo)**.** Un **campo finito** $\mathbb{F}_p$ (donde p es primo) es el conjunto $\{0, 1, 2, \ldots, p-1\}$ con dos operaciones:

- **Suma**: $a + b \mod p$
- **Multiplicación**: $a \cdot b \mod p$

que satisfacen los axiomas de campo (asociatividad, conmutatividad, distributividad, existencia de neutros e inversos).

Ejemplo 1.1 (Campo $\mathbb{F}_7$)**.** En $\mathbb{F}_7 = \{0, 1, 2, 3, 4, 5, 6\}$:

$$\begin{aligned} 3 + 5 = 8 &\mod 7 = 1 \\ 3 \cdot 5 = 15 &\mod 7 = 1 \\ 4 + 6 = 10 &\mod 7 = 3 \end{aligned}$$

Observa que $3 \cdot 5 = 1$, lo que significa que 5 es el **inverso multiplicativo** de 3.

Inversos Multiplicativos

Definición 1.3 (Inverso Multiplicativo)**.** Para $a \in \mathbb{F}_p$ con $a \neq 0$, el **inverso multiplicativo** a^{-1} es el único elemento tal que:

$$a \cdot a^{-1} \equiv 1 \pmod{p}$$

Matemáticas

El inverso existe para todo $a \neq 0$ porque p es primo (por el teorema de Bézout: $\gcd(a, p) = 1$ implica que existen x, y con $ax + py = 1$, entonces $ax \equiv 1 \pmod{p}$).

Hay dos métodos principales para calcular inversos:

Método 1: Algoritmo de Euclides Extendido

```
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(a, p):
    _, x, _ = extended_gcd(a % p, p)
    return x % p
```

Método 2: Pequeño Teorema de Fermat

Teorema 1.1 (Pequeño Teorema de Fermat)**.** *Si p es primo y $a \not\equiv 0 \pmod{p}$, entonces:*

$$a^{p-1} \equiv 1 \pmod{p}$$

Por lo tanto:

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Ejemplo 1.2 (Calculando 3^{-1} en $\mathbb{F}_7$)**.** Usando Fermat: $3^{-1} = 3^{7-2} = 3^5 \mod 7$

$$\begin{aligned} 3^2 &= 9 \mod 7 = 2 \\ 3^4 &= 2^2 = 4 \\ 3^5 &= 3^4 \cdot 3 = 4 \cdot 3 = 12 \mod 7 = 5 \end{aligned}$$

Verificación: $3 \cdot 5 = 15 \mod 7 = 1$ ✓

Orden de Elementos

Definición 1.4 (Orden de un Elemento)**.** El **orden** de $a \in \mathbb{F}_p^*$ (donde $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$) es el menor entero positivo k tal que:

$$a^k \equiv 1 \pmod{p}$$

Se denota $\text{ord}(a) = k$.

Teorema 1.2 (Lagrange)**.** *El orden de cualquier elemento divide a $|\mathbb{F}_p^*| = p - 1$.*

Definición 1.5 (Generador / Raíz Primitiva)**.** Un elemento $g \in \mathbb{F}_p^*$ es un **generador** (o raíz primitiva) si $\text{ord}(g) = p - 1$. Esto significa que:

$$\mathbb{F}_p^* = \{g^0, g^1, g^2, \ldots, g^{p-2}\} = \{1, g, g^2, \ldots, g^{p-2}\}$$

Ejemplo 1.3 (Generador en $\mathbb{F}_7$)**.** Probemos si $g = 3$ es generador:

$$\begin{aligned} 3^1 &= 3 \\ 3^2 &= 9 \mod 7 = 2 \\ 3^3 &= 6 \\ 3^4 &= 18 \mod 7 = 4 \\ 3^5 &= 12 \mod 7 = 5 \\ 3^6 &= 15 \mod 7 = 1 \end{aligned}$$

Las potencias generan $\{1, 2, 3, 4, 5, 6\} = \mathbb{F}_7^*$. Por tanto, 3 es generador con orden $6 = p - 1$.

En Banco ZK

En Banco ZK, los campos finitos que usamos tienen primos especiales. Por ejemplo, el campo de BN254:

$p = 21888242871839275222246405745257275088696311157297823662689037894645226208583$

Este primo de 254 bits fue elegido para que las curvas elípticas sobre $\mathbb{F}_p$ tengan propiedades específicas para pairings.

1.2.2 Extensiones de Campo $\mathbb{F}_{p^k}$

Intuición

Así como extendemos $\mathbb{R}$ a $\mathbb{C}$ añadiendo $i = \sqrt{-1}$, podemos extender $\mathbb{F}_p$ añadiendo raíces de polinomios irreducibles. Esto nos da campos más grandes con estructura rica, esenciales para pairings.

Definición 1.6 (Extensión de Campo)**.** Sea $f(x) \in \mathbb{F}_p[x]$ un polinomio irreducible de grado k. La **extensión** $\mathbb{F}_{p^k}$ se construye como:

$$\mathbb{F}_{p^k} = \mathbb{F}_p[x]/(f(x))$$

Los elementos son polinomios de grado $< k$ con coeficientes en $\mathbb{F}_p$.

Extensión Cuadrática $\mathbb{F}_{p^2}$

Ejemplo 1.4 (Construcción de $\mathbb{F}_{p^2}$)**.** Si -1 no es un cuadrado en $\mathbb{F}_p$ (i.e., $p \equiv 3 \pmod 4$), podemos usar $f(x) = x^2 + 1$:

$$\mathbb{F}_{p^2} = \mathbb{F}_p[i] \quad \text{donde } i^2 = -1$$

Los elementos tienen la forma $a + bi$ con $a, b \in \mathbb{F}_p$.

Aritmética en $\mathbb{F}_{p^2}$:

$$(a+bi)+(c+di)=(a+c)+(b+d)i$$
$$(a+bi)\cdot(c+di)=(ac-bd)+(ad+bc)i$$

Matemáticas

La multiplicación requiere 4 multiplicaciones en $\mathbb{F}_p$. Con el truco de Karatsuba se reduce a 3:

$$ac-bd=ac-bd$$
$$ad+bc=(a+b)(c+d)-ac-bd$$

Calculamos ac, bd, $(a+b)(c+d)$: solo 3 multiplicaciones.

Extensión $\mathbb{F}_{p^{12}}$ para Pairings

Intuición

Los pairings en curvas BN y BLS requieren $\mathbb{F}_{p^{12}}$. ¿Por qué 12? Es el *embedding degree*: el menor k tal que $r|p^k-1$, donde r es el orden del subgrupo de la curva.

La construcción típica usa una **torre de extensiones**:

$$\mathbb{F}_p \to \mathbb{F}_{p^2} \to \mathbb{F}_{p^6} \to \mathbb{F}_{p^{12}}$$

Ejemplo 1.5 (Torre para BN254)**.**

$$\mathbb{F}_{p^2}=\mathbb{F}_p[u]/(u^2+1)$$
$$\mathbb{F}_{p^6}=\mathbb{F}_{p^2}[v]/(v^3-(9+u))$$
$$\mathbb{F}_{p^{12}}=\mathbb{F}_{p^6}[w]/(w^2-v)$$

Un elemento de $\mathbb{F}_{p^{12}}$ se representa con 12 elementos de $\mathbb{F}_p$.

Advertencia

La aritmética en $\mathbb{F}_{p^{12}}$ es costosa: una multiplicación requiere ~ 54 multiplicaciones en $\mathbb{F}_p$ (con optimizaciones). Por eso los pairings son operaciones "caras" comparadas con operaciones en curvas.

Idea Clave

Resumen de extensiones:

- $|\mathbb{F}_{p^k}|=p^k$ elementos
- $\mathbb{F}_{p^k}^*$ es cíclico de orden p^k-1

- Toda extensión finita de $\mathbb{F}_p$ tiene la forma $\mathbb{F}_{p^k}$
- Pairings mapean $\mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T \subset \mathbb{F}^*_{p^{12}}$

1.2.3 Polinomios: Interpolación y Evaluación

Los polinomios son la herramienta fundamental en ZK-SNARKs y STARKs. Permiten codificar computaciones completas en objetos matemáticos compactos.

Definición 1.7 (Polinomio sobre $\mathbb{F}_p$)**.** Un polinomio de grado $\leq n$ sobre $\mathbb{F}_p$ es:

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n, \quad a_i \in \mathbb{F}_p$$

El conjunto de todos estos polinomios se denota $\mathbb{F}_p[x]_{\leq n}$.

Evaluación de Polinomios

Definición 1.8 (Evaluación)**.** Dado $f(x) \in \mathbb{F}_p[x]$ y un punto $\alpha \in \mathbb{F}_p$, la **evaluación** $f(\alpha)$ sustituye x por α.

Método de Horner (evaluación eficiente en $O(n)$):

$$f(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x \cdot a_n)))$$

```
def horner_eval(coeffs, x, p):
    """Evalúa polinomio en x módulo p usando Horner"""
    result = 0
    for coeff in reversed(coeffs):
        result = (result * x + coeff) % p
    return result
```

Interpolación de Lagrange

Teorema 1.3 (Interpolación de Lagrange)**.** *Dados $n+1$ puntos distintos $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ con $x_i \in \mathbb{F}_p$ distintos, existe un **único** polinomio $f(x)$ de grado $\leq n$ tal que $f(x_i) = y_i$ para todo i.*

Definición 1.9 (Polinomios Base de Lagrange)**.** Para un conjunto de puntos $\{x_0, \ldots, x_n\}$, el i-ésimo **polinomio base de Lagrange** es:

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Satisface: $L_i(x_j) = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$

El polinomio interpolante es:

$$f(x) = \sum_{i=0}^{n} y_i \cdot L_i(x)$$

Ejemplo 1.6 (Interpolación en $\mathbb{F}_{11}$)**.** Interpolar los puntos $(1, 3), (2, 7), (4, 5)$:

Bases de Lagrange:

$$L_0(x) = \frac{(x-2)(x-4)}{(1-2)(1-4)} = \frac{(x-2)(x-4)}{(-1)(-3)} = \frac{(x-2)(x-4)}{3}$$
$$L_1(x) = \frac{(x-1)(x-4)}{(2-1)(2-4)} = \frac{(x-1)(x-4)}{-2}$$
$$L_2(x) = \frac{(x-1)(x-2)}{(4-1)(4-2)} = \frac{(x-1)(x-2)}{6}$$

En $\mathbb{F}_{11}$: $3^{-1} = 4$, $(-2)^{-1} = 9^{-1} = 5$, $6^{-1} = 2$

$$f(x) = 3 \cdot 4(x-2)(x-4) + 7 \cdot 5(x-1)(x-4) + 5 \cdot 2(x-1)(x-2)$$

Expandiendo y simplificando módulo 11: $f(x) = 2x^2 + 9x + 3$

Verificación: $f(1) = 2 + 9 + 3 = 14 \equiv 3$, $f(2) = 8 + 18 + 3 = 29 \equiv 7$, $f(4) = 32 + 36 + 3 = 71 \equiv 5$ ✓

Idea Clave

Dualidad evaluación-coeficientes:

- Un polinomio de grado n se representa con $n + 1$ coeficientes
- Alternativamente, con $n + 1$ evaluaciones en puntos distintos
- Lagrange convierte evaluaciones → coeficientes
- Horner convierte coeficientes → evaluación

Esta dualidad es fundamental en SNARKs: el prover trabaja con coeficientes, el verifier con evaluaciones.

Polinomio de Desvanecimiento (Vanishing Polynomial)

Definición 1.10 (Vanishing Polynomial)**.** Para un conjunto $H = \{h_1, \ldots, h_n\} \subset \mathbb{F}_p$, el **polinomio de desvanecimiento** es:

$$Z_H(x) = \prod_{i=1}^{n} (x - h_i)$$

Satisface: $Z_H(h) = 0$ para todo $h \in H$.

En Banco ZK

En PLONK, H es el dominio de evaluación (típicamente raíces de la unidad). Si $|H| = n$ y ω es raíz n-ésima primitiva:

$$Z_H(x) = x^n - 1$$

¡Un polinomio de grado n con solo 2 coeficientes no nulos! Esto hace que dividir por Z_H sea extremadamente eficiente.

1.2.4 FFT/NTT: Transformada Rápida

Intuición

La interpolación de Lagrange ingenua cuesta $O(n^2)$. Para circuitos con millones de gates, esto es prohibitivo. La FFT/NTT reduce el costo a $O(n \log n)$, haciendo posibles los SNARKs prácticos.

Raíces de la Unidad

Definición 1.11 (Raíz n-ésima de la Unidad)**.** En $\mathbb{F}_p$, una **raíz n-ésima primitiva de la unidad** es un elemento ω tal que:

1. $\omega^n = 1$
2. $\omega^k \neq 1$ para $0 < k < n$

Las n raíces n-ésimas son $\{1, \omega, \omega^2, \ldots, \omega^{n-1}\}$.

Teorema 1.4 (Existencia de Raíces)**.** $\mathbb{F}_p$ *contiene una raíz n-ésima primitiva si y solo si* $n|(p-1)$.

Ejemplo 1.7 (Raíces en $\mathbb{F}_{17}$)**.** Como $17 - 1 = 16 = 2^4$, existen raíces n-ésimas para $n \in \{1, 2, 4, 8, 16\}$.

Para $n = 4$: buscamos ω con $\omega^4 = 1$ pero $\omega^2 \neq 1$.

- $2^4 = 16 \equiv -1 \pmod{17}$, así que $2^8 \equiv 1$. Entonces $\omega = 2^2 = 4$ tiene orden 4.

Verificación: $4^1 = 4$, $4^2 = 16 \equiv -1$, $4^3 = -4 \equiv 13$, $4^4 = 1$ ✓

Las 4 raíces cuartas son: $\{1, 4, 16, 13\} = \{4^0, 4^1, 4^2, 4^3\}$.

Number Theoretic Transform (NTT)

La NTT es la versión de la FFT sobre campos finitos.

Definición 1.12 (NTT)**.** Sea ω raíz n-ésima primitiva en $\mathbb{F}_p$. La **NTT** de un vector $(a_0, \ldots, a_{n-1})$ es:

$$\hat{a}_k = \sum_{j=0}^{n-1} a_j \cdot \omega^{jk} \mod p$$

Equivalentemente, si $f(x) = \sum_{j=0}^{n-1} a_j x^j$, entonces:

$$\text{NTT}(a) = (f(1), f(\omega), f(\omega^2), \ldots, f(\omega^{n-1}))$$

Idea Clave

La NTT convierte coeficientes en evaluaciones sobre raíces de la unidad.

- NTT: coeficientes $\rightarrow$ evaluaciones en $\{1, \omega, \omega^2, \ldots\}$
- INTT: evaluaciones $\rightarrow$ coeficientes

Algoritmo Cooley-Tukey

La eficiencia viene de la estructura recursiva. Si $n = 2m$:

$$f(x) = f_{\text{even}}(x^2) + x \cdot f_{\text{odd}}(x^2)$$

donde f_{even} tiene los coeficientes de índice par y f_{odd} los impares.

```
def ntt(a, omega, p):
    """NTT recursiva - O(n log n)"""
    n = len(a)
    if n == 1:
        return a

    # Dividir en pares e impares
    a_even = ntt(a[0::2], pow(omega, 2, p), p)
    a_odd = ntt(a[1::2], pow(omega, 2, p), p)

    # Combinar (butterfly)
    result = [0] * n
    omega_k = 1
    for k in range(n // 2):
        result[k] = (a_even[k] + omega_k * a_odd[k]) % p
        result[k + n//2] = (a_even[k] - omega_k * a_odd[k]) % p
        omega_k = (omega_k * omega) % p

    return result
```

Matemáticas

Complejidad:

- Evaluación ingenua en n puntos: $O(n^2)$
- NTT: $O(n \log n)$

- Para $n = 2^{20}$ (un millón de gates):
  - Ingenuo: 10^{12} operaciones
  - NTT: 20×10^6 operaciones
  - ¡Speedup de $50,000\times$!

Multiplicación de Polinomios con NTT

Teorema 1.5 (Convolución vía NTT)**.** *Para multiplicar dos polinomios f, g de grado $< n$:*

1. *Pad a longitud $2n$*
2. *$\hat{f} = NTT(f)$, $\hat{g} = NTT(g)$*
3. *$\hat{h}_k = \hat{f}_k \cdot \hat{g}_k$ (multiplicación punto a punto)*
4. *$h = INTT(\hat{h})$*

Costo total: $O(n \log n)$ vs $O(n^2)$ ingenuo.

En Banco ZK

¿Por qué importa en SNARKs/STARKs?
PLONK: Necesita multiplicar y dividir polinomios de grado $\sim n$ (número de gates). Sin NTT, generar una prueba para un circuito de 1 millón de gates tomaría horas. Con NTT, toma segundos.
STARKs: Usan FRI (Fast Reed-Solomon IOP) que requiere evaluar polinomios en dominios exponencialmente más grandes. La NTT hace esto factible.
KZG Commitments: El trusted setup genera evaluaciones de τ^i que se combinan con NTT para commits eficientes.

Advertencia

Para usar NTT necesitas que $n|(p-1)$ donde n es potencia de 2. Los primos "amigables con SNARK" se eligen específicamente para esto. Ejemplos:

- BN254: $p - 1 = 2^{28} \cdot \ldots$ (soporta NTT hasta 2^{28})
- Goldilocks: $p = 2^{64} - 2^{32} + 1$, $p - 1 = 2^{32} \cdot \ldots$
- BLS12-381: $p - 1 = 2^{32} \cdot \ldots$

1.2.5 Grupos Cíclicos y Supuestos de Seguridad

Esta sección formaliza la notación de grupos que usaremos constantemente y establece los problemas computacionalmente difíciles que fundamentan la seguridad criptográfica.

Grupos Cíclicos

Definición 1.13 (Grupo Cíclico)**.** Un grupo $\mathbb{G}$ es **cíclico** si existe un elemento $g \in \mathbb{G}$ (llamado **generador**) tal que:

$$\mathbb{G} = \langle g \rangle = \{g^0, g^1, g^2, \ldots, g^{n-1}\}$$

donde $n = |\mathbb{G}|$ es el **orden** del grupo.

Observación 1.1 (Notación). Usamos dos notaciones según el contexto:

- **Multiplicativa**: g^x (típica en $\mathbb{F}_p^*$, grupos abstractos)
- **Aditiva**: $[x]G = xG = G + G + \cdots + G$ (x veces) (típica en curvas elípticas)

Son equivalentes: $g^x \leftrightarrow [x]G$.

Ejemplo 1.8 (Grupos Cíclicos Comunes)**.**

1. $\mathbb{Z}_n = (\{0, 1, \ldots, n-1\}, +)$: grupo aditivo, generador 1
2. $\mathbb{F}_p^* = (\{1, 2, \ldots, p-1\}, \cdot)$: grupo multiplicativo, generador = raíz primitiva
3. $E(\mathbb{F}_p)$: puntos de curva elíptica (bajo ciertas condiciones)

El Problema del Logaritmo Discreto (DLOG)

Definición 1.14 (Problema DLOG)**.** Sea $\mathbb{G} = \langle g \rangle$ un grupo cíclico de orden n. Dado $h \in \mathbb{G}$, el **problema del logaritmo discreto** es encontrar $x \in \mathbb{Z}_n$ tal que:

$$g^x = h$$

El valor x se llama el **logaritmo discreto** de h en base g, denotado $x = \log_g h$.

Intuición

Calcular g^x dado g y x es fácil (exponenciación rápida: $O(\log x)$ multiplicaciones). Pero encontrar x dado g y g^x es difícil en ciertos grupos. Esta *asimetría* es la base de la criptografía de clave pública.

Definición 1.15 (Supuesto DLOG)**.** El **supuesto DLOG** afirma que para todo adversario probabilístico de tiempo polinomial $\mathcal{A}$:

$$\Pr\left[\mathcal{A}(g, g^x) = x \ : \ x \leftarrow \mathbb{Z}_n\right] \leq \mathsf{negl}(\lambda)$$

donde λ es el parámetro de seguridad y $\mathsf{negl}(\lambda)$ es una función negligible.

Advertencia

DLOG **no** es difícil en todos los grupos:

- En $(\mathbb{Z}_n, +)$: $\log_1 x = x$ (trivial)
- En $\mathbb{F}_p^*$ con p pequeño: algoritmo de Pohlig-Hellman

- En $\mathbb{F}_p^*$ con p grande pero $(p-1)$ suave: también vulnerable

Necesitamos grupos donde el mejor algoritmo conocido sea exponencial.

Supuestos Relacionados

Definición 1.16 (Computational Diffie-Hellman (CDH))**.** Dados g, g^a, g^b (con a, b aleatorios), es difícil computar g^{ab}.

Definición 1.17 (Decisional Diffie-Hellman (DDH))**.** Es difícil distinguir (g, g^a, g^b, g^{ab}) de (g, g^a, g^b, g^c) donde c es aleatorio.

Teorema 1.6 (Relación entre supuestos)**.**

$$DLOG \implies CDH \implies DDH$$

(Si puedes resolver DLOG, puedes resolver CDH; si puedes resolver CDH, puedes resolver DDH.)

Definición 1.18 (Knowledge of Exponent Assumption (KEA))**.** Si un algoritmo $\mathcal{A}$ produce $(C, \hat{C})$ tales que $\hat{C} = C^\alpha$ (donde α es desconocido para $\mathcal{A}$), entonces $\mathcal{A}$ "conoce" c tal que $C = g^c$.

Formalmente: existe un extractor $\mathcal{E}$ que, con acceso a $\mathcal{A}$, puede extraer c.

Idea Clave

KEA es fundamental para SNARKs: permite argumentar que el prover "conoce" el witness, no solo que produjo una prueba válida. Groth16 y otros sistemas usan variantes de KEA.

Tabla de Grupos y Seguridad

Grupo	DLOG	DDH	Mejor ataque conocido
$\mathbb{F}_p^*$	Difícil	Difícil	Index calculus: $L_p[1/3]$
$E(\mathbb{F}_p)$ (curva genérica)	Difícil	Difícil	Pollard-rho: $O(\sqrt{n})$
$E(\mathbb{F}_p)$ (con pairing)	Difícil	**Fácil**	Pairing $\to \mathbb{F}_{p^k}^*$

Matemáticas

En curvas con pairing, DDH es fácil porque:

$$e(g^a, g^b) = e(g, g)^{ab}$$

Podemos verificar si $g^c = g^{ab}$ chequeando $e(g, g^c) \stackrel{?}{=} e(g^a, g^b)$.
Esto no rompe DLOG ni CDH, pero significa que debemos usar otros supuestos (como co-CDH o q-SDH) para seguridad en sistemas basados en pairings.

Notación Estándar

Resumimos la notación que usaremos en el resto del libro:

Símbolo	Significado
$\mathbb{G}$	Grupo cíclico genérico
g, h	Generadores del grupo
n u $\lvert\mathbb{G}\rvert$	Orden del grupo
$\mathbb{Z}_n$	Exponentes (enteros módulo n)
g^x o $[x]g$	Exponenciación / multiplicación escalar
$\mathbb{G}_1, \mathbb{G}_2$	Grupos fuente de pairing
$\mathbb{G}_T$	Grupo objetivo de pairing
$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$	Pairing bilineal
$\mathsf{negl}(\lambda)$	Función negligible en parámetro de seguridad
$x \leftarrow S$	x muestreado uniformemente de S
$x \leftarrow \mathcal{A}(\cdot)$	x es output del algoritmo $\mathcal{A}$

En Banco ZK

Con esta base algebraica, estamos preparados para entender:

- **Pedersen Commitments**: $C = g^v h^r$ (hiding por DLOG)
- **KZG Commitments**: evaluaciones en τ secreto, verificación con pairing
- **PLONK**: polinomios sobre $\mathbb{F}_p$, NTT para eficiencia, KZG para commits
- **STARKs**: campos grandes, FRI usando NTT extensiva
- **Groth16**: pairings para verificación en 3 operaciones

Todo se construye sobre los conceptos de esta sección.

La Construcción Merkle-Damgård

SHA-256 utiliza la construcción **Merkle-Damgård**, propuesta independientemente por Ralph Merkle e Ivan Damgård en 1989. Este descubrimiento fue fundamental porque resolvió cómo construir una función hash segura para mensajes de cualquier longitud a partir de una función de compresión que solo procesa bloques de tamaño fijo.

Definición 1.19 (Función de Compresión)**.** Una **función de compresión** es una función determinística:

$$f : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n$$

que toma dos entradas:

- Un estado de n bits (llamado *chaining value*)
- Un bloque de mensaje de b bits

y produce una salida de n bits (el nuevo estado).

Definición 1.20 (Construcción Merkle-Damgård)**.** Sea $f : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n$ una función de compresión resistente a colisiones. La construcción MD define una función hash $H : \{0,1\}^* \to \{0,1\}^n$ como sigue:

Entrada: Mensaje M de longitud arbitraria

Proceso:

1. **Padding:** Extender M a longitud múltiplo de b bits:

$$M' = M \,||\, 1 \,||\, 0^k \,||\, \langle |M| \rangle_{64}$$

donde k es el mínimo tal que $|M| + 1 + k + 64 \equiv 0 \pmod{b}$

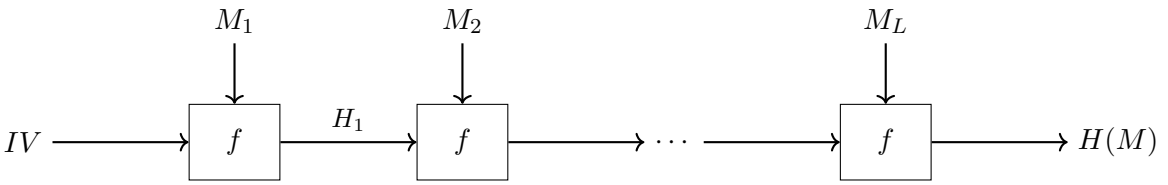
2. **División:** Particionar M' en bloques de b bits:

$$M' = M_1 \,||\, M_2 \,||\, \cdots \,||\, M_L$$

3. **Iteración:** Comenzando con un valor inicial fijo $H_0 = IV$:

$$H_1 = f(H_0, M_1) = f(IV, M_1) \tag{1.1}$$
$$H_2 = f(H_1, M_2) \tag{1.2}$$
$$\vdots \tag{1.3}$$
$$H_L = f(H_{L-1}, M_L) \tag{1.4}$$

4. **Salida:** $H(M) = H_L$



Intuición

Analogía de la trituradora de papel:
Imagina una trituradora de documentos especial:

1. Comienzas con una "semilla" específica en la máquina (IV)
2. Introduces la primera página del documento (M_1)
3. La máquina mezcla la semilla con la página y produce un nuevo estado interno (H_1)
4. Introduces la segunda página (M_2), se mezcla con H_1 produciendo H_2
5. Continúas hasta procesar todas las páginas

6. El estado final de la máquina es tu "hash"

Lo crucial es que cada página afecta todas las siguientes (efecto cascada), no puedes retroceder, y cambiar cualquier página cambia completamente el resultado final.

[Merkle-Damgård con mensaje "Hola"] Usaremos parámetros simplificados para ilustrar:

Paso 1: Mensaje original

```
"Hola" en ASCII:
H = 0x48, o = 0x6F, l = 0x6C, a = 0x61
Mensaje M = 32 bits
```

Paso 2: Calcular padding (para bloques de 512 bits)

$$\text{Longitud} = 32 \text{ bits} \tag{1.5}$$

$$\text{Padding} = 1 \,||\, 0^{423} \,||\, \langle 32 \rangle_{64} \tag{1.6}$$

Paso 3: Aplicar función de compresión

$$H_1 = f(IV, M_1) = f(\texttt{0x6a09e667...}, \texttt{[bloque con padding]}) \tag{1.7}$$

Como solo hay un bloque: $H(\text{"Hola"}) = H_1$

Parámetros Específicos de SHA-256

Parámetro	**Valor en SHA-256**
Tamaño de salida	256 bits
Tamaño de bloque	512 bits
Tamaño de palabra	32 bits
Número de rondas	64
Tamaño del estado interno	256 bits (8 palabras de 32 bits)
Campo de longitud	64 bits

Paso 1: Padding del Mensaje

Definición 1.21 (Padding SHA-256)**.** Dado un mensaje M de longitud L bits:

1. Añadir un bit '1' al final: $M \leftarrow M||1$
2. Añadir k bits '0', donde k es el mínimo tal que $L + 1 + k \equiv 448 \pmod{512}$
3. Añadir la longitud original L como entero de 64 bits (big-endian)

[Padding del mensaje "abc"] El mensaje "abc" en ASCII es:

```
a = 0x61 = 01100001
b = 0x62 = 01100010
c = 0x63 = 01100011
```

Mensaje original: 24 bits. Padding:

1. Añadir bit '1': ahora 25 bits
2. $25 + k \equiv 448 \pmod{512}$, entonces $k = 423$ ceros
3. Añadir longitud $L = 24$ como 64 bits: `0x0000000000000018`

Resultado (512 bits = 1 bloque):

```
61626380 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000018
```

Paso 2: Vector de Inicialización

Definición 1.22 (IV de SHA-256)**.** El estado inicial consiste en 8 palabras de 32 bits, derivadas de las partes fraccionarias de las raíces cuadradas de los primeros 8 primos:

$$H_0^{(0)} = \texttt{0x6a09e667} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{2}) \rfloor \tag{1.8}$$
$$H_1^{(0)} = \texttt{0xbb67ae85} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{3}) \rfloor \tag{1.9}$$
$$H_2^{(0)} = \texttt{0x3c6ef372} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{5}) \rfloor \tag{1.10}$$
$$H_3^{(0)} = \texttt{0xa54ff53a} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{7}) \rfloor \tag{1.11}$$
$$H_4^{(0)} = \texttt{0x510e527f} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{11}) \rfloor \tag{1.12}$$
$$H_5^{(0)} = \texttt{0x9b05688c} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{13}) \rfloor \tag{1.13}$$
$$H_6^{(0)} = \texttt{0x1f83d9ab} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{17}) \rfloor \tag{1.14}$$
$$H_7^{(0)} = \texttt{0x5be0cd19} \leftarrow \lfloor 2^{32} \times \text{frac}(\sqrt{19}) \rfloor \tag{1.15}$$

Intuición

¿Por qué raíces cuadradas de primos? Es una forma de generar constantes "nothing-up-my-sleeve" (nada bajo la manga). Cualquiera puede verificar que estas constantes no fueron elegidas maliciosamente — simplemente se derivan de matemáticas públicas.

Paso 3: Constantes de Ronda

SHA-256 usa 64 constantes $K_0, \ldots, K_{63}$, derivadas de las partes fraccionarias de las raíces cúbicas de los primeros 64 primos:

$$K_i = \lfloor 2^{32} \times \text{frac}(\sqrt[3]{p_i}) \rfloor$$

```
K[0..7]   = 428a2f98 71374491 b5c0fbcf e9b5dba5
            3956c25b 59f111f1 923f82a4 ab1c5ed5
K[8..15]  = d807aa98 12835b01 243185be 550c7dc3
            72be5d74 80deb1fe 9bdc06a7 c19bf174
    ... (continúa hasta K[63])
```

Paso 4: Expansión del Mensaje (Message Schedule)

Cada bloque de 512 bits se divide en 16 palabras de 32 bits $(W_0, \ldots, W_{15})$, y luego se expanden a 64 palabras:

Definición 1.23 (Message Schedule)**.**

$$W_t = M_t^{(i)} \quad \text{para } t = 0, \ldots, 15 \tag{1.16}$$

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \quad \text{para } t = 16, \ldots, 63 \tag{1.17}$$

Donde las funciones σ son:

$$\sigma_0(x) = \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x) \tag{1.18}$$

$$\sigma_1(x) = \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x) \tag{1.19}$$

Intuición

La expansión de mensaje asegura que cada bit del mensaje original influya en muchas rondas diferentes, amplificando el efecto avalancha.

Paso 5: Función de Compresión (64 Rondas)

Definición 1.24 (Funciones Auxiliares)**.**

$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z) \quad (\text{“choice”}) \tag{1.20}$$

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \quad (\text{“majority”}) \tag{1.21}$$

$$\Sigma_0(x) = \text{ROTR}^2(x) \oplus \text{ROTR}^{13}(x) \oplus \text{ROTR}^{22}(x) \tag{1.22}$$

$$\Sigma_1(x) = \text{ROTR}^6(x) \oplus \text{ROTR}^{11}(x) \oplus \text{ROTR}^{25}(x) \tag{1.23}$$

Intuición

- $Ch(x, y, z)$: “Si x, entonces y, sino z” — cada bit de x elige entre el bit correspondiente de y o z
- $Maj(x, y, z)$: Voto mayoritario bit a bit — el resultado es 1 si al menos 2 de los 3 bits son 1

Definición 1.25 (Una Ronda de SHA-256)**.** Dado el estado (a, b, c, d, e, f, g, h), en la ronda t:

$$T_1 = h + \Sigma_1(e) + Ch(e, f, g) + K_t + W_t \tag{1.24}$$

$$T_2 = \Sigma_0(a) + Maj(a, b, c) \tag{1.25}$$

$$h \leftarrow g, \quad g \leftarrow f, \quad f \leftarrow e, \quad e \leftarrow d + T_1 \tag{1.26}$$

$$d \leftarrow c, \quad c \leftarrow b, \quad b \leftarrow a, \quad a \leftarrow T_1 + T_2 \tag{1.27}$$

Ejemplo Completo: SHA-256(“abc”)

[Cálculo paso a paso] **Input:** “abc” (3 bytes)

Paso 1 - Padding:

```
61626380 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000018
```

Paso 2 - Estado inicial:

```
a = 6a09e667    e = 510e527f
b = bb67ae85    f = 9b05688c
c = 3c6ef372    g = 1f83d9ab
d = a54ff53a    h = 5be0cd19
```

Paso 3 - Primera ronda (t=0):

$$W_0 = \texttt{61626380} \tag{1.28}$$

$$K_0 = \texttt{428a2f98} \tag{1.29}$$

$$\Sigma_1(e) = \texttt{3a6fe667} \tag{1.30}$$

$$Ch(e, f, g) = \texttt{1f85c98c} \tag{1.31}$$

$$T_1 = h + \Sigma_1(e) + Ch(e, f, g) + K_0 + W_0 = \texttt{5d6aebb1} \tag{1.32}$$

(Continúa por 64 rondas...)

Resultado final:

```
SHA-256("abc") = ba7816bf8f01cfea414140de5dae2223
                 b00361a396177a9cb410ff61f20015ad
```

Efecto Avalancha

El **efecto avalancha** significa que cambiar un solo bit del input cambia aproximadamente el 50% de los bits del output.

[Demostración del efecto avalancha]

```
SHA-256("abc") = ba7816bf8f01cfea414140de5dae2223...
SHA-256("abd") = 6ca13d52ca70c883e0f0bb101e425a89...
```

Diferencia: 130 de 256 bits cambiaron (50.8%). Solo cambió 'c' por 'd'.

```
SHA-256("abc") = ba7816bf...
SHA-256("Abc") = 06d90109... (solo mayúscula)
```

Diferencia: 125 bits (48.8%)

Propiedades de Seguridad

Propiedad	Seguridad Ideal	Mejor Ataque Conocido
Preimagen	2^{256}	2^{254} (teórico, no práctico)
Segunda preimagen	2^{256}	2^{256}
Colisión	2^{128}	2^{128} (birthday attack)

Intuición

La seguridad contra colisiones es 2^{128} (no 2^{256}) debido a la **paradoja del cumpleaños**: en un grupo de 23 personas, hay >50% de probabilidad de que dos compartan cumpleaños. Análogamente, después de $\sqrt{2^{256}} = 2^{128}$ hashes, hay buena probabilidad de encontrar una colisión.

Vulnerabilidad: Ataque de Extensión de Longitud

Advertencia

La construcción Merkle-Damgård tiene una vulnerabilidad inherente: el **ataque de extensión de longitud**.

Definición 1.26 (Length Extension Attack)**.** Conociendo $H(M)$ y la longitud de M (pero NO el contenido de M), un atacante puede calcular:

$$H(M||\text{padding}||M')$$

para cualquier sufijo M' elegido por el atacante.

Intuición

El hash $H(M)$ es exactamente el estado interno después de procesar M. Si conocemos este estado y la longitud, podemos "continuar" el proceso de hashing con bloques adicionales, sin conocer M.

[Explotación práctica] **Escenario:** Un servidor web autentica requests usando:

```
token = SHA-256(secret_key || user_data)
```

Situación del atacante:

- Conoce un token válido para `user_data = "user=alice&role=guest"`
- Sabe que `secret_key` tiene 16 bytes
- Quiere crear token para `"...&role=admin"`

Ataque:

1. Calcula el padding que SHA-256 añadió internamente

2. Construye: `user_data || padding || "&role=admin"`
3. Usa el token original como estado inicial (en lugar de IV)
4. Calcula el nuevo hash para "&role=admin"

¡El servidor aceptará este token como válido, sin conocer `secret_key`!
Mitigaciones:

Mitigación	Descripción
HMAC	$\text{HMAC}(K, M) = H((K \oplus \text{opad}) \,\|\|\, H((K \oplus \text{ipad}) \,\|\|\, M))$
SHA-3	Construcción sponge, inmune por diseño
BLAKE2/3	Incluye contador de bytes, previene extensión
Truncamiento	SHA-224 es SHA-256 truncado con IV diferente

Operaciones Bitwise

Definición 1.27 (Operaciones en SHA-256)**.**

- **ROTR**$^n(x)$: Rotación circular a la derecha por n bits

$$\text{ROTR}^n(x) = (x \gg n) \vee (x \ll (32 - n))$$

- **SHR**$^n(x)$: Shift lógico a la derecha (rellena con ceros)

$$\text{SHR}^n(x) = x \gg n$$

- **XOR** ($\oplus$): OR exclusivo bit a bit
- **Suma mod** 2^{32}: Aritmética modular, descartando overflow

[Rotación vs Shift] Sea $x =$ `0x80000001` (bit más significativo y menos significativo en 1)
En binario: `10000000 00000000 00000000 00000001`

$$\text{ROTR}^1(x) = \texttt{0xC0000000} \quad \text{(el bit 1 “envuelve” a la izquierda)} \tag{1.33}$$

$$\text{SHR}^1(x) = \texttt{0x40000000} \quad \text{(el bit 1 se pierde)} \tag{1.34}$$

Aplicaciones en Criptomonedas

Bitcoin: Proof of Work

Definición 1.28 (Mining de Bitcoin)**.** Los mineros buscan un `nonce` tal que:

$$\text{SHA-256}(\text{SHA-256}(\text{block_header}||\text{nonce})) < \text{target}$$

Intuición

Bitcoin usa SHA-256 **doble** (SHA-256d) principalmente como defensa contra ataques de extensión de longitud.

[Dificultad de Bitcoin] En 2024, el target requiere aproximadamente 79 ceros iniciales en binario:

- Probabilidad de éxito por intento: $\approx 2^{-79}$
- Hashrate de la red: ≈ 500 EH/s (5×10^{20} hashes/segundo)
- Tiempo promedio para encontrar bloque: 10 minutos

Árboles de Merkle en Bitcoin Las transacciones se organizan en un árbol binario donde cada nodo es:

$$H_{ij} = \text{SHA-256d}(H_i || H_j)$$

Esto permite **SPV (Simplified Payment Verification)**: verificar una transacción con solo $\log_2(n)$ hashes.

Direcciones de Bitcoin

```
public_key (65 bytes)
    ↓ SHA-256
hash_1 (32 bytes)
    ↓ RIPEMD-160
hash_2 (20 bytes)
    ↓ + version + checksum (SHA-256d)
    ↓ Base58Check
Dirección Bitcoin (25-34 caracteres)
```

Comparación con Otras Funciones Hash

Función	Output	Bloques	Construcción	Estado
MD5	128 bits	512 bits	Merkle-Damgård	ROTO
SHA-1	160 bits	512 bits	Merkle-Damgård	ROTO
SHA-256	256 bits	512 bits	Merkle-Damgård	Seguro
SHA-512	512 bits	1024 bits	Merkle-Damgård	Seguro
SHA-3-256	256 bits	1088 bits	Sponge	Seguro
BLAKE3	256 bits	64 bytes	Merkle tree	Seguro

Rendimiento

Plataforma	Velocidad Típica	Notas
CPU moderno (software)	200-500 MB/s	Con SHA-NI: 2+ GB/s
GPU (RTX 3090)	∼1.5 GH/s	Optimizado para mining
ASIC (Antminer S19)	∼110 TH/s	Hardware especializado

Aplicación en Banco ZK

En Banco ZK

¿Por qué Banco ZK NO usa SHA-256 internamente?

Aunque SHA-256 es excelente para uso general, tiene un problema crítico para pruebas de conocimiento cero: su estructura de operaciones bitwise (rotaciones, XOR) es extremadamente costosa de representar en circuitos aritméticos.

Función Hash	Restricciones en Circuito ZK
SHA-256	∼25,000
Keccak	∼150,000
Poseidon	∼250

Decisión de diseño en Banco ZK:

- **Dentro del circuito ZK:** Poseidon (250 restricciones)
- **Interfaz con L1/externos:** SHA-256 cuando sea necesario
- **Nunca:** Construcciones vulnerables a length extension para autenticación

La reducción de 25,000 a 250 restricciones (100x) significa que las pruebas se generan mucho más rápido y son más baratas de verificar on-chain.

Implementación de Referencia (Pseudocódigo)

```
function sha256(message):
    # Paso 1: Padding
    padded = pad(message)
    blocks = split_into_512bit_blocks(padded)

    # Paso 2: Inicializar estado
    H = [0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
         0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19]

    # Paso 3: Procesar cada bloque
    for block in blocks:
        W = expand_message(block)  # 16 → 64 palabras
        a,b,c,d,e,f,g,h = H

        for t in range(64):
            T1 = h + 1(e) + Ch(e,f,g) + K[t] + W[t]
            T2 = 0(a) + Maj(a,b,c)
            h,g,f,e,d,c,b,a = g,f,e,(d+T1),c,b,a,(T1+T2)
```

```
        H = [x + y for x,y in zip(H, [a,b,c,d,e,f,g,h])]

    return concatenate(H)
```

Ejercicios Propuestos

1. Calcular manualmente el padding para el mensaje “Hello” (5 bytes).
2. Verificar que SHA-256(“”) (string vacío) = `e3b0c442...`
3. Implementar las funciones Ch, Maj, Σ_0, Σ_1 en tu lenguaje favorito.
4. Demostrar el efecto avalancha: hashear “test0”, “test1”, ..., “test9” y contar bits diferentes entre hashes consecutivos.
5. ¿Por qué Bitcoin usa SHA-256 doble en lugar de simple?
6. Calcular el padding completo para “ZK” (2 bytes) usando parámetros de SHA-256.
7. ¿Cuántos bloques de 512 bits se necesitan para hashear 447 bits? ¿Y para 448 bits? ¿Por qué la diferencia?

1.2.6 Keccak (SHA-3)

Definición 1.29 (Keccak)**.** Función hash basada en construcción esponja, estandarizada como SHA-3.

Arquitectura esponja:

- Estado de 1600 bits
- Parámetros: rate r y capacity c ($r + c = 1600$)
- Fase de absorción: XOR input con estado
- Fase de exprimido: extraer output
- Permutación Keccak-f[1600]: 24 rondas

Ventajas sobre SHA-2:

- Resistente a ataques de extensión de longitud
- Diseño completamente diferente (diversidad)
- Parámetros flexibles (SHAKE128, SHAKE256)

Usado en: Ethereum (direcciones, state root), aplicaciones post-SHA-2.

Contexto Histórico: La Competencia SHA-3

Intuición

En 2007, NIST (National Institute of Standards and Technology) anunció una competencia pública para seleccionar un nuevo estándar de hash llamado SHA-3. La motivación no era que SHA-2 estuviera roto, sino que SHA-2 compartía diseño similar con SHA-1 (que mostraba debilidades). NIST quería un "plan B" con arquitectura completamente diferente.

La competencia SHA-3 fue una de las más importantes en la historia de la criptografía:

Fase	Detalle
2007	Convocatoria abierta
2008	64 candidatos presentados
2009	Ronda 1: 51 candidatos aceptados, 14 pasan a Ronda 2
2010	Ronda 2: 5 finalistas seleccionados
2012	**Keccak** declarado ganador
2015	Publicación oficial de SHA-3 (FIPS 202)

Los 5 finalistas fueron:

- **Keccak** (Bertoni, Daemen, Peeters, Van Assche) — *Ganador*
- BLAKE (Aumasson et al.) — Muy rápido en software
- Grøstl (Knudsen et al.) — Basado en AES
- JH (Wu) — Diseño elegante
- Skein (Schneier et al.) — Basado en Threefish

Observación 1.2. Joan Daemen, uno de los creadores de Keccak, también co-diseñó AES (Rijndael). Su experiencia en diseño de primitivas criptográficas fue crucial para la seguridad de Keccak.

La Construcción Sponge

A diferencia de Merkle-Damgård (usada en SHA-2), Keccak usa la **construcción sponge** (esponja), una arquitectura fundamentalmente diferente.

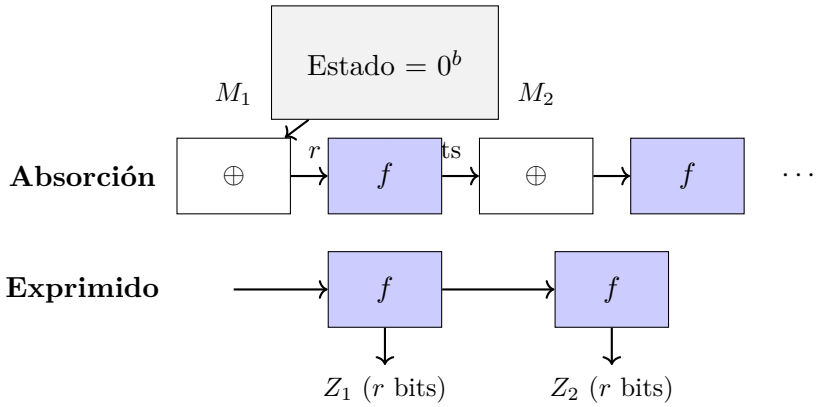
Definición 1.30 (Construcción Sponge)**.** Una construcción sponge es una función criptográfica que:

1. Mantiene un **estado interno** de b bits, dividido en:
   - **Rate** (r bits): parte que interactúa con input/output
   - **Capacity** (c bits): parte que *nunca* se expone directamente
2. Opera en dos fases:
   - **Absorción**: el mensaje se "absorbe" en el estado

- **Exprimido**: el hash se "exprime" del estado

3. Usa una **permutación** f para mezclar el estado

Construcción Sponge



Idea Clave

La seguridad de la construcción sponge depende de la **capacity** c. Esta parte del estado **nunca** se expone ni se modifica directamente por el input. Un atacante tendría que adivinar $2^{c/2}$ estados posibles para montar un ataque de colisión.
En SHA-3-256: $c = 512$ bits $\Rightarrow$ seguridad de 2^{256} contra colisiones.

Intuición

Analogía de la esponja real:
Imagina una esponja de cocina:

1. **Absorción**: Sumerges la esponja en agua (tu mensaje). El agua entra gradualmente, mezclándose con lo que ya había dentro.

2. **Exprimido**: Aprietas la esponja y sale agua. Pero no sale *toda* el agua — parte queda atrapada en el interior (la capacity).

La parte que "queda atrapada" es lo que hace segura a la construcción: aunque veas lo que sale (el hash), no puedes saber el estado interno completo.

Parámetros de SHA-3

Variante	Output	Rate r	Capacity c	Seguridad
SHA-3-224	224 bits	1152	448	112 bits
SHA-3-256	256 bits	1088	512	128 bits
SHA-3-384	384 bits	832	768	192 bits
SHA-3-512	512 bits	576	1024	256 bits
SHAKE128	Variable	1344	256	128 bits
SHAKE256	Variable	1088	512	256 bits

Matemáticas

La relación fundamental es:

$$r + c = b = 1600 \text{ bits}$$

El **nivel de seguridad** contra ataques genéricos es:

- Preimagen: $\min(2^{output}, 2^c)$
- Colisión: $\min(2^{output/2}, 2^{c/2})$

Por ejemplo, SHA-3-256 tiene $c = 512$, por lo que la seguridad contra colisiones es $\min(2^{128}, 2^{256}) = 2^{128}$.

El Estado Interno: Estructura 3D

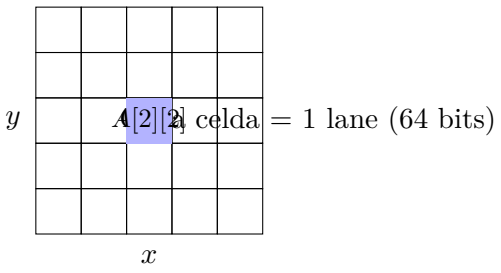
Definición 1.31 (Estado de Keccak)**.** El estado de 1600 bits se organiza como un arreglo tridimensional:

$$A[x][y][z] \quad \text{donde } x, y \in \{0, 1, 2, 3, 4\}, \quad z \in \{0, 1, \ldots, 63\}$$

Esto forma:

- $5 \times 5 = 25$ **lanes** (carriles) de 64 bits cada uno
- $25 \times 64 = 1600$ bits totales
- Visualización: un cubo de $5 \times 5 \times 64$

Vista superior del estado



La Permutación Keccak-f: 24 Rondas

La permutación f que mezcla el estado consiste en 24 rondas, cada una aplicando 5 operaciones en secuencia:

Definición 1.32 (Ronda de Keccak-f)**.** Cada ronda R_i consiste en:

$$R_i = \iota \circ \chi \circ \pi \circ \rho \circ \theta$$

Aplicadas en este orden:

1. θ (theta) — Paridad y difusión lineal

2. ρ (rho) — Rotación de bits dentro de cada lane
3. π (pi) — Permutación de posiciones de lanes
4. χ (chi) — Transformación no lineal
5. ι (iota) — Adición de constante de ronda

Operación θ (Theta): Difusión

Definición 1.33 (Theta)**.**

$$C[x] = A[x][0] \oplus A[x][1] \oplus A[x][2] \oplus A[x][3] \oplus A[x][4] \tag{1.35}$$
$$D[x] = C[x-1] \oplus \mathrm{ROT}(C[x+1], 1) \tag{1.36}$$
$$A'[x][y] = A[x][y] \oplus D[x] \tag{1.37}$$

donde los índices son módulo 5.

Intuición

θ calcula la "paridad" de cada columna y la propaga a las columnas vecinas. Esto asegura que cambiar un solo bit afecte a múltiples posiciones — el inicio del efecto avalancha.

Operación ρ (Rho): Rotación

Definición 1.34 (Rho)**.** Cada lane $A[x][y]$ se rota por un número diferente de bits:

$$A'[x][y][z] = A[x][y][z - r[x][y] \mod 64]$$

Los offsets de rotación $r[x][y]$ son:

$r[x][y]$	$y=0$	$y=1$	$y=2$	$y=3$	$y=4$
$x=0$	0	36	3	41	18
$x=1$	1	44	10	45	2
$x=2$	62	6	43	15	61
$x=3$	28	55	25	21	56
$x=4$	27	20	39	8	14

Operación π (Pi): Permutación

Definición 1.35 (Pi)**.** Los lanes se reordenan según:

$$A'[x][y] = A[x + 3y \mod 5][x]$$

Esta es una permutación de las 25 posiciones (x, y) en el plano.

Operación χ (Chi): No Linealidad

Definición 1.36 (Chi)**.**

$$A'[x][y][z] = A[x][y][z] \oplus ((\neg A[x+1][y][z]) \wedge A[x+2][y][z])$$

Idea Clave

χ es la **única operación no lineal** en Keccak. Sin ella, todo sería lineal (sumas y XORs) y fácilmente invertible. La no linealidad es esencial para la seguridad criptográfica — impide que un atacante resuelva ecuaciones simples para revertir el hash.

Operación ι (Iota): Constante de Ronda

Definición 1.37 (Iota)**.**

$$A'[0][0] = A[0][0] \oplus RC_i$$

donde RC_i es una constante diferente para cada ronda $i \in \{0, \ldots, 23\}$.

Intuición

Sin ι, todas las rondas serían idénticas. Un atacante podría explotar esta simetría. Las constantes RC_i "rompen la simetría" y aseguran que cada ronda sea única.

Algoritmo Completo

Ejemplo 1.9 (Procesamiento de un mensaje)**. Input:** Mensaje M de longitud arbitraria

Paso 1: Padding

Keccak usa padding simple: $M' = M \,||\, 0x06 \,||\, 0x00^* \,||\, 0x80$

(El padding de SHA-3 es ligeramente diferente: añade bits específicos para distinguir SHA-3 de SHAKE)

Paso 2: Inicializar estado

$$S = 0^{1600} \quad \text{(1600 bits en cero)}$$

Paso 3: Absorción

Dividir M' en bloques de r bits: $M' = M_1 || M_2 || \cdots || M_k$

```
for i = 1 to k:
    S[0..r-1] = S[0..r-1] XOR M_i    # XOR con rate
    S = Keccak-f(S)                   # Aplicar permutación
```

Paso 4: Exprimido

```
Z = ""
while len(Z) < output_length:
    Z = Z || S[0..r-1]    # Extraer rate
    S = Keccak-f(S)       # Aplicar permutación
return Z[0..output_length-1]
```

¿Por Qué es Inmune al Ataque de Extensión de Longitud?

Advertencia

Recordemos que SHA-256 (Merkle-Damgård) es vulnerable: conociendo $H(M)$ y $|M|$, se puede calcular $H(M||pad||M')$ sin conocer M.

Idea Clave

Keccak es **inmune** a este ataque porque el hash NO revela el estado interno completo. En Merkle-Damgård:

$$H(M) = \text{estado completo después de procesar } M$$

En Sponge:

$$H(M) = \text{solo los primeros } r \text{ bits del estado}$$

La capacity (c bits) **nunca se revela**. Sin conocer esos c bits, el atacante no puede continuar el cálculo.

Ejemplo 1.10 (Por qué falla el ataque)**.** Supongamos SHA-3-256 ($r = 1088$, $c = 512$):

1. El atacante conoce $H(M) = 256$ bits
2. El estado interno tiene 1600 bits
3. El atacante conoce como máximo 256 bits (el hash)
4. Faltan $1600 - 256 = 1344$ bits desconocidos
5. Para continuar el cálculo, necesitaría adivinar al menos los 512 bits de capacity
6. Complejidad: 2^{512} intentos — completamente impracticable

SHAKE: Funciones de Salida Extensible (XOF)

Definición 1.38 (XOF - Extendable Output Function)**.** Una XOF es una función hash que puede producir salida de longitud arbitraria:

$$\text{SHAKE}n(M, d) : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^d$$

donde d es la longitud de salida deseada.

Ejemplo 1.11 (Usos de SHAKE)**.**
```
SHAKE128("password", 256)  -> 256 bits de output
SHAKE128("password", 512)  -> 512 bits de output
SHAKE128("password", 1024) -> 1024 bits de output
SHAKE256("key_material", 32768) -> 32KB de output
```
Todos con el mismo mensaje pero diferente longitud de salida.

Idea Clave

SHAKE es posible gracias a la construcción sponge. Durante la fase de exprimido, puedes seguir "exprimiendo" tantos bloques como necesites. En Merkle-Damgård esto no es posible porque el output es exactamente el estado final, de tamaño fijo.

Aplicaciones de XOF:

- **Derivación de claves**: Generar múltiples claves desde un secreto master
- **Padding criptográfico**: Generar máscaras de longitud variable
- **Generación de números pseudoaleatorios**: CSPRNG basado en SHAKE
- **Firmas post-cuánticas**: SPHINCS+, Dilithium usan SHAKE internamente

Comparación SHA-2 vs SHA-3

Característica	SHA-2 (SHA-256)	SHA-3 (Keccak)
Construcción	Merkle-Damgård	Sponge
Operaciones	AND, OR, XOR, ROT, ADD	XOR, ROT, AND, NOT
Estado interno	256 bits	1600 bits
Rondas	64	24
Extension attack	Vulnerable	Inmune
Salida variable	No	Sí (SHAKE)
Velocidad (software)	Más rápido	Más lento
Velocidad (hardware)	Similar	Muy eficiente
Madurez	Más analizado	Más nuevo

Observación 1.3. SHA-3 es generalmente más lento que SHA-256 en CPUs convencionales, pero puede ser muy eficiente en hardware dedicado debido a su estructura regular. En ASICs, Keccak puede superar a SHA-256.

Keccak en Ethereum

Intuición

Ethereum usa una versión de Keccak que es **ligeramente diferente** del SHA-3 estandarizado. Esto se debe a que Ethereum adoptó Keccak antes de que NIST finalizara SHA-3 con cambios menores en el padding.

Definición 1.39 (Keccak-256 vs SHA-3-256)**.**

- **Keccak-256** (Ethereum): Padding $0x01$
- **SHA-3-256** (NIST): Padding $0x06$

Para el mismo input, producen hashes **diferentes**.

Ejemplo 1.12 (Diferencia práctica)**.** `Keccak-256("") = c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470`

```
SHA-3-256("")  = a7ffc6f8bf1ed76651c14756a061d662
                 f580ff4de43b49fa82d80a4b80f8434a
```

¡Hashes completamente diferentes para el string vacío!

Usos de Keccak-256 en Ethereum:

- **Direcciones**: address = Keccak-256(pubkey)[12..31] (últimos 20 bytes)
- **State root**: Raíz del Merkle Patricia Trie
- **Transaction hash**: Identificador único de transacciones
- **Function selectors**: selector = Keccak-256(signature)[0..3] (primeros 4 bytes)

Ejemplo 1.13 (Selector de función en Solidity). `function transfer(address to, uint256 amount)`

```
signature = "transfer(address,uint256)"
selector = Keccak-256(signature)[0:4] = 0xa9059cbb
```

Por eso en los calldata de transacciones ERC-20 siempre ves `0xa9059cbb...`

Rendimiento

Plataforma	SHA-256	SHA-3-256	Ratio
Intel x86-64 (software)	500 MB/s	300 MB/s	SHA-2 1.7x
Intel con SHA-NI	2000 MB/s	300 MB/s	SHA-2 6.7x
ARM Cortex-A72	200 MB/s	350 MB/s	SHA-3 1.75x
FPGA (optimizado)	Variable	Muy eficiente	SHA-3 gana
Circuito ZK (R1CS)	25,000	150,000	SHA-2 6x

Advertencia

En **circuitos ZK**, tanto SHA-256 como Keccak son extremadamente costosos:

- SHA-256: ∼25,000 restricciones
- Keccak-256: ∼150,000 restricciones

Ambos usan operaciones bitwise (XOR, AND, rotaciones) que son muy ineficientes en aritmética de campos finitos. Por esto se desarrollaron hashes "ZK-friendly" como Poseidon.

Seguridad: Estado del Arte

Ataque	SHA-3-256 Completo	Mejor Ataque Conocido
Preimagen	2^{256}	2^{256} (no hay ataque)
Colisión	2^{128}	2^{128} (no hay ataque)
Distinguisher	—	8 rondas (de 24)

Idea Clave

SHA-3 tiene un margen de seguridad muy amplio. Los mejores ataques conocidos solo afectan versiones reducidas (8 de 24 rondas). El diseño con 24 rondas tiene un factor de seguridad de 3x sobre los mejores ataques.

Aplicación en Banco ZK

En Banco ZK

Rol de Keccak en Banco ZK:

1. Interoperabilidad con Ethereum: Para verificar datos de Ethereum dentro de circuitos ZK (como probar que una transacción existe), necesitamos verificar hashes Keccak. Esto es costoso (∼150,000 restricciones) pero necesario para bridges y verificación cross-chain.

2. Decisión de diseño:

Uso	Hash	Restricciones
Estado interno	Poseidon	∼250
Nullifiers	Poseidon	∼250
Merkle trees	Poseidon	∼250
Verificar Ethereum	Keccak	∼150,000
Compatibilidad Bitcoin	SHA-256	∼25,000

3. Optimización: Cuando es posible, movemos la verificación de Keccak/SHA-256 fuera del circuito. Por ejemplo, verificamos el hash en la capa de contrato inteligente y solo pasamos el resultado al circuito ZK como input público.

4. Futuro: Proyectos como "Keccak circuits optimizados" y "precompiles de hash en L2" están reduciendo estos costos. Algunos ZK-rollups ya ofrecen precompiles que reducen el costo de Keccak a ∼50,000 restricciones.

Ejercicios

1. Calcular manualmente una ronda de θ para un estado simplificado de 5×5 bits (en lugar de $5 \times 5 \times 64$).

2. Si SHA-3-256 tiene $c = 512$ bits de capacity, ¿cuál es la complejidad de un ataque de colisión? ¿Y de preimagen?

3. Explicar por qué SHAKE puede producir salida de cualquier longitud mientras que SHA-256 no puede.

4. Un sistema usa: `token = Keccak(secret || data)`. ¿Es vulnerable a ataques de extensión de longitud? Justificar.

5. Dado que Ethereum usa Keccak-256 (no SHA-3-256), ¿qué sucedería si un desarrollador usa una librería de SHA-3 estándar para verificar direcciones Ethereum?

6. Calcular el function selector para `approve(address,uint256)` en Solidity.

7. Si Keccak cuesta 150,000 restricciones en un circuito ZK y Poseidon cuesta 250, ¿cuántas operaciones Poseidon equivalen a una operación Keccak? ¿Qué implicaciones tiene esto para el diseño de protocolos ZK?

1.2.7 Poseidon

Definición 1.40 (Poseidon)**.** Función hash algebraica diseñada para eficiencia en circuitos ZK.

Arquitectura:

- Opera sobre campos finitos $\mathbb{F}_p$ nativamente
- S-box: $x \mapsto x^\alpha$ (típicamente $\alpha = 5$)
- Capa lineal: multiplicación por matriz MDS
- Rondas parciales + rondas completas

Matemáticas

Costo en constraints:

- Poseidon: ∼250 constraints
- SHA-256: ∼25,000 constraints
- Keccak: ∼150,000 constraints

Por esto Poseidon es el estándar en ZK.

Otros hashes algebraicos:

- **MiMC:** $x \mapsto x^3$, simple pero lento
- **Rescue:** Alternancia de S-boxes
- **Griffin:** Optimizado para STARKs
- **Anemoi:** Nuevo, muy eficiente

El Problema: ¿Por Qué Necesitamos Hashes Nuevos?

Intuición

Las pruebas de conocimiento cero funcionan convirtiendo computaciones en ecuaciones polinomiales sobre campos finitos. Cada operación del programa original se traduce en "restricciones" (constraints) que el prover debe satisfacer.
El problema es que SHA-256 y Keccak usan operaciones **bitwise**: AND, OR, XOR, rotaciones. Estas operaciones son baratas en CPUs pero **extremadamente costosas** en aritmética de campos finitos.

Ejemplo 1.14 (Costo de operaciones bitwise en ZK)**.** Para representar XOR de dos valores de 256 bits en un circuito aritmético:

Paso 1: Descomponer cada número en 256 bits individuales

$$x = \sum_{i=0}^{255} b_i \cdot 2^i \quad \text{donde } b_i \in \{0, 1\}$$

Esto requiere 256 constraints para verificar que cada b_i es binario: $b_i \cdot (1 - b_i) = 0$

Paso 2: Calcular XOR bit a bit

$$(a \oplus b)_i = a_i + b_i - 2 \cdot a_i \cdot b_i$$

256 constraints más.

Paso 3: Recomponer el resultado Otros 256 constraints.

Total para UN XOR de 256 bits: ∼768 constraints

SHA-256 tiene cientos de operaciones XOR, AND, rotaciones... Por eso cuesta ∼25,000 constraints.

Idea Clave

La solución es diseñar funciones hash que usen operaciones **nativas** en campos finitos:

- Suma: $a + b \mod p \to 0$ constraints (gratis)
- Multiplicación: $a \times b \mod p \to 1$ constraint
- Potenciación: $x^5 \to 4$ multiplicaciones $\to 4$ constraints

Poseidon está diseñado específicamente para minimizar el número de multiplicaciones.

Historia y Contexto

Poseidon fue presentado en 2019 por un equipo de investigadores:

- Lorenzo Grassi (Universidad de Graz)
- Dmitry Khovratovich (Ethereum Foundation)
- Christian Rechberger (TU Graz)
- Arnab Roy (Fujitsu Labs)
- Markus Schofnegger (TU Graz)

Observación 1.4. El nombre “Poseidon” viene del dios griego del mar. Los autores usaron una temática de dioses griegos para sus hashes: **Poseidon** (hash general), **Neptune** (para compromisos), **Starkad** y **Poseidon**$^\pi$ (variantes optimizadas).

La evolución de hashes ZK-friendly:

Año	Hash	Constraints (aprox.)	Innovación
2016	MiMC	∼700	Primera propuesta algebraica
2019	Poseidon	∼250	Rondas parciales
2019	Rescue	∼400	S-boxes alternadas
2022	Griffin	∼150	Optimizado para STARKs
2022	Anemoi	∼120	Estado del arte

Arquitectura de Poseidon

Definición 1.41 (Estructura de Poseidon)**.** Poseidon opera sobre un estado de t elementos de campo $\mathbb{F}_p$:

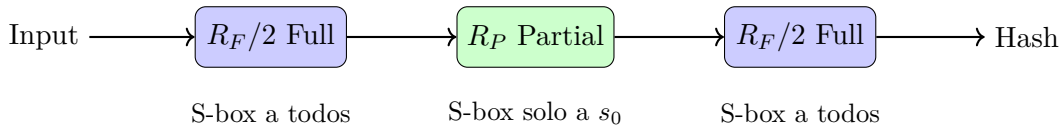
$$\text{state} = (s_0, s_1, \ldots, s_{t-1}) \in \mathbb{F}_p^t$$

Cada ronda aplica tres operaciones:

1. **AddRoundConstants (ARC):** $s_i \leftarrow s_i + c_i$
2. **S-box:** $s_i \leftarrow s_i^\alpha$ (o solo a s_0 en rondas parciales)
3. **MDS:** state $\leftarrow M \cdot$ state

La estructura de rondas es:

$$R_F/2 \text{ rondas completas} \to R_P \text{ rondas parciales} \to R_F/2 \text{ rondas completas}$$



La S-box: $x \mapsto x^\alpha$

Definición 1.42 (S-box de Poseidon)**.** La S-box es la función no lineal:

$$S(x) = x^\alpha$$

donde α es el menor entero positivo tal que $\gcd(\alpha, p-1) = 1$.
Valores típicos:

- $\alpha = 5$ para la mayoría de campos primos usados en SNARKs
- $\alpha = 3$ no sirve cuando $3|(p-1)$ (común en curvas elípticas)
- $\alpha = 7$ usado en algunos campos específicos

Matemáticas

¿Por qué necesitamos $\gcd(\alpha, p-1) = 1$?
Para que la S-box sea una **permutación** (biyectiva), necesitamos que $x \mapsto x^\alpha$ sea invertible. Esto ocurre si y solo si α es coprimo con $p-1$.

La inversa es:

$$S^{-1}(y) = y^{\alpha^{-1} \mod (p-1)}$$

Por ejemplo, si $\alpha = 5$ y $p - 1 = 4 \cdot k$ para algún k, entonces $\gcd(5, p-1)$ podría no ser 1. Hay que verificar para cada campo.

Ejemplo 1.15 (Cálculo de S-box)**.** Sea $p = 17$ (campo $\mathbb{F}_{17}$) y $\alpha = 5$.

Verificación: $p - 1 = 16$, y $\gcd(5, 16) = 1$

Calculamos $S(x) = x^5 \mod 17$:

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x^5	1	15	5	4	14	7	11	9	16	10	6	3	13	2	12	8

Observamos que es una permutación: cada valor de 1 a 16 aparece exactamente una vez.

Idea Clave

El costo de x^5 en un circuito R1CS es solo **4 constraints**:

$$x^2 = x \cdot x \quad \text{(1 constraint)} \tag{1.38}$$

$$x^4 = x^2 \cdot x^2 \quad \text{(1 constraint)} \tag{1.39}$$

$$x^5 = x^4 \cdot x \quad \text{(1 constraint)} \tag{1.40}$$

Más un constraint auxiliar para el routing del resultado. Comparado con los ∼768 constraints de un XOR, esto es 200x más eficiente.

La Matriz MDS

Definición 1.43 (Matriz MDS)**.** Una matriz $M \in \mathbb{F}_p^{t \times t}$ es **MDS** (Maximum Distance Separable) si todas sus submatrices cuadradas son invertibles.

Esto garantiza **difusión óptima**: cambiar un elemento del estado afecta a todos los demás después de una multiplicación por M.

Intuición

La matriz MDS es como un "mezclador perfecto". Imagina t líquidos de colores en t vasos. Multiplicar por una matriz MDS es como mezclarlos de tal forma que cada vaso final contiene partes de **todos** los líquidos originales.
Sin esta mezcla, los bits de entrada podrían "viajar" por el estado sin interactuar, debilitando la seguridad.

Ejemplo 1.16 (Matriz MDS para $t = 3$)**.** Una construcción común es la matriz de Cauchy. Para $t = 3$:

$$M = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

Verificación de que es MDS (todas las submatrices $k \times k$ son invertibles):

- Submatrices 1×1: $\det(2) = 2 \neq 0$
- Submatrices 2×2: $\det \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = 3 \neq 0$
- Matriz completa: $\det(M) = 4 \neq 0$

Matemáticas

En la práctica, Poseidon usa matrices MDS construidas a partir de:

- **Matrices de Cauchy**: $M_{ij} = \frac{1}{x_i - y_j}$ para secuencias x, y disjuntas
- **Matrices circulantes**: Eficientes para implementación

La elección depende del campo y del tamaño del estado. Las matrices se precomputan y son parte de la especificación de cada instancia de Poseidon.

Rondas Completas vs Parciales

Definición 1.44 (Tipos de Rondas)**. Ronda Completa (Full Round):**

1. Sumar constantes: $s_i \leftarrow s_i + c_i$ para todo i
2. Aplicar S-box: $s_i \leftarrow s_i^5$ para **todo** i
3. Multiplicar por MDS: state $\leftarrow M \cdot$ state

Ronda Parcial (Partial Round):

1. Sumar constantes: $s_i \leftarrow s_i + c_i$ para todo i
2. Aplicar S-box: $s_0 \leftarrow s_0^5$ **solo al primer elemento**
3. Multiplicar por MDS: state $\leftarrow M \cdot$ state

Idea Clave

Esta es la innovación clave de Poseidon. Las rondas parciales ahorran $(t-1)$ S-boxes por ronda, manteniendo la seguridad gracias a la difusión de la matriz MDS.
Costo comparativo para $t = 3$:

- Ronda completa: $3 \times 4 = 12$ constraints (3 S-boxes)
- Ronda parcial: $1 \times 4 = 4$ constraints (1 S-box)

Ahorro: 67% por ronda parcial.

Ejemplo 1.17 (Parámetros típicos para seguridad de 128 bits)**.** Para campo BN254 (usado en muchos SNARKs) con $t = 3$:

Parámetro	Símbolo	Valor	Constraints
Rondas completas	R_F	8	$8 \times 3 \times 4 = 96$
Rondas parciales	R_P	57	$57 \times 1 \times 4 = 228$
Constantes de ronda	—	$65 \times 3 = 195$	0 (precomputadas)
Multiplicación MDS	—	65 matrices	$\sim$18
Total			$\sim$342

En la práctica, con optimizaciones, se llega a $\sim$250 constraints.

Algoritmo Completo

Ejemplo 1.18 (Poseidon Hash paso a paso)**. Parámetros:**

- Campo: $\mathbb{F}_p$ con $p = 101$ (ejemplo pequeño)
- Estado: $t = 3$
- S-box: $\alpha = 5$
- Rondas: $R_F = 2$, $R_P = 1$ (simplificado)
- Matriz MDS: $M = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$

Input: Mensaje $m = (15, 27)$

Paso 1: Inicializar estado

$$\text{state} = (0, 15, 27)$$

(El primer elemento es el "capacity", los otros son el rate)

Paso 2: Ronda completa 1

$$\text{ARC}: \quad (0 + 3, 15 + 7, 27 + 11) = (3, 22, 38) \tag{1.41}$$

$$\text{S-box}: \quad (3^5, 22^5, 38^5) \mod 101 = (42, 79, 85) \tag{1.42}$$

$$\text{MDS}: \quad M \cdot (42, 79, 85)^T = (89, 67, 52) \tag{1.43}$$

Paso 3: Ronda parcial

$$\text{ARC}: \quad (89 + 13, 67 + 17, 52 + 19) = (1, 84, 71) \tag{1.44}$$

$$\text{S-box (solo } s_0): \quad (1^5, 84, 71) = (1, 84, 71) \tag{1.45}$$

$$\text{MDS}: \quad M \cdot (1, 84, 71)^T = (56, 68, 57) \tag{1.46}$$

Paso 4: Ronda completa 2

$$\text{ARC}: \quad (56 + 23, 68 + 29, 57 + 31) = (79, 97, 88) \tag{1.47}$$

$$\text{S-box}: \quad (79^5, 97^5, 88^5) \mod 101 = (31, 12, 44) \tag{1.48}$$

$$\text{MDS}: \quad M \cdot (31, 12, 44)^T = (18, 99, 32) \tag{1.49}$$

Salida: $H(15, 27) = 18$ (primer elemento del estado final)

Modos de Operación

Definición 1.45 (Construcción Sponge en Poseidon)**.** Poseidon típicamente usa una construcción sponge con:

- **Capacity**: Primer(os) elemento(s) del estado (nunca absorbemos/exprimimos aquí)
- **Rate**: Resto del estado (interactúa con input/output)

Para $t = 3$ con capacity 1 y rate 2:

```
state = [capacity | rate_0 | rate_1]
             ↑          ↑        ↑
          nunca      absorbe/  absorbe/
          expuesto   exprime   exprime
```

Modos comunes:

- **Hash**: Absorber mensaje, exprimir un elemento
- **Merkle tree**: $H(left, right) \to root$
- **Commitment**: $C = H(value, randomness)$
- **PRF**: $k = H(secret, nonce)$

Seguridad

Idea Clave

Poseidon está diseñado para resistir tres tipos principales de ataques:

1. **Ataques algebraicos**: Resolver el sistema de ecuaciones polinomiales
2. **Ataques diferenciales**: Explotar diferencias que se propagan de forma predecible
3. **Ataques de interpolación**: Aproximar la función con polinomios de bajo grado

Definición 1.46 (Márgenes de Seguridad)**.** Para Poseidon con 128 bits de seguridad:

- **Ataques algebraicos**: Requieren resolver sistemas de grado 5^R donde R es el número de rondas con S-boxes activas
- **Ataques diferenciales**: La matriz MDS asegura que diferencias se propaguen a todos los elementos
- **Interpolación**: El grado del polinomio después de todas las rondas excede 2^{128}

Advertencia

Poseidon es relativamente nuevo (2019) comparado con SHA-256 (2001). Aunque ha sido bien analizado, algunos criptógrafos recomiendan usar parámetros conservadores (más rondas) en aplicaciones críticas.
En 2023, se publicaron ataques de interpolación mejorados que no rompen Poseidon con parámetros estándar, pero redujeron el margen de seguridad. Los diseñadores respondieron recomendando parámetros actualizados.

Otros Hashes Algebraicos

MiMC (2016)

Definición 1.47 (MiMC)**.** MiMC es el primer hash algebraico ampliamente usado:

$$F(x) = (x + k + c_i)^3$$

aplicado iterativamente por r rondas (típicamente $r \approx 82$ para 128 bits de seguridad).

	MiMC	**Poseidon**
S-box	x^3	x^5
Rondas	∼82	∼65 (8 full + 57 partial)
Constraints	∼700	∼250
Estructura	Una sola rama	Estado de t elementos

Observación 1.5. MiMC tiene estructura más simple pero es menos eficiente porque no tiene el concepto de rondas parciales.

Rescue (2019)

Definición 1.48 (Rescue)**.** Rescue alterna entre dos S-boxes:

- $x \mapsto x^\alpha$ (forward)
- $x \mapsto x^{1/\alpha}$ (backward, es decir $x^{\alpha^{-1} \mod (p-1)}$)

Esto proporciona seguridad simétrica: es igualmente difícil atacar en dirección forward o backward.

Matemáticas

La inversión $x^{1/\alpha}$ es costosa de calcular pero verifica que el prover realmente hizo el trabajo. Esto es útil en esquemas donde queremos que el prover trabaje más que el verifier (asimetría favorable).

Griffin (2022)

Definición 1.49 (Griffin)**.** Griffin está optimizado para pruebas STARKs (que usan AIR en lugar de R1CS):

- Usa una combinación de operaciones lineales y no lineales
- Aprovecha que STARKs tienen estructura de constraints diferente
- Especialmente eficiente para campos pequeños (ej: $p = 2^{64} - 2^{32} + 1$)

Anemoi (2022)

Definición 1.50 (Anemoi)**.** Anemoi es el estado del arte en eficiencia:

- S-box: Combinación de $x \mapsto x^{1/5}$ y operaciones cuadráticas
- Estructura de "mariposa" (butterfly)
- Optimizado tanto para R1CS como para AIR

Costo: $\sim$120 constraints para seguridad de 128 bits.

Hash	Constraints	Rondas	Madurez
MiMC	$\sim$700	82	Alta (2016)
Rescue	$\sim$400	10	Media (2019)
Poseidon	$\sim$250	65	Alta (2019)
Griffin	$\sim$150	12	Baja (2022)
Anemoi	$\sim$120	10	Baja (2022)

Intuición

El campo de hashes ZK-friendly evoluciona rápidamente. La tendencia es hacia diseños más eficientes, pero con el trade-off de menor tiempo de análisis criptográfico. Poseidon representa un buen balance: significativamente mejor que MiMC, con 5+ años de escrutinio por la comunidad.

Implementación Práctica

Ejemplo 1.19 (Poseidon en Circom)**.** `pragma circom 2.0.0;`

```
include "poseidon.circom";

template HashExample() {
    signal input a;
    signal input b;
    signal output hash;
```

```
    component poseidon = Poseidon(2);  // 2 inputs
    poseidon.inputs[0] <== a;
    poseidon.inputs[1] <== b;
    hash <== poseidon.out;
}

component main = HashExample();
```

Este circuito produce ∼250 constraints y calcula $H(a, b)$.

Ejemplo 1.20 (Poseidon en Rust (Arkworks)).

```
use ark_crypto_primitives::sponge::poseidon::{
    PoseidonConfig, PoseidonSponge
};
use ark_crypto_primitives::sponge::CryptographicSponge;
use ark_bn254::Fr;  // Campo del BN254

fn hash_two_elements(a: Fr, b: Fr) -> Fr {
    let params = PoseidonConfig::new(...);  // Parámetros precomputados
    let mut sponge = PoseidonSponge::new(&params);

    sponge.absorb(&[a, b]);
    sponge.squeeze_field_elements(1)[0]
}
```

Parámetros para Diferentes Curvas

Curva/Campo	t	R_F	R_P	Uso
BN254	3	8	57	Ethereum, zkSync
BLS12-381	3	8	57	Filecoin, Zcash
Pallas	3	8	56	Zcash Orchard
Vesta	3	8	56	Zcash Orchard
Goldilocks ($p = 2^{64} - 2^{32} + 1$)	12	8	22	Plonky2

Observación 1.6. Cada instancia de Poseidon requiere parámetros específicos: constantes de ronda y matriz MDS. Estos se generan de forma determinista a partir de una semilla, permitiendo verificación independiente.

Aplicación en Banco ZK

En Banco ZK

Poseidon es el hash principal de Banco ZK. Lo usamos en:

1. **Árboles de Merkle del estado**
   - Profundidad: 20 niveles ($2^{20} \approx 1M$ notas)
   - Cada nivel: 1 hash Poseidon

- Costo total del path: $20 \times 250 = 5,000$ constraints
- Comparación con SHA-256: $20 \times 25,000 = 500,000$ constraints (100x más)

2. Compromisos a notas

$$\text{note_commitment} = \text{Poseidon}(\text{value}, \text{owner_pubkey}, \text{randomness}) \tag{1.50}$$

3. Derivación de nullifiers

$$\text{nullifier} = \text{Poseidon}(\text{note_commitment}, \text{spending_key}) \tag{1.51}$$

4. Por qué elegimos Poseidon sobre alternativas:

Criterio	**Poseidon**	**Anemoi**
Constraints	250	120
Años de análisis	5+	2
Librerías maduras	Muchas	Pocas
Auditorías	Múltiples	Limitadas
Decisión	✓Elegido	Para futuro

En Banco ZK priorizamos seguridad probada sobre eficiencia marginal. Cuando Anemoi tenga más escrutinio, consideraremos migrarlo.

5. Optimizaciones implementadas:

- Precomputación de constantes de ronda
- Matrices MDS optimizadas para multiplicación rápida
- Caché de estados intermedios para pruebas batch

Comparación Final: El Costo Real

Ejemplo 1.21 (Impacto en tiempo de prueba)**.** Para un circuito de transferencia privada típico:

Componente	**Con SHA-256**	**Con Poseidon**	**Ahorro**
Merkle path (20 niveles)	500,000	5,000	99%
2 nullifiers	50,000	500	99%
2 commitments	50,000	500	99%
Lógica adicional	10,000	10,000	0%
Total	610,000	16,000	97%

Tiempo de generación de prueba (estimado en laptop):

- Con SHA-256: ∼30 segundos

- Con Poseidon: $\sim$1 segundo

Esta diferencia de 30x hace que las pruebas sean prácticas para usuarios móviles.

1.3 Curvas Elipticas

1.3.1 Definición y Ley de Grupo

Definición 1.51 (Curva elíptica)**.** Sobre un campo K, una curva elíptica en forma de Weierstrass es:

$$E : y^2 = x^3 + ax + b \quad \text{con } 4a^3 + 27b^2 \neq 0$$

Los puntos de E forman un grupo abeliano con:

- Elemento neutro: punto en el infinito $\mathcal{O}$
- Suma geométrica: línea por P, Q intersecta en R, reflejar
- Inverso: $-P = (x, -y)$

Contexto Histórico

Intuición

Las curvas elípticas tienen una historia fascinante que abarca más de 2000 años. Originalmente estudiadas en el contexto de calcular la longitud del arco de una elipse (de ahí el nombre "elípticas"), hoy son fundamentales para la criptografía moderna.

Línea temporal:

Época	Desarrollo
Siglo III a.C.	Diofanto estudia ecuaciones similares
Siglo XVII	Fermat, Newton estudian curvas cúbicas
1901	Poincaré descubre la estructura de grupo
1985	Koblitz y Miller proponen ECC independientemente
2009	Bitcoin usa secp256k1
2016	Ethereum adopta la misma curva

Observación 1.7. Neal Koblitz y Victor Miller, trabajando independientemente, propusieron usar curvas elípticas para criptografía en 1985. Su insight fue que el problema del logaritmo discreto en curvas elípticas (ECDLP) era más difícil que en grupos multiplicativos, permitiendo claves más cortas con la misma seguridad.

La Ecuación de Weierstrass

Definición 1.52 (Forma de Weierstrass Generalizada)**.** La forma más general de una curva elíptica es:

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

Para campos de característica $\neq 2, 3$, esto se simplifica a la **forma corta de Weierstrass**:

$$E : y^2 = x^3 + ax + b$$

Matemáticas

La condición $\Delta = 4a^3 + 27b^2 \neq 0$ es el **discriminante**. Garantiza que la curva es "suave" (no tiene puntos singulares como cúspides o nodos).
Geométricamente:

- Si $\Delta = 0$: la curva tiene un punto donde se "cruza" consigo misma o tiene una "punta"
- Si $\Delta \neq 0$: la curva es suave, lo que permite definir tangentes en todos los puntos

Ejemplo 1.22 (Curvas válidas e inválidas)**. Curva válida:** $E : y^2 = x^3 - 3x + 3$

$$a = -3, \quad b = 3 \tag{1.52}$$

$$\Delta = 4(-3)^3 + 27(3)^2 = 4(-27) + 27(9) = -108 + 243 = 135 \neq 0 \quad \checkmark \tag{1.53}$$

Curva inválida (singular): $E : y^2 = x^3$

$$a = 0, \quad b = 0 \tag{1.54}$$

$$\Delta = 4(0)^3 + 27(0)^2 = 0 \quad \text{(tiene cúspide en el origen)} \tag{1.55}$$

Curva inválida (nodo): $E : y^2 = x^3 + x^2$

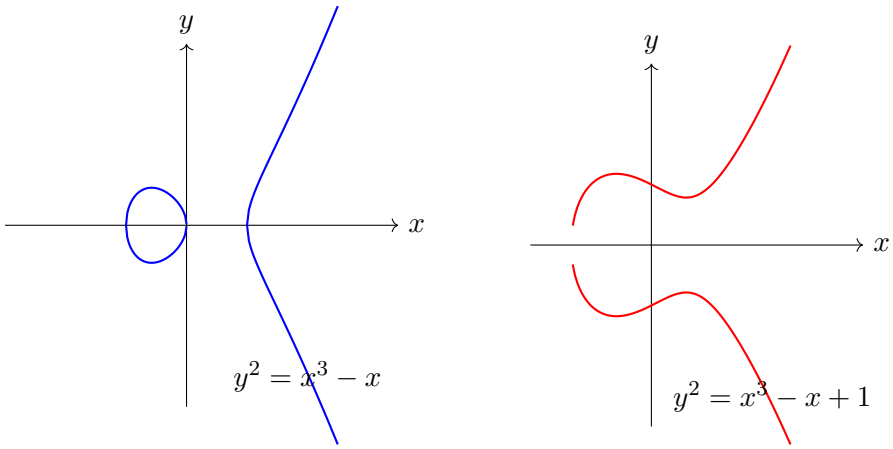
$$a = 0, \quad b = 0 \quad \text{(forma } y^2 = x^2(x+1)\text{)} \tag{1.56}$$

El origen es un punto donde la curva se cruza consigo misma.

Visualización Geométrica

Intuición

Sobre los números reales $\mathbb{R}$, una curva elíptica tiene una forma característica simétrica respecto al eje x. Dependiendo de los coeficientes, puede tener una o dos componentes conexas.



Observación 1.8. En criptografía **no** trabajamos sobre $\mathbb{R}$, sino sobre campos finitos $\mathbb{F}_p$. En estos campos, la "curva" es realmente un conjunto discreto de puntos que satisfacen la ecuación módulo p. La visualización geométrica sobre $\mathbb{R}$ es útil para intuición, pero los cálculos reales son en aritmética modular.

El Punto en el Infinito

Definición 1.53 (Punto en el Infinito)**.** El punto $\mathcal{O}$ (punto en el infinito) es un punto "especial" que no tiene coordenadas finitas. Se puede pensar como el punto donde todas las líneas verticales se encuentran.

Formalmente, $\mathcal{O}$ es el elemento neutro del grupo:

$$P + \mathcal{O} = \mathcal{O} + P = P \quad \text{para todo punto } P$$

Intuición

Imagina que extiendes el plano hacia arriba y hacia abajo al infinito. Todas las líneas verticales, que parecen paralelas, eventualmente se "encuentran" en un único punto en el infinito $\mathcal{O}$.
En coordenadas proyectivas, $\mathcal{O} = [0 : 1 : 0]$. En la práctica, lo representamos con un flag especial o coordenadas $(0, 0)$ con una marca.

La Ley de Grupo: Suma de Puntos

Idea Clave

Los puntos de una curva elíptica forman un **grupo abeliano** bajo una operación de "suma" definida geométricamente. Esta estructura algebraica es lo que hace a las curvas elípticas útiles para criptografía.

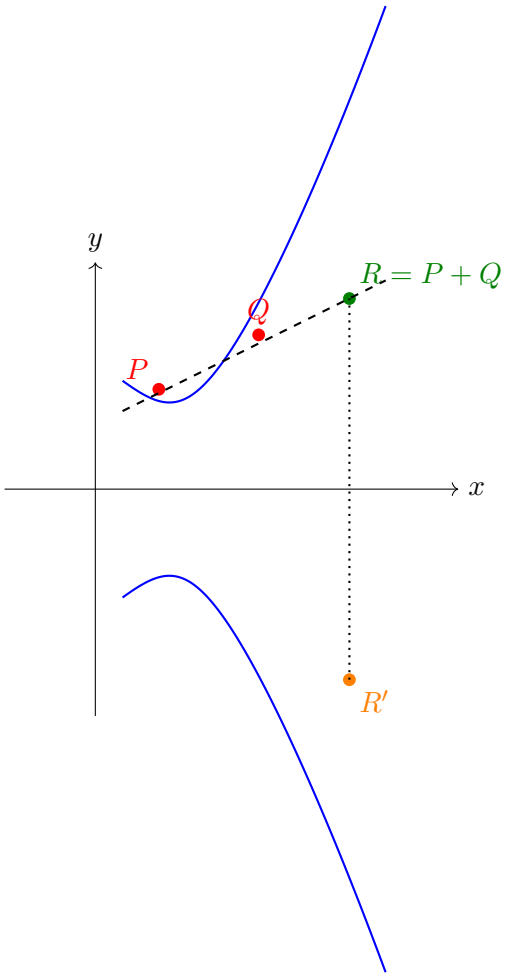
Definición 1.54 (Suma de Puntos - Definición Geométrica)**.** Dados dos puntos P y Q en la curva E:

1. Trazar la línea ℓ que pasa por P y Q
2. Esta línea intersecta la curva en un tercer punto R'

3. Reflejar R' sobre el eje x para obtener $R = P + Q$

Casos especiales:

- Si $P = Q$: usar la línea tangente a la curva en P
- Si $P = -Q$ (misma x, opuesta y): la línea es vertical, $P + Q = \mathcal{O}$



Fórmulas Algebraicas Explícitas

Definición 1.55 (Fórmulas de Suma)**.** Para $P = (x_1, y_1)$ y $Q = (x_2, y_2)$ en $E : y^2 = x^3 + ax + b$:

Caso 1: $P \neq Q$ (suma de puntos distintos)

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \tag{1.57}$$

$$x_3 = \lambda^2 - x_1 - x_2 \tag{1.58}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \tag{1.59}$$

Caso 2: $P = Q$ (duplicación)

$$\lambda = \frac{3x_1^2 + a}{2y_1} \tag{1.60}$$

$$x_3 = \lambda^2 - 2x_1 \tag{1.61}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \tag{1.62}$$

Casos especiales:

- Si $x_1 = x_2$ pero $y_1 = -y_2$: $P + Q = \mathcal{O}$
- Si $P = \mathcal{O}$: $P + Q = Q$
- Si $Q = \mathcal{O}$: $P + Q = P$
- Si $P = Q$ y $y_1 = 0$: $2P = \mathcal{O}$ (tangente vertical)

Matemáticas

El valor λ es la **pendiente** de la línea:

- En el caso $P \neq Q$: pendiente de la secante entre P y Q
- En el caso $P = Q$: pendiente de la tangente en P, obtenida por diferenciación implícita de $y^2 = x^3 + ax + b$:

$$2y \cdot \frac{dy}{dx} = 3x^2 + a \implies \frac{dy}{dx} = \frac{3x^2 + a}{2y}$$

Ejemplo Completo sobre Campo Finito

Ejemplo 1.23 (Suma de puntos en $E(\mathbb{F}_{17})$)**.** Consideremos la curva $E : y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$.

Paso 1: Verificar que la curva es válida

$$\Delta = 4(2)^3 + 27(2)^2 = 32 + 108 = 140 \equiv 4 \pmod{17} \neq 0 \quad \checkmark \tag{1.63}$$

Paso 2: Encontrar puntos en la curva

Probamos valores de x y verificamos si $x^3 + 2x + 2$ es un cuadrado en $\mathbb{F}_{17}$:

x	$x^3 + 2x + 2$	¿Cuadrado?	Puntos (x, y)
0	2	$2 \equiv 6^2 = 36 \equiv 2$? No	—
1	$1 + 2 + 2 = 5$	5: $6^2 = 2, 7^2 = 15$, no	—
2	$8 + 4 + 2 = 14$	14: no	—
3	$27 + 6 + 2 = 35 \equiv 1$	$1 = 1^2$	$(3, 1), (3, 16)$
5	$125 + 10 + 2 = 137 \equiv 1$	$1 = 1^2$	$(5, 1), (5, 16)$
6	$216 + 12 + 2 = 230 \equiv 9$	$9 = 3^2$	$(6, 3), (6, 14)$

Paso 3: Sumar $P = (3, 1)$ **y** $Q = (6, 3)$

$$\lambda = \frac{3-1}{6-3} = \frac{2}{3} \pmod{17} \tag{1.64}$$

Necesitamos $3^{-1} \mod 17$. Como $3 \cdot 6 = 18 \equiv 1 \pmod{17}$, tenemos $3^{-1} = 6$.

$$\lambda = 2 \cdot 6 = 12 \pmod{17} \tag{1.65}$$

Ahora calculamos las coordenadas:

$$x_3 = \lambda^2 - x_1 - x_2 = 12^2 - 3 - 6 = 144 - 9 = 135 \equiv 16 \pmod{17} \tag{1.66}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 12(3 - 16) - 1 = 12(-13) - 1 \tag{1.67}$$

$$= 12 \cdot 4 - 1 = 48 - 1 = 47 \equiv 13 \pmod{17} \tag{1.68}$$

Resultado: $P + Q = (3, 1) + (6, 3) = (16, 13)$

Verificación: ¿Está $(16, 13)$ en la curva?

$$y^2 = 13^2 = 169 \equiv 16 \pmod{17} \tag{1.69}$$

$$x^3 + 2x + 2 = 16^3 + 2(16) + 2 = 4096 + 32 + 2 = 4130 \equiv 16 \pmod{17} \quad \checkmark \tag{1.70}$$

Duplicación de Puntos

Ejemplo 1.24 (Duplicar $P = (3, 1)$ en $E : y^2 = x^3 + 2x + 2$ sobre $\mathbb{F}_{17}$)**.** Usamos la fórmula de duplicación con $a = 2$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{3(3)^2 + 2}{2(1)} = \frac{27 + 2}{2} = \frac{29}{2} \equiv \frac{12}{2} \pmod{17} \tag{1.71}$$

Necesitamos $2^{-1} \mod 17$. Como $2 \cdot 9 = 18 \equiv 1$, tenemos $2^{-1} = 9$.

$$\lambda = 12 \cdot 9 = 108 \equiv 6 \pmod{17} \tag{1.72}$$

Calculamos:

$$x_3 = \lambda^2 - 2x_1 = 6^2 - 2(3) = 36 - 6 = 30 \equiv 13 \pmod{17} \tag{1.73}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 6(3 - 13) - 1 = 6(-10) - 1 = -61 \equiv 7 \pmod{17} \tag{1.74}$$

Resultado: $2P = (13, 7)$

Verificación:

$$y^2 = 7^2 = 49 \equiv 15 \pmod{17} \tag{1.75}$$

$$x^3 + 2x + 2 = 13^3 + 26 + 2 = 2197 + 28 = 2225 \equiv 15 \pmod{17} \quad \checkmark \tag{1.76}$$

Propiedades del Grupo

Idea Clave

Las curvas elípticas forman un grupo abeliano. Esto significa que la operación de suma satisface:

1. **Clausura**: Si $P, Q \in E$, entonces $P + Q \in E$
2. **Asociatividad**: $(P + Q) + R = P + (Q + R)$
3. **Elemento neutro**: Existe $\mathcal{O}$ tal que $P + \mathcal{O} = P$
4. **Inversos**: Para cada P, existe $-P$ tal que $P + (-P) = \mathcal{O}$
5. **Conmutatividad**: $P + Q = Q + P$

Definición 1.56 (Inverso de un Punto)**.** Para $P = (x, y)$, su inverso es:

$$-P = (x, -y)$$

En un campo finito $\mathbb{F}_p$, esto es:

$$-P = (x, p - y)$$

Ejemplo 1.25 (Inversos en $\mathbb{F}_{17}$)**.** Si $P = (3, 1)$, entonces:

$$-P = (3, 17 - 1) = (3, 16)$$

Verificación: $P + (-P) = (3, 1) + (3, 16)$
Como $x_1 = x_2 = 3$ pero $y_1 = 1 \neq -y_2 = -16 \equiv 1$...
Espera, $y_2 = 16 \equiv -1 \pmod{17}$, y $-y_2 = -16 \equiv 1 = y_1$. No, $y_1 \neq -y_2$ en este caso...
Realmente: $y_1 = 1$ y $-y_2 = -16 \equiv 1 \pmod{17}$, entonces $y_1 = -y_2$
Corrección: $-y_2 = 17 - 16 = 1 = y_1$, así que... los puntos SON opuestos cuando $y_2 = p - y_1$.
Por lo tanto: $(3, 1) + (3, 16) = \mathcal{O}$

Multiplicación Escalar

Definición 1.57 (Multiplicación Escalar)**.** Para un punto P y un entero n, definimos:

$$nP = \underbrace{P + P + \cdots + P}_{n \text{ veces}}$$

También:

- $0 \cdot P = \mathcal{O}$
- $(-n) \cdot P = n \cdot (-P)$

Idea Clave

La multiplicación escalar es la operación fundamental en criptografía de curvas elípticas:

- **Fácil**: Dado n y P, calcular $Q = nP$ (usando double-and-add)
- **Difícil**: Dado P y Q, encontrar n tal que $Q = nP$ (ECDLP)

Esta asimetría es la base de la seguridad de ECC.

Definición 1.58 (Algoritmo Double-and-Add)**.** Para calcular nP eficientemente:

```
function scalar_mult(n, P):
    if n == 0: return O
    if n < 0: return scalar_mult(-n, -P)

    R = O
    Q = P
    while n > 0:
        if n es impar:
            R = R + Q
        Q = 2*Q  # Duplicación
        n = n // 2
    return R
```

Complejidad: $O(\log n)$ operaciones de grupo.

Ejemplo 1.26 (Calcular $7P$ usando double-and-add)**.** $7 = 111_2$ (en binario)

$$7P = 1 \cdot P + 1 \cdot 2P + 1 \cdot 4P \tag{1.77}$$
$$= P + 2P + 4P \tag{1.78}$$

Pasos:

1. $2P$ = duplicar P
2. $4P = 2(2P)$ = duplicar $2P$
3. $P + 2P = 3P$
4. $3P + 4P = 7P$

Total: 2 duplicaciones + 2 sumas = 4 operaciones (vs. 6 sumas ingenuas)

El Orden del Grupo

Definición 1.59 (Orden del Grupo)**.** El **orden** de $E(\mathbb{F}_p)$, denotado $\#E(\mathbb{F}_p)$ o $|E(\mathbb{F}_p)|$, es el número total de puntos en la curva, incluyendo $\mathcal{O}$.

Teorema 1.7 (Teorema de Hasse)**.** *Para una curva elíptica E sobre $\mathbb{F}_p$:*

$$|p + 1 - \#E(\mathbb{F}_p)| \leq 2\sqrt{p}$$

Equivalentemente:

$$p + 1 - 2\sqrt{p} \leq \#E(\mathbb{F}_p) \leq p + 1 + 2\sqrt{p}$$

Intuición

El teorema de Hasse dice que el número de puntos está "cerca" de $p+1$. Para p grande, el número de puntos es aproximadamente p, con una desviación de como mucho $2\sqrt{p}$. Por ejemplo, para $p \approx 2^{256}$:

- $\#E \approx 2^{256} \pm 2^{129}$
- El "error" es insignificante en términos relativos

Definición 1.60 (Orden de un Punto)**.** El **orden** de un punto P, denotado $\text{ord}(P)$, es el menor entero positivo n tal que:

$$nP = \mathcal{O}$$

Por el teorema de Lagrange, $\text{ord}(P)$ divide a $\#E(\mathbb{F}_p)$.

Curvas Usadas en Criptografía

Curva	Ecuación	Campo	Uso
secp256k1	$y^2 = x^3 + 7$	$\mathbb{F}_p$, $p = 2^{256} - 2^{32} - 977$	Bitcoin, Ethereum
P-256	$y^2 = x^3 - 3x + b$	$\mathbb{F}_p$, $p = 2^{256} - 2^{224} + \ldots$	TLS, NIST
Curve25519	$y^2 = x^3 + 486662x^2 + x$	$\mathbb{F}_p$, $p = 2^{255} - 19$	Signal, SSH
BN254	Barreto-Naehrig	$\mathbb{F}_p$, $p \approx 2^{254}$	SNARKs
BLS12-381	Barreto-Lynn-Scott	$\mathbb{F}_p$, $p \approx 2^{381}$	Ethereum 2.0

Observación 1.9. **secp256k1** (usada en Bitcoin/Ethereum) tiene la forma simple $y^2 = x^3 + 7$ porque:

- $a = 0$ simplifica las fórmulas (menos multiplicaciones)
- $b = 7$ es el menor valor que da una curva segura con $a = 0$
- Los parámetros son "nothing-up-my-sleeve": claramente no hay backdoor

Representación de Puntos

Definición 1.61 (Coordenadas Afines vs Proyectivas)**. Coordenadas afines:** (x, y) donde $y^2 = x^3 + ax + b$

Coordenadas proyectivas: $(X : Y : Z)$ donde $Y^2Z = X^3 + aXZ^2 + bZ^3$

Relación: $(x, y) \leftrightarrow (x : y : 1)$ y $(X : Y : Z) \leftrightarrow (X/Z, Y/Z)$ si $Z \neq 0$

El punto en el infinito: $\mathcal{O} = (0 : 1 : 0)$

Idea Clave

Las coordenadas proyectivas evitan divisiones (inversiones modulares), que son costosas. En su lugar, se trabaja con multiplicaciones y se hace una sola inversión al final para convertir a coordenadas afines.
En implementaciones optimizadas, esto puede dar speedups de 10x o más.

Definición 1.62 (Coordenadas Jacobianas)**.** Una variante popular: $(X : Y : Z)$ donde $(x, y) = (X/Z^2, Y/Z^3)$

La ecuación de la curva se transforma en:

$$Y^2 = X^3 + aXZ^4 + bZ^6$$

Las fórmulas de duplicación son especialmente eficientes en esta representación.

Compresión de Puntos

Definición 1.63 (Compresión de Puntos)**.** Un punto (x, y) puede representarse de forma comprimida como $(x, \text{signo}(y))$, donde el signo indica cuál de las dos raíces cuadradas elegir.

- **Sin comprimir**: $04 \,||\, x \,||\, y$ (65 bytes para curvas de 256 bits)
- **Comprimido**: $02/03 \,||\, x$ (33 bytes)
  - 02 si y es par
  - 03 si y es impar

Ejemplo 1.27 (Compresión en secp256k1)**.** Clave pública sin comprimir (65 bytes):

```
04 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B
   16F81798 483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A6855419
   9C47D08FFB10D4B8
```

Clave pública comprimida (33 bytes):

```
02 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B
   16F81798
```

El 02 indica que y es par. Para descomprimir, se calcula $y = \sqrt{x^3 + 7}$ y se elige la raíz par.

Aplicación en Banco ZK

En Banco ZK

Las curvas elípticas son fundamentales en Banco ZK:

1. **Firmas digitales (secp256k1)**
   - Compatible con Ethereum para verificación de transacciones
   - Usuarios firman con sus claves existentes de Ethereum

2. Curvas para SNARKs (BN254, BLS12-381)

- Verificación de pruebas ZK requiere operaciones de pairing
- BN254: 190 bits de seguridad, muy eficiente
- BLS12-381: 128 bits de seguridad, usado en Ethereum 2.0

3. Compromisos de Pedersen

$$C = v \cdot G + r \cdot H$$

donde G y H son generadores de la curva. Usamos esto para ocultar valores de transacciones.

4. Ciclos de curvas (Pallas/Vesta)

Para pruebas recursivas eficientes, necesitamos "ciclos" de curvas donde el orden de una curva es el tamaño del campo de la otra:

- Pallas: $E_1(\mathbb{F}_p)$ con $\#E_1 = q$
- Vesta: $E_2(\mathbb{F}_q)$ con $\#E_2 = p$

Esto permite verificar una prueba sobre E_1 dentro de un circuito sobre E_2 eficientemente.

Decisión de diseño:

Componente	Curva	Razón
Firmas de usuario	secp256k1	Compatibilidad Ethereum
Verificación SNARK	BN254	Eficiencia, soporte EVM
Pruebas recursivas	Pallas/Vesta	Ciclo nativo
Compromisos	Misma curva del SNARK	Eficiencia

1.3.2 secp256k1

Curva usada en Bitcoin y Ethereum.

- Ecuación: $y^2 = x^3 + 7$ sobre $\mathbb{F}_p$
- $p = 2^{256} - 2^{32} - 977$
- Orden: $n \approx 2^{256}$
- Curva de Koblitz: parámetros especiales para eficiencia

Origen y Nombre

Intuición

El nombre "secp256k1" sigue la convención de SECG (Standards for Efficient Cryptography Group):

- **sec**: Standards for Efficient Cryptography
- **p**: Campo primo (vs. "2" para campos binarios)
- **256**: Tamaño del campo en bits
- **k**: Curva de Koblitz (parámetros especiales)
- **1**: Primera curva de este tipo en el estándar

Observación 1.10. Antes de Bitcoin, secp256k1 era relativamente poco usada. La mayoría de aplicaciones preferían secp256r1 (P-256), recomendada por NIST. Satoshi Nakamoto eligió secp256k1 en 2008, posiblemente porque:

1. Sus parámetros son "nothing-up-my-sleeve" (no hay sospecha de backdoors)
2. La forma $y^2 = x^3 + 7$ (con $a = 0$) permite optimizaciones
3. No fue generada por NIST/NSA (después de las revelaciones de Snowden, esto se valoró más)

Parámetros Completos

Definición 1.64 (Parámetros de secp256k1)**.** La curva secp256k1 está definida por:

Ecuación:

$$E : y^2 = x^3 + 7$$

Campo base $\mathbb{F}_p$ con:

$$\begin{aligned} p &= 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1 && (1.79)\\ &= 2^{256} - 2^{32} - 977 && (1.80)\\ &= \texttt{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF} && (1.81)\\ &\quad \texttt{FFFFFFFF FFFFFFFF FFFFFFFE FFFFFC2F} && (1.82) \end{aligned}$$

Orden del grupo (número de puntos):

$$\begin{aligned} n &= \texttt{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE} && (1.83)\\ &\quad \texttt{BAAEDCE6 AF48A03B BFD25E8C D0364141} && (1.84) \end{aligned}$$

Cofactor:

$$h = 1$$

Generador $G = (G_x, G_y)$:

$$G_x = \texttt{79BE667E F9DCBBAC 55A06295 CE870B07} \tag{1.85}$$

$$\texttt{029BFCDB 2DCE28D9 59F2815B 16F81798} \tag{1.86}$$

$$G_y = \texttt{483ADA77 26A3C465 5DA4FBFC 0E1108A8} \tag{1.87}$$

$$\texttt{FD17B448 A6855419 9C47D08F FB10D4B8} \tag{1.88}$$

Matemáticas

Verificación del discriminante:

$$a = 0, \quad b = 7 \tag{1.89}$$

$$\Delta = 4a^3 + 27b^2 = 0 + 27 \cdot 49 = 1323 \neq 0 \quad \checkmark \tag{1.90}$$

La curva es no singular, como se requiere.

¿Por Qué Esta Forma Especial?

Idea Clave

La ecuación $y^2 = x^3 + 7$ tiene $a = 0$, lo que elimina el término ax y simplifica significativamente las operaciones:
Duplicación estándar (con a genérico):

$$\lambda = \frac{3x_1^2 + a}{2y_1}$$

Duplicación en secp256k1 ($a = 0$):

$$\lambda = \frac{3x_1^2}{2y_1}$$

Ahorro: una multiplicación por constante menos por cada duplicación. En millones de operaciones criptográficas, esto suma.

Matemáticas

Las fórmulas optimizadas para secp256k1 en coordenadas Jacobianas (X, Y, Z) donde $(x, y) = (X/Z^2, Y/Z^3)$:
Duplicación de $P = (X_1, Y_1, Z_1)$:

$$S = 4X_1Y_1^2 \tag{1.91}$$

$$M = 3X_1^2 \quad \text{(no hay término } aZ_1^4 \text{ porque } a = 0) \tag{1.92}$$

$$X_3 = M^2 - 2S \tag{1.93}$$

$$Y_3 = M(S - X_3) - 8Y_1^4 \tag{1.94}$$

$$Z_3 = 2Y_1Z_1 \tag{1.95}$$

Costo: 1S + 8M (1 cuadrado, 8 multiplicaciones), vs. 1S + 10M con $a \neq 0$.

El Primo p: Estructura Especial

Definición 1.65 (Primo de Mersenne Generalizado)**.** El primo de secp256k1:

$$p = 2^{256} - 2^{32} - 977$$

tiene una estructura que permite **reducción rápida** módulo p.

Intuición

La reducción modular normalmente requiere división, que es costosa. Pero con primos de forma especial, podemos usar trucos algebraicos.
Para $p = 2^{256} - c$ donde c es pequeño:

$$x \mod p = x \mod (2^{256} - c)$$

Si $x = x_{\text{high}} \cdot 2^{256} + x_{\text{low}}$, entonces:

$$x \equiv x_{\text{low}} + c \cdot x_{\text{high}} \pmod{p} \tag{1.96}$$

Como $c = 2^{32} + 977$ es pequeño (33 bits), la multiplicación $c \cdot x_{\text{high}}$ es mucho más barata que una división completa.

Ejemplo 1.28 (Reducción rápida)**.** Supongamos que tenemos un producto de 512 bits (resultado de multiplicar dos números de 256 bits):

$$x = x_1 \cdot 2^{256} + x_0$$

donde $x_1, x_0 < 2^{256}$.
Paso 1:

$$x \equiv x_0 + (2^{32} + 977) \cdot x_1 \pmod{p}$$

Paso 2: El resultado tiene como mucho $256 + 33 = 289$ bits. Repetimos si es necesario.
Paso 3: Después de 2-3 iteraciones, el resultado está en $[0, p)$.
Esto es ∼10x más rápido que división general.

El Orden n y Seguridad

Definición 1.66 (Orden del Grupo)**.**

$$n = \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141}$$

En decimal:

$$n = 115792089237316195423570985008687907852837564279074904382605163141518161494337$$

Esto es aproximadamente $2^{256} - 4.32 \times 10^{38}$.

Idea Clave

El orden n es **primo**. Esto es crucial porque:

1. El grupo generado por G es cíclico de orden n
2. Todo punto $\neq \mathcal{O}$ tiene orden n (no hay subgrupos pequeños)
3. El cofactor es $h = \#E(\mathbb{F}_p)/n = 1$

Si hubiera subgrupos pequeños, ciertos ataques (como el ataque de Pohlig-Hellman) serían posibles.

Matemáticas

Nivel de seguridad:
El mejor ataque conocido contra ECDLP es el algoritmo rho de Pollard, que requiere $O(\sqrt{n})$ operaciones:

$$\sqrt{n} \approx \sqrt{2^{256}} = 2^{128}$$

Por lo tanto, secp256k1 ofrece **128 bits de seguridad**, equivalente a:

- AES-128
- RSA de ∼3072 bits
- Hash de 256 bits (para colisiones)

El Generador G

Definición 1.67 (Punto Generador)**.** El generador G es un punto específico de la curva tal que:

$$\{G, 2G, 3G, \ldots, nG\} = E(\mathbb{F}_p) \setminus \{\mathcal{O}\}$$

Es decir, G genera todo el grupo (excepto el punto en el infinito).

Observación 1.11. ¿Cómo se eligió G? El estándar SECG especifica que G se obtiene hasheando una semilla conocida. Esto es un procedimiento "nothing-up-my-sleeve": cualquiera puede verificar que G no fue elegido maliciosamente.

Para secp256k1, el proceso fue:

1. Elegir la coordenada x como el menor entero tal que $x^3 + 7$ es un cuadrado en $\mathbb{F}_p$
2. La coordenada y es la raíz cuadrada con cierta paridad

Ejemplo 1.29 (Verificar que G está en la curva)**.**

$$G_x = \texttt{0x79BE667E...16F81798} \tag{1.97}$$

$$G_y = \texttt{0x483ADA77...FB10D4B8} \tag{1.98}$$

Verificamos: $G_y^2 \stackrel{?}{=} G_x^3 + 7 \pmod{p}$

```
>>> Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
>>> Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
>>> p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
>>> (Gy**2 - Gx**3 - 7) % p
0
```

G está en la curva

Claves Públicas y Privadas

Definición 1.68 (Par de Claves en secp256k1).

- **Clave privada**: Un entero aleatorio $d \in [1, n-1]$
- **Clave pública**: El punto $Q = dG$

La seguridad se basa en que conocer Q y G no permite recuperar d (problema ECDLP).

Ejemplo 1.30 (Generación de claves en Bitcoin/Ethereum). **Paso 1:** Generar 256 bits aleatorios seguros

```
private_key = 0xE8F32E723DECF4051AEFAC8E2C93C9C5B214313817CDB01A1494B917C8436B35
```

Paso 2: Verificar que está en rango válido

```
assert 1 <= private_key < n
```

Paso 3: Calcular clave pública

```
public_key = private_key * G  # Multiplicación escalar
# Resultado: (Qx, Qy)
```

Paso 4: Serializar

```
# Sin comprimir (65 bytes): 04 || Qx || Qy
# Comprimido (33 bytes): 02/03 || Qx (según paridad de Qy)
```

Advertencia

Generación de claves privadas:
La clave privada DEBE ser:

- Generada con un CSPRNG (generador criptográficamente seguro)
- Verificada que esté en $[1, n-1]$
- Nunca reutilizada entre sistemas diferentes
- Almacenada de forma segura

Fallos históricos:

- Android (2013): Bug en SecureRandom causó reutilización de nonces, perdiendo fondos
- Blockchain.info (2014): Generador débil predecible
- Numerosas wallets con entropía insuficiente

Direcciones de Bitcoin

Definición 1.69 (Derivación de Dirección Bitcoin)**.** Desde la clave pública, se deriva la dirección así:

1. pubkey_hash = RIPEMD160(SHA256(pubkey)) (20 bytes)
2. versioned = version_byte $||$ pubkey_hash (21 bytes)
3. checksum = SHA256(SHA256(versioned))$[0:4]$ (4 bytes)
4. address = Base58(versioned $||$ checksum)

Version bytes:

- `0x00`: Mainnet P2PKH (empieza con "1")
- `0x05`: Mainnet P2SH (empieza con "3")
- `0x6F`: Testnet P2PKH (empieza con "m" o "n")

Ejemplo 1.31 (Derivación completa)**.**

```
1. Clave pública (comprimida):
   02 50863AD64A87AE8A2FE83C1AF1A8403CB53F53E486D8511DAD8A04887E5B2352

2. SHA256:
   0B7C28C9B7290C98D7438E70B3D3F7C848FBD7D1DC194FF83F4F7CC9B1378E98

3. RIPEMD160:
   F54A5851E9372B87810A8E60CDD2E7CFD80B6E31

4. Añadir version byte (0x00 para mainnet):
   00 F54A5851E9372B87810A8E60CDD2E7CFD80B6E31

5. Doble SHA256 y tomar primeros 4 bytes (checksum):
   C7F18FE8

6. Concatenar:
   00 F54A5851E9372B87810A8E60CDD2E7CFD80B6E31 C7F18FE8

7. Base58 encode:
   1PMycacnJaSqwwJqjawXBErnLsZ7RkXUAs
```

Direcciones de Ethereum

Definición 1.70 (Derivación de Dirección Ethereum)**.** Ethereum usa un proceso más simple:

1. Clave pública **sin comprimir, sin prefijo** (64 bytes, solo $x \,||\, y$)
2. hash = Keccak256(pubkey) (32 bytes)
3. address = hash[12 : 32] (últimos 20 bytes)
4. Representación: "0x" + hexadecimal (42 caracteres)

Ejemplo 1.32 (Dirección Ethereum)**.** `1. Clave pública (64 bytes, sin 04):`
`50863AD64A87AE8A2FE83C1AF1A8403CB53F53E486D8511DAD8A04887E5B2352`
`2CD470243453A299FA9E77237716103ABC11A1DF38855ED6F2EE187E9C582BA6`

`2. Keccak256:`
`FC12AD814631BA689F7ADE2B6F9F4D17C8B6B6B4BF1D7E1E0C1C5D8F9A0B1C2D...`

`3. Últimos 20 bytes:`
`C8B6B6B4BF1D7E1E0C1C5D8F9A0B1C2D3E4F5A6B`

`4. Dirección:`
`0xC8B6B6B4BF1D7E1E0C1C5D8F9A0B1C2D3E4F5A6B`

Observación 1.12. Ethereum añade un checksum opcional mediante capitalización mixta (EIP-55):

```
Sin checksum: 0xc8b6b6b4bf1d7e1e0c1c5d8f9a0b1c2d3e4f5a6b
Con checksum: 0xC8b6B6b4bF1D7e1E0c1c5d8F9A0b1c2D3e4F5A6B
```

Las letras se capitalizan según los bits del hash de la dirección.

Comparación con Otras Curvas

Curva	a	b	Bits	Origen
secp256k1	0	7	256	SECG
secp256r1 (P-256)	-3	variable	256	NIST
Curve25519	–	–	255	Bernstein
BN254	0	3	254	Barreto-Naehrig
BLS12-381	0	4	381	BLS

Idea Clave

¿Por qué no P-256 (secp256r1)?
P-256 es más popular en TLS, certificados, etc. Pero:

1. Sus parámetros fueron generados por NIST/NSA con proceso opaco

2. El coeficiente b parece aleatorio, sin justificación clara

3. Después de Snowden, hay sospechas de posibles "backdoors"

secp256k1 tiene parámetros transparentes: $a = 0$, $b = 7$ (el menor b que da curva segura con $a = 0$).

Aspecto	**secp256k1**	**P-256**
Transparencia	Alta ("nothing-up-my-sleeve")	Baja (proceso NIST opaco)
Velocidad ($a = 0$)	Más rápida	Estándar
Adopción cripto	Bitcoin, Ethereum	Rara
Adopción TLS/PKI	Rara	Dominante
Librerías	libsecp256k1 (muy optimizada)	Muchas

Vulnerabilidades y Consideraciones

Advertencia

Curvas potencialmente vulnerables:
secp256k1 es segura contra ataques conocidos, pero algunas clases de curvas son vulnerables:

- **Curvas anómalas**: $\#E(\mathbb{F}_p) = p \to$ ECDLP se reduce a DLP en $\mathbb{F}_p$ (resuelto en tiempo lineal)
- **Curvas supersingulares**: Vulnerable al ataque MOV (embedding en campo de extensión)
- **Embedding degree bajo**: Similar al ataque MOV

secp256k1 no tiene ninguna de estas propiedades.

Observación 1.13. **Computación cuántica:**

El algoritmo de Shor puede resolver ECDLP en tiempo polinomial con una computadora cuántica suficientemente grande. Para secp256k1:

- Se necesitan ∼2500 qubits lógicos
- Con corrección de errores: millones de qubits físicos
- Estimación optimista: 10-20 años para amenaza real

Por esto, se investigan alternativas "post-cuánticas" (lattices, hashes, códigos).

Implementación: libsecp256k1

Intuición

La biblioteca `libsecp256k1`, desarrollada por Pieter Wuille para Bitcoin Core, es considerada la implementación de referencia. Sus características:

- Escrita en C puro, sin dependencias
- Optimizaciones de ensamblador para x86_64, ARM
- Tiempo constante (resistente a side-channels)
- Verificación formal parcial
- Usada por Bitcoin, Ethereum, y cientos de proyectos

Ejemplo 1.33 (Uso de libsecp256k1 en C)**.** `#include <secp256k1.h>`

```
// Crear contexto
secp256k1_context* ctx = secp256k1_context_create(
    SECP256K1_CONTEXT_SIGN | SECP256K1_CONTEXT_VERIFY);

// Generar par de claves
unsigned char seckey[32];
// ... llenar seckey con 32 bytes aleatorios seguros ...

secp256k1_pubkey pubkey;
int result = secp256k1_ec_pubkey_create(ctx, &pubkey, seckey);

// Serializar clave pública (comprimida)
unsigned char pubkey_ser[33];
size_t len = 33;
secp256k1_ec_pubkey_serialize(ctx, pubkey_ser, &len, &pubkey,
    SECP256K1_EC_COMPRESSED);

// Firmar mensaje
unsigned char msg_hash[32] = /* SHA256 del mensaje */;
secp256k1_ecdsa_signature sig;
secp256k1_ecdsa_sign(ctx, &sig, msg_hash, seckey, NULL, NULL);

// Verificar firma
int valid = secp256k1_ecdsa_verify(ctx, &sig, msg_hash, &pubkey);

secp256k1_context_destroy(ctx);
```

Optimizaciones Específicas

Definición 1.71 (Endomorfismo GLV)**.** secp256k1 admite el endomorfismo eficiente:

$$\phi(x, y) = (\beta x, y)$$

donde β es una raíz cúbica de la unidad en $\mathbb{F}_p$ (existe porque $p \equiv 1 \pmod 3$).

Este endomorfismo satisface $\phi^2 + \phi + 1 = 0$ en el anillo de endomorfismos, correspondiente a un λ tal que:

$$\phi(P) = \lambda P$$

Idea Clave

Speedup GLV:

Para calcular nP, descomponemos:

$$n = n_1 + n_2\lambda$$

donde $n_1, n_2 \approx \sqrt{n}$ (128 bits cada uno).

Entonces:

$$nP = n_1P + n_2\phi(P) = n_1P + n_2(\beta x_P, y_P)$$

Usando double-and-add simultáneo sobre ambos sumandos, reducimos el número de duplicaciones a la mitad. Speedup: $\sim$33%.

Matemáticas

Los valores concretos para secp256k1:

$$\beta = \texttt{0x7AE96A2B657C07106E64479EAC3434E99CF0497512F58995C1396C28719501EE} \tag{1.99}$$

$$\lambda = \texttt{0x5363AD4CC05C30E0A5261C028812645A122E22EA20816678DF02967C1B23BD72} \tag{1.100}$$

Verificación: $\beta^3 = 1 \pmod p$ y $\lambda^2 + \lambda + 1 = 0 \pmod n$.

Aplicación en Banco ZK

En Banco ZK

secp256k1 es fundamental para la interoperabilidad de Banco ZK con Ethereum:

1. **Autenticación de usuarios**
   - Los usuarios conectan sus wallets Ethereum existentes
   - Firman transacciones de Banco ZK con sus claves secp256k1
   - No necesitan crear nuevas claves ni aprender nuevos procesos
2. **Verificación de firmas en circuitos ZK**

Verificar una firma ECDSA dentro de un circuito ZK es costoso pero necesario:

Operación	Constraints
Multiplicación escalar en secp256k1	∼500,000
Verificación ECDSA completa	∼600,000

3. Estrategia de optimización

En lugar de verificar firmas secp256k1 dentro del SNARK, usamos:

1. Verificar firma en el contrato L1 (barato en gas: ∼3000 gas)
2. Pasar la clave pública como input público al circuito
3. El circuito verifica que el nullifier corresponde a esa clave

Esto reduce de 600,000 a ∼1,000 constraints.

4. Bridge de fondos

- Depósitos: Usuario firma con secp256k1 para autorizar
- Retiros: El contrato verifica firma antes de liberar fondos
- La misma clave protege fondos en Ethereum y en Banco ZK

5. Compatibilidad futura

Consideramos migrar a EdDSA (ed25519) para nuevas funcionalidades porque:

- Firmas determinísticas (no hay riesgo de nonce)
- Más eficiente en circuitos ZK
- Batch verification nativa

Pero mantenemos secp256k1 para compatibilidad con el ecosistema Ethereum existente.

Constantes Útiles

Matemáticas

Resumen de constantes secp256k1:

```
# Campo base
p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F

# Orden del grupo
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141

# Coeficientes de la curva
a = 0
```

```
b = 7

# Generador G
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8

# Para endomorfismo GLV
beta = 0x7AE96A2B657C07106E64479EAC3434E99CF0497512F58995C1396C28719501EE
lambda = 0x5363AD4CC05C30E0A5261C028812645A122E22EA20816678DF02967C1B23BD72

# Cofactor
h = 1
```

1.3.3 BN254 (alt_bn128)

Curva pairing-friendly usada en ZK-SNARKs de Ethereum.

- Embedding degree: 12
- Seguridad: ∼100 bits (suficiente para SNARKs actuales)
- Precompilados en EVM: EIP-196, EIP-197
- Usada en: Groth16, PLONK en Ethereum

¿Qué es una Curva Pairing-Friendly?

Intuición

Las curvas "normales" como secp256k1 permiten operaciones de grupo (suma, multiplicación escalar), pero nada más. Las curvas **pairing-friendly** tienen una propiedad mágica adicional: permiten "multiplicar" puntos de curvas elípticas de una forma que no es posible en curvas ordinarias.
Esta operación extra, llamada **pairing** o emparejamiento bilineal, es lo que hace posible los SNARKs eficientes.

Definición 1.72 (Pairing Bilineal)**.** Un pairing es una función:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

donde $\mathbb{G}_1, \mathbb{G}_2$ son grupos de puntos de curva elíptica y $\mathbb{G}_T$ es un grupo multiplicativo (subgrupo del campo de extensión), que satisface:

Bilinealidad:

$$e(aP, bQ) = e(P, Q)^{ab}$$

para todos los puntos $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$ y escalares a, b.

Idea Clave

La bilinealidad permite "mover" escalares entre argumentos:

$$e(aP, Q) = e(P, Q)^a = e(P, aQ) \tag{1.101}$$

Esto es imposible en curvas normales y es la base de:

- Verificación de SNARKs en tiempo constante
- Firmas BLS agregables
- Identity-Based Encryption (IBE)
- Attribute-Based Encryption (ABE)

Ejemplo 1.34 (Verificación de Groth16)**.** En Groth16, la prueba consiste en tres puntos $\pi = (A, B, C)$ donde $A, C \in \mathbb{G}_1$ y $B \in \mathbb{G}_2$.

La verificación comprueba:

$$e(A, B) = e(\alpha, \beta) \cdot e(\sum_i x_i L_i, \gamma) \cdot e(C, \delta)$$

Sin pairings, esta ecuación no tendría sentido. Con pairings, verificar una prueba de cualquier computación compleja requiere solo 3-4 pairings, independientemente de la complejidad del circuito.

Historia de BN254

Observación 1.14. BN254 pertenece a la familia de curvas Barreto-Naehrig (BN), propuesta por Paulo Barreto y Michael Naehrig en 2005. Estas curvas se caracterizan por:

- Construcción paramétrica simple
- Embedding degree $k = 12$
- Eficiencia en implementación

BN254 específicamente fue popularizada por la biblioteca `bn128` y adoptada por Ethereum en 2017 mediante los EIPs 196 y 197 para habilitar SNARKs on-chain.

Intuición

El nombre "alt_bn128" que aparece en documentación de Ethereum es un nombre alternativo histórico. El "128" se refería originalmente a la seguridad estimada en bits, pero análisis posteriores mostraron que la seguridad real es ∼100 bits debido a avances en ataques a pairings (NFS modificado).

Parámetros de BN254

Definición 1.73 (Construcción Barreto-Naehrig)**.** Las curvas BN se parametrizan por un entero u. Para BN254:

$$u = \texttt{0x44E992B44A6909F1}$$

A partir de u se derivan todos los parámetros:

$$p = 36u^4 + 36u^3 + 24u^2 + 6u + 1 \quad \text{(primo del campo base)} \tag{1.102}$$

$$n = 36u^4 + 36u^3 + 18u^2 + 6u + 1 \quad \text{(orden del grupo)} \tag{1.103}$$

Matemáticas

Parámetros explícitos de BN254:

Campo base $\mathbb{F}_p$:

```
p = 21888242871839275222246405745257275088696311157297823662689037894645226208583
```

En hexadecimal:

```
p = 0x30644E72E131A029B85045B68181585D97816A916871CA8D3C208C16D87CFD47
```

Orden del grupo n:

```
n = 21888242871839275222246405745257275088548364400416034343698204186575808495617
```

Ecuación de la curva $E(\mathbb{F}_p)$:

$$y^2 = x^3 + 3$$

Generador G_1:

$$G_{1x} = 1 \tag{1.104}$$

$$G_{1y} = 2 \tag{1.105}$$

Observación 1.15. La simplicidad de $G_1 = (1, 2)$ y $b = 3$ no es coincidencia. Las curvas BN se diseñan buscando parámetros pequeños cuando es posible, lo que facilita la implementación y verificación.

Los Tres Grupos: $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$

Definición 1.74 (Grupos en BN254)**.** $\mathbb{G}_1$: Puntos de la curva sobre el campo base

$$\mathbb{G}_1 \subset E(\mathbb{F}_p) : y^2 = x^3 + 3$$

$\mathbb{G}_2$: Puntos de la curva sobre una extensión cuadrática

$$\mathbb{G}_2 \subset E(\mathbb{F}_p^2) : y^2 = x^3 + \frac{3}{9+i}$$

donde $\mathbb{F}_p^2 = \mathbb{F}_p[i]/(i^2 + 1)$ (adjuntando $\sqrt{-1}$).

$\mathbb{G}_T$: Subgrupo multiplicativo del campo de extensión

$$\mathbb{G}_T \subset \mathbb{F}_p^{12*}$$

Intuición

- $\mathbb{G}_1$: Puntos "normales", coordenadas de 256 bits
- $\mathbb{G}_2$: Puntos "extendidos", coordenadas de 512 bits (par de elementos de $\mathbb{F}_p$)
- $\mathbb{G}_T$: Elemento del campo de extensión, 3072 bits (12×256)

El pairing toma un punto "pequeño" y uno "grande" y produce un elemento "muy grande".

El Embedding Degree

Definición 1.75 (Embedding Degree)**.** El **embedding degree** k es el menor entero positivo tal que:

$$n \mid p^k - 1$$

donde n es el orden del grupo de la curva y p es el primo del campo base.

Idea Clave

El embedding degree determina:

1. **Seguridad**: El pairing mapea ECDLP a DLP en $\mathbb{F}_p^k$. Para seguridad, necesitamos que DLP en $\mathbb{F}_p^k$ sea difícil.
2. **Eficiencia**: k más pequeño = computación más rápida, pero también potencialmente menos seguridad.

Para BN254: $k = 12$, lo que significa que el grupo target $\mathbb{G}_T$ vive en $\mathbb{F}_p^{12}$ ($\sim$3072 bits).

Matemáticas

¿Por qué $k = 12$ da $\sim$100 bits de seguridad?
La seguridad del sistema se determina por el eslabón más débil:

- ECDLP en $\mathbb{G}_1$: $\sqrt{n} \approx 2^{127}$ operaciones (Pollard rho)
- DLP en $\mathbb{F}_p^{12}$: $\sim 2^{100}$ operaciones (NFS especial para pairings)

El ataque al campo de extensión resultó ser más eficiente de lo esperado inicialmente, reduciendo la seguridad a $\sim$100 bits.

Precompilados de Ethereum: EIP-196 y EIP-197

Definición 1.76 (Precompilados EVM para BN254)**.** Ethereum incluye contratos precompilados (código nativo, no EVM) para operaciones BN254:

EIP-196 (direcciones 0x06, 0x07):

- `0x06`: Suma de puntos en $\mathbb{G}_1$
- `0x07`: Multiplicación escalar en $\mathbb{G}_1$

EIP-197 (dirección 0x08):

- `0x08`: Verificación de producto de pairings

Ejemplo 1.35 (Costos de gas)**.**

Operación	Dirección	Gas
Suma $\mathbb{G}_1$	0x06	150
Multiplicación escalar $\mathbb{G}_1$	0x07	6,000
Pairing (por par)	0x08	34,000 + 45,000 × pares

Para verificar un Groth16 típico (3 pairings):

$$\text{Gas} \approx 34,000 + 3 \times 45,000 = 169,000 \text{ gas}$$

Idea Clave

Sin precompilados, verificar un pairing en EVM puro costaría millones de gas (si fuera posible). Los precompilados hacen SNARKs on-chain económicamente viables.
Esto es por qué BN254 domina en Ethereum: es la única curva con soporte nativo en la EVM.

Ejemplo 1.36 (Llamada al precompilado de pairing en Solidity)**.**

```
function verifyPairing(
    uint256[2] memory a,      // Punto A en G1
    uint256[2][2] memory b,   // Punto B en G2
    uint256[2] memory c,      // Punto C en G1
    uint256[2] memory alpha,  // Parámetro de setup
    uint256[2][2] memory beta // Parámetro de setup
) internal view returns (bool) {
    uint256[24] memory input;
    // Codificar los puntos...

    assembly {
        // Llamar al precompilado 0x08
        let success := staticcall(
            sub(gas(), 2000),
            0x08,  // Dirección del precompilado
            input,
            768,   // 24 * 32 bytes
```

```
            input,
            32      // Resultado: 0 o 1
        )
        // ...
    }
}
```

El Pairing Óptimo Ate

Definición 1.77 (Pairing Ate Óptimo)**.** BN254 usa el pairing "Optimal Ate", que es la versión más eficiente conocida para curvas BN:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

El cálculo involucra:

1. **Miller loop**: Construye una función racional evaluada en los puntos

2. **Final exponentiation**: Eleva el resultado a $(p^{12} - 1)/n$

Matemáticas

La complejidad del pairing Ate en BN254:

- Miller loop: $O(\log n)$ iteraciones, cada una con operaciones en $\mathbb{F}_p^{12}$
- Final exponentiation: Varias multiplicaciones y exponenciaciones en $\mathbb{F}_p^{12}$

Tiempo típico en CPU moderno: $\sim$1-2 ms por pairing.

Seguridad: ¿Son Suficientes 100 Bits?

Advertencia

La seguridad de ∼100 bits de BN254 está por debajo del estándar moderno de 128 bits. Sin embargo:

Argumentos a favor de seguir usando BN254:

1. Los ataques conocidos requieren 2^{100} operaciones, aún impracticables
2. Las pruebas ZK típicamente tienen validez temporal corta
3. El costo de migrar el ecosistema Ethereum es enorme
4. Para aplicaciones no críticas, es suficiente

Argumentos para migrar a BLS12-381:

1. 128 bits es el estándar de la industria
2. El hardware mejora; el margen de seguridad se reduce
3. Nuevos protocolos deberían usar curvas más seguras

Observación 1.16. La comunidad de Ethereum está considerando añadir precompilados para BLS12-381 (EIP-2537), pero aún no está implementado en mainnet. Mientras tanto, verificar BLS12-381 en EVM es prohibitivamente caro.

Aplicación en Banco ZK

En Banco ZK

BN254 es la curva principal de Banco ZK para verificación on-chain:

1. Verificación de pruebas Groth16

- El verificador on-chain usa los precompilados 0x06, 0x07, 0x08
- Costo: ∼200,000 gas por verificación
- Independiente del tamaño del circuito

2. Por qué elegimos BN254 sobre BLS12-381

Criterio	BN254	BLS12-381
Precompilados EVM	Sí	No
Gas para verificar	∼200K	∼3M+ (estimado)
Seguridad	∼100 bits	128 bits

La diferencia de costo es 15x, haciendo BLS12-381 impracticable actualmente.

3. Estructura de la prueba

```
struct Proof {
    // Elementos en G1 (256 bits cada coordenada)
    uint256[2] a;      // Punto A
```

```
    uint256[2] c;      // Punto C

    // Elemento en G2 (512 bits cada coordenada)
    uint256[2][2] b;   // Punto B
}
```

Tamaño total de prueba: $2 + 2 + 4 = 8$ elementos de campo = 256 bytes.

4. Migración futura

Cuando EIP-2537 se active:

1. Desplegaremos verificadores BLS12-381 para nuevos circuitos
2. Mantendremos verificadores BN254 para compatibilidad
3. Los usuarios no notarán diferencia (abstracción en contrato)

Librerías y Herramientas

Lenguaje	Librería	Notas
Rust	arkworks (ark-bn254)	Referencia, muy completa
Rust	bellman	Usada por Zcash
Go	gnark	Muy rápida
JavaScript	snarkjs	Para desarrollo/testing
Solidity	Precompilados nativos	Producción

1.3.4 BLS12-381

Curva de mayor seguridad para pairings.

- Embedding degree: 12
- Seguridad: ∼128 bits
- Usada en: Ethereum 2.0 (firmas BLS), Zcash Sapling
- Más segura que BN254, pero sin precompilados EVM

Origen y Motivación

Intuición

BLS12-381 fue diseñada específicamente para abordar las limitaciones de seguridad de curvas anteriores como BN254. El nombre proviene de:

- **BLS**: Barreto-Lynn-Scott (los diseñadores de la familia de curvas)
- **12**: Embedding degree
- **381**: Tamaño del campo base en bits

Observación 1.17. BLS12-381 fue propuesta por Sean Bowe (Zcash) en 2017 como parte del trabajo para Zcash Sapling. Los parámetros fueron elegidos cuidadosamente para:

1. Alcanzar 128 bits de seguridad
2. Maximizar eficiencia dentro de esa restricción
3. Permitir implementaciones de tiempo constante
4. Tener buenas propiedades para FFT (para SNARKs)

Parámetros de BLS12-381

Definición 1.78 (Construcción BLS12)**.** Las curvas BLS12 se parametrizan por un entero x. Para BLS12-381:

$$x = -\texttt{0xD201000000010000}$$

(Valor negativo, lo que da aritmética eficiente)
A partir de x:

$$p = (x-1)^2(x^4 - x^2 + 1)/3 + x \tag{1.106}$$

$$n = x^4 - x^2 + 1 \tag{1.107}$$

Matemáticas

Parámetros explícitos de BLS12-381:
Campo base $\mathbb{F}_p$ (381 bits):

```
p = 0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f624
      1eabfffeb153ffffb9feffffffffaaab
```

Orden del subgrupo n (255 bits):

```
n = 0x73eda753299d7d483339d80809a1d80553bda402fffe5bfeffffffff00000001
```

Ecuación de la curva:

$$E(\mathbb{F}_p) : y^2 = x^3 + 4$$

Twist para $\mathbb{G}_2$:

$$E'(\mathbb{F}_p^2) : y^2 = x^3 + 4(1+i)$$

donde $\mathbb{F}_p^2 = \mathbb{F}_p[i]/(i^2+1)$.
Cofactores:

$$h_1 = (x-1)^2/3 = \texttt{0x396c8c005555e1568c00aaab0000aaab} \tag{1.108}$$

$$h_2 = (x^8 - 4x^7 + 5x^6 - 4x^4 + 6x^3 - 4x^2 - 4x + 13)/9 \tag{1.109}$$

Advertencia

A diferencia de BN254 donde el cofactor es 1, BLS12-381 tiene **cofactores grandes**. Esto significa que no todos los puntos de la curva están en el subgrupo de orden n. Las implementaciones deben:

1. Multiplicar por el cofactor para proyectar al subgrupo correcto, o
2. Verificar membresía en el subgrupo explícitamente

Fallar en esto puede causar vulnerabilidades de seguridad.

Comparación de Tamaños

Elemento	BN254	BLS12-381
Campo base p	254 bits	381 bits
Orden del grupo n	254 bits	255 bits
Elemento de $\mathbb{G}_1$	64 bytes	96 bytes
Elemento de $\mathbb{G}_2$	128 bytes	192 bytes
Elemento de $\mathbb{G}_T$	384 bytes	576 bytes
Campo de extensión	$\mathbb{F}_p^{12}$ (3072 bits)	$\mathbb{F}_p^{12}$ (4572 bits)

Intuición

BLS12-381 usa ∼50% más espacio que BN254 para cada elemento. Esto impacta:

- Tamaño de claves públicas y firmas
- Ancho de banda para transmitir pruebas
- Almacenamiento on-chain (más gas en calldata)

Pero la seguridad adicional (128 vs 100 bits) generalmente vale este costo.

Seguridad: Los 128 Bits

Idea Clave

BLS12-381 fue diseñada específicamente para 128 bits de seguridad considerando:

1. **ECDLP en** $\mathbb{G}_1, \mathbb{G}_2$: $\sqrt{n} \approx 2^{127.5}$ operaciones
2. **DLP en** $\mathbb{G}_T \subset \mathbb{F}_p^{12}$: NFS especial requiere $\sim 2^{128}$ operaciones

Ambos ataques están balanceados, sin eslabón débil.

Matemáticas

El tamaño del campo (381 bits) fue elegido precisamente para que:

$$\text{Seguridad de } \mathbb{F}_p^{12} \approx \text{Seguridad de ECDLP} \approx 128 \text{ bits}$$

Si el campo fuera más pequeño (como en BN254), el ataque al campo de extensión sería más fácil.

Firmas BLS

Definición 1.79 (Firma BLS)**.** El esquema de firmas BLS aprovecha los pairings:

Generación de claves:

- Clave privada: $sk \in \mathbb{Z}_n$ aleatorio
- Clave pública: $pk = sk \cdot G_2 \in \mathbb{G}_2$

Firma:

$$\sigma = sk \cdot H(m) \in \mathbb{G}_1$$

donde $H : \{0,1\}^* \to \mathbb{G}_1$ es un hash-to-curve.

Verificación:

$$e(\sigma, G_2) \stackrel{?}{=} e(H(m), pk)$$

Matemáticas

¿Por qué funciona?

$$\begin{aligned} e(\sigma, G_2) &= e(sk \cdot H(m), G_2) && (1.110)\\ &= e(H(m), G_2)^{sk} \quad \text{(bilinealidad)} && (1.111)\\ &= e(H(m), sk \cdot G_2) && (1.112)\\ &= e(H(m), pk) \quad \checkmark && (1.113) \end{aligned}$$

Agregación de Firmas

Idea Clave

La propiedad más poderosa de BLS es la **agregación de firmas**:
Mismo mensaje: Si n firmantes firman el mismo mensaje m:

$$\sigma_{\text{agg}} = \sigma_1 + \sigma_2 + \cdots + \sigma_n$$

Verificación:

$$e(\sigma_{\text{agg}}, G_2) = e(H(m), pk_1 + pk_2 + \cdots + pk_n)$$

¡Una sola verificación de pairing para n firmas!

Mensajes diferentes (más complejo):

$$\sigma_{\text{agg}} = \sigma_1 + \sigma_2 + \cdots + \sigma_n$$

$$e(\sigma_{\text{agg}}, G_2) = \prod_{i=1}^{n} e(H(m_i), pk_i)$$

Requiere n pairings, pero solo una firma que transmitir.

Ejemplo 1.37 (Ethereum 2.0 Beacon Chain)**.** La Beacon Chain tiene ∼900,000 validadores. Sin agregación:

- 900,000 firmas ECDSA × 64 bytes = 57.6 MB por bloque
- Verificar 900,000 firmas individualmente

Con agregación BLS:

- Los validadores se agrupan en comités de 128
- Cada comité produce una firma agregada (48 bytes)
- ∼2,000 firmas agregadas por slot
- Reducción de ∼450x en datos y verificación

Uso en Zcash Sapling

Observación 1.18. Zcash migró de BN128 (usado en Sprout) a BLS12-381 (en Sapling) por razones de seguridad. El protocolo Sapling usa:

- **Groth16** sobre BLS12-381 para pruebas de transacciones privadas
- **RedDSA** (variante de Schnorr) sobre Jubjub (curva embebida en BLS12-381)

Definición 1.80 (Curva Jubjub)**.** Jubjub es una curva de Edwards embebida en BLS12-381:

$$-x^2 + y^2 = 1 + d \cdot x^2 y^2$$

Su orden es $n_{\text{BLS12-381}}$, lo que permite operaciones de curva elíptica **dentro** de circuitos SNARK sobre BLS12-381 de forma eficiente.

EIP-2537: Precompilados BLS12-381

Definición 1.81 (EIP-2537)**.** Propuesta para añadir precompilados BLS12-381 a Ethereum:

Dirección	Operación	Gas propuesto
0x0a	Suma $\mathbb{G}_1$	500
0x0b	Multiplicación escalar $\mathbb{G}_1$	12,000
0x0c	Multi-scalar multiplication $\mathbb{G}_1$	Variable
0x0d	Suma $\mathbb{G}_2$	800
0x0e	Multiplicación escalar $\mathbb{G}_2$	45,000
0x0f	Multi-scalar multiplication $\mathbb{G}_2$	Variable
0x10	Pairing	43,000 + 32,600 $\times$ pares
0x11	Map to $\mathbb{G}_1$	5,500
0x12	Map to $\mathbb{G}_2$	75,000

Observación 1.19. EIP-2537 ha estado en desarrollo desde 2020 pero aún no se ha incluido en un hard fork. Las razones incluyen:

- Complejidad de implementación y testing
- Preocupaciones sobre aumento del tamaño del cliente
- Debate sobre si los costos de gas son apropiados
- Competencia con otras mejoras prioritarias

Mientras tanto, verificar BLS12-381 en EVM requiere implementación en Solidity puro (∼millones de gas).

Rendimiento Comparativo

Operación	BN254	BLS12-381	Ratio
Suma $\mathbb{G}_1$	0.5 µs	0.8 µs	1.6x
Mult. escalar $\mathbb{G}_1$	50 µs	80 µs	1.6x
Suma $\mathbb{G}_2$	1.5 µs	2.5 µs	1.7x
Mult. escalar $\mathbb{G}_2$	150 µs	250 µs	1.7x
Pairing	1.5 ms	2 ms	1.3x

Intuición

BLS12-381 es aproximadamente 1.3-1.7x más lento que BN254 para la mayoría de operaciones. Este overhead es aceptable considerando el aumento de seguridad de 100 a 128 bits.

Comparación Final: BN254 vs BLS12-381

Característica	BN254	BLS12-381
Seguridad	∼100 bits	∼128 bits
Campo base	254 bits	381 bits
Precompilados EVM	Sí (desde 2017)	No (propuesto)
Tamaño de $\mathbb{G}_1$ punto	64 bytes	96 bytes
Tamaño de $\mathbb{G}_2$ punto	128 bytes	192 bytes
Velocidad	Más rápida	∼1.5x más lenta
Ethereum usage	SNARKs on-chain	Beacon Chain (off-EVM)
Zcash	Sprout (deprecado)	Sapling (actual)
Cofactor h_1	1	Grande
Madurez	Alta	Alta

Aplicación en Banco ZK

En Banco ZK

Estrategia dual de curvas en Banco ZK:

1. **Actualmente (BN254)**
   - Todos los verificadores on-chain usan BN254
   - Aprovechamos precompilados para costos bajos
   - Aceptamos 100 bits de seguridad como compromiso práctico
2. **Infraestructura preparada para BLS12-381**
   - Circuitos compilados para ambas curvas
   - Verificadores BLS12-381 listos (esperando EIP-2537)
   - Testing continuo en redes de prueba
3. **Casos de uso por curva**

Componente	Curva	Razón
Verificación SNARK (L1)	BN254	Precompilados disponibles
Agregación de firmas (L2)	BLS12-381	Mejor seguridad, off-chain
Firmas de validadores	BLS12-381	Estándar Ethereum 2.0
Pruebas recursivas	BN254/BLS12-381	Depende del caso

4. **Plan de migración post-EIP-2537**

Cuando los precompilados BLS12-381 estén disponibles:

1. Desplegar verificador BLS12-381
2. Período de transición con ambos verificadores activos

3. Deprecar verificador BN254 para nuevas pruebas

4. Mantener BN254 solo para verificación de pruebas históricas

5. Consideraciones de diseño

Para maximizar flexibilidad, nuestro contrato verificador es una interfaz:

```
interface IVerifier {
    function verify(
        bytes calldata proof,
        uint256[] calldata publicInputs
    ) external view returns (bool);
}

contract BN254Verifier is IVerifier { ... }
contract BLS12381Verifier is IVerifier { ... }  // Futuro
```

El contrato principal puede actualizar la implementación del verificador sin cambiar la lógica de negocio.

Librerías y Herramientas para BLS12-381

Lenguaje	Librería	Notas
Rust	arkworks (ark-bls12-381)	Referencia
Rust	blst	Muy optimizada (usado en Ethereum 2.0)
Go	gnark-crypto	Producción
Go	bls12-381 (herumi)	Optimizada
C	blst	Ensamblador optimizado
JavaScript	noble-bls12-381	Audited, pura JS
Python	py_ecc	Educacional

Observación 1.20. La librería `blst` ("blast") desarrollada por Supranational es considerada la implementación más rápida y segura. Es usada por la mayoría de clientes de Ethereum 2.0 (Lighthouse, Prysm, Teku, etc.).

1.3.5 Pasta (Pallas y Vesta)

Ciclo de curvas para recursión eficiente.

Definición 1.82 (Ciclo de curvas)**.** Dos curvas E_1, E_2 forman un ciclo si:

- El orden de E_1 es el campo base de E_2
- El orden de E_2 es el campo base de E_1
- **Pallas:** $y^2 = x^3 + 5$ sobre $\mathbb{F}_p$

- **Vesta:** $y^2 = x^3 + 5$ sobre $\mathbb{F}_q$
- Donde $p = |E_{\text{Vesta}}|$ y $q = |E_{\text{Pallas}}|$
- Usadas en: Halo2, Mina, recursión de pruebas

El Problema de la Recursión

Intuición

Imagina que quieres verificar una prueba ZK *dentro* de otra prueba ZK. Por ejemplo:

1. Tienes una prueba π_1 que demuestra "Conozco x tal que $f(x) = y$"
2. Quieres crear una prueba π_2 que demuestra "Verifiqué correctamente π_1"
3. Y luego π_3 que demuestra "Verifiqué π_2"...

Esto es **recursión de pruebas**, fundamental para:

- Comprimir cadenas largas de verificaciones
- Rollups que acumulan miles de transacciones
- Blockchains comprimidas (como Mina)

Advertencia

El problema técnico:
Si usas una curva como BN254, verificar una prueba requiere operaciones de curva elíptica (multiplicaciones escalares, pairings). Pero dentro de un circuito SNARK sobre BN254, las operaciones de campo son módulo p (el orden del grupo), no módulo el campo base de BN254.
Resultado: verificar una prueba BN254 dentro de un circuito BN254 requiere **aritmética de campo no nativa**, que es extremadamente costosa (∼1,000,000+ constraints para una verificación).

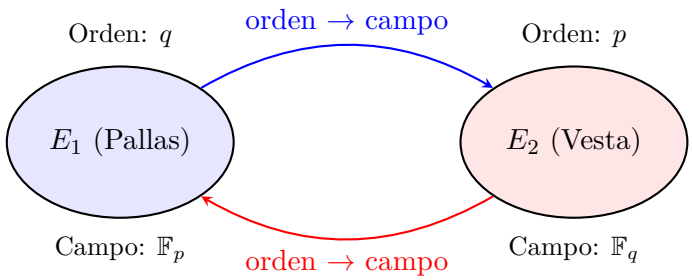
La Solución: Ciclos de Curvas

Definición 1.83 (Ciclo de Curvas (2-cycle))**.** Dos curvas elípticas E_1 y E_2 forman un **2-ciclo** si:

$$\#E_1(\mathbb{F}_p) = q \tag{1.114}$$
$$\#E_2(\mathbb{F}_q) = p \tag{1.115}$$

donde p es el campo base de E_1 y q es el campo base de E_2.
Dicho de otra forma:

- El **orden del grupo** de E_1 es el **campo base** de E_2
- El **orden del grupo** de E_2 es el **campo base** de E_1



Idea Clave

Con un ciclo de curvas:

1. Creas una prueba π_1 sobre E_1 (Pallas)
2. Verificas π_1 **dentro** de un circuito sobre E_2 (Vesta)
3. La verificación usa aritmética de $\mathbb{F}_q$, que es el campo nativo de E_2
4. Las operaciones de curva de E_1 (que tienen orden q) son **nativas** en E_2
5. Y viceversa para verificar pruebas de E_2 en circuitos de E_1

No hay aritmética de campo no nativa. La verificación es eficiente.

Historia: De MNT a Pasta

Observación 1.21. Los ciclos de curvas no son nuevos, pero encontrar ciclos con buenas propiedades criptográficas es difícil:

2001 - Curvas MNT: Miyaji, Nakabayashi, y Takano descubrieron familias de curvas que forman ciclos. Usadas en los primeros sistemas recursivos.

2014 - Ben-Sasson et al.: Proponen usar ciclos MNT para SNARKs recursivos en el paper "Scalable Zero Knowledge via Cycles of Elliptic Curves".

2019 - Bowe et al. (Zcash): Diseñan las curvas **Pasta** (Pallas + Vesta) específicamente para Halo 2, optimizadas para eficiencia moderna.

2020 - Mina Protocol: Usa Pasta para crear una blockchain de tamaño constante (∼22 KB) mediante recursión.

Las Curvas Pasta: Pallas y Vesta

Definición 1.84 (Curvas Pasta)**. Pallas** (la "verde"):

$$E_{\text{Pallas}} : y^2 = x^3 + 5 \quad \text{sobre } \mathbb{F}_p \tag{1.116}$$

$$p = 2^{254} + 45560315531419706090280762371685220353 \tag{1.117}$$

Vesta (la "roja"):

$$E_{\text{Vesta}} : y^2 = x^3 + 5 \quad \text{sobre } \mathbb{F}_q \tag{1.118}$$

$$q = 2^{254} + 45560315531506369815346746415080538113 \tag{1.119}$$

La propiedad de ciclo:

$$\#E_{\text{Pallas}}(\mathbb{F}_p) = q \tag{1.120}$$

$$\#E_{\text{Vesta}}(\mathbb{F}_q) = p \tag{1.121}$$

Matemáticas

Parámetros explícitos:

```
# Pallas
p = 0x40000000000000000000000000000000224698fc094cf91b992d30ed00000001
# Orden de Pallas = q (campo base de Vesta)
q = 0x40000000000000000000000000000000224698fc0994a8dd8c46eb2100000001

# Ecuación: y² = x³ + 5
a = 0
b = 5

# Generador de Pallas
Gx = -1 (mod p)
Gy = 2
```

Ambas curvas tienen la misma ecuación ($y^2 = x^3 + 5$) pero sobre campos diferentes.

Intuición

¿Por qué el nombre "Pasta"?
Es un juego de palabras: **Pa**llas + Ve**sta** = Pasta
Los nombres individuales vienen de asteroides/diosas:

- **Pallas**: Segundo asteroide descubierto (1802), nombrado por Palas Atenea
- **Vesta**: Cuarto asteroide descubierto (1807), nombrado por la diosa romana del hogar

La comunidad ZK tiene tradición de nombres creativos (Groth16, PLONK, Halo, etc.).

¿Por Qué Ambas Curvas son $y^2 = x^3 + 5$?

Idea Clave

Tener la **misma ecuación** para ambas curvas simplifica enormemente:

1. **Código compartido**: Las mismas fórmulas de punto funcionan para ambas
2. **Menos errores**: Una implementación, dos curvas
3. **Optimizaciones compartidas**: Mejoras aplican a ambas

La única diferencia es el campo sobre el que operan.

Ejemplo 1.38 (Suma de puntos - mismo código)**.**

```
// Funciona para Pallas Y Vesta
fn point_add<F: PrimeField>(p1: Point<F>, p2: Point<F>) -> Point<F> {
    let lambda = (p2.y - p1.y) / (p2.x - p1.x);
    let x3 = lambda * lambda - p1.x - p2.x;
    let y3 = lambda * (p1.x - x3) - p1.y;
    Point { x: x3, y: y3 }
}

// Para Pallas: F = Fp
// Para Vesta: F = Fq
```

Propiedades de Seguridad

Propiedad	Pallas	Vesta
Tamaño del campo	254 bits	254 bits
Orden del grupo	254 bits	254 bits
Seguridad ECDLP	∼127 bits	∼127 bits
Cofactor	1	1
Embedding degree	Alto	Alto
Discriminante CM	Grande	Grande

Idea Clave

Ambas curvas tienen **cofactor 1**, lo que significa:

- Todo punto (excepto $\mathcal{O}$) tiene orden n (el orden del grupo)
- No hay subgrupos pequeños que explotar
- No se necesita multiplicación por cofactor para seguridad

Esto simplifica implementaciones y evita clases enteras de ataques.

Observación 1.22. Las curvas Pasta fueron diseñadas para **no** ser pairing-friendly (embedding degree alto). Esto es intencional:

1. Evita ataques de MOV/FR que explotan pairings
2. Simplifica la implementación (no hay código de pairing)
3. Para sistemas como Halo 2, los pairings no son necesarios

Recursión en Acción

Ejemplo 1.39 (Cadena de verificación recursiva)**. Objetivo:** Demostrar que verificaste correctamente 1000 pruebas.

Sin recursión:

- Verificar 1000 pruebas on-chain
- Costo: $1000 \times 200,000 = 200$ millones de gas
- Imposible en un solo bloque

Con recursión usando Pasta:

Paso 1: Crear prueba π_1 sobre Pallas que demuestra "Verifiqué pruebas 1-10"

Paso 2: Crear prueba π_2 sobre **Vesta** que demuestra:

- "π_1 es válida" (verificación nativa, eficiente)
- "También verifiqué pruebas 11-20"

Paso 3: Crear prueba π_3 sobre **Pallas** que demuestra:

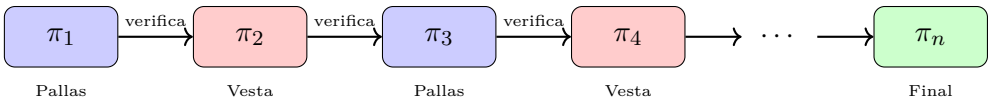
- "π_2 es válida"
- "También verifiqué pruebas 21-30"

... continúa alternando ...

Paso 100: Prueba final π_{100} que demuestra todo.

Resultado:

- Una sola prueba de tamaño constante
- Verifica 1000 pruebas originales
- Costo on-chain: ∼200,000 gas (igual que una prueba)



Halo 2: El Sistema que Usa Pasta

Definición 1.85 (Halo 2)**.** Halo 2 es un sistema de pruebas desarrollado por Zcash que:

- Usa las curvas Pasta para recursión
- No requiere trusted setup (es "transparent")
- Basado en el esquema de compromiso IPA (Inner Product Argument)
- Implementa un sistema de constraints llamado PLONKish

Idea Clave

¿Por qué Halo 2 no necesita pairings?

Los sistemas como Groth16 usan pairings para verificación:

$$e(A, B) = e(C, D) \cdot e(E, F) \cdots$$

Halo 2 usa una técnica diferente basada en **acumuladores**:

1. En lugar de verificar completamente cada prueba, "acumula" el trabajo
2. Las verificaciones parciales se combinan en un "acumulador"
3. Solo al final se hace una verificación completa
4. Esta verificación final es eficiente y no requiere pairings

Esto evita la necesidad de curvas pairing-friendly, permitiendo usar curvas más simples como Pasta.

Observación 1.23. El paper original "Halo: Recursive Proof Composition without a Trusted Setup" (2019) de Bowe, Grigg, y Hopwood fue revolucionario al demostrar que la recursión eficiente era posible sin pairings y sin trusted setup.

Mina Protocol: Blockchain de 22 KB

Intuición

Mina Protocol usa Pasta para crear una blockchain donde el estado completo siempre es de ∼22 KB, independientemente del historial.

¿Cómo?

Cada bloque contiene una prueba recursiva que dice: "El bloque anterior era válido, y esta nueva transición de estado también es válida"

Verificar la cadena completa = verificar UNA prueba.

Ejemplo 1.40 (Comparación de tamaños de blockchain)**.**

Blockchain	Tamaño para verificar	Tiempo sync
Bitcoin	∼500 GB	Horas/días
Ethereum	∼1 TB+	Días
Mina	∼22 KB	Segundos

Mina logra esto acumulando todas las verificaciones en una sola prueba recursiva usando Pasta.

Comparación con Otros Ciclos

Ciclo	Tamaño (bits)	Seguridad	Uso
MNT4/MNT6	298/753	∼80/∼128 bits	Celo, antiguo Zcash
Pasta (Pallas/Vesta)	254/254	∼127 bits	Halo 2, Mina
Curve-5/Curve-6	Variable	Variable	Experimental

Idea Clave

Ventajas de Pasta sobre MNT:

1. **Tamaños balanceados**: Ambas curvas de 254 bits (MNT tiene una curva mucho más grande)
2. **Misma ecuación**: Código compartido
3. **Cofactor 1**: Sin complicaciones de subgrupos
4. **Diseño moderno**: Optimizado para hardware actual

Implementación: Coordinación de Campos

Matemáticas

El truco de implementación:
Cuando trabajas con Pasta, constantemente cambias entre dos contextos:

- Aritmética en $\mathbb{F}_p$ (campo base de Pallas, campo escalar de Vesta)
- Aritmética en $\mathbb{F}_q$ (campo base de Vesta, campo escalar de Pallas)

Un elemento puede tener dos "interpretaciones":

- Como coordenada de un punto Pallas (vive en $\mathbb{F}_p$)
- Como escalar para multiplicación en Vesta (vive en $\mathbb{F}_q$ = orden de Pallas)

Pero $p \neq q$ (difieren en $\sim 2^{126}$), así que hay que tener cuidado con las conversiones.

Ejemplo 1.41 (Estructura de datos en Rust (Halo 2)). `use pasta_curves::{Fp, Fq, pallas, vesta};`

```
// Punto en Pallas - coordenadas en Fp
let pallas_point: pallas::Point = pallas::Point::generator();

// Escalar para Pallas - vive en Fq (el orden de Pallas)
let pallas_scalar: Fq = Fq::from(42u64);

// Multiplicación escalar en Pallas
let result = pallas_point * pallas_scalar;
```

```
// Punto en Vesta - coordenadas en Fq
let vesta_point: vesta::Point = vesta::Point::generator();

// Escalar para Vesta - vive en Fp (el orden de Vesta)
let vesta_scalar: Fp = Fp::from(42u64);

// Aquí está la magia del ciclo:
// Fp es TANTO el campo base de Pallas COMO el campo escalar de Vesta
// Fq es TANTO el campo base de Vesta COMO el campo escalar de Pallas
```

Consideraciones de Seguridad

Advertencia

Riesgos específicos de ciclos de curvas:

1. Curvas relacionadas: Un ataque que debilite una curva podría afectar a la otra, ya que comparten estructura algebraica.

2. Errores de implementación: Confundir $\mathbb{F}_p$ con $\mathbb{F}_q$ puede causar bugs sutiles que compilan pero son incorrectos.

3. Side-channels amplificados: Si hay un leak de información en una curva, podría revelar información sobre la otra.

Mitigaciones:

- Tipos de datos fuertes que distinguen $\mathbb{F}_p$ de $\mathbb{F}_q$
- Tests extensivos de ida y vuelta
- Auditorías de seguridad específicas para sistemas recursivos

Rendimiento

Operación	Pallas/Vesta	Comparación
Suma de puntos	∼0.3 µs	Similar a secp256k1
Multiplicación escalar	∼40 µs	Similar a secp256k1
MSM (1024 puntos)	∼15 ms	Optimizado para Halo 2
Verificación recursiva	∼10-50 ms	Depende del circuito

Observación 1.24. Las curvas Pasta están optimizadas para operaciones de **Multi-Scalar Multiplication (MSM)**, que dominan el tiempo de prueba en sistemas como Halo 2. Implementaciones como la de Zcash usan:

- Algoritmo de Pippenger para MSM
- Paralelización en múltiples cores
- Representaciones de punto optimizadas (coordenadas extendidas)

Futuro: Más Allá de 2-Ciclos

Intuición

Los 2-ciclos (como Pasta) alternan entre dos curvas. Pero hay investigación en:

- **1-ciclos**: Una sola curva que "cicla" consigo misma (teóricamente posible pero no encontrado aún con buena seguridad)
- **k-ciclos**: Cadenas más largas de curvas
- **Ciclos con pairings**: Combinar recursión eficiente con verificación por pairings

Observación 1.25. Encontrar un 1-ciclo seguro sería un avance significativo: eliminaría la necesidad de alternar entre curvas y simplificaría aún más las implementaciones. Sin embargo, las restricciones matemáticas hacen esto muy difícil.

Aplicación en Banco ZK

En Banco ZK

Las curvas Pasta son parte de nuestra estrategia de recursión:

1. Agregación de pruebas

En lugar de verificar cada prueba de transacción individualmente on-chain:

1. Generamos pruebas individuales sobre Pallas
2. Las agregamos recursivamente alternando Pallas/Vesta
3. Enviamos una sola prueba agregada al L1

Beneficio: 1000 transacciones $\rightarrow$ 1 verificación on-chain.

2. Arquitectura del sistema

Capa	Curva	Función
Transacción individual	Pallas	Prueba de validez
Agregación nivel 1	Vesta	Combinar 10 pruebas
Agregación nivel 2	Pallas	Combinar 10 agregados
$\vdots$	Alterna	$\vdots$
Prueba final	BN254	Para verificar on-chain

3. Conversión Pasta $\rightarrow$ BN254

La prueba final debe verificarse en Ethereum (que tiene precompilados BN254). Hacemos una "traducción" final:

1. Prueba recursiva final está sobre Pallas o Vesta
2. Creamos una prueba BN254 que dice "Verifiqué la prueba Pasta"
3. Esta verificación incluye emular aritmética de Pasta en BN254

4. Costoso (∼1M constraints), pero solo se hace una vez

4. Trade-offs considerados

Opción	Ventaja	Desventaja
Solo BN254	Simple, directo a L1	Recursión cara
Pasta + traducción	Recursión eficiente	Traducción costosa
Solo Pasta	Más eficiente	No hay precompilados L1

Elegimos Pasta + traducción porque la eficiencia de recursión compensa el costo de traducción cuando hay muchas transacciones.

5. Código de ejemplo

```
// Estructura de agregación recursiva
struct RecursiveProof {
    // Punto de acumulación en Pallas o Vesta
    accumulator: CurvePoint,
    // Prueba de que el acumulador es válido
    proof: Proof,
    // Metadata
    num_proofs_accumulated: u64,
}

fn aggregate(proofs: Vec<Proof>) -> RecursiveProof {
    let mut acc = RecursiveProof::empty();

    for (i, proof) in proofs.iter().enumerate() {
        if i % 2 == 0 {
            // Verificar en circuito Vesta
            acc = verify_and_accumulate_on_vesta(acc, proof);
        } else {
            // Verificar en circuito Pallas
            acc = verify_and_accumulate_on_pallas(acc, proof);
        }
    }

    acc
}
```

Librerías y Herramientas

Librería	Lenguaje	Notas
pasta_curves	Rust	Implementación oficial de Zcash
halo2	Rust	Sistema de pruebas completo
arkworks (ark-pallas)	Rust	Parte del ecosistema arkworks
o1js (SnarkyJS)	TypeScript	Para desarrollo en Mina

Ejemplo 1.42 (Uso básico de pasta_curves)**.**

```
use pasta_curves::{
    pallas, vesta,
    group::Group,
    arithmetic::CurveAffine,
};

fn main() {
    // Generadores
    let g_pallas = pallas::Point::generator();
    let g_vesta = vesta::Point::generator();

    // Escalares (notar los tipos cruzados)
    let scalar_for_pallas: pallas::Scalar = pallas::Scalar::from(42u64);
    let scalar_for_vesta: vesta::Scalar = vesta::Scalar::from(42u64);

    // Multiplicación escalar
    let p = g_pallas * scalar_for_pallas;
    let v = g_vesta * scalar_for_vesta;

    // Conversión a coordenadas afines
    let p_affine = p.to_affine();
    println!("Pallas point: ({:?}, {:?})", p_affine.x, p_affine.y);
}
```

Constantes de Referencia

Matemáticas

Resumen de constantes Pasta:

```
# Pallas
p_pallas = 0x40000000000000000000000000000000224698fc094cf91b992d30ed00000001
# = 2^254 + 45560315531419706090280762371685220353

# Orden de Pallas (= campo base de Vesta)
q_pallas = 0x40000000000000000000000000000000224698fc0994a8dd8c46eb2100000001
# = 2^254 + 45560315531506369815346746415080538113
```

```
# Ecuación de ambas curvas
a = 0
b = 5

# Generador de Pallas
Gx_pallas = p_pallas - 1  # = -1 mod p
Gy_pallas = 2

# Generador de Vesta
Gx_vesta = q_pallas - 1   # = -1 mod q
Gy_vesta = 2

# Verificación del ciclo
assert orden_pallas == campo_base_vesta
assert orden_vesta == campo_base_pallas
```

1.4 Firmas Digitales

1.4.1 ECDSA

Definición 1.86 (ECDSA)**.** Elliptic Curve Digital Signature Algorithm.

Algoritmo:

1. **Keygen:** $sk \leftarrow \mathbb{Z}_n$, $pk = sk \cdot G$
2. **Sign:**
   - Elegir nonce $k \leftarrow \mathbb{Z}_n$
   - $R = kG$, sea $r = x_R \mod n$
   - $s = k^{-1}(H(m) + r \cdot sk) \mod n$
   - Firma: (r, s)
3. **Verify:**
   - $u_1 = H(m) \cdot s^{-1}$, $u_2 = r \cdot s^{-1}$
   - Verificar $x_{u_1 G + u_2 \cdot pk} \equiv r \pmod n$

Idea Clave

Peligro crítico: Reutilizar el nonce k expone la clave privada. Si dos firmas (r, s_1) y (r, s_2) usan el mismo k:

$$sk = \frac{s_1 H(m_2) - s_2 H(m_1)}{r(s_2 - s_1)}$$

Esto causó pérdidas millonarias en la historia de Bitcoin.

Contexto Histórico

Intuición

Las firmas digitales son el equivalente criptográfico de una firma manuscrita: demuestran que el poseedor de cierta clave privada autorizó un mensaje específico. ECDSA es la variante de curva elíptica del algoritmo DSA (Digital Signature Algorithm), estandarizado por NIST en 1991.

Observación 1.26. **Línea temporal de las firmas digitales:**

Año	Desarrollo
1976	Diffie-Hellman introduce firmas digitales (concepto)
1978	RSA - primera implementación práctica
1985	ElGamal propone firmas basadas en logaritmo discreto
1991	DSA estandarizado por NIST (FIPS 186)
1992	Vanstone propone ECDSA
1999	ECDSA incluido en ANSI X9.62
2005	ECDSA en FIPS 186-3
2009	Bitcoin usa ECDSA sobre secp256k1
2015	Ethereum adopta el mismo esquema

¿Por Qué ECDSA sobre DSA Clásico?

Idea Clave

ECDSA ofrece la misma seguridad que DSA con claves mucho más cortas:

Seguridad	RSA/DSA	ECDSA
80 bits	1024 bits	160 bits
112 bits	2048 bits	224 bits
128 bits	3072 bits	256 bits
256 bits	15360 bits	512 bits

Para 128 bits de seguridad:

- Clave RSA: 3072 bits = 384 bytes
- Clave ECDSA: 256 bits = 32 bytes (12x más pequeña)

El Algoritmo en Detalle

Parámetros del Sistema

Definición 1.87 (Parámetros de Dominio ECDSA)**.** Para usar ECDSA, se fijan:

- Curva elíptica E sobre $\mathbb{F}_p$
- Punto generador $G \in E(\mathbb{F}_p)$ de orden primo n

- Función hash $H : \{0,1\}^* \to \mathbb{Z}_n$

Para Bitcoin/Ethereum (secp256k1):

- $E : y^2 = x^3 + 7$ sobre $\mathbb{F}_p$
- $p = 2^{256} - 2^{32} - 977$
- $n =$ `0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFE...`
- $H =$ SHA-256 (Bitcoin) o Keccak-256 (Ethereum)

Generación de Claves

Definición 1.88 (KeyGen)**.** 1. Elegir $sk \xleftarrow{\$} [1, n-1]$ uniformemente aleatorio

2. Calcular $pk = sk \cdot G$
3. Publicar pk, mantener sk secreto

Intuición

La clave pública pk es un punto en la curva. Aunque el atacante conoce pk y G, no puede encontrar sk porque esto requeriría resolver el problema del logaritmo discreto en curvas elípticas (ECDLP), que es computacionalmente intratable.

Firma

Definición 1.89 (Sign(sk, m))**.** Para firmar un mensaje m con clave privada sk:

1. Calcular el hash del mensaje: $e = H(m)$
2. Elegir nonce aleatorio: $k \xleftarrow{\$} [1, n-1]$
3. Calcular el punto efímero: $R = k \cdot G = (x_R, y_R)$
4. Calcular $r = x_R \mod n$
   - Si $r = 0$, volver al paso 2
5. Calcular $s = k^{-1}(e + r \cdot sk) \mod n$
   - Si $s = 0$, volver al paso 2
6. Retornar firma $\sigma = (r, s)$

Matemáticas

Descomposición de la fórmula de s:

$$s = k^{-1}(e + r \cdot sk) \mod n$$

- $e = H(m)$: el mensaje "entra" en la firma

- $r \cdot sk$: enlaza con la clave privada
- k^{-1}: "oculta" los componentes usando el nonce

La seguridad depende de que k sea secreto y único para cada firma.

Verificación

Definición 1.90 (Verify(pk, m, σ))**.** Para verificar una firma $\sigma = (r, s)$ sobre mensaje m con clave pública pk:

1. Verificar que $r, s \in [1, n-1]$
2. Calcular $e = H(m)$
3. Calcular $w = s^{-1} \mod n$
4. Calcular $u_1 = e \cdot w \mod n$
5. Calcular $u_2 = r \cdot w \mod n$
6. Calcular el punto $R' = u_1 \cdot G + u_2 \cdot pk$
7. Verificar que $x_{R'} \equiv r \pmod{n}$

Si la verificación pasa, la firma es válida.

¿Por Qué Funciona?

Matemáticas

Demostración de correctitud:
Queremos mostrar que $x_{R'} = r$ cuando la firma es válida.
Partimos de:

$$R' = u_1 \cdot G + u_2 \cdot pk$$

Sustituyendo:

$$\begin{aligned} R' &= (e \cdot w) \cdot G + (r \cdot w) \cdot pk && (1.122)\\ &= w \cdot (e \cdot G + r \cdot pk) && (1.123)\\ &= s^{-1} \cdot (e \cdot G + r \cdot sk \cdot G) \quad (\text{porque } pk = sk \cdot G) && (1.124)\\ &= s^{-1} \cdot (e + r \cdot sk) \cdot G && (1.125) \end{aligned}$$

Recordando que: $s = k^{-1}(e + r \cdot sk)$, entonces $s^{-1} = k \cdot (e + r \cdot sk)^{-1}$

$$R' = k \cdot (e + r \cdot sk)^{-1} \cdot (e + r \cdot sk) \cdot G \tag{1.126}$$
$$= k \cdot G \tag{1.127}$$
$$= R \tag{1.128}$$

Por lo tanto, $x_{R'} = x_R = r$. □

Ejemplo Numérico Completo

Ejemplo 1.43 (ECDSA sobre una curva pequeña)**.** Usaremos una curva pequeña para ilustrar (NO segura para uso real).

Parámetros:

- Curva: $E : y^2 = x^3 + 7$ sobre $\mathbb{F}_{17}$
- Generador: $G = (6, 11)$
- Orden: $n = 18$ (simplificado)

1. Generación de claves:

$$sk = 7 \quad \text{(elegido aleatoriamente)} \tag{1.129}$$
$$pk = 7 \cdot G = 7 \cdot (6, 11) = (13, 10) \tag{1.130}$$

2. Firma del mensaje m donde $H(m) = 5$:

$$k = 11 \quad \text{(nonce aleatorio)} \tag{1.131}$$
$$R = 11 \cdot G = (7, 8) \tag{1.132}$$
$$r = x_R = 7 \tag{1.133}$$
$$k^{-1} = 11^{-1} \mod 18 = 5 \quad (\text{porque } 11 \cdot 5 = 55 \equiv 1 \mod 18) \tag{1.134}$$
$$s = k^{-1}(e + r \cdot sk) = 5(5 + 7 \cdot 7) = 5(5 + 49) = 5 \cdot 54 \tag{1.135}$$
$$= 5 \cdot 54 \mod 18 = 270 \mod 18 = 0 \tag{1.136}$$

¡$s = 0$! Esto es inválido, debemos elegir otro k.

2b. Segundo intento con $k = 13$:

$$R = 13 \cdot G = (10, 2) \tag{1.137}$$
$$r = 10 \tag{1.138}$$
$$k^{-1} = 13^{-1} \mod 18 = 7 \quad (\text{porque } 13 \cdot 7 = 91 \equiv 1 \mod 18) \tag{1.139}$$
$$s = 7(5 + 10 \cdot 7) = 7(5 + 70) = 7 \cdot 75 = 7 \cdot 3 = 21 \equiv 3 \mod 18 \tag{1.140}$$

Firma: $(r, s) = (10, 3)$

3. Verificación:

$$w = s^{-1} = 3^{-1} \mod 18 = 6 \quad (\text{porque } 3 \cdot 6 = 18 \equiv 0... \tag{1.141}$$

$$\tag{1.142}$$

Hmm, $3 \cdot 6 = 18 \equiv 0 \mod 18$, así que 3^{-1} no existe... Esto es porque $\gcd(3, 18) = 3 \neq 1$.

Nota: Este ejemplo simplificado tiene problemas porque $n = 18$ no es primo. En la práctica, n siempre es primo, garantizando que todos los elementos no nulos tienen inverso.

Ejemplo 1.44 (ECDSA con parámetros realistas (valores truncados))**. Usando secp256k1:**

Clave privada:

```
sk = 0xc9afa9d845ba75166b5c215767b1d6934e50c3db36e89b127b8a622b120f6721
```

Clave pública: $pk = sk \cdot G$

```
pk.x = 0x60fed4ba255a9d31c961eb74c6356d68c049b8923b61fa6ce669622e60f29fb6
pk.y = 0x7903fe1008b8bc99a41ae9e95628bc64f2f1b20c2d7e9f5177a3c294d4462299
```

Mensaje: "Hello, ECDSA!"

```
e = SHA256("Hello, ECDSA!")
  = 0x7846e3be8abd2e089ed812e9d8c5e23a5ad76d5af9d65e0f9d89e8c76a5f8b12
```

Firma (usando RFC 6979 para k determinístico):

```
r = 0x2b826c8c5f1e8e7d3a9b2c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f
s = 0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b
```

Verificación:

```
w = s^(-1) mod n
u1 = e * w mod n
u2 = r * w mod n
R' = u1*G + u2*pk
assert R'.x == r  #  Firma válida
```

El Peligro Mortal: Reutilización del Nonce

Advertencia

LA REGLA MÁS IMPORTANTE DE ECDSA:

NUNCA, JAMÁS, BAJO NINGUNA CIRCUNSTANCIA, REUTILIZAR EL NONCE k

Si dos firmas usan el mismo k, la clave privada queda expuesta inmediatamente.

Matemáticas

Demostración del ataque:

Supongamos que firmamos dos mensajes m_1, m_2 con el mismo nonce k:

$$s_1 = k^{-1}(e_1 + r \cdot sk) \mod n \tag{1.143}$$

$$s_2 = k^{-1}(e_2 + r \cdot sk) \mod n \tag{1.144}$$

Nota que r es igual en ambas (porque $R = kG$ es igual).

Restando:

$$s_1 - s_2 = k^{-1}(e_1 - e_2) \mod n \tag{1.145}$$

Despejando k:

$$k = (e_1 - e_2)(s_1 - s_2)^{-1} \mod n \tag{1.146}$$

Una vez conocido k, recuperamos sk:

$$sk = (s_1 \cdot k - e_1) \cdot r^{-1} \mod n \tag{1.147}$$

O equivalentemente:

$$sk = \frac{s_1 e_2 - s_2 e_1}{r(s_2 - s_1)} \mod n$$

El atacante solo necesita ver dos firmas con el mismo r para robar todos los fondos.

Ejemplo 1.45 (El hack de Sony PlayStation 3 (2010))**.** Sony usaba ECDSA para firmar el software de PS3. Su error fatal:

```
// Código de Sony (simplificado)
k = 4;  // ¡CONSTANTE! El mismo para TODAS las firmas
```

El grupo "fail0verflow" observó que todas las firmas tenían el mismo valor r. Aplicando el ataque descrito, recuperaron la clave privada de Sony.

Resultado: Cualquiera podía firmar software como si fuera Sony, permitiendo piratería masiva y homebrew.

Lección: Sony perdió el control de su plataforma por un error de una línea de código.

Ejemplo 1.46 (Pérdidas en Bitcoin por nonces repetidos)**. Android SecureRandom Bug (2013):**

El generador de números aleatorios de Android en ciertas versiones producía outputs predecibles. Múltiples wallets Android generaron nonces idénticos para diferentes transacciones.

Resultado:

- Miles de claves privadas comprometidas
- 55 BTC robados en las primeras horas del descubrimiento

- Todas las wallets afectadas vaciadas eventualmente

Blockchain.info Bug (2014):
Un error en el código de generación de nonces causó colisiones.
Resultado:

- Cientos de BTC robados
- La empresa tuvo que compensar a usuarios

Solución: Nonces Determinísticos (RFC 6979)

Definición 1.91 (RFC 6979)**.** Para evitar la catastrófica reutilización de nonces, RFC 6979 define un método **determinístico** para generar k:

$$k = \text{HMAC-DRBG}(sk, H(m))$$

El nonce se deriva de:

- La clave privada sk
- El hash del mensaje $H(m)$

Propiedades:

1. Mismo mensaje + misma clave = misma firma (determinístico)
2. Diferente mensaje = diferente k (único)
3. Sin necesidad de generador aleatorio (portable)
4. Imposible para atacante predecir k sin conocer sk

Intuición

RFC 6979 elimina completamente el riesgo de nonce repetido:

- Si firmas el mismo mensaje dos veces, obtienes la misma firma (no revela nada nuevo)
- Si firmas mensajes diferentes, los nonces son diferentes (seguro)
- No dependes de un generador aleatorio que podría fallar

Hoy en día, **todas** las implementaciones serias usan RFC 6979 o similar.

Ejemplo 1.47 (Pseudocódigo RFC 6979)**.**
```
def generate_k_rfc6979(sk, message):
    # Paso 1: Hash del mensaje
    h1 = SHA256(message)

    # Paso 2: Inicializar HMAC-DRBG
    V = b'\x01' * 32
```

```
K = b'\x00' * 32

# Paso 3: Actualizar con clave privada y hash
K = HMAC_SHA256(K, V + b'\x00' + sk + h1)
V = HMAC_SHA256(K, V)
K = HMAC_SHA256(K, V + b'\x01' + sk + h1)
V = HMAC_SHA256(K, V)

# Paso 4: Generar k
while True:
    V = HMAC_SHA256(K, V)
    k = int.from_bytes(V, 'big')
    if 1 <= k < n:
        return k
    K = HMAC_SHA256(K, V + b'\x00')
    V = HMAC_SHA256(K, V)
```

Maleabilidad de Firmas

Advertencia

ECDSA tiene un problema de **maleabilidad**: dada una firma válida (r, s), cualquiera puede crear otra firma válida $(r, n - s)$ para el mismo mensaje.

¿Por qué?

$$s' = n - s = -s \mod n$$

La verificación usa s^{-1}:

$$(s')^{-1} = (-s)^{-1} = -s^{-1} \mod n$$

Los signos se cancelan y la verificación pasa.

Idea Clave

Impacto de la maleabilidad:

En Bitcoin, cada transacción se identifica por su hash (txid). Si alguien modifica la firma de una transacción no confirmada:

- La transacción sigue siendo válida
- Pero el txid cambia
- Esto puede romper cadenas de transacciones dependientes

Solución (BIP-62, BIP-146):

Bitcoin requiere que $s \leq n/2$ ("low-S"). Esto elimina la maleabilidad porque solo una de s o $n - s$ puede ser $\leq n/2$.

Ejemplo 1.48 (Normalización low-S)**.**
```
def normalize_signature(r, s, n):
    """Asegurar que s <= n/2 para evitar maleabilidad"""
    if s > n // 2:
        s = n - s
    return (r, s)

# Ejemplo
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
s = 0xDEADBEEFDEADBEEFDEADBEEFDEADBEEFDEADBEEFDEADBEEFDEADBEEFDEADBEEF

if s > n // 2:
    s_normalized = n - s
    print("Firma normalizada a low-S")
```

Formato de Firma

Definición 1.92 (Formatos de Firma ECDSA)**. DER (Distinguished Encoding Rules):**

```
30 <length>
   02 <r-length> <r>
   02 <s-length> <s>
```

Tamaño variable: 70-72 bytes típicamente.

Compacto (raw):

```
<r: 32 bytes> <s: 32 bytes>
```

Tamaño fijo: 64 bytes.

Ethereum (con recovery id):

```
<r: 32 bytes> <s: 32 bytes> <v: 1 byte>
```

Tamaño: 65 bytes.

Observación 1.27. El byte v en Ethereum (27 o 28, o 0/1 post-EIP-155) permite **recuperar la clave pública** a partir de la firma. Esto es útil porque:

- No necesitas incluir la clave pública en cada transacción
- El contrato puede derivar la dirección del firmante
- Ahorra espacio en la blockchain

Recuperación de Clave Pública (ecrecover)

Definición 1.93 (ECDSA Public Key Recovery)**.** Dada una firma (r, s) y el hash del mensaje e, se puede recuperar la clave pública pk (con hasta 4 candidatos, reducidos a 2 con el recovery id):

1. Calcular $R = (r, y_R)$ donde y_R se deriva de r (dos opciones)

2. Calcular $pk = r^{-1}(s \cdot R - e \cdot G)$

Ejemplo 1.49 (ecrecover en Solidity)**.**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ECDSAExample {
    function recoverSigner(
        bytes32 messageHash,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public pure returns (address) {
        // ecrecover es un precompilado de la EVM
        address signer = ecrecover(messageHash, v, r, s);
        require(signer != address(0), "Invalid signature");
        return signer;
    }

    function verifySignature(
        bytes32 messageHash,
        bytes memory signature,
        address expectedSigner
    ) public pure returns (bool) {
        bytes32 r;
        bytes32 s;
        uint8 v;

        assembly {
            r := mload(add(signature, 32))
            s := mload(add(signature, 64))
            v := byte(0, mload(add(signature, 96)))
        }

        return recoverSigner(messageHash, v, r, s) == expectedSigner;
    }
}
```

ECDSA vs Otros Esquemas de Firma

Propiedad	ECDSA	Schnorr	EdDSA	BLS
Tamaño de firma	64-72 B	64 B	64 B	48 B (G1)
Determinístico	Opcional	Sí	Sí	Sí
Maleabilidad	Sí	No	No	No
Agregación nativa	No	Sí (MuSig)	Limitada	Sí
Batch verification	No	Sí	Sí	Sí
Prueba de seguridad	ROM	ROM/Standard	Standard	ROM
Adopción	Alta	Media	Alta	Media

Idea Clave

¿Por qué Bitcoin/Ethereum eligieron ECDSA sobre Schnorr?
En 2008-2015, las firmas Schnorr estaban bajo patente (expirada en 2008, pero la incertidumbre legal persistía). ECDSA era el estándar "seguro" establecido.
Taproot (2021) añadió Schnorr a Bitcoin (BIP-340), ofreciendo:

- Firmas más pequeñas
- Agregación de firmas (multisig más eficiente)
- Sin maleabilidad

Ethereum planea adoptar BLS para Ethereum 2.0 (agregación de firmas de validadores).

Verificación en Circuitos ZK

Matemáticas

Verificar una firma ECDSA dentro de un circuito ZK es costoso porque involucra:

1. Multiplicación escalar en curva elíptica: $u_1 \cdot G$
2. Otra multiplicación escalar: $u_2 \cdot pk$
3. Suma de puntos: $u_1 G + u_2 \cdot pk$
4. Inversión modular: s^{-1}
5. Hash del mensaje (SHA-256 o Keccak)

Costo aproximado en R1CS:

Operación	**Constraints**
Multiplicación escalar (secp256k1)	∼250,000
Inversión modular	∼10,000
SHA-256	∼25,000
Total ECDSA verify	∼600,000

Aplicación en Banco ZK

En Banco ZK

ECDSA es la interfaz principal entre usuarios de Ethereum y Banco ZK:

1. Autenticación de usuarios

- Los usuarios firman transacciones de Banco ZK con sus claves Ethereum existentes
- No necesitan crear nuevas claves ni aprender nuevos procesos
- Compatibilidad total con MetaMask, Ledger, etc.

2. Verificación de firmas

Opción A: Fuera del circuito (lo que hacemos)

1. El contrato L1 verifica la firma ECDSA (barato: ∼3,000 gas)
2. Pasa la dirección del firmante como input público al circuito
3. El circuito verifica que los nullifiers corresponden a esa dirección

Opción B: Dentro del circuito (costoso)

- Verificar ECDSA en el circuito: ∼600,000 constraints
- Solo útil si necesitas privacidad sobre QUIÉN firmó

3. Flujo de depósito

```
1. Usuario firma: sign(sk, "deposit 1 ETH to commitment C")
2. Contrato verifica: ecrecover(hash, v, r, s) == msg.sender
3. Contrato ejecuta: deposit(commitment)
4. El circuito ZK opera sobre el commitment, no sobre la firma
```

4. Flujo de retiro

```
1. Usuario genera prueba ZK de que conoce el preimagen de un nullifier
2. Usuario firma el retiro: sign(sk, "withdraw to address X")
3. Contrato verifica la firma ECDSA
4. Contrato verifica la prueba ZK
5. Contrato envía fondos a X
```

5. Por qué no usamos solo EdDSA o Schnorr internamente

Aunque son más eficientes en circuitos ZK, mantener ECDSA tiene ventajas:

Aspecto	ECDSA	EdDSA
Wallets compatibles	Todas	Pocas
Hardware wallets	Sí	Limitado
UX de usuario	Conocida	Nueva
Gas en L1	3,000 (precompilado)	∼30,000 (sin precompilado)

6. Consideración de seguridad

Nuestros contratos SIEMPRE verifican:

```
require(s <= secp256k1_n / 2, "Malleable signature");
```

Esto previene ataques de maleabilidad que podrían alterar el txid.

7. Migración futura

Cuando Account Abstraction (EIP-4337) esté más maduro:

- Usuarios podrán usar cualquier esquema de firma
- Banco ZK soportará Schnorr, EdDSA, BLS
- La interfaz permanecerá igual para el usuario

Implementación de Referencia

Ejemplo 1.50 (ECDSA en Python (educacional))**.**

```
import hashlib
import secrets
from dataclasses import dataclass

# Parámetros secp256k1 (simplificados)
p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC2F
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

```
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8

@dataclass
class Point:
    x: int
    y: int

def mod_inverse(a, m):
    return pow(a, m - 2, m)

def point_add(P, Q):
    if P is None: return Q
    if Q is None: return P
    if P.x == Q.x and P.y != Q.y: return None

    if P.x == Q.x:  # Duplicación
        lam = (3 * P.x * P.x) * mod_inverse(2 * P.y, p) % p
    else:  # Suma
        lam = (Q.y - P.y) * mod_inverse(Q.x - P.x, p) % p

    x = (lam * lam - P.x - Q.x) % p
    y = (lam * (P.x - x) - P.y) % p
    return Point(x, y)

def scalar_mult(k, P):
    result = None
    addend = P
    while k:
        if k & 1:
            result = point_add(result, addend)
        addend = point_add(addend, addend)
        k >>= 1
    return result

G = Point(Gx, Gy)

def keygen():
    sk = secrets.randbelow(n - 1) + 1
    pk = scalar_mult(sk, G)
    return sk, pk
```

```
def sign(sk, message):
    e = int(hashlib.sha256(message).hexdigest(), 16) % n

    while True:
        k = secrets.randbelow(n - 1) + 1
        R = scalar_mult(k, G)
        r = R.x % n
        if r == 0: continue

        s = mod_inverse(k, n) * (e + r * sk) % n
        if s == 0: continue

        # Normalizar a low-S
        if s > n // 2:
            s = n - s

        return r, s

def verify(pk, message, r, s):
    if not (1 <= r < n and 1 <= s < n):
        return False

    e = int(hashlib.sha256(message).hexdigest(), 16) % n
    w = mod_inverse(s, n)
    u1 = (e * w) % n
    u2 = (r * w) % n

    R = point_add(scalar_mult(u1, G), scalar_mult(u2, pk))
    if R is None:
        return False

    return R.x % n == r

# Uso
sk, pk = keygen()
message = b"Hello, ECDSA!"
r, s = sign(sk, message)
assert verify(pk, message, r, s), "Verification failed!"
print("Signature verified successfully!")
```

Resumen de Buenas Prácticas

Idea Clave

Checklist de seguridad ECDSA:

- ☐ **NUNCA** reutilizar nonces. Usar RFC 6979.
- ☐ Verificar que $r, s \in [1, n-1]$ antes de procesar.
- ☐ Normalizar firmas a low-S ($s \leq n/2$).
- ☐ Usar librerías auditadas (libsecp256k1, etc.).
- ☐ Generar claves con CSPRNG de calidad.
- ☐ Validar claves públicas (verificar que están en la curva).
- ☐ Proteger claves privadas en memoria (zeroing después de uso).
- ☐ Considerar timing attacks en implementaciones.

1.4.2 Schnorr

Definición 1.94 (Firma Schnorr)**.** Esquema de firma con prueba de conocimiento del logaritmo discreto.

Algoritmo:

1. Elegir nonce k, calcular $R = kG$
2. Challenge: $e = H(R \| pk \| m)$
3. Response: $s = k + e \cdot sk$
4. Firma: (R, s) o (e, s)
5. Verificar: $sG = R + e \cdot pk$

Ventajas:

- Prueba de seguridad más simple
- Linealidad permite agregación: $\sum s_i G = \sum R_i + e \sum pk_i$
- MuSig: firmas multisig que parecen firmas normales

Contexto Histórico

Intuición

Las firmas Schnorr son elegantes, eficientes y matemáticamente más simples que ECDSA. Fueron inventadas en 1989 por Claus-Peter Schnorr, un criptógrafo alemán. Entonces, ¿por qué Bitcoin no las usó desde el principio?
La respuesta: **patentes**.

Observación 1.28. **La saga de las patentes Schnorr:**

Año	Evento
1989	Claus Schnorr publica el esquema
1990	Patente US 4,995,082 otorgada
1991	NIST estandariza DSA (diseñado para evitar la patente)
2008	Patente expira
2008	Satoshi elige ECDSA (incertidumbre legal residual)
2018	BIP-340 propuesto para Bitcoin
2021	Taproot activa Schnorr en Bitcoin

DSA (y por extensión ECDSA) fue diseñado específicamente por NIST para evitar la patente de Schnorr. Es por eso que ECDSA tiene una estructura más complicada.

Idea Clave

Ahora que las patentes expiraron, Schnorr está siendo adoptado ampliamente:

- **Bitcoin**: Taproot (BIP-340, BIP-341) desde noviembre 2021
- **EdDSA**: Variante de Schnorr sobre curvas de Edwards (ed25519)
- **Monero**: Usa firmas Schnorr para ring signatures
- **Polkadot**: Sr25519 (Schnorr sobre Ristretto)

El Protocolo Schnorr: Identificación Interactiva

Intuición

Antes de entender las firmas, es útil ver el **protocolo de identificación** de Schnorr. Es un protocolo interactivo donde un prover demuestra que conoce un secreto sin revelarlo.

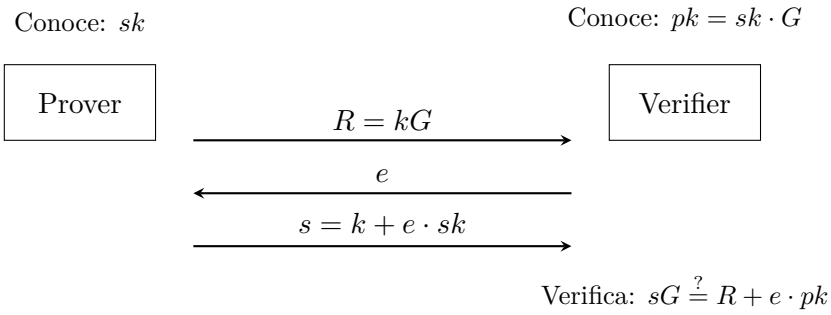
Definición 1.95 (Protocolo de Identificación Schnorr)**. Setup:** Prover tiene sk, Verifier conoce $pk = sk \cdot G$.

Protocolo:

1. **Commitment:** Prover elige $k \xleftarrow{\$} \mathbb{Z}_n$, envía $R = k \cdot G$
2. **Challenge:** Verifier elige $e \xleftarrow{\$} \mathbb{Z}_n$, envía e

3. **Response:** Prover calcula $s = k + e \cdot sk$, envía s

4. **Verificación:** Verifier acepta si $s \cdot G = R + e \cdot pk$



Matemáticas

¿Por qué funciona?

$$sG = (k + e \cdot sk) \cdot G \tag{1.148}$$
$$= k \cdot G + e \cdot sk \cdot G \tag{1.149}$$
$$= R + e \cdot pk \quad \checkmark \tag{1.150}$$

¿Por qué es seguro?

Un impostor que no conoce sk tendría que:

- Adivinar e antes de enviar R (probabilidad $1/n \approx 2^{-256}$), o
- Calcular sk a partir de pk (problema ECDLP)

De Identificación a Firma: Transformación Fiat-Shamir

Definición 1.96 (Transformación Fiat-Shamir)**.** Para convertir un protocolo interactivo en una firma no interactiva, reemplazamos el challenge aleatorio del verifier por el hash de la transcripción:

$$e = H(R \,\|\, pk \,\|\, m)$$

donde m es el mensaje a firmar.

Intuición

La idea es que el hash actúa como un "verifier virtual" que genera un challenge impredecible basado en el commitment R. El prover no puede "hacer trampa" eligiendo R después de conocer e, porque cambiar R cambiaría e.

Idea Clave

La firma Schnorr es esencialmente una **prueba de conocimiento cero no interactiva (NIZK)** de que el firmante conoce el logaritmo discreto de pk.
Esto conecta directamente con los SNARKs: las firmas Schnorr son el caso más simple

de “prueba de conocimiento”.

El Algoritmo de Firma Schnorr

Definición 1.97 (Firma Schnorr)**. Parámetros:** Curva E, generador G, orden n, hash H.

KeyGen:

1. $sk \xleftarrow{\$} [1, n-1]$
2. $pk = sk \cdot G$

Sign(sk, m)**:**

1. Elegir nonce: $k \xleftarrow{\$} [1, n-1]$ (o determinístico vía RFC 6979)
2. Calcular commitment: $R = k \cdot G$
3. Calcular challenge: $e = H(R \,\|\, pk \,\|\, m) \mod n$
4. Calcular response: $s = k + e \cdot sk \mod n$
5. Retornar $\sigma = (R, s)$ o $\sigma = (e, s)$

Verify(pk, m, σ)**:**

1. Parsear $\sigma = (R, s)$
2. Calcular $e = H(R \,\|\, pk \,\|\, m) \mod n$
3. Verificar: $s \cdot G \stackrel{?}{=} R + e \cdot pk$

Observación 1.29. Hay dos formatos comunes de firma Schnorr:

- (R, s): 64 bytes (32 para x_R, 32 para s). Usado en BIP-340.
- (e, s): 64 bytes (32 para e, 32 para s). Más compacto pero requiere recalcular R en verificación.

Bitcoin usa (R, s) porque simplifica la verificación por lotes (batch verification).

Comparación con ECDSA

Aspecto	ECDSA	Schnorr
Firma	$s = k^{-1}(e + r \cdot sk)$	$s = k + e \cdot sk$
Verificación	$R' = s^{-1}(eG + r \cdot pk)$	$sG = R + e \cdot pk$
Inversiones	2 (en firma y verificación)	0
Prueba de seguridad	Modelo del oráculo aleatorio	Más simple, más fuerte
Linealidad	No	Sí
Agregación nativa	No	Sí
Maleabilidad	Sí ((r, s) y $(r, n-s)$)	No
Batch verification	No eficiente	Sí, nativo

Idea Clave

La diferencia clave es la **linealidad**:

- ECDSA: $s = k^{-1}(e + r \cdot sk)$ — la inversión k^{-1} rompe la linealidad
- Schnorr: $s = k + e \cdot sk$ — completamente lineal

Esta linealidad permite sumar firmas, lo cual es imposible con ECDSA.

Ejemplo Numérico

Ejemplo 1.51 (Firma Schnorr paso a paso)**. Parámetros simplificados:**

- Campo: $\mathbb{F}_{97}$
- Curva: $y^2 = x^3 + 7$ (como secp256k1 pero pequeña)
- Generador: $G = (5, 32)$
- Orden: $n = 79$ (asumamos que es primo para el ejemplo)

1. Generación de claves:

$$sk = 23 \quad \text{(secreto)} \tag{1.151}$$

$$pk = 23 \cdot G = 23 \cdot (5, 32) = (41, 67) \quad \text{(calculado)} \tag{1.152}$$

2. Firma del mensaje m donde $H(R\|pk\|m) = 17$:

$$k = 47 \quad \text{(nonce aleatorio)} \tag{1.153}$$

$$R = 47 \cdot G = (62, 15) \tag{1.154}$$

$$e = H(R\|pk\|m) = 17 \quad \text{(simplificado)} \tag{1.155}$$

$$s = k + e \cdot sk = 47 + 17 \cdot 23 = 47 + 391 = 438 \equiv 43 \pmod{79} \tag{1.156}$$

Firma: $\sigma = (R, s) = ((62, 15), 43)$

3. Verificación:

Calcular ambos lados de $sG = R + e \cdot pk$:

Lado izquierdo:

$$sG = 43 \cdot G = 43 \cdot (5, 32) = (?, ?)$$

Lado derecho:

$$R + e \cdot pk = (62, 15) + 17 \cdot (41, 67) = (62, 15) + (?, ?) = (?, ?)$$

Si ambos son iguales, la firma es válida.

Seguridad: El Nonce Sigue Siendo Crítico

Advertencia

Al igual que ECDSA, **reutilizar el nonce en Schnorr expone la clave privada**. Si dos firmas (R, s_1) y (R, s_2) usan el mismo k (mismo R):

$$s_1 = k + e_1 \cdot sk \tag{1.157}$$

$$s_2 = k + e_2 \cdot sk \tag{1.158}$$

Restando:

$$s_1 - s_2 = (e_1 - e_2) \cdot sk$$

Despejando:

$$sk = \frac{s_1 - s_2}{e_1 - e_2} \mod n$$

¡Incluso más simple que el ataque a ECDSA!

Idea Clave

Solución: Usar nonces determinísticos.
BIP-340 especifica:

$$k = H_{\text{aux}}(sk \,\|\, \text{aux_rand}) \oplus H_{\text{nonce}}(sk \,\|\, pk \,\|\, m)$$

Esto combina:

- Determinismo (derivado de sk, pk, m)
- Aleatoriedad opcional (`aux_rand`) para protección contra side-channels

La Propiedad Mágica: Linealidad

Idea Clave

La ecuación de verificación Schnorr es **lineal**:

$$s \cdot G = R + e \cdot pk$$

Esto significa que podemos **sumar** firmas:

$$(s_1 + s_2) \cdot G = (R_1 + R_2) + e \cdot (pk_1 + pk_2)$$

¡Una firma agregada que verifica contra una clave pública agregada!

Ejemplo 1.52 (Agregación simple (ingenua, insegura))**.** Alice y Bob quieren firmar conjuntamente:

- Alice: sk_A, $pk_A = sk_A \cdot G$

- Bob: sk_B, $pk_B = sk_B \cdot G$

Clave pública conjunta:

$$pk_{AB} = pk_A + pk_B = (sk_A + sk_B) \cdot G$$

Firma conjunta:

1. Alice elige k_A, envía $R_A = k_A G$
2. Bob elige k_B, envía $R_B = k_B G$
3. $R = R_A + R_B$
4. $e = H(R \| pk_{AB} \| m)$
5. Alice calcula $s_A = k_A + e \cdot sk_A$
6. Bob calcula $s_B = k_B + e \cdot sk_B$
7. $s = s_A + s_B$

Firma final: (R, s)
Verificación:

$$sG = (s_A + s_B)G = s_A G + s_B G = (R_A + e \cdot pk_A) + (R_B + e \cdot pk_B) = R + e \cdot pk_{AB} \quad \checkmark$$

Advertencia

¡La agregación ingenua es INSEGURA!
El ataque "Rogue Key": Un atacante puede elegir su "clave pública" como:

$$pk_{\text{attack}} = pk'_{\text{attack}} - pk_{\text{victim}}$$

Entonces:

$$pk_{\text{joint}} = pk_{\text{victim}} + pk_{\text{attack}} = pk'_{\text{attack}}$$

El atacante controla completamente la clave conjunta y puede firmar solo.
Solución: Protocolos seguros como MuSig.

MuSig: Agregación Segura de Firmas

Definición 1.98 (MuSig)**.** MuSig (Multi-Signature Schnorr) es un protocolo seguro para crear firmas n-de-n que son indistinguibles de firmas individuales.

Idea clave: Cada clave pública se multiplica por un coeficiente derivado de todas las claves, previniendo ataques de clave rogue.

Ejemplo 1.53 (MuSig simplificado (2-de-2))**. Setup:**

- Alice: sk_A, pk_A

- Bob: sk_B, pk_B

Paso 1: Calcular coeficientes

$$L = H(pk_A \| pk_B) \quad \text{(lista ordenada de claves)} \tag{1.159}$$
$$a_A = H(L \| pk_A) \tag{1.160}$$
$$a_B = H(L \| pk_B) \tag{1.161}$$

Paso 2: Clave pública agregada

$$\tilde{pk} = a_A \cdot pk_A + a_B \cdot pk_B$$

Paso 3: Firma (3 rondas)

1. Cada parte elige nonce k_i, envía commitment $H(R_i)$
2. Revelar R_i, calcular $R = R_A + R_B$
3. Calcular $e = H(R \| \tilde{pk} \| m)$
4. Cada parte calcula $s_i = k_i + e \cdot a_i \cdot sk_i$
5. $s = s_A + s_B$

Firma final: (R, s) — ¡indistinguible de firma individual!

Observación 1.30. **Evolución de MuSig:**

- **MuSig** (2018): 3 rondas de comunicación
- **MuSig2** (2020): 2 rondas, más eficiente
- **MuSig-DN** (2020): Nonces determinísticos
- **FROST** (2020): Threshold signatures (t-de-n)

MuSig2: Reduciendo Rondas

Definición 1.99 (MuSig2)**.** MuSig2 reduce las rondas de comunicación de 3 a 2 usando múltiples nonces por firmante.

Idea: Cada firmante envía 2 nonces $(R_{i,1}, R_{i,2})$ en la primera ronda. El nonce efectivo se combina con un coeficiente derivado del mensaje.

Matemáticas

Por qué funcionan 2 rondas:
En MuSig original, la primera ronda es solo para commitments de nonces. Esto previene que un atacante vea los nonces de otros y ajuste el suyo.
MuSig2 usa una técnica diferente:

1. Enviar $(R_{i,1}, R_{i,2})$ directamente

2. El nonce efectivo es $R_i = R_{i,1} + b \cdot R_{i,2}$ donde $b = H(\text{todos los } R_{j,k}, m)$
3. El atacante no puede predecir b al elegir sus nonces

Batch Verification

Idea Clave

Otra ventaja de la linealidad: verificar n firmas Schnorr es más rápido que verificarlas individualmente.
Verificación individual: n verificaciones, cada una con 2 multiplicaciones escalares $= 2n$ mult. escalares.
Batch verification: Combinar en una sola ecuación usando pesos aleatorios.

Definición 1.100 (Batch Verification)**.** Para verificar firmas $(R_1, s_1), \ldots, (R_n, s_n)$ sobre mensajes $m_1, \ldots, m_n$ con claves $pk_1, \ldots, pk_n$:

1. Elegir pesos aleatorios $\alpha_1, \ldots, \alpha_n \xleftarrow{\$} \mathbb{Z}_n$
2. Calcular challenges: $e_i = H(R_i \| pk_i \| m_i)$
3. Verificar:
$$\left(\sum_{i=1}^{n} \alpha_i s_i\right) \cdot G = \sum_{i=1}^{n} \alpha_i R_i + \sum_{i=1}^{n} \alpha_i e_i \cdot pk_i$$

Si la ecuación se cumple, todas las firmas son válidas (con alta probabilidad).

Matemáticas

Análisis de eficiencia:

	Individual	**Batch**
Mult. escalares	$2n$	$n + 1$
Sumas de puntos	n	$2n$

Las multiplicaciones escalares son mucho más costosas que las sumas. Para n grande, batch verification es $\sim$2x más rápido.
Los pesos aleatorios α_i previenen que un atacante cree firmas inválidas que se "cancelen" en la suma.

BIP-340: Schnorr en Bitcoin

Definición 1.101 (BIP-340)**.** Bitcoin Improvement Proposal 340 especifica Schnorr para Bitcoin (activado con Taproot en noviembre 2021).

Características:

- Curva: secp256k1 (misma que ECDSA)
- Tamaño de firma: 64 bytes (vs. 71-72 para ECDSA DER)

- Clave pública: 32 bytes (solo coordenada x)
- Hash: SHA256 con tags para domain separation

Matemáticas

Innovaciones de BIP-340:

1. Solo coordenada x para claves y R:

En lugar de transmitir (x, y), BIP-340 usa solo x y asume que y es par. Si y es impar, se niega la clave.

Ahorro: 32 bytes por clave pública.

2. Tagged hashes:

$$H_{\text{tag}}(m) = \text{SHA256}(\text{SHA256}(\text{tag})\|\text{SHA256}(\text{tag})\|m)$$

Tags usados:

- "BIP0340/aux" para mezclar `aux_rand`
- "BIP0340/nonce" para generar nonce
- "BIP0340/challenge" para el challenge e

Esto previene colisiones entre diferentes usos del hash.

Ejemplo 1.54 (Firma BIP-340)**.**

```
# Inputs
secret_key: 32 bytes
message: arbitrary length
aux_rand: 32 bytes (optional randomness)

# Key generation
d = int(secret_key)
P = d * G
if P.y is odd:
    d = n - d  # Negate to make y even
pk = P.x  # Just x-coordinate, 32 bytes

# Nonce generation
t = xor(secret_key, tagged_hash("BIP0340/aux", aux_rand))
k' = int(tagged_hash("BIP0340/nonce", t || pk || message))
k' = k' mod n
R = k' * G
if R.y is odd:
    k = n - k'  # Negate to make y even
else:
    k = k'
```

```
# Challenge
e = int(tagged_hash("BIP0340/challenge", R.x || pk || message)) mod n

# Signature
s = (k + e * d) mod n
sig = R.x || s  # 64 bytes total
```

Taproot: Schnorr + MAST

Intuición

Taproot combina Schnorr con Merkelized Abstract Syntax Trees (MAST) para crear condiciones de gasto flexibles y privadas.
Idea: Una dirección Taproot puede gastarse de dos formas:

1. **Key path:** Firma Schnorr simple (parece una transacción normal)
2. **Script path:** Revelar un script oculto en un árbol Merkle

Si todos los participantes cooperan, usan el key path y nadie sabe que había otras condiciones posibles.

Definición 1.102 (Dirección Taproot)**.** Una dirección Taproot codifica:

$$P = Q + H(Q \| \text{script_root}) \cdot G$$

donde:

- Q es la clave pública interna (para key path)
- script_root es la raíz del árbol Merkle de scripts

Para gastar por key path, se firma con $q + H(Q \| \text{script_root})$ donde q es la clave privada de Q.

Schnorr vs EdDSA

Observación 1.31. EdDSA (Ed25519, Ed448) es técnicamente una variante de Schnorr, pero con diferencias importantes:

Aspecto	Schnorr (BIP-340)	EdDSA (Ed25519)
Curva	secp256k1	Curve25519
Forma de curva	Weierstrass	Edwards
Nonce	Determinístico + aux	Determinístico puro
Challenge	$H(R\|\|pk\|\|m)$	$H(R\|\|pk\|\|m)$
Firma	$(R.x, s)$	(R, s)
Clave pública	32 bytes (solo x)	32 bytes (comprimido)
Cofactor	1	8

Idea Clave

EdDSA es más popular fuera de Bitcoin porque:

1. Curve25519 es más rápida que secp256k1
2. Implementaciones de referencia bien auditadas (libsodium)
3. Sin historial de patentes
4. Usado en SSH, TLS 1.3, Signal, etc.

Bitcoin usa Schnorr sobre secp256k1 para compatibilidad con infraestructura existente.

Verificación en Circuitos ZK

Matemáticas

Verificar Schnorr en un circuito ZK es significativamente más eficiente que ECDSA:

Operación	**ECDSA**	**Schnorr**
Inversiones modulares	2	0
Multiplicaciones escalares	2	2
Estructura	No lineal	Lineal
Constraints totales	∼600,000	∼400,000

La ausencia de inversiones y la estructura lineal reducen los constraints en ∼30%.

Adapter Signatures (Firmas Adaptadas)

Definición 1.103 (Adapter Signature)**.** Una firma adaptada es una firma Schnorr "incompleta" que se completa al revelar un secreto.

Setup:

- Alice quiere crear una firma que Bob solo puede completar si revela un secreto t
- Alice conoce $T = t \cdot G$ (el "adaptor point")

Protocolo:

1. Alice crea firma adaptada: $\tilde{\sigma} = (R, \tilde{s})$ donde R incluye T
2. Bob puede verificar que $\tilde{\sigma}$ se convertirá en firma válida con t
3. Cuando Alice publica la firma completa $\sigma = (R, s)$
4. Bob calcula $t = s - \tilde{s}$

Intuición

Las adapter signatures permiten **atomic swaps sin HTLCs**:

1. Alice crea firma adaptada para enviar BTC a Bob
2. Bob crea firma adaptada para enviar ETH a Alice, usando el mismo adaptor T
3. Cuando Alice reclama su ETH (revelando t), Bob puede extraer t y reclamar su BTC

Todo ocurre sin timelocks ni scripts complejos.

Aplicación en Banco ZK

En Banco ZK

Las firmas Schnorr ofrecen varias ventajas para Banco ZK:

1. Eficiencia en circuitos

Verificar Schnorr en un SNARK cuesta ∼30% menos que ECDSA:

Esquema	Constraints	Tiempo de prueba
ECDSA	∼600,000	∼10s
Schnorr	∼400,000	∼7s
EdDSA (sobre curva embebida)	∼50,000	∼1s

2. Agregación de firmas para batch deposits

Múltiples usuarios pueden agregar sus firmas de depósito:

```
// 100 usuarios depositan
// Sin agregación: 100 verificaciones on-chain
// Con MuSig: 1 verificación on-chain

aggregate_pubkey = sum(coeff_i * pk_i)
aggregate_signature = (sum(R_i), sum(s_i))

// El contrato verifica UNA firma
verify(aggregate_pubkey, message, aggregate_signature)
```

Ahorro: $100 \times 3000 = 300,000$ gas $\rightarrow 3,000$ gas

3. Threshold signatures para governance

El protocolo puede requerir t-de-n firmas para cambios críticos:

- Actualización de verificadores: 3-de-5 multisig
- Emergency pause: 2-de-3 multisig
- Treasury movements: 4-de-7 multisig

Con FROST (Schnorr threshold), estas parecen firmas normales on-chain.

4. Por qué aún soportamos ECDSA

A pesar de las ventajas de Schnorr:

- No hay precompilado de Schnorr en EVM
- Wallets Ethereum usan ECDSA nativamente
- Cambiar requiere nuevo estándar de firma (EIP)

Estrategia híbrida:

Uso	Esquema
Firmas de usuario (L1)	ECDSA (compatibilidad)
Firmas internas (L2)	Schnorr/EdDSA
Multisig de governance	MuSig2
Dentro de circuitos ZK	EdDSA sobre Jubjub

5. Adapter signatures para puentes

Para puentes trustless entre Banco ZK y otras cadenas:

1. Usuario crea adapter signature para lock en Banco ZK
2. Relayer crea adapter signature para mint en destino
3. Al completar una, la otra se puede completar atómicamente

Esto elimina la necesidad de confiar en el relayer.

6. Implementación actual

```
// Usamos la librería schnorr-signatures de noble
import { schnorr } from '@noble/curves/secp256k1';

// Para firmas de governance (off-chain, luego verificadas)
const signature = schnorr.sign(messageHash, privateKey);
const isValid = schnorr.verify(signature, messageHash, publicKey);

// La verificación on-chain usa un contrato custom
// (no hay precompilado)
contract SchnorrVerifier {
    function verify(
        bytes32 px,
        bytes32 rx,
        bytes32 s,
        bytes32 e
    ) public pure returns (bool) {
```

```
        // Implementación en Solidity...
        // ~30,000 gas (vs 3,000 para ECDSA precompilado)
    }
}
```

Resumen: ¿Por Qué Schnorr es Mejor?

Idea Clave

Ventajas de Schnorr sobre ECDSA:

1. **Simplicidad matemática**: Sin inversiones, estructura lineal
2. **Prueba de seguridad**: Reducción más directa al problema DL
3. **Agregación**: Múltiples firmas → una firma
4. **Batch verification**: Verificar n firmas casi tan rápido como 1
5. **Sin maleabilidad**: No hay firmas equivalentes
6. **Firmas adaptadas**: Atomic swaps elegantes
7. **Privacidad**: Multisig parece firma normal
8. **Eficiencia en ZK**: Menos constraints

Desventajas:

1. Adopción más reciente (menos battle-tested)
2. Sin precompilado EVM
3. El nonce sigue siendo crítico (mismo riesgo que ECDSA)

Veredicto: Para nuevos sistemas, Schnorr es generalmente preferible. Para sistemas existentes con ECDSA, la migración tiene costos.

Implementación de Referencia

Ejemplo 1.55 (Schnorr en Python (educacional)). `import hashlib`
`import secrets`

```
# Parámetros (simplificados, usar librerías reales en producción)
p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
```

```
def tagged_hash(tag, data):
    tag_hash = hashlib.sha256(tag.encode()).digest()
    return hashlib.sha256(tag_hash + tag_hash + data).digest()

def schnorr_sign(sk, message):
    # Asegurar que pk tiene y par
    pk_point = scalar_mult(sk, G)
    if pk_point.y % 2 != 0:
        sk = n - sk
    pk = pk_point.x.to_bytes(32, 'big')

    # Generar nonce determinístico
    k = int.from_bytes(
        tagged_hash("BIP0340/nonce", sk.to_bytes(32, 'big') + pk + message),
        'big'
    ) % n

    R = scalar_mult(k, G)
    if R.y % 2 != 0:
        k = n - k

    # Challenge
    e = int.from_bytes(
        tagged_hash("BIP0340/challenge", R.x.to_bytes(32, 'big') + pk + message),
        'big'
    ) % n

    # Response
    s = (k + e * sk) % n

    return R.x.to_bytes(32, 'big') + s.to_bytes(32, 'big')

def schnorr_verify(pk_bytes, message, sig):
    rx = int.from_bytes(sig[:32], 'big')
    s = int.from_bytes(sig[32:], 'big')

    # Reconstruir R (asumiendo y par)
    R = lift_x(rx)
    if R is None:
        return False

    # Challenge
```

```
    e = int.from_bytes(
        tagged_hash("BIP0340/challenge", sig[:32] + pk_bytes + message),
        'big'
    ) % n

    # Verificar: sG = R + e*pk
    pk_point = lift_x(int.from_bytes(pk_bytes, 'big'))
    lhs = scalar_mult(s, G)
    rhs = point_add(R, scalar_mult(e, pk_point))

    return lhs.x == rhs.x and lhs.y == rhs.y
```

1.4.3 EdDSA (Ed25519)

Definición 1.104 (EdDSA)**.** Schnorr sobre curvas de Edwards con nonce determinístico.

Características:

- Curva Edwards: $-x^2 + y^2 = 1 + dx^2y^2$
- Nonce: $k = H(h_{sk}\|m)$ (determinístico, elimina riesgo)
- Ed25519: curva Curve25519, hash SHA-512
- Muy rápido, usado en: Signal, SSH, Solana

Contexto y Motivación

Intuición

EdDSA (Edwards-curve Digital Signature Algorithm) fue diseñado por Daniel J. Bernstein y sus colaboradores con un objetivo claro: crear un esquema de firmas que sea **imposible de implementar mal**.
Los problemas históricos con ECDSA (nonces repetidos, side-channels, maleabilidad) motivaron un rediseño desde cero con seguridad como prioridad absoluta.

Observación 1.32. **El equipo detrás de EdDSA:**

- Daniel J. Bernstein (djb) - Creador de Curve25519, qmail, NaCl
- Niels Duif
- Tanja Lange
- Peter Schwabe
- Bo-Yin Yang

El paper "High-speed high-security signatures" (2011) introdujo Ed25519, que rápidamente se convirtió en el estándar de facto para firmas modernas.

Año	Adopción
2011	Publicación de Ed25519
2013	OpenSSH añade soporte
2014	GnuPG añade soporte
2016	Signal Protocol adopta Ed25519
2017	TLS 1.3 incluye Ed25519
2018	Solana usa Ed25519 nativamente
2020	FIDO2/WebAuthn usa Ed25519

Curvas de Edwards

Definición 1.105 (Curva de Edwards)**.** Una curva de Edwards sobre un campo $\mathbb{F}$ tiene la forma:

$$E : x^2 + y^2 = 1 + d \cdot x^2 y^2$$

donde $d \in \mathbb{F}$ es un no-cuadrado (para que la curva sea no singular).
La forma **twisted Edwards** generaliza esto:

$$E : ax^2 + y^2 = 1 + d \cdot x^2 y^2$$

Idea Clave

¿Por qué curvas de Edwards sobre Weierstrass?
Las curvas de Edwards tienen propiedades algebraicas superiores:

1. **Ley de grupo unificada**: Una sola fórmula para suma y duplicación
2. **Fórmulas completas**: Funcionan para TODOS los puntos, sin casos especiales
3. **Más rápidas**: Menos operaciones de campo por operación de grupo
4. **Resistentes a side-channels**: Ejecución en tiempo constante natural

Matemáticas

Ley de grupo en curvas de Edwards:
Para $P_1 = (x_1, y_1)$ y $P_2 = (x_2, y_2)$:

$$P_1 + P_2 = \left(\frac{x_1 y_2 + x_2 y_1}{1 + d x_1 x_2 y_1 y_2}, \frac{y_1 y_2 - a x_1 x_2}{1 - d x_1 x_2 y_1 y_2} \right)$$

Esta fórmula:

- Funciona para suma de puntos distintos
- Funciona para duplicación ($P_1 = P_2$)
- Funciona cuando uno de los puntos es el neutro $(0, 1)$
- Nunca tiene divisiones por cero (si d es no-cuadrado)

Compárese con Weierstrass, que tiene 3+ casos especiales.

Curve25519 y Ed25519

Definición 1.106 (Curve25519)**.** Curve25519 es una curva de Montgomery:

$$y^2 = x^3 + 486662x^2 + x$$

sobre $\mathbb{F}_p$ donde $p = 2^{255} - 19$.
Su forma twisted Edwards equivalente (Edwards25519) es:

$$-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2$$

El número 25519 viene del primo $p = 2^{255} - 19$.

Matemáticas

Parámetros de Ed25519:

```
# Campo
p = 2^255 - 19
  = 0x7fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffed

# Curva (twisted Edwards)
a = -1
d = -121665/121666 mod p
  = 0x52036cee2b6ffe738cc740797779e89800700a4d4141d8ab75eb4dca135978a3

# Orden del subgrupo
n = 2^252 + 27742317777372353535851937790883648493
  = 0x1000000000000000000000000000000014def9dea2f79cd65812631a5cf5d3ed

# Cofactor
h = 8

# Punto base B
B_x = 0x216936d3cd6e53fec0a4e231fdd6dc5c692cc7609525a7b2c9562d608f25d51a
B_y = 0x6666666666666666666666666666666666666666666666666666666666666658
```

Observación 1.33. **¿Por qué** $p = 2^{255} - 19$**?**

1. Es primo
2. Permite aritmética muy rápida (reducción módulo $2^{255} - 19$ es eficiente)
3. 255 bits caben perfectamente en 32 bytes con 1 bit de sobra

4. 19 es el menor entero positivo que hace $2^{255} - 19$ primo

El Algoritmo EdDSA

Definición 1.107 (EdDSA - Algoritmo Completo)**. Parámetros:** Curva Edwards con punto base B de orden n, hash H.

KeyGen:

1. Generar semilla aleatoria: $seed \xleftarrow{\$} \{0,1\}^{256}$
2. Expandir con hash: $(h_L, h_R) = H(seed)$ (64 bytes)
3. Procesar h_L: ajustar bits para clamping
4. Clave privada efectiva: $s = \text{clamp}(h_L)$
5. Clave pública: $A = s \cdot B$

Sign$(seed, A, m)$**:**

1. Calcular nonce determinístico: $r = H(h_R \| m) \mod n$
2. Calcular commitment: $R = r \cdot B$
3. Calcular challenge: $k = H(R \| A \| m) \mod n$
4. Calcular response: $S = (r + k \cdot s) \mod n$
5. Retornar firma: $\sigma = (R, S)$

Verify(A, m, σ)**:**

1. Parsear $\sigma = (R, S)$
2. Calcular $k = H(R \| A \| m) \mod n$
3. Verificar: $8 \cdot S \cdot B \stackrel{?}{=} 8 \cdot R + 8 \cdot k \cdot A$

Idea Clave

El nonce determinístico es la innovación clave:

$$r = H(h_R \| m)$$

Donde h_R es la mitad derecha del hash de la semilla (nunca revelada).
Propiedades:

- **Determinístico**: Mismo mensaje = mismo nonce = misma firma
- **Impredecible**: Sin conocer h_R, no se puede calcular r
- **Sin riesgo de repetición**: Diferentes mensajes → diferentes nonces
- **Sin PRNG**: No depende de la calidad del generador aleatorio

Advertencia

El cofactor 8:
Ed25519 tiene cofactor $h = 8$, lo que significa que hay subgrupos pequeños. La verificación multiplica por 8 para proyectar al subgrupo principal:

$$8 \cdot S \cdot B = 8 \cdot R + 8 \cdot k \cdot A$$

Sin esto, un atacante podría crear firmas "válidas" usando puntos de orden pequeño.
Variante: **Ed25519ctx** y **Ed25519ph** añaden domain separation.

Clamping: ¿Qué y Por Qué?

Definición 1.108 (Clamping de clave privada)**.** El "clamping" ajusta los bits de h_L antes de usarlo como escalar:

```
def clamp(h):
    h[0] &= 248    # Limpiar los 3 bits inferiores
    h[31] &= 127   # Limpiar el bit más alto
    h[31] |= 64    # Establecer el segundo bit más alto
    return h
```

Resultado: El escalar s siempre tiene:

- Bits 0, 1, 2 = 0 (divisible por 8)
- Bit 254 = 1
- Bit 255 = 0

Matemáticas

¿Por qué clamping?
1. Divisible por cofactor (8): Garantiza que $s \cdot B$ está en el subgrupo principal, evitando ataques de subgrupo pequeño.
2. Bit 254 = 1: Garantiza que la multiplicación escalar siempre toma el mismo número de pasos (tiempo constante).
3. Bit 255 = 0: El escalar cabe en 255 bits, evitando overflow.

Comparación de Rendimiento

Operación	Ed25519	ECDSA P-256	RSA-2048
KeyGen	30 µs	80 µs	300 ms
Sign	50 µs	100 µs	10 ms
Verify	120 µs	200 µs	0.3 ms
Firma tamaño	64 B	64 B	256 B
Clave pública	32 B	64 B	256 B
Clave privada	32 B	32 B	256+ B

Intuición

Ed25519 es ∼2x más rápido que ECDSA y órdenes de magnitud más rápido que RSA. Además, las claves son más pequeñas, lo que reduce ancho de banda y almacenamiento.

Ed25519 en la Práctica

Ejemplo 1.56 (Ed25519 en diferentes lenguajes)**. Python (usando PyNaCl):**

```
from nacl.signing import SigningKey, VerifyKey

# KeyGen
sk = SigningKey.generate()
pk = sk.verify_key

# Sign
message = b"Hello, Ed25519!"
signed = sk.sign(message)
signature = signed.signature  # 64 bytes

# Verify
try:
    pk.verify(signed)
    print("Valid signature!")
except:
    print("Invalid signature!")
```

JavaScript (usando @noble/ed25519):

```
import * as ed from '@noble/ed25519';

// KeyGen
const privateKey = ed.utils.randomPrivateKey();
const publicKey = await ed.getPublicKey(privateKey);

// Sign
const message = new TextEncoder().encode("Hello, Ed25519!");
const signature = await ed.sign(message, privateKey);

// Verify
const isValid = await ed.verify(signature, message, publicKey);
```

Rust (usando ed25519-dalek):

```
use ed25519_dalek::{Keypair, Signature, Signer, Verifier};
use rand::rngs::OsRng;
```

```
// KeyGen
let mut csprng = OsRng{};
let keypair = Keypair::generate(&mut csprng);

// Sign
let message = b"Hello, Ed25519!";
let signature = keypair.sign(message);

// Verify
assert!(keypair.public.verify(message, &signature).is_ok());
```

Variantes de EdDSA

Variante	Curva	Hash	Uso
Ed25519	Curve25519	SHA-512	General, más común
Ed25519ctx	Curve25519	SHA-512 + context	Cuando se necesita domain separation
Ed25519ph	Curve25519	SHA-512(SHA-512(m))	Para mensajes muy grandes
Ed448	Curve448	SHAKE256	Mayor seguridad (224 bits)

Observación 1.34. **Ed448 (Goldilocks):**

Para aplicaciones que necesitan más de 128 bits de seguridad, Ed448 ofrece ∼224 bits usando la curva Curve448 (también diseñada por djb):

$$p = 2^{448} - 2^{224} - 1$$

Es más lento que Ed25519 pero sigue siendo muy rápido comparado con alternativas.

Ristretto: Resolviendo el Problema del Cofactor

Definición 1.109 (Ristretto255)**.** Ristretto es una técnica para construir un grupo primo a partir de una curva con cofactor.

Para Curve25519 (cofactor 8), Ristretto255 proporciona:

- Un grupo de orden primo n (sin subgrupos pequeños)
- Codificación canónica de puntos
- Compatibilidad con implementaciones existentes de Curve25519

Intuición

El problema del cofactor causa bugs sutiles:

- Diferentes codificaciones del mismo punto
- Ataques de subgrupo pequeño si no se verifica membresía

- Complejidad en protocolos multi-party

Ristretto "quotients out" los puntos de torsión, dando una abstracción limpia de grupo primo.

Observación 1.35. **Sr25519 (Schnorrkel):**

Polkadot usa Sr25519, que es Schnorr sobre Ristretto255. Combina:

- La velocidad de Curve25519
- La limpieza de un grupo primo (Ristretto)
- Las ventajas de Schnorr (agregación, etc.)

Ed25519 en Circuitos ZK

Matemáticas

Ed25519 es particularmente eficiente para verificación en circuitos ZK, especialmente cuando se usa una curva "embebida".

Curva embebida: Una curva cuyo orden es el campo escalar del sistema SNARK. Para SNARKs sobre BLS12-381, se usa **Jubjub**:

- Curva twisted Edwards sobre $\mathbb{F}_q$ (campo escalar de BLS12-381)
- Orden del subgrupo primo $\approx 2^{251}$
- Operaciones de Jubjub son "nativas" en el circuito

Costo comparativo:

Verificación	Constraints
ECDSA (secp256k1) en BN254	∼600,000
EdDSA (Ed25519) en BN254	∼400,000
EdDSA (Jubjub) en BLS12-381	∼15,000

¡40x más eficiente con curva embebida!

Aplicación en Banco ZK

En Banco ZK

EdDSA es el esquema de firma preferido para operaciones internas de Banco ZK:

1. Firmas dentro de circuitos

Para verificar firmas de usuario dentro del SNARK, usamos EdDSA sobre una curva embebida:

SNARK base	Curva de firma	Constraints
BN254 (Groth16)	Baby Jubjub	∼20,000
BLS12-381 (Halo2)	Jubjub	∼15,000

Comparado con verificar ECDSA (∼600,000), esto es 30-40x más eficiente.

2. Por qué no usamos Ed25519 para todo

Aspecto	Ed25519	ECDSA secp256k1
Wallets compatibles	Pocas	Todas (Ethereum)
Precompilado EVM	No	Sí (ecrecover)
Verificación on-chain	∼50,000 gas	∼3,000 gas
En circuito ZK	Eficiente	Muy costoso

3. Arquitectura híbrida

```
Usuario (L1) ECDSA> Contrato

                    [verificación L1]

                          v
Circuito ZK <EdDSA/Jubjub> Estado interno
```

El usuario firma con ECDSA (compatible con MetaMask), pero las operaciones internas del protocolo usan EdDSA sobre curva embebida.

4. Migración futura

Con Account Abstraction (EIP-4337), los usuarios podrán usar wallets Ed25519:

- Verificación delegada a contratos custom
- Sin dependencia de precompilados
- Mejor UX móvil (Ed25519 más rápido en ARM)

1.4.4 BLS

Definición 1.110 (Firma BLS)**.** Firma basada en pairings con agregación nativa.

Sea $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ un pairing.

1. **Keygen:** $sk \in \mathbb{Z}_q$, $pk = sk \cdot g_2 \in \mathbb{G}_2$
2. **Sign:** $\sigma = sk \cdot H(m) \in \mathbb{G}_1$
3. **Verify:** $e(\sigma, g_2) = e(H(m), pk)$

Idea Clave

Agregación: Múltiples firmas se combinan en una:

$$\sigma_{\text{agg}} = \sum_i \sigma_i, \quad \text{verifica con} \quad \sum_i pk_i$$

Ethereum 2.0 agrega miles de firmas de validadores en una sola.

Contexto Histórico

Intuición

Las firmas BLS (Boneh-Lynn-Shacham) fueron propuestas en 2001 y representan un paradigma diferente: usan **pairings** en curvas elípticas, una operación matemática más poderosa que permite cosas imposibles con firmas tradicionales.

Observación 1.36. **Los autores:**

- **Dan Boneh**: Profesor en Stanford, pionero en criptografía de pairings
- **Ben Lynn**: PhD de Stanford, ahora en Google
- **Hovav Shacham**: Profesor en UT Austin

El paper original "Short Signatures from the Weil Pairing" (2001) mostró que los pairings permitían firmas más cortas que cualquier esquema conocido hasta entonces.

Año	Desarrollo
2001	Paper original de BLS
2003	Agregación de firmas propuesta
2017	Ethereum 2.0 elige BLS para consenso
2018	Zcash Sapling usa BLS12-381 (para SNARKs)
2020	Beacon Chain lanzada con firmas BLS
2022	The Merge: BLS en producción masiva

Pairings: El Ingrediente Mágico

Definición 1.111 (Pairing Bilineal (Repaso))**.** Un pairing es una función:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$$

que satisface **bilinealidad**:

$$e(aP, bQ) = e(P, Q)^{ab}$$

Propiedades adicionales:

- **No degenerado**: $e(G_1, G_2) \neq 1$ para generadores

- **Eficientemente computable**: $O(\log n)$ operaciones de campo

Idea Clave

Los pairings permiten una forma de "multiplicación" en curvas elípticas que no existe de otra forma:

- Sin pairing: podemos sumar puntos y multiplicar por escalares
- Con pairing: podemos "combinar" información de dos puntos de manera bilineal

Esto permite construcciones criptográficas imposibles sin pairings, como:

- Firmas cortas (BLS)
- Identity-Based Encryption (IBE)
- Verificación de SNARKs en tiempo constante

El Algoritmo BLS en Detalle

Definición 1.112 (BLS Signature Scheme)**. Setup:** Curva pairing-friendly con grupos $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ de orden q, generadores $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$, y hash-to-curve $H : \{0,1\}^* \to \mathbb{G}_1$.

KeyGen:

1. $sk \xleftarrow{\$} \mathbb{Z}_q$
2. $pk = sk \cdot g_2 \in \mathbb{G}_2$

Sign(sk, m)**:**

1. Hashear mensaje a punto de curva: $h = H(m) \in \mathbb{G}_1$
2. Firma: $\sigma = sk \cdot h \in \mathbb{G}_1$

Verify(pk, m, σ)**:**

1. Calcular $h = H(m)$
2. Verificar: $e(\sigma, g_2) \stackrel{?}{=} e(h, pk)$

Matemáticas

¿Por qué funciona?

$$
\begin{aligned}
e(\sigma, g_2) &= e(sk \cdot H(m), g_2) && (1.162)\\
&= e(H(m), g_2)^{sk} \quad \text{(bilinealidad)} && (1.163)\\
&= e(H(m), sk \cdot g_2) && (1.164)\\
&= e(H(m), pk) \quad \checkmark && (1.165)
\end{aligned}
$$

Intuición

La firma BLS es increíblemente simple:

- KeyGen: multiplicar generador por escalar
- Sign: multiplicar hash por escalar
- Verify: comparar dos pairings

¡No hay nonces! Esto elimina completamente el riesgo de nonce repetido que afecta a ECDSA y Schnorr.

Hash-to-Curve

Advertencia

BLS requiere hashear mensajes a puntos de curva: $H : \{0,1\}^* \to \mathbb{G}_1$.
Esto es más complejo que hashear a enteros:

1. Hashear a un valor de campo: $h' = H'(m) \in \mathbb{F}_p$
2. Mapear a la curva usando un algoritmo como Simplified SWU
3. Multiplicar por cofactor si es necesario

Un hash-to-curve incorrecto puede causar vulnerabilidades graves.

Observación 1.37. El estándar **RFC 9380** ("Hashing to Elliptic Curves") especifica métodos seguros para hash-to-curve. Para BLS12-381:

- $\mathbb{G}_1$: hash_to_field + map_to_curve (Simplified SWU) + clear_cofactor
- $\mathbb{G}_2$: Similar pero sobre la extensión cuadrática

Agregación de Firmas: El Superpoder de BLS

Idea Clave

La característica más importante de BLS es la **agregación nativa**:
Múltiples firmas sobre el mismo mensaje:

$$\sigma_{\text{agg}} = \sigma_1 + \sigma_2 + \cdots + \sigma_n = \sum_i sk_i \cdot H(m) \tag{1.166}$$

$$pk_{\text{agg}} = pk_1 + pk_2 + \cdots + pk_n = \sum_i pk_i \tag{1.167}$$

Verificación:

$$e(\sigma_{\text{agg}}, g_2) = e(H(m), pk_{\text{agg}})$$

¡Una sola verificación de pairing para n firmas!

Matemáticas

¿Por qué funciona?

$$e(\sigma_{\text{agg}}, g_2) = e\left(\sum_i sk_i \cdot H(m), g_2\right) \tag{1.168}$$

$$= e\left(H(m), g_2\right)^{\sum_i sk_i} \tag{1.169}$$

$$= e\left(H(m), \sum_i sk_i \cdot g_2\right) \tag{1.170}$$

$$= e(H(m), pk_{\text{agg}}) \quad \checkmark \tag{1.171}$$

Ejemplo 1.57 (Agregación en Ethereum 2.0)**. Contexto:** La Beacon Chain tiene ∼900,000 validadores. Cada slot (12 segundos), miles de validadores deben atestiguar (firmar) un bloque.

Sin agregación:

- ∼100,000 firmas por slot
- 48 bytes × 100,000 = 4.8 MB por slot
- 100,000 verificaciones de pairing por slot

Con agregación BLS:

- Validadores se agrupan en comités de ∼128
- Cada comité produce UNA firma agregada
- ∼800 firmas por slot
- 48 bytes × 800 = 38.4 KB por slot
- 800 verificaciones por slot

Mejora: 125x menos datos, 125x menos verificaciones.

Agregación de Mensajes Diferentes

Definición 1.113 (Agregación General)**.** Cuando cada firmante firma un mensaje diferente:

$$\sigma_{\text{agg}} = \sum_i \sigma_i = \sum_i sk_i \cdot H(m_i)$$

Verificación (n pairings):

$$e(\sigma_{\text{agg}}, g_2) \stackrel{?}{=} \prod_i e(H(m_i), pk_i)$$

Requiere n pairings pero solo transmite UNA firma.

Matemáticas

Verificación batch:
Con técnicas de multi-pairing, verificar n pairings es más rápido que n verificaciones individuales:

$$\text{Costo de } n \text{ pairings} \approx 1 + 0.1 \times n \text{ pairings individuales}$$

Esto hace que la agregación de mensajes diferentes siga siendo muy eficiente.

El Ataque Rogue Key y Proof of Possession

Advertencia

Ataque Rogue Key en BLS:
Similar a Schnorr, un atacante puede elegir su clave pública maliciosamente:

$$pk_{\text{attack}} = sk_{\text{attack}} \cdot g_2 - pk_{\text{victim}}$$

Entonces:

$$pk_{\text{agg}} = pk_{\text{victim}} + pk_{\text{attack}} = sk_{\text{attack}} \cdot g_2$$

¡El atacante puede firmar por la "víctima" sin su consentimiento!

Definición 1.114 (Proof of Possession (PoP))**.** Para prevenir ataques rogue key, cada participante debe demostrar que conoce la clave privada correspondiente a su clave pública:

$$\text{PoP} = sk \cdot H_{\text{pop}}(pk)$$

Esto se verifica una vez al registrar la clave:

$$e(\text{PoP}, g_2) = e(H_{\text{pop}}(pk), pk)$$

Después de esto, la clave puede usarse para agregación segura.

Observación 1.38. Ethereum 2.0 requiere que cada validador registre un PoP al hacer el depósito de 32 ETH. Sin el PoP, el depósito es rechazado.

BLS vs Otros Esquemas

Propiedad	ECDSA	Schnorr	EdDSA	BLS
Tamaño de firma	64-72 B	64 B	64 B	48 B
Sin nonces	No	No	Sí	Sí
Agregación nativa	No	Requiere MuSig	No	Sí
Determinístico	Opcional	Opcional	Sí	Sí
Requiere pairing	No	No	No	Sí
Velocidad sign	Rápido	Rápido	Muy rápido	Más lento
Velocidad verify	Rápido	Rápido	Rápido	Lento
Verify agregada	N/A	Batch	Batch	1 pairing

Idea Clave

Cuándo usar BLS:

- Muchos firmantes necesitan agregarse (consenso, multisig masivo)
- El tamaño de firma es crítico
- La velocidad de verificación individual no es el cuello de botella

Cuándo NO usar BLS:

- Firmas individuales sin agregación
- Dispositivos con recursos limitados (pairings son costosos)
- Cuando no hay curvas pairing-friendly disponibles

Curvas para BLS

Curva	Seguridad	Firma	Clave pública
BN254	~100 bits	32 B ($\mathbb{G}_1$)	64 B ($\mathbb{G}_2$)
BLS12-381	~128 bits	48 B ($\mathbb{G}_1$)	96 B ($\mathbb{G}_2$)
BLS12-377	~128 bits	48 B	96 B

Observación 1.39. **¿Por qué firma en $\mathbb{G}_1$ y clave en $\mathbb{G}_2$?**

En BLS12-381:

- $\mathbb{G}_1$: puntos de 48 bytes (comprimidos)
- $\mathbb{G}_2$: puntos de 96 bytes (comprimidos)

Como transmitimos muchas más firmas que claves públicas, ponemos firmas en el grupo más pequeño ($\mathbb{G}_1$).

Ethereum 2.0 usa esta configuración. Alternativamente, se pueden intercambiar si hay más verificaciones que firmas.

Threshold BLS

Definición 1.115 (Threshold BLS)**.** Firma t-de-n donde cualquier t participantes pueden crear una firma válida, pero menos de t no pueden.

Setup: Shamir Secret Sharing distribuye sk en n shares $sk_1, \ldots, sk_n$.

Firma (t participantes):

1. Cada participante i calcula $\sigma_i = sk_i \cdot H(m)$
2. Se combinan t firmas parciales usando interpolación de Lagrange:

$$\sigma = \sum_{i \in S} \lambda_i \sigma_i$$

donde S es el conjunto de t participantes y λ_i son coeficientes de Lagrange.

La firma resultante es indistinguible de una firma BLS normal.

Intuición

Threshold BLS es más simple que threshold Schnorr porque:

1. No hay nonces que coordinar
2. Cada firma parcial es determinística
3. La combinación es simplemente suma ponderada

Esto lo hace ideal para protocolos distribuidos.

BLS en Ethereum 2.0

Matemáticas

Especificación de Ethereum 2.0:
Curva: BLS12-381 **Hash:** SHA-256 + hash_to_G1 **Domain separation:** Diferentes dominios para diferentes tipos de mensajes
Tipos de firmas:

- **Attestations**: Validadores atestiguan bloques (agregadas por comité)
- **Sync committee**: 512 validadores rotativos firman headers
- **Block proposals**: Firma individual del proposer
- **Voluntary exits**: Validador sale del protocolo
- **Slashings**: Evidencia de comportamiento malicioso

Flujo de attestation:

```
1. Validador recibe bloque B en slot S
2. Calcula: msg = (slot, block_root, source, target)
```

```
3. Firma:  = sk · H(domain || msg)
4. Broadcast: (attestation_data, , validator_index)
5. Aggregador colecta , , ... de su comité
6. Produce: _agg =  , bitfield de participantes
7. Beacon chain verifica: e(_agg, g) = e(H(msg),  pk)
```

Rendimiento de BLS

Operación (BLS12-381)	Tiempo	Comparación
Hash to $\mathbb{G}_1$	∼0.3 ms	—
Multiplicación escalar $\mathbb{G}_1$	∼0.5 ms	5x más lento que Ed25519
Multiplicación escalar $\mathbb{G}_2$	∼1.5 ms	—
Pairing	∼2 ms	—
Verify 1 firma	∼2.5 ms	20x más lento que Ed25519
Verify 100 firmas (agregadas)	∼3 ms	80x más rápido que individual

Idea Clave

BLS es lento para firmas individuales pero brilla con agregación:

- 1 firma: ∼2.5 ms (20x más lento que Ed25519)
- 100 firmas agregadas: ∼3 ms (prácticamente igual que 1)
- 100 firmas Ed25519: ∼12 ms (4x más lento)

El crossover está alrededor de 4-5 firmas: con más de eso, BLS agregado gana.

Aplicación en Banco ZK

En Banco ZK

BLS juega varios roles importantes en Banco ZK:

1. Firmas de validadores L2

Nuestro sequencer tiene un comité de validadores que firman batches:

```
// Comité de 21 validadores
// Requiere 14/21 (2/3+1) para finalizar un batch

batch_hash = keccak256(transactions, state_root, batch_number)
signatures = [validator.sign(batch_hash) for validator in committee]
agg_sig = aggregate(signatures)

// On-chain: verificar UNA firma en lugar de 14
verify(agg_sig, batch_hash, agg_pubkey)
```

Ahorro:

- Sin BLS: 14 × ecrecover = 42,000 gas
- Con BLS: 1 verificación = ∼100,000 gas (más caro individualmente)

¿Por qué BLS es más caro? No hay precompilado BLS en EVM. Pero el ahorro viene en:

- Calldata: 14 × 65 bytes = 910 bytes → 48 bytes
- Con EIP-2537: ∼50,000 gas (mejor que ECDSA)

2. Agregación de depósitos
Múltiples usuarios pueden agregar sus firmas de depósito off-chain:

```
// 100 usuarios quieren depositar
// Cada uno firma: "deposit(amount, commitment)"

off_chain_aggregate:
    agg_sig = sum(user_signatures)
    agg_pk = sum(user_pubkeys)

on_chain:
    verify_aggregate(agg_sig, deposit_data, agg_pk)
    process_deposits(deposit_list)
```

3. Sync committee para bridges
Para verificar estado de Ethereum en otras cadenas (o viceversa):

- 512 validadores firman headers (sync committee)
- La firma agregada permite light client verification
- Banco ZK puede verificar estado de Ethereum L1 eficientemente

4. Por qué no usamos BLS para todo

Uso	Esquema elegido	Razón
Usuarios (L1)	ECDSA	Wallets existentes
Validadores L2	BLS	Agregación
Dentro de SNARK	EdDSA	Eficiencia en circuito
Multisig governance	BLS threshold	Sin coordinación de nonces

5. Futuro: BLS para usuarios
Con Account Abstraction, los usuarios podrán optar por BLS:

- Wallet contract verifica firma BLS
- Múltiples operaciones del mismo usuario se agregan
- Ahorro significativo en gas para usuarios activos

Librerías BLS

Lenguaje	Librería	Notas
C	blst (Supranational)	La más rápida, usada en Ethereum
Rust	blst-rs	Bindings de blst
Rust	arkworks bls12-381	Más flexible
Go	gnark-crypto/bls12-381	Producción
JavaScript	@noble/curves	Pura JS, auditada
Python	py_ecc	Educacional

Ejemplo 1.58 (BLS con @noble/curves)**.** `import { bls12_381 as bls } from '@noble/curves/bls12-38`

```
// KeyGen
const privateKey = bls.utils.randomPrivateKey();
const publicKey = bls.getPublicKey(privateKey);

// Sign
const message = new TextEncoder().encode("Hello, BLS!");
const signature = bls.sign(message, privateKey);

// Verify
const isValid = bls.verify(signature, message, publicKey);

// Aggregation (same message)
const sk1 = bls.utils.randomPrivateKey();
const sk2 = bls.utils.randomPrivateKey();
const pk1 = bls.getPublicKey(sk1);
const pk2 = bls.getPublicKey(sk2);

const sig1 = bls.sign(message, sk1);
const sig2 = bls.sign(message, sk2);

const aggSig = bls.aggregateSignatures([sig1, sig2]);
const aggPk = bls.aggregatePublicKeys([pk1, pk2]);

const isAggValid = bls.verify(aggSig, message, aggPk);
console.log("Aggregated signature valid:", isAggValid);
```

Resumen: El Ecosistema de Firmas

Idea Clave

Guía de selección de esquema de firmas:

Caso de uso	Esquema recomendado
Compatibilidad Ethereum/Bitcoin	ECDSA
Nuevos proyectos, velocidad máxima	EdDSA (Ed25519)
Agregación de muchas firmas	BLS
Multisig que parece normal	Schnorr (MuSig2)
Threshold sin coordinación	BLS threshold
Dentro de circuitos ZK	EdDSA sobre curva embebida
Post-cuántico	Dilithium, SPHINCS+

En la práctica, los sistemas modernos como Banco ZK usan **múltiples esquemas** optimizando para cada contexto.

1.5 Merkle Trees

Definición 1.116 (Merkle Tree)**.** Árbol binario donde cada nodo interno es el hash de sus hijos.

Propiedades:

- La raíz compromete a todos los datos
- Prueba de inclusión: $O(\log n)$ hashes
- Actualización: $O(\log n)$
- Usado en: Bitcoin (tx), Ethereum (state), ZK (commitments)

1.5.1 Contexto Histórico

Intuición

Los Merkle trees son una de las estructuras de datos más importantes en criptografía aplicada. Permiten **comprometer** grandes cantidades de datos en un solo hash (la raíz) y después **probar** la inclusión de cualquier elemento sin revelar todo el conjunto.

Observación 1.40. **Ralph Merkle y el nacimiento del hashing:**

Ralph Merkle propuso esta estructura en 1979 como parte de su tesis doctoral en Stanford, en el mismo período que inventó el concepto de criptografía de clave pública (independientemente de Diffie-Hellman).

Año	Desarrollo
1979	Merkle propone el árbol hash
1980	Patente US 4,309,569
1991	Uso en timestamps digitales (Haber-Stornetta)
2008	Bitcoin usa Merkle trees para transacciones
2015	Ethereum usa Patricia Merkle Tries para estado
2016	Zcash usa Merkle trees para notas privadas

Curiosamente, los papers de Haber y Stornetta sobre timestamping (que usaban Merkle trees) están citados en el whitepaper de Bitcoin.

1.5.2 Construcción Básica

Definición 1.117 (Construcción de Merkle Tree)**.** Dado un conjunto de n elementos (hojas) $L_0, L_1, \ldots, L_{n-1}$ y una función hash H:

Nivel 0 (hojas):

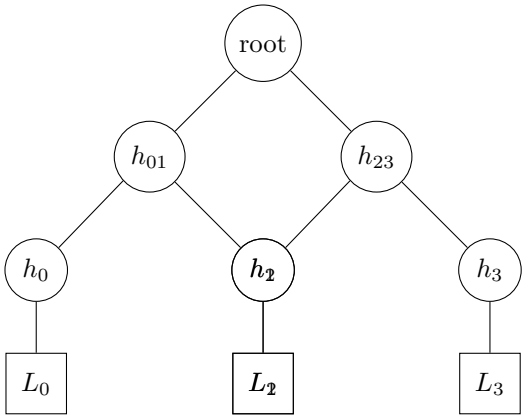
$$h_i^{(0)} = H(L_i)$$

Niveles superiores:

$$h_i^{(k+1)} = H(h_{2i}^{(k)} \| h_{2i+1}^{(k)})$$

Raíz:

$$\text{root} = h_0^{(\lceil \log_2 n \rceil)}$$



Ejemplo 1.59 (Construcción paso a paso)**. Datos:** 4 transacciones $\text{tx}_0, \text{tx}_1, \text{tx}_2, \text{tx}_3$

Nivel 0 (hojas):

$$h_0 = H(\text{tx}_0) = \texttt{a1b2...} \tag{1.172}$$

$$h_1 = H(\text{tx}_1) = \texttt{c3d4...} \tag{1.173}$$

$$h_2 = H(\text{tx}_2) = \texttt{e5f6...} \tag{1.174}$$

$$h_3 = H(\text{tx}_3) = \texttt{7890...} \tag{1.175}$$

Nivel 1:

$$h_{01} = H(h_0 \| h_1) = H(\texttt{a1b2...} \| \texttt{c3d4...}) = \texttt{1234...} \tag{1.176}$$

$$h_{23} = H(h_2 \| h_3) = H(\texttt{e5f6...} \| \texttt{7890...}) = \texttt{5678...} \tag{1.177}$$

Nivel 2 (raíz):

$$\text{root} = H(h_{01} \| h_{23}) = H(\texttt{1234...} \| \texttt{5678...}) = \texttt{abcd...}$$

La raíz `abcd...` compromete a las 4 transacciones.

1.5.3 Merkle Proof (Prueba de Inclusión)

Para probar que L_i está en el árbol:

1. Proporcionar los "hermanos" en el camino a la raíz
2. Verificador recalcula hashes hasta la raíz
3. Compara con raíz conocida

Tamaño de prueba: $\log_2(n)$ hashes.

Definición 1.118 (Merkle Proof)**.** Una prueba de inclusión para el elemento L_i en un árbol con raíz r consiste en:

1. El elemento L_i (o su hash h_i)
2. El **authentication path**: secuencia de hashes hermanos desde la hoja hasta la raíz
3. Los **direction bits**: indica si cada hermano va a la izquierda o derecha

$$\text{proof} = (L_i, [(s_0, d_0), (s_1, d_1), \ldots, (s_{k-1}, d_{k-1})])$$

donde s_j es el hash hermano y $d_j \in \{L, R\}$ es su posición.

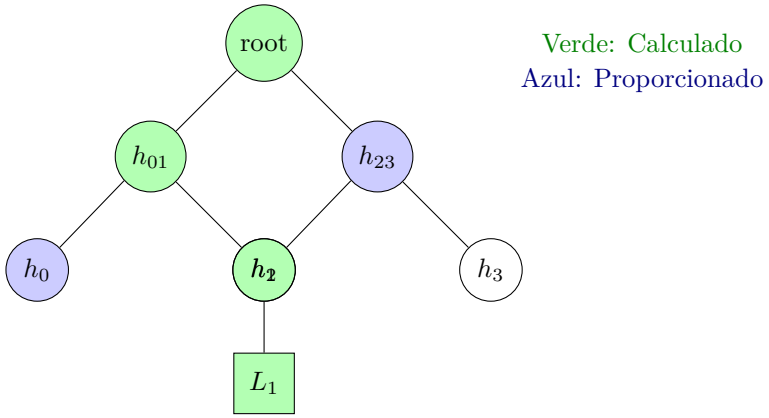
Ejemplo 1.60 (Merkle proof para L_1)**.** Queremos probar que tx_1 está en el árbol.
Authentication path:

1. Hermano de h_1: h_0 (a la izquierda)
2. Hermano de h_{01}: h_{23} (a la derecha)

Proof: $(\text{tx}_1, [(h_0, L), (h_{23}, R)])$
Verificación:

1. $h_1 = H(\text{tx}_1)$
2. $h_{01} = H(h_0 \| h_1)$ (hermano h_0 a la izquierda)
3. $\text{root}' = H(h_{01} \| h_{23})$ (hermano h_{23} a la derecha)

4. Verificar: $\text{root}' \stackrel{?}{=} \text{root}_{\text{conocida}}$



1.5.4 Propiedades y Complejidad

Idea Clave

Complejidad de operaciones en Merkle Trees:

Operación	Tiempo	Espacio
Construir árbol	$O(n)$	$O(n)$
Calcular raíz	$O(n)$	$O(\log n)$ (streaming)
Generar proof	$O(\log n)$	$O(\log n)$
Verificar proof	$O(\log n)$	$O(\log n)$
Actualizar hoja	$O(\log n)$	$O(\log n)$

Para $n = 2^{20} \approx 1$ millón de elementos, el proof tiene solo 20 hashes.

Matemáticas

Propiedades de seguridad:

Sea H una función hash resistente a colisiones.

1. Binding: Es computacionalmente imposible encontrar dos conjuntos de hojas diferentes que produzcan la misma raíz. (Se reduce a resistencia a colisiones de H.)

2. Soundness de proofs: Un proof válido para un elemento L con raíz r implica que L estaba en el conjunto original (salvo encontrar colisiones en H).

3. Hiding (opcional): El árbol por sí solo no oculta información. Para privacidad, se pueden añadir hojas aleatorias o usar hashes con salt.

1.5.5 Árboles con Número Impar de Hojas

Advertencia

¿Qué pasa si n no es potencia de 2?

Opción 1: Duplicar último elemento

$$h_n = h_{n-1}$$

Problema: Dos conjuntos diferentes pueden tener la misma raíz.

Opción 2: Padding con hash especial

$$h_n = H(\texttt{0x00})$$

Problema: Inconsistencia si el conjunto crece.

Opción 3: Árbol desbalanceado Permitir nodos con un solo hijo, propagando el hash directamente.

Opción 4: Fijar profundidad (común en ZK) Usar un árbol de profundidad fija d, con hojas vacías = hash de cero.

Ejemplo 1.61 (Árbol de profundidad fija)**.** Para un árbol de profundidad 3 (capacidad 8) con 5 elementos:

```
Hojas: [L0, L1, L2, L3, L4, EMPTY, EMPTY, EMPTY]
donde EMPTY = H(0)
```

Esto es común en protocolos ZK donde la profundidad del árbol es un parámetro del circuito.

1.5.6 Merkle Trees en Bitcoin

Definición 1.119 (Merkle Root de Bitcoin)**.** Cada bloque de Bitcoin contiene un Merkle root que compromete todas las transacciones del bloque.

```
Block Header:
  - version
  - prev_block_hash
  - merkle_root        <-- Hash de todas las transacciones
  - timestamp
  - difficulty_target
  - nonce
```

Idea Clave

¿Por qué Merkle trees en Bitcoin?

1. Verificación SPV (Simplified Payment Verification):

Un nodo ligero puede verificar que una transacción está en un bloque sin descargar todas las transacciones:

- Descarga solo headers (∼80 bytes/bloque)
- Pide Merkle proof de la transacción específica
- Verifica proof contra merkle_root del header

2. **Eficiencia:**
   - Header: 80 bytes (fijo)
   - Bloque completo: ∼1-2 MB
   - Proof para 1 tx en bloque de 4000 tx: ∼12 hashes = 384 bytes

Ejemplo 1.62 (Verificación SPV)**.** Alice quiere verificar que su pago está confirmado.

Sin Merkle tree:

1. Descargar bloque completo (∼1 MB)
2. Buscar transacción
3. Verificar

Con Merkle tree:

1. Pedir Merkle proof (∼400 bytes)
2. Verificar proof contra header (∼80 bytes)

Reducción: 1 MB → 500 bytes (∼2000x menos datos).

Observación 1.41. Bitcoin usa SHA-256 doble: $H(x) = \text{SHA256}(\text{SHA256}(x))$

Esto fue una decisión de Satoshi, posiblemente para prevenir ataques de extensión de longitud (aunque SHA-256 truncado hubiera sido suficiente).

1.5.7 Merkle Patricia Trie en Ethereum

Definición 1.120 (Patricia Trie)**.** Un Patricia Trie (Practical Algorithm To Retrieve Information Coded In Alphanumeric) es un trie comprimido donde nodos con un solo hijo se fusionan con su padre.

Definición 1.121 (Merkle Patricia Trie)**.** Ethereum combina Patricia Tries con Merkle Trees para crear una estructura que permite:

- Búsqueda eficiente por clave
- Pruebas de inclusión Y exclusión
- Compromiso a todo el estado en una raíz

Tres tipos de nodos:

1. **Branch node:** 17 elementos [16 hijos + valor]

2. **Extension node:** [prefijo compartido, siguiente nodo]
3. **Leaf node:** [sufijo de clave, valor]

Intuición

Mientras Bitcoin solo necesita probar "esta transacción está en el bloque", Ethereum necesita probar cosas más complejas:

- "La cuenta 0x1234... tiene balance X"
- "El storage slot Y del contrato Z tiene valor W"
- "La cuenta 0xABCD... NO existe" (prueba de exclusión)

El Merkle Patricia Trie permite todo esto con pruebas logarítmicas.

Observación 1.42. Ethereum tiene múltiples MPTs:

- **State trie:** Mapea direcciones a cuentas (balance, nonce, codeHash, storageRoot)
- **Storage trie:** Para cada contrato, mapea slots a valores
- **Transaction trie:** Transacciones del bloque indexadas por posición
- **Receipt trie:** Receipts indexados por posición

El header del bloque contiene las raíces de todos estos tries.

1.5.8 Sparse Merkle Trees

Definición 1.122 (Sparse Merkle Tree (SMT))**.** Un SMT es un Merkle tree de profundidad fija d donde la mayoría de las hojas están vacías.

Propiedades:

- 2^d hojas posibles, pero solo $n \ll 2^d$ tienen valores no vacíos
- Permite pruebas de NO inclusión
- Se pueden precomputar los hashes de subárboles vacíos

Idea Clave

Truco de eficiencia para SMTs:
Los hashes de subárboles vacíos son determinísticos:

$$\text{empty}_0 = H(\texttt{0x00}) \quad \text{(hoja vacía)} \tag{1.178}$$
$$\text{empty}_1 = H(\text{empty}_0 \| \text{empty}_0) \tag{1.179}$$
$$\text{empty}_2 = H(\text{empty}_1 \| \text{empty}_1) \tag{1.180}$$
$$\vdots \tag{1.181}$$
$$\text{empty}_d = H(\text{empty}_{d-1} \| \text{empty}_{d-1}) \tag{1.182}$$

Estos valores se precomputan una vez. Un subárbol completamente vacío se representa con su hash precomputado, sin almacenar los nodos individuales.

Ejemplo 1.63 (SMT para un conjunto de 3 elementos)**.** Profundidad $d = 32$ (soporta $2^{32} \approx 4$ mil millones de claves).

Elementos: clave 5 → valor A, clave 1000 → valor B, clave 7 → valor C

```
Almacenamiento real:
- Camino a clave 5: ~32 nodos
- Camino a clave 1000: ~32 nodos
- Camino a clave 7: ~32 nodos
- Nodos compartidos entre caminos

Total: << 2^32 nodos
```

Los subárboles sin elementos almacenan solo una referencia al hash vacío precomputado.

Matemáticas

Prueba de no inclusión en SMT:
Para probar que la clave k NO está en el árbol:

1. Proporcionar el camino desde la raíz hasta la posición de k
2. Mostrar que la hoja en esa posición está vacía (= hash vacío)

Esto es imposible en un Merkle tree normal sin profundidad fija.

1.5.9 Indexed Merkle Trees

Definición 1.123 (Indexed Merkle Tree)**.** Variante donde cada hoja contiene no solo un valor, sino también información sobre el siguiente elemento en orden:

$$\text{leaf} = (k, v, \text{next_key}, \text{next_idx})$$

Esto permite pruebas de no inclusión más eficientes que SMTs completos.

Observación 1.43. Los Indexed Merkle Trees son usados en Aztec Protocol para su árbol de nullifiers, combinando las ventajas de:

- Merkle trees normales: tamaño proporcional a elementos insertados
- SMTs: pruebas de no inclusión

1.5.10 Append-Only Merkle Trees

Definición 1.124 (Append-Only Merkle Tree)**.** Un Merkle tree donde solo se permiten inserciones al final, nunca modificaciones o eliminaciones.

Propiedades adicionales:

- **Consistency proofs:** Dado root antiguo r_1 y nuevo r_2, probar que el árbol viejo es prefijo del nuevo
- **Tamper evidence:** Cualquier modificación histórica es detectable

Intuición

Certificate Transparency (CT) usa append-only Merkle trees para crear un log público de certificados TLS. Si una CA emite un certificado malicioso, queda registrado públicamente y es detectable.

1.5.11 Hash Functions para Merkle Trees

Hash	Bitcoin	Ethereum	ZK SNARKs	Costo ZK
SHA-256				∼25,000
Keccak-256				∼150,000
Poseidon				∼250
Pedersen				∼1,000
MiMC				∼700

Idea Clave

La elección del hash importa enormemente en ZK:
Para un Merkle tree de profundidad 20 (1M hojas):

- Con SHA-256: $20 \times 25,000 = 500,000$ constraints
- Con Keccak: $20 \times 150,000 = 3,000,000$ constraints
- Con Poseidon: $20 \times 250 = 5,000$ constraints

¡El mismo árbol puede costar 100-600x más con un hash inadecuado!

1.5.12 Merkle Trees en Sistemas ZK

Matemáticas

Los Merkle trees son fundamentales en casi todos los protocolos ZK:
Tornado Cash:

- Árbol de profundidad 20 con compromisos de depósitos
- Probar: "Conozco el secreto de algún commitment en el árbol"
- Sin revelar CUÁL commitment

Zcash:

- Árbol de notas (notes = UTXOs privados)
- Probar: "Esta nota existe y puedo gastarla"

- Nullifier previene doble gasto sin revelar la nota

ZK Rollups:

- Árbol de estado (cuentas y balances)
- Probar: "Las transacciones actualizaron el estado correctamente"
- Raíz antigua → raíz nueva

Ejemplo 1.64 (Merkle proof en circuito ZK)**. Inputs públicos:**

- Merkle root: r

Inputs privados (witness):

- Hoja: L
- Índice: i
- Authentication path: $[s_0, s_1, \ldots, s_{d-1}]$

Circuito:

```
h = Hash(L)
for j in 0..d:
    bit = (i >> j) & 1
    if bit == 0:
        h = Hash(h, siblings[j])
    else:
        h = Hash(siblings[j], h)

assert h == root
```

El circuito verifica el path sin revelar L, i, ni los siblings.

1.5.13 Optimizaciones

Idea Clave

1. Merkle Mountain Ranges (MMR):
Para datos append-only, MMR permite agregar elementos en $O(1)$ amortizado (vs $O(\log n)$ para árboles balanceados).

2. Verkle Trees:
Usan compromisos polinomiales (KZG) en lugar de hashes. Los proofs son de tamaño constante independiente de la profundidad. Ethereum planea migrar de MPT a Verkle trees.

3. Árboles de aridad alta:
En lugar de árboles binarios, usar árboles con 4, 8 o 16 hijos por nodo. Reduce la

profundidad pero aumenta el tamaño de cada nodo en el proof.

Matemáticas

Trade-off de aridad:
Para n hojas con árbol de aridad a:

- Profundidad: $\log_a(n)$
- Tamaño de proof: $\log_a(n) \times (a-1)$ hashes

Aridad	Profundidad	Hashes en proof
2	$\log_2(n) = 20$	20
4	$\log_4(n) = 10$	30
8	$\log_8(n) \approx 7$	49
16	$\log_{16}(n) = 5$	75

Para $n = 2^{20}$. El árbol binario es óptimo en tamaño de proof.

1.5.14 Ataques y Consideraciones de Seguridad

Advertencia

1. Second preimage attacks:
Si el hash no distingue hojas de nodos internos, un atacante podría presentar un nodo interno como hoja.
Solución: Prefijos diferentes para hojas y nodos:

$$H_{\text{leaf}}(x) = H(\texttt{0x00} \| x) \tag{1.183}$$
$$H_{\text{node}}(l, r) = H(\texttt{0x01} \| l \| r) \tag{1.184}$$

2. Length extension en Merkle roots:
Si se usa un hash vulnerable a length extension (SHA-256 raw), cuidado con cómo se usa la raíz.
3. Índice fuera de rango:
Verificar que el índice proporcionado está en rango válido.

1.5.15 Implementación

Ejemplo 1.65 (Merkle Tree en Python)**.** `import hashlib`

```
def hash_node(left, right):
    """Hash two nodes together"""
    return hashlib.sha256(b'\x01' + left + right).digest()
```

```
def hash_leaf(data):
    """Hash a leaf node"""
    return hashlib.sha256(b'\x00' + data).digest()

class MerkleTree:
    def __init__(self, leaves):
        self.leaves = [hash_leaf(leaf) for leaf in leaves]
        self.layers = [self.leaves]
        self._build()

    def _build(self):
        """Build tree from leaves up"""
        current = self.leaves
        while len(current) > 1:
            next_layer = []
            for i in range(0, len(current), 2):
                left = current[i]
                right = current[i+1] if i+1 < len(current) else left
                next_layer.append(hash_node(left, right))
            self.layers.append(next_layer)
            current = next_layer

    @property
    def root(self):
        return self.layers[-1][0]

    def get_proof(self, index):
        """Get authentication path for leaf at index"""
        proof = []
        for layer in self.layers[:-1]:
            sibling_idx = index ^ 1  # Flip last bit
            if sibling_idx < len(layer):
                sibling = layer[sibling_idx]
            else:
                sibling = layer[index]
            direction = 'L' if index % 2 == 1 else 'R'
            proof.append((sibling, direction))
            index //= 2
        return proof

def verify_proof(leaf, proof, root):
    """Verify a Merkle proof"""
```

```
    current = hash_leaf(leaf)
    for sibling, direction in proof:
        if direction == 'L':
            current = hash_node(sibling, current)
        else:
            current = hash_node(current, sibling)
    return current == root

# Uso
leaves = [b'tx0', b'tx1', b'tx2', b'tx3']
tree = MerkleTree(leaves)
proof = tree.get_proof(1)  # Proof para tx1
assert verify_proof(b'tx1', proof, tree.root)
```

Ejemplo 1.66 (Merkle Tree en Solidity)**.**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

library MerkleProof {
    function verify(
        bytes32[] memory proof,
        bytes32 root,
        bytes32 leaf,
        uint256 index
    ) internal pure returns (bool) {
        bytes32 computedHash = leaf;

        for (uint256 i = 0; i < proof.length; i++) {
            bytes32 proofElement = proof[i];

            if (index % 2 == 0) {
                // Sibling is on the right
                computedHash = keccak256(
                    abi.encodePacked(computedHash, proofElement)
                );
            } else {
                // Sibling is on the left
                computedHash = keccak256(
                    abi.encodePacked(proofElement, computedHash)
                );
            }

            index = index / 2;
        }
```

```
        return computedHash == root;
    }
}

contract MerkleAirdrop {
    bytes32 public merkleRoot;
    mapping(address => bool) public claimed;

    constructor(bytes32 _merkleRoot) {
        merkleRoot = _merkleRoot;
    }

    function claim(
        uint256 index,
        address account,
        uint256 amount,
        bytes32[] calldata proof
    ) external {
        require(!claimed[account], "Already claimed");

        bytes32 leaf = keccak256(
            abi.encodePacked(index, account, amount)
        );
        require(
            MerkleProof.verify(proof, merkleRoot, leaf, index),
            "Invalid proof"
        );

        claimed[account] = true;
        // Transfer tokens...
    }
}
```

1.5.16 Aplicación en Banco ZK

En Banco ZK

Los Merkle trees son el corazón de la arquitectura de Banco ZK:

1. Árbol de Notas (Note Tree)

Cada depósito crea un "commitment" que se inserta en el árbol:

$$\text{commitment} = \text{Poseidon}(\text{value}, \text{pubkey}, \text{randomness})$$

Parámetro	Valor
Profundidad	20
Capacidad	$2^{20} \approx 1$ millón de notas
Hash	Poseidon
Tipo	Append-only

2. Árbol de Nullifiers (Nullifier Tree)

Para prevenir doble gasto sin revelar qué nota se gastó:

$$\text{nullifier} = \text{Poseidon}(\text{commitment}, \text{spending_key})$$

Parámetro	Valor
Profundidad	32
Tipo	Sparse (indexed)
Función	Detectar doble gasto

3. Flujo de transacción privada

```
DEPOSITAR:
1. Usuario genera: commitment = Poseidon(value, pk, r)
2. Contrato inserta commitment en Note Tree
3. Usuario guarda (value, pk, r) localmente

TRANSFERIR (privadamente):
1. Usuario prueba en ZK:
   - "Conozco una nota en el Note Tree" (Merkle proof)
   - "El nullifier de esta nota no está usado"
   - "Estoy creando nueva(s) nota(s) válida(s)"
2. Publica: nullifier, nuevos commitments, prueba ZK
3. Contrato:
   - Verifica prueba
   - Añade nullifier al Nullifier Tree
   - Añade nuevos commitments al Note Tree

RETIRAR:
1. Similar a transferir, pero el output es público
2. Contrato envía fondos a la dirección especificada
```

4. Costo en circuitos

Componente	Hash	Constraints
Note tree proof (depth 20)	Poseidon	$20 \times 250 = 5,000$
Nullifier computation	Poseidon	250
Commitment computation	Poseidon	250
Output commitments ($\times 2$)	Poseidon	500
Total hashing		$\sim$6,000

Comparado con usar SHA-256 ($\sim$500,000), $Poseidon reduce el costo 80x$.

5. On-chain storage

El contrato almacena:

- Note Tree root: 32 bytes (actualizado por batch)
- Nullifier Tree root: 32 bytes (actualizado por batch)
- Historical roots: Array para verificar pruebas antiguas

```
contract BancoZK {
    bytes32 public noteTreeRoot;
    bytes32 public nullifierTreeRoot;

    // Permitir pruebas contra roots recientes
    bytes32[100] public historicalRoots;
    uint256 public rootIndex;

    function isKnownRoot(bytes32 root) public view returns (bool) {
        if (root == noteTreeRoot) return true;
        for (uint i = 0; i < 100; i++) {
            if (historicalRoots[i] == root) return true;
        }
        return false;
    }
}
```

6. Actualización de árboles

Las actualizaciones se procesan en batches:

1. Sequencer colecta transacciones
2. Genera prueba ZK de transición de estado válida
3. Publica: nueva raíz + prueba
4. Contrato verifica prueba y actualiza raíz

7. Consideraciones de privacidad
Para máxima privacidad:

- El árbol tiene hojas aleatorias (no todas son depósitos reales)
- Las inserciones son en posiciones semi-aleatorias
- El timing de operaciones se ofusca donde es posible

1.5.17 Resumen

Idea Clave

Merkle Trees: Puntos clave

1. **Compromiso eficiente:** Una raíz de 32 bytes compromete millones de elementos
2. **Proofs logarítmicos:** $O(\log n)$ hashes para probar inclusión
3. **Variantes para diferentes usos:**
   - Normal: inclusión
   - Sparse: inclusión y exclusión
   - Indexed: eficiencia intermedia
   - Append-only: logs y auditoría
4. **Hash matters:** En ZK, elegir Poseidon sobre SHA-256 puede ahorrar 100x en constraints
5. **Ubicuidad:** Bitcoin, Ethereum, Zcash, rollups, airdrops, certificate transparency...

1.6 Merkle Patricia Tries

Definición 1.125 (Patricia Trie)**.** Trie comprimido donde caminos sin bifurcación se colapsan.

Definición 1.126 (Merkle Patricia Trie)**.** Patricia Trie donde cada nodo está hasheado (Merkleizado).

Usado en Ethereum para:

- State trie: cuentas y storage
- Transaction trie: transacciones por bloque
- Receipt trie: logs y resultados

Tipos de nodos:

- **Branch:** 16 hijos (hex) + valor opcional

- **Extension:** prefijo compartido + siguiente nodo
- **Leaf:** resto del path + valor

Ventajas:

- Pruebas de inclusión Y exclusión
- Actualizaciones eficientes
- State root en header de bloque

1.6.1 Contexto: ¿Por Qué No un Merkle Tree Simple?

Intuición

Bitcoin usa Merkle trees simples para transacciones: funciona porque solo necesita probar "esta transacción está en el bloque". Pero Ethereum necesita algo más poderoso. Ethereum es una **máquina de estados**: cada bloque transiciona el estado global (balances, storage de contratos, código). Necesitamos:

1. Probar que una cuenta tiene cierto balance
2. Probar que una variable de contrato tiene cierto valor
3. Probar que una cuenta **NO existe** (prueba de exclusión)
4. Actualizar eficientemente cuando el estado cambia

Un Merkle tree simple no puede hacer el punto 3 (exclusión) ni el 4 eficientemente.

Idea Clave

El Merkle Patricia Trie combina tres conceptos:

1. **Trie**: Estructura de árbol indexada por claves (permite búsqueda)
2. **Patricia**: Compresión de caminos sin bifurcación (eficiencia espacial)
3. **Merkle**: Cada nodo está hasheado (compromiso criptográfico)

Resultado: Una estructura que soporta búsquedas, actualizaciones, pruebas de inclusión Y exclusión, todo con compromiso criptográfico.

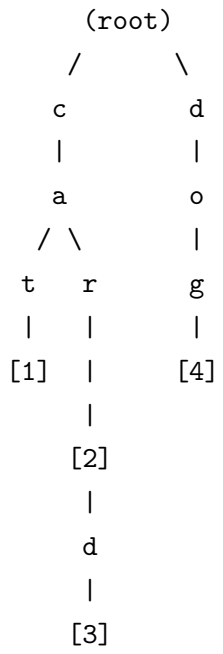
1.6.2 Fundamentos: El Trie Básico

Definición 1.127 (Trie (Prefix Tree))**.** Un trie es un árbol donde:

- Cada arista está etiquetada con un carácter
- La clave de un nodo es la concatenación de etiquetas desde la raíz
- Los valores se almacenan en los nodos

Para claves que son strings sobre alfabeto Σ, cada nodo tiene hasta $|\Sigma|$ hijos.

Ejemplo 1.67 (Trie simple con palabras)**.** Almacenando: "cat" $\to$ 1, "car" $\to$ 2, "card" $\to$ 3, "dog" $\to$ 4



Para buscar "car": seguir c $\to$ a $\to$ r, encontrar valor 2.

Matemáticas

Complejidad del trie básico:

- Búsqueda: $O(|k|)$ donde $|k|$ es la longitud de la clave
- Inserción: $O(|k|)$
- Espacio: $O(n \cdot |\Sigma| \cdot |k|)$ en el peor caso

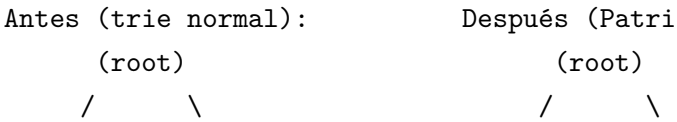
El problema: con alfabeto grande ($|\Sigma| = 256$ para bytes), cada nodo usa mucha memoria aunque tenga pocos hijos.

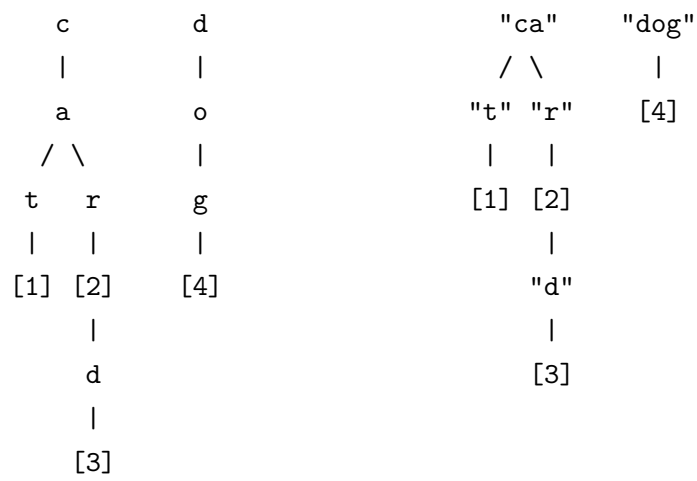
1.6.3 Patricia Trie: Compresión de Caminos

Definición 1.128 (Patricia Trie)**.** PATRICIA = Practical Algorithm To Retrieve Information Coded In Alphanumeric

Un Patricia trie comprime cadenas de nodos con un solo hijo en un solo nodo con una etiqueta multi-carácter.

Ejemplo 1.68 (Compresión Patricia)**.** El trie anterior comprimido:





Nodos reducidos de 8 a 6. Con claves más largas, la compresión es mayor.

Idea Clave

La compresión Patricia es especialmente efectiva cuando:

- Las claves son largas (como hashes de 32 bytes)
- Las claves comparten prefijos largos
- El conjunto de claves es disperso (sparse)

En Ethereum, las claves son hashes Keccak de 32 bytes. Sin compresión, el árbol tendría profundidad 64 (nibbles hexadecimales). Con Patricia, la profundidad promedio es mucho menor.

1.6.4 Merkle Patricia Trie: Añadiendo Hashes

Definición 1.129 (Merkleización)**.** Para convertir un Patricia trie en un Merkle Patricia Trie:

1. Serializar cada nodo usando RLP (Recursive Length Prefix)
2. Si el RLP del nodo tiene ≥ 32 bytes, almacenar su hash Keccak-256
3. Si tiene < 32 bytes, almacenar el nodo directamente (inline)
4. La raíz del trie es el hash del nodo raíz

Intuición

El hash de la raíz "compromete" a todo el contenido del trie. Cualquier cambio en cualquier valor propagará cambios de hash hasta la raíz.
Esto permite incluir un solo hash de 32 bytes en el header del bloque que compromete todo el estado de Ethereum (∼100 GB de datos).

1.6.5 Tipos de Nodos en Detalle

Definición 1.130 (Nodo Branch)**.** Un nodo branch tiene 17 elementos:

$$[\text{child}_0, \text{child}_1, \ldots, \text{child}_{15}, \text{value}]$$

- child_i: referencia al hijo para el nibble i (0-9, a-f)
- value: valor almacenado si alguna clave termina en este nodo

Cada child_i es:

- Vacío (null) si no hay hijo
- Hash del hijo si el hijo serializado ≥ 32 bytes
- El hijo serializado directamente (inline) si < 32 bytes

Definición 1.131 (Nodo Extension)**.** Un nodo extension tiene 2 elementos:

$$[\text{encoded_path}, \text{next_node}]$$

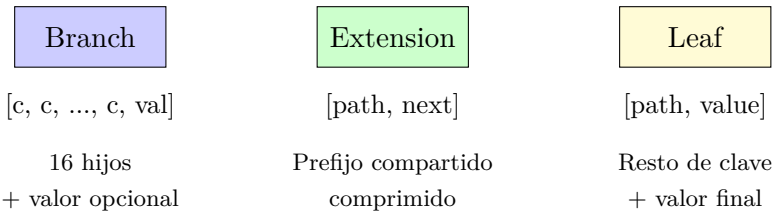
- encoded_path: prefijo compartido comprimido (con hex-prefix encoding)
- next_node: referencia al siguiente nodo (hash o inline)

Se usa cuando múltiples claves comparten un prefijo largo.

Definición 1.132 (Nodo Leaf)**.** Un nodo leaf tiene 2 elementos:

$$[\text{encoded_path}, \text{value}]$$

- encoded_path: resto del path desde el nodo padre (con hex-prefix)
- value: el valor almacenado



1.6.6 Hex-Prefix Encoding

Definición 1.133 (Hex-Prefix Encoding)**.** Para distinguir nodos Extension de Leaf y manejar paths de longitud impar, se usa un prefijo especial:

Tipo de nodo	Longitud path	Prefijo
Extension	Par	0x00
Extension	Impar	0x1
Leaf	Par	0x20
Leaf	Impar	0x3

Si la longitud es impar, el primer nibble del path se combina con el prefijo.

Ejemplo 1.69 (Hex-Prefix encoding)**. Path = [1, 2, 3, 4] (par), tipo Leaf:**

```
Prefijo: 0x20
Encoded: [0x20, 0x12, 0x34]
```

Path = [1, 2, 3] (impar), tipo Leaf:

```
Prefijo: 0x3, combinado con primer nibble
Encoded: [0x31, 0x23]
```

Path = [a, b, c, d] (par), tipo Extension:

```
Prefijo: 0x00
Encoded: [0x00, 0xab, 0xcd]
```

1.6.7 RLP Encoding

Definición 1.134 (Recursive Length Prefix (RLP))**.** RLP es el formato de serialización usado en Ethereum para codificar datos estructurados.

Reglas básicas:

- Byte simple $[0x00, 0x7f]$: se codifica como sí mismo
- String corto (≤ 55 bytes): $[0x80 + \text{len}] + \text{data}$
- String largo (> 55 bytes): $[0xb7 + \text{len_of_len}] + \text{len} + \text{data}$
- Lista corta: $[0xc0 + \text{len}] + \text{items}$
- Lista larga: $[0xf7 + \text{len_of_len}] + \text{len} + \text{items}$

Ejemplo 1.70 (RLP de nodo Leaf)**.**

```
Leaf node:
  path = [0x20, 0x12, 0x34]  (3 bytes)
  value = "hello"           (5 bytes)

RLP([path, value]):
  path_rlp = [0x83, 0x20, 0x12, 0x34]  (0x83 = 0x80 + 3)
  value_rlp = [0x85] + "hello"         (0x85 = 0x80 + 5)

  list_content = path_rlp + value_rlp = 12 bytes
  list_prefix = 0xc0 + 12 = 0xcc

Final RLP: [0xcc, 0x83, 0x20, 0x12, 0x34, 0x85, 'h', 'e', 'l', 'l', 'o']
```

Como tiene 12 bytes (< 32), este nodo puede ser inline en su padre.

1.6.8 Ejemplo Completo de MPT

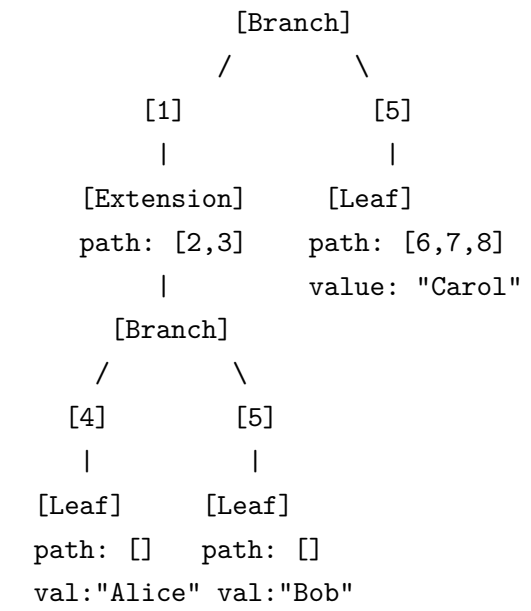
Ejemplo 1.71 (Construcción de MPT)**.** Almacenemos estos pares clave-valor (claves en hex):

Clave	Valor
0x1234	"Alice"
0x1235	"Bob"
0x5678	"Carol"

Paso 1: Convertir claves a nibbles

```
0x1234 → [1, 2, 3, 4]
0x1235 → [1, 2, 3, 5]
0x5678 → [5, 6, 7, 8]
```

Paso 2: Construir trie



Paso 3: Merkleizar (bottom-up)

```
1. Leaf "Alice": RLP([hp([]), "Alice"]) → hash
2. Leaf "Bob":   RLP([hp([]), "Bob"])   → hash
3. Branch en [1,2,3]:
   children[4] = hash
   children[5] = hash
   RLP → hash
4. Extension [2,3]:
   RLP([hp([2,3], ext), hash]) → hash
5. Leaf "Carol":
   RLP([hp([6,7,8], leaf), "Carol"]) → hash
6. Root Branch:
```

```
children[1] = hash
children[5] = hash
RLP → ROOT
```

1.6.9 Operaciones

Búsqueda (Get)

Definición 1.135 (Algoritmo de Búsqueda)**.** `function get(node, path):`

```
    if path is empty:
        if node is Leaf or Branch with value:
            return node.value
        else:
            return NOT_FOUND

    if node is Branch:
        child = node.children[path[0]]
        if child is null:
            return NOT_FOUND
        return get(resolve(child), path[1:])

    if node is Extension:
        if path starts with node.path:
            return get(resolve(node.next), path[len(node.path):])
        else:
            return NOT_FOUND

    if node is Leaf:
        if path == node.path:
            return node.value
        else:
            return NOT_FOUND
```

Inserción (Put)

Matemáticas

La inserción puede requerir:

1. Crear nuevo Leaf (clave no existe, sin conflictos)
2. Dividir un Extension (nuevo prefijo compartido más corto)
3. Dividir un Leaf (claves comparten prefijo parcial)
4. Actualizar valor existente (clave ya existe)

Cada caso requiere recalcular hashes hasta la raíz.

Eliminación (Delete)

Advertencia

La eliminación es la operación más compleja porque puede requerir:

1. Eliminar Leaf y posiblemente colapsar Branch padre
2. Fusionar Extension con hijo si queda un solo camino
3. Actualizar hashes hasta la raíz

La complejidad de mantener el trie "comprimido" después de deletes hace que la implementación sea propensa a bugs.

1.6.10 Pruebas de Inclusión y Exclusión

Idea Clave

A diferencia de los Merkle trees simples, los MPT permiten **pruebas de exclusión**: demostrar que una clave NO está en el trie.

Definición 1.136 (Prueba de Inclusión)**.** Para probar que la clave k tiene valor v:

1. Proporcionar todos los nodos en el camino desde la raíz hasta el Leaf de k
2. El verificador reconstruye los hashes y verifica que la raíz coincide

Definición 1.137 (Prueba de Exclusión)**.** Para probar que la clave k NO está en el trie:

1. Proporcionar el camino hasta donde la búsqueda "falla"
2. Mostrar que:
   - Un Branch no tiene hijo para el siguiente nibble, o
   - Un Extension tiene un prefijo que no coincide, o
   - Un Leaf tiene un path diferente

Ejemplo 1.72 (Prueba de exclusión)**.** En el MPT anterior, probar que `0x1236` NO existe:

```
Camino de búsqueda:
1. Root Branch → child[1] existe → seguir
2. Extension [2,3] → coincide → seguir
3. Branch → child[6] = null → ¡NO EXISTE!

Proof:
- Root Branch (mostrando child[1] = hash)
- Extension con path [2,3]
- Branch (mostrando child[6] = null)

Verificador:
1. Verifica que los hashes son consistentes con la raíz conocida
2. Observa que child[6] está vacío
3. Concluye: 0x1236 no puede existir
```

1.6.11 Ethereum: Los Cuatro Tries

Definición 1.138 (State Trie)**.** Mapea direcciones (20 bytes) a cuentas (Account objects).
Clave: Keccak256(address) **Valor:** RLP([nonce, balance, storageRoot, codeHash])

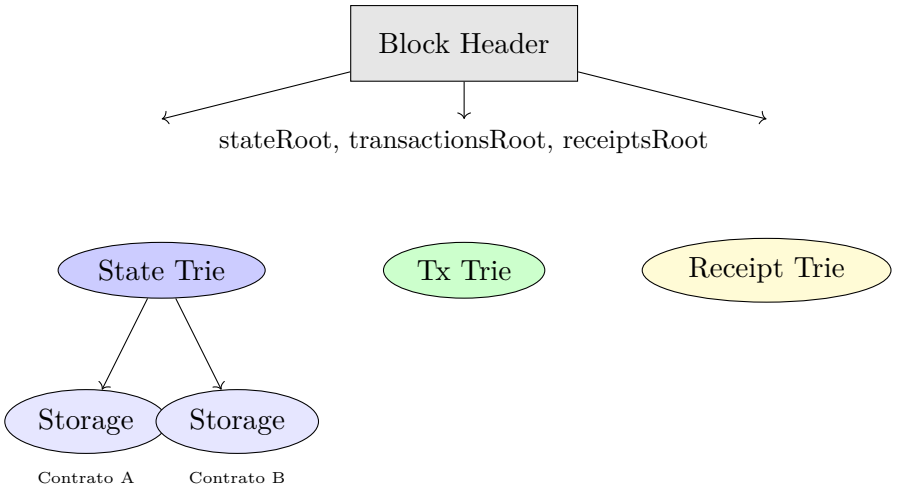
```
Account:
  nonce: número de transacciones enviadas
  balance: wei poseídos
  storageRoot: raíz del Storage Trie de la cuenta
  codeHash: Keccak256(código) para contratos,
            Keccak256("") para EOAs
```

Definición 1.139 (Storage Trie)**.** Cada contrato tiene su propio Storage Trie.
Clave: Keccak256(slot) donde slot es de 32 bytes **Valor:** RLP del valor (32 bytes)
Los slots se asignan según las reglas del compilador Solidity:

- Variables simples: slots consecutivos 0, 1, 2, ...
- Mappings: Keccak256(key‖slot)
- Arrays dinámicos: elementos en Keccak256(slot) + i

Definición 1.140 (Transaction Trie)**.** Contiene las transacciones de un bloque.
Clave: RLP(índice de transacción): 0, 1, 2, ... **Valor:** RLP(transacción)
Raíz incluida en block header como `transactionsRoot`.

Definición 1.141 (Receipt Trie)**.** Contiene los receipts de transacciones.
Clave: RLP(índice de transacción) **Valor:** RLP(receipt) con status, gasUsed, logs, etc.
Raíz incluida en block header como `receiptsRoot`.



1.6.12 Tamaño del Estado de Ethereum

Matemáticas

Estado de Ethereum (circa 2024):

Métrica	Valor aproximado
Cuentas únicas	∼250 millones
Contratos	∼50 millones
Tamaño del state trie	∼100 GB
Nodos en el trie	∼500 millones
Profundidad promedio	∼8 niveles
Profundidad máxima	64 nibbles (32 bytes)

El state trie es la principal razón por la que los nodos completos de Ethereum requieren $>$ 1 TB de almacenamiento.

1.6.13 Problemas y Limitaciones

Advertencia

1. State bloat:
El estado de Ethereum crece constantemente. Cada cuenta nueva, cada variable de storage, añade al trie permanentemente. No hay "garbage collection" automático.

2. Proof size:
Una prueba de inclusión/exclusión puede requerir varios KB de datos (múltiples nodos del camino).

3. Witness size para statelessness:
Para "stateless clients" (clientes que no almacenan el estado), cada transacción necesita incluir pruebas de todos los accesos al estado. Esto puede ser 100+ KB por transacción.

4. Complejidad de implementación:
La combinación de Patricia compression + hex-prefix + RLP + Merkleization hace

que las implementaciones sean complejas y propensas a bugs.

1.6.14 Verkle Trees: El Futuro

Definición 1.142 (Verkle Tree)**.** Verkle trees (Vector commitment Merkle trees) reemplazan hashes con **compromisos polinomiales** (usualmente KZG).

Ventajas sobre MPT:

- Proofs de tamaño constante (∼150 bytes vs ∼4 KB)
- Agregación de proofs
- Mejor para stateless clients

Observación 1.44. Ethereum tiene planeado migrar de MPT a Verkle trees en una actualización futura (probablemente post-2025). Esto:

1. Reducirá drásticamente el tamaño de los witnesses
2. Habilitará stateless clients prácticos
3. Requerirá una migración masiva del estado existente

1.6.15 Implementación

Ejemplo 1.73 (MPT simplificado en Python)**.**

```
from typing import Optional, List, Union
import hashlib

def keccak256(data: bytes) -> bytes:
    return hashlib.sha3_256(data).digest()

class MPT:
    def __init__(self):
        self.db = {}  # hash -> node
        self.root = None

    def _hash_node(self, node: bytes) -> bytes:
        if len(node) < 32:
            return node  # Inline
        h = keccak256(node)
        self.db[h] = node
        return h

    def _nibbles(self, key: bytes) -> List[int]:
        """Convert bytes to nibbles (hex digits)"""
        result = []
```

```
        for byte in key:
            result.append(byte >> 4)
            result.append(byte & 0x0f)
        return result

    def get(self, key: bytes) -> Optional[bytes]:
        """Get value for key, or None if not found"""
        if self.root is None:
            return None
        nibbles = self._nibbles(keccak256(key))
        return self._get(self.root, nibbles)

    def _get(self, node_ref, path: List[int]) -> Optional[bytes]:
        node = self._resolve(node_ref)
        if node is None:
            return None

        node_type, *data = self._decode_node(node)

        if node_type == 'branch':
            if not path:
                return data[16]  # Value at branch
            child = data[path[0]]
            if child is None:
                return None
            return self._get(child, path[1:])

        elif node_type == 'extension':
            ext_path, next_node = data
            if path[:len(ext_path)] != ext_path:
                return None
            return self._get(next_node, path[len(ext_path):])

        elif node_type == 'leaf':
            leaf_path, value = data
            if path == leaf_path:
                return value
            return None

    def _resolve(self, ref):
        """Resolve hash reference to node data"""
        if ref is None:
```

```
            return None
        if len(ref) < 32:
            return ref  # Inline node
        return self.db.get(ref)

    # put() and delete() omitted for brevity - they're complex!
```

Ejemplo 1.74 (Verificación de prueba en Solidity)**.** `// SPDX-License-Identifier: MIT`
`pragma solidity ^0.8.0;`

```
library MPTVerifier {
    // Verifica una prueba de inclusión en un MPT
    function verifyInclusionProof(
        bytes32 root,
        bytes memory key,
        bytes memory value,
        bytes[] memory proof
    ) internal pure returns (bool) {
        bytes32 keyHash = keccak256(key);
        bytes memory path = _toNibbles(keyHash);

        bytes32 currentHash = root;
        uint256 pathIndex = 0;

        for (uint256 i = 0; i < proof.length; i++) {
            bytes memory node = proof[i];

            // Verify current node hash
            if (keccak256(node) != currentHash &&
                node.length >= 32) {
                return false;
            }

            // Decode and traverse node
            // (simplified - real implementation needs RLP decoding)
            // ...
        }

        return true;
    }

    function _toNibbles(bytes32 data)
        internal pure returns (bytes memory)
```

```
    {
        bytes memory result = new bytes(64);
        for (uint256 i = 0; i < 32; i++) {
            result[i*2] = bytes1(uint8(data[i]) >> 4);
            result[i*2+1] = bytes1(uint8(data[i]) & 0x0f);
        }
        return result;
    }
}
```

1.6.16 Aplicación en Banco ZK

En Banco ZK

Banco ZK interactúa con MPT de Ethereum para verificar estado L1:

1. Verificación de depósitos

Cuando un usuario deposita en el contrato L1:

1. El evento Deposit se emite y va al Receipt Trie
2. El sequencer de L2 verifica el receipt proof
3. Una vez verificado, acredita fondos en L2

```
// Verificar que un depósito ocurrió
function verifyDeposit(
    bytes32 receiptRoot,    // Del block header
    bytes[] calldata proof, // MPT proof
    uint256 txIndex,
    address depositor,
    uint256 amount
) internal returns (bool) {
    // Decodificar receipt y verificar evento Deposit
    // ...
}
```

2. Verificación de estado para bridges

Para verificar balances o storage de contratos L1:

```
// Probar que una cuenta tiene cierto balance
struct AccountProof {
    bytes[] accountProof;  // Proof en state trie
    bytes32 stateRoot;     // Del block header
}

function verifyBalance(
```

```
    AccountProof calldata proof,
    address account,
    uint256 expectedBalance
) internal pure returns (bool) {
    // Verificar MPT proof
    // Decodificar Account struct
    // Comparar balance
}
```

3. Por qué NO usamos MPT internamente

Para el estado interno de L2, usamos Sparse Merkle Trees con Poseidon en lugar de MPT:

Aspecto	MPT (Ethereum)	SMT (Banco ZK)
Hash	Keccak-256	Poseidon
Costo en ZK	∼150K constraints/hash	∼250 constraints/hash
Prueba típica	∼4 KB	∼1 KB
Complejidad	Alta (3 tipos nodos)	Baja (solo branch/leaf)

4. Interoperabilidad L1 ↔ L2

```
L1 (Ethereum)                    L2 (Banco ZK)

  MPT State                      SMT State
  - Keccak                       - Poseidon
  - RLP                          - Simple


        Bridge

         MPT proofs verificados
         en circuito ZK (costoso)
```

Verificar MPT proofs en ZK es costoso (∼150K constraints por hash Keccak), pero solo lo hacemos para:

- Verificar depósitos desde L1
- Sincronizar con estado L1 cuando es necesario

El estado interno de L2 usa estructuras ZK-friendly.

5. Light client verification

Para que usuarios verifiquen el estado de L2 sin confiar en el sequencer:

1. Publicamos state root de L2 en contrato L1

2. Usuario puede pedir proof de su balance en L2
3. Proof se verifica contra state root publicado

Esto permite "exits" de emergencia si el sequencer se vuelve malicioso.

1.6.17 Resumen

Idea Clave

Merkle Patricia Tries: Puntos clave

1. **Combina tres conceptos:** Trie (búsqueda) + Patricia (compresión) + Merkle (hashing)
2. **Tres tipos de nodos:** Branch (16 hijos), Extension (prefijo compartido), Leaf (valor final)
3. **Cuatro tries en Ethereum:** State, Storage (por contrato), Transaction, Receipt
4. **Ventaja única:** Pruebas de inclusión Y exclusión
5. **Desventajas:** Proofs grandes, estado creciente, implementación compleja
6. **Futuro:** Verkle trees para proofs más pequeños y stateless clients
7. **En ZK:** Costoso verificar (Keccak), preferimos SMT con Poseidon para estado interno

1.7 Commitment Schemes

1.7.1 Qué son y para qué sirven

Definición 1.143 (Esquema de compromiso)**.** Protocolo de dos fases:

1. **Commit:** $c = \text{Commit}(m, r)$
2. **Open:** Revelar (m, r), verificar c

Propiedades:

- **Hiding:** c no revela información sobre m
- **Binding:** No se puede abrir a otro mensaje

Intuición: El Sobre Sellado

Intuición

Un commitment scheme es como poner un mensaje en un sobre sellado:

1. **Commit:** Escribes tu predicción, la metes en un sobre, lo sellas y lo entregas
2. **Open:** Más tarde, abres el sobre para revelar tu predicción

Propiedades del sobre:

- **Hiding:** Nadie puede ver a través del sobre (tu predicción está oculta)
- **Binding:** No puedes cambiar el contenido después de sellarlo (sin romperlo)

Los commitments criptográficos logran esto matemáticamente, sin sobres físicos.

Ejemplo 1.75 (Juego de "piedra, papel, tijera" remoto)**.** Alice y Bob quieren jugar piedra-papel-tijera por internet. Sin commitments:

- Si Alice envía primero, Bob puede ver y elegir para ganar
- Si envían simultáneamente, ¿cómo garantizarlo?

Con commitments:

1. Alice: $c_A = \text{Commit}(\text{“piedra”}, r_A)$, envía c_A
2. Bob: $c_B = \text{Commit}(\text{“papel”}, r_B)$, envía c_B
3. Alice revela: $(\text{“piedra”}, r_A)$
4. Bob revela: $(\text{“papel”}, r_B)$
5. Ambos verifican los commitments
6. Bob gana (papel vence piedra)

Ninguno puede hacer trampa: no pueden cambiar su elección después de ver el commitment del otro.

Definición Formal

Definición 1.144 (Commitment Scheme)**.** Un esquema de compromiso consiste en tres algoritmos:

Setup$() \to$ params: Genera parámetros públicos (si los hay).

Commit$(\text{params}, m) \to (c, d)$:

- Input: mensaje m del espacio de mensajes $\mathcal{M}$
- Output: commitment c y decommitment (opening) d

Verify$(\text{params}, c, m, d) \to \{0, 1\}$:

- Input: commitment c, mensaje m, decommitment d
- Output: 1 (aceptar) o 0 (rechazar)

Definición 1.145 (Propiedades de Seguridad)**. Hiding (Ocultamiento):**
Un commitment es *hiding* si el commitment c no revela información sobre m.

- **Perfectly hiding:** Incluso con poder computacional ilimitado, c no revela nada sobre m. Formalmente: $\forall m_0, m_1$, las distribuciones de Commit(m_0) y Commit(m_1) son idénticas.
- **Computationally hiding:** Un adversario PPT no puede distinguir Commit(m_0) de Commit(m_1) con ventaja no negligible.

Binding (Vinculante):
Un commitment es *binding* si no se puede abrir a dos mensajes diferentes.

- **Perfectly binding:** Incluso con poder ilimitado, no existe (m', d') con $m' \neq m$ tal que Verify$(c, m', d') = 1$.
- **Computationally binding:** Un adversario PPT no puede encontrar tal (m', d') con probabilidad no negligible.

Matemáticas

Teorema de imposibilidad:
Ningún esquema de compromiso puede ser simultáneamente **perfectly hiding** Y **perfectly binding**.
Demostración informal:

- Si es perfectly hiding, entonces para cada c, múltiples mensajes m son "compatibles" con c
- Pero entonces un adversario con poder ilimitado podría encontrar aperturas para diferentes mensajes
- Contradiciendo perfect binding

Por lo tanto, debemos elegir cuál propiedad es perfecta y cuál es computacional.

El Commitment más Simple: Hash

Definición 1.146 (Hash Commitment)**.** Usando una función hash H:

$$c = H(m \| r)$$

donde r es aleatoriedad (randomness) elegida uniformemente.

Idea Clave

Propiedades del hash commitment:

- **Computationally hiding:** Asumiendo H es un oráculo aleatorio, c no revela m (por r)
- **Computationally binding:** Encontrar $m' \neq m$ y r' con $H(m'\|r') = H(m\|r)$ requiere romper resistencia a colisiones

Ni perfectly hiding ni perfectly binding, pero ambas propiedades computacionalmente fuertes.

Ejemplo 1.76 (Hash commitment en la práctica)**.**

```
import hashlib
import secrets

def commit(message: bytes) -> tuple[bytes, bytes]:
    r = secrets.token_bytes(32)  # 256 bits de aleatoriedad
    c = hashlib.sha256(message + r).digest()
    return (c, r)  # commitment y decommitment

def verify(c: bytes, message: bytes, r: bytes) -> bool:
    return hashlib.sha256(message + r).digest() == c

# Uso
msg = b"Mi prediccion: Bitcoin a $100,000"
commitment, opening = commit(msg)

print(f"Commitment: {commitment.hex()[:16]}...")
# Más tarde...
assert verify(commitment, msg, opening)
```

Aplicaciones de Commitments

Idea Clave

Los commitments son un **building block** fundamental en criptografía:

Aplicación	Uso del commitment
Coin flipping	Comprometer a bit antes de revelar
Zero-knowledge proofs	Prover compromete a witness
Auction protocols	Comprometer a bid sellado
Voting	Comprometer a voto antes de contar
Timestamping	Comprometer a documento en blockchain
Confidential transactions	Comprometer a monto sin revelarlo

1.7.2 Pedersen Commitments

Definición 1.147 (Pedersen)**.** Con generadores g, h donde $\log_g h$ es desconocido:

$$C = g^m \cdot h^r$$

- **Perfectly hiding:** Para cualquier c y m', existe r'
- **Computationally binding:** Encontrar dos aperturas = DLP
- **Homomórfico:** $C_1 \cdot C_2 = \text{Commit}(m_1 + m_2, r_1 + r_2)$

Usado en: Confidential Transactions, Bulletproofs, ZK.

Contexto Histórico

Observación 1.45. Torben Pedersen propuso este esquema en 1991 en el paper "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing".

El esquema se basa en el trabajo previo de:

- Diffie-Hellman (1976): Problema del logaritmo discreto
- ElGamal (1985): Cifrado basado en DLP
- Feldman (1987): VSS verificable

La innovación de Pedersen fue combinar la estructura multiplicativa con dos generadores independientes para lograr perfect hiding.

Construcción Detallada

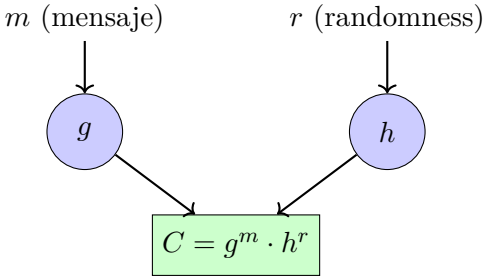
Definición 1.148 (Pedersen Commitment - Formal)**. Setup:**

1. Elegir grupo cíclico $\mathbb{G}$ de orden primo q
2. Elegir dos generadores $g, h \in \mathbb{G}$ tales que $\log_g h$ es desconocido
3. Publicar $(q, \mathbb{G}, g, h)$

Commit(m, r) donde $m \in \mathbb{Z}_q$ y $r \xleftarrow{\$} \mathbb{Z}_q$:

$$C = g^m \cdot h^r$$

Open: Revelar (m, r)
Verify(C, m, r): Aceptar si $C = g^m \cdot h^r$



Análisis de Seguridad

Matemáticas

Perfect Hiding:
Para cualquier commitment C y cualquier mensaje objetivo m', existe un r' tal que $C = g^{m'}h^{r'}$.
Demostración:

$$C = g^m h^r \tag{1.185}$$
$$g^{m'}h^{r'} = C \tag{1.186}$$
$$h^{r'} = C \cdot g^{-m'} = g^m h^r \cdot g^{-m'} = g^{m-m'}h^r \tag{1.187}$$
$$r' = r + (m - m') \cdot \log_g h \tag{1.188}$$

Para cada m', existe exactamente un r' que satisface la ecuación. Por lo tanto, viendo solo C, todos los mensajes son igualmente probables.
Nota: Esto asume que no conocemos $\log_g h$. Si lo conociéramos, podríamos calcular r', pero aún no podríamos determinar el m original solo viendo C.

Matemáticas

Computational Binding:
Supongamos que un adversario encuentra (m_1, r_1) y (m_2, r_2) con $m_1 \neq m_2$ tales que:

$$g^{m_1}h^{r_1} = g^{m_2}h^{r_2}$$

Entonces:

$$g^{m_1-m_2} = h^{r_2-r_1} \tag{1.189}$$
$$g^{(m_1-m_2)/(r_2-r_1)} = h \tag{1.190}$$

Por lo tanto: $\log_g h = (m_1 - m_2)/(r_2 - r_1)$
¡El adversario ha resuelto el DLP entre g y h! Si el DLP es difícil en $\mathbb{G}$, esto es computacionalmente imposible.

La Propiedad Homomórfica

Idea Clave

El superpoder de Pedersen: Los commitments son **aditivamente homomórficos**.

$$C_1 \cdot C_2 = (g^{m_1}h^{r_1}) \cdot (g^{m_2}h^{r_2}) = g^{m_1+m_2}h^{r_1+r_2} = \text{Commit}(m_1 + m_2, r_1 + r_2)$$

Podemos sumar commitments sin conocer los mensajes originales.

Ejemplo 1.77 (Verificación de sumas sin revelar valores)**. Escenario:** Alice tiene 3 valores secretos que suman 100. Quiere probar esto sin revelar los valores individuales.

```
Valores secretos: a=30, b=25, c=45
Randomness: r_a, r_b, r_c (elegidos aleatoriamente)

Commitments:
C_a = g^30 · h^(r_a)
C_b = g^25 · h^(r_b)
C_c = g^45 · h^(r_c)

Alice publica: C_a, C_b, C_c

Verificador calcula:
C_total = C_a · C_b · C_c = g^(30+25+45) · h^(r_a+r_b+r_c)
        = g^100 · h^(r_total)

Alice revela: r_total = r_a + r_b + r_c

Verificador comprueba:
C_total ?= g^100 · h^(r_total)

Si coincide, la suma es 100 (sin saber los valores individuales).
```

Pedersen en Notación Aditiva (Curvas Elípticas)

Definición 1.149 (Pedersen sobre Curvas Elípticas)**.** En una curva elíptica con puntos base G, H donde $\log_G H$ es desconocido:

$$C = m \cdot G + r \cdot H$$

Es la misma construcción, pero en notación aditiva.

Observación 1.46. En la práctica, casi siempre usamos curvas elípticas para Pedersen porque:

1. Claves más cortas para mismo nivel de seguridad
2. Operaciones más rápidas
3. Compatibilidad con otros protocolos (ECDSA, Schnorr)

Generación de G y H

Advertencia

Crítico: Nadie debe conocer $\log_G H$.
Si alguien conoce x tal que $H = xG$:

$$C = mG + rH = mG + r(xG) = (m + rx)G \tag{1.191}$$

Puede abrir el commitment a cualquier m' usando:

$$r' = (m - m' + rx)/x$$

$$C = m'G + r'H = m'G + r'(xG) = (m' + r'x)G = (m + rx)G \quad \checkmark$$

¡El commitment ya no es binding!

Idea Clave

Métodos para generar H de forma segura:

1. Hash-to-curve (Nothing-up-my-sleeve):

$$H = \text{HashToCurve}(\text{“Pedersen generator H”})$$

Nadie puede conocer $\log_G H$ porque tendría que invertir el hash.

2. Trusted setup con múltiples partes:

Cada parte i elige x_i secreto:

$$H = \sum_i x_i \cdot G$$

Si al menos una parte es honesta y descarta su x_i, nadie conoce $\log_G H$.

3. Usar punto estándar:

Curvas como secp256k1 tienen un segundo generador estándar derivado de forma verificable.

Ejemplo Numérico

Ejemplo 1.78 (Pedersen sobre curva pequeña)**. Parámetros (toy example):**

- Grupo: $\mathbb{Z}_{23}^*$ (multiplicativo módulo 23)
- $g = 5$ (generador)
- $h = 7$ (segundo generador, $\log_5 7$ desconocido)
- Orden: $q = 22$

Commit al mensaje $m = 10$ con $r = 15$:

$$\begin{aligned} C &= g^m \cdot h^r \mod 23 && (1.192)\\ &= 5^{10} \cdot 7^{15} \mod 23 && (1.193)\\ &= 9765625 \cdot 4747561509943 \mod 23 && (1.194)\\ &\equiv 9 \cdot 12 \mod 23 && (1.195)\\ &= 108 \mod 23 && (1.196)\\ &= 16 && (1.197) \end{aligned}$$

Verificación:

$$5^{10} \cdot 7^{15} \mod 23 = 16 \quad \checkmark$$

Perfect hiding: Para $m' = 5$, ¿existe r'?

$$16 = 5^5 \cdot 7^{r'} \mod 23 \tag{1.198}$$
$$16 = 3125 \cdot 7^{r'} \mod 23 \tag{1.199}$$
$$16 = 20 \cdot 7^{r'} \mod 23 \tag{1.200}$$
$$7^{r'} = 16 \cdot 20^{-1} = 16 \cdot 15 = 240 \equiv 10 \mod 23 \tag{1.201}$$

Buscamos r' tal que $7^{r'} \equiv 10 \pmod{23}$. Por prueba: $7^{12} = 10 \mod 23$.

Por lo tanto, $(m', r') = (5, 12)$ también abre al commitment 16.

¡Esto demuestra perfect hiding! Pero encontrar este r' requirió resolver DLP (computationally hard en grupos grandes).

Vector Pedersen Commitments

Definición 1.150 (Vector Pedersen)**.** Para comprometer a un vector $\vec{m} = (m_1, \ldots, m_n)$:

$$C = \sum_{i=1}^{n} m_i \cdot G_i + r \cdot H$$

donde $G_1, \ldots, G_n, H$ son generadores independientes.

Idea Clave

Los vector Pedersen commitments son fundamentales para:

- **Bulletproofs**: Prueban que un valor está en un rango $[0, 2^n)$
- **Inner Product Arguments**: Prueban $\langle \vec{a}, \vec{b} \rangle = c$
- **Polynomial commitments**: Comprometer a coeficientes de polinomio

Aplicaciones

Observación 1.47. **Confidential Transactions (Monero, Mimblewimble):**

En lugar de montos públicos, cada output tiene un Pedersen commitment al monto:

$$C = m \cdot G + r \cdot H$$

Propiedad clave: Se puede verificar que inputs = outputs sin conocer montos:

$$\sum C_{\text{inputs}} - \sum C_{\text{outputs}} = 0 \cdot G + r_{\text{fee}} \cdot H$$

Solo funciona si la suma de montos es correcta (homomorfismo).

Problema: Sin restricciones adicionales, alguien podría crear monto negativo.

Solución: Range proofs (Bulletproofs) prueban $m \in [0, 2^{64})$.

1.7.3 KZG Commitments

Definición 1.151 (KZG)**.** Compromiso a polinomio con evaluación verificable.

Setup: $\{g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d}\}$ donde τ es secreto (toxic waste).

- Commit: $C = g^{p(\tau)} = \prod_i (g^{\tau^i})^{a_i}$
- Evaluación: Probar $p(z) = y$ con prueba $\pi = g^{q(\tau)}$
- Donde $q(x) = \frac{p(x)-y}{x-z}$
- Verificar: $e(C - g^y, g) = e(\pi, g^{\tau} - g^z)$

Tamaño de prueba: 1 elemento de grupo (48 bytes en BLS12-381).

Contexto y Motivación

Intuición

KZG permite hacer algo mágico: comprometer a un polinomio entero en un solo punto de curva, y después probar evaluaciones individuales sin revelar el polinomio completo. Es como tener una "caja negra" que contiene un polinomio $p(x)$:

- La caja es pequeña (48 bytes)
- Puedes probar "mi caja contiene un polinomio donde $p(5) = 17$"
- La prueba también es pequeña (48 bytes)
- Sin revelar ningún otro punto del polinomio

Observación 1.48. **Kate-Zaverucha-Goldberg (2010):**

Aniket Kate, Gregory Zaverucha, y Ian Goldberg propusieron este esquema en "Constant-Size Commitments to Polynomials and Their Applications".

El paper original lo llamaba "polynomial commitment scheme", pero la comunidad lo nombró KZG por los autores.

Adopción:

Protocolo	**Uso de KZG**
PLONK	Commitment a polinomios del circuito
Groth16	Verificación de polinomios
Ethereum (Danksharding)	Data availability sampling
Scroll, Polygon zkEVM	zkRollups basados en PLONK

El Setup: Structured Reference String (SRS)

Definición 1.152 (Trusted Setup KZG)**.** Para polinomios de grado hasta d, el setup genera:

En $\mathbb{G}_1$:

$$\{g_1, g_1^{\tau}, g_1^{\tau^2}, \ldots, g_1^{\tau^d}\}$$

En $\mathbb{G}_2$:

$$\{g_2, g_2^{\tau}\}$$

donde $\tau \xleftarrow{\$} \mathbb{F}_p$ es el "toxic waste" que debe destruirse.

Advertencia

El Toxic Waste:
Si alguien conoce τ, puede:

1. Crear commitments falsos a cualquier polinomio
2. Crear pruebas de evaluación falsas
3. Romper completamente la seguridad del sistema

Por esto se llama "toxic waste": debe destruirse inmediatamente después del setup.

Idea Clave

Ceremonia de Powers of Tau:
Para generar el SRS sin que nadie conozca τ:

1. Participante 1 elige τ_1, computa $(g^{\tau_1}, g^{\tau_1^2}, \ldots)$, destruye τ_1
2. Participante 2 recibe, elige τ_2, computa $(g^{\tau_1\tau_2}, g^{(\tau_1\tau_2)^2}, \ldots)$, destruye τ_2
3. ...continúa con n participantes

El τ final es $\tau = \tau_1 \cdot \tau_2 \cdots \tau_n$.
Seguridad: Si AL MENOS UN participante es honesto y destruye su τ_i, nadie conoce τ completo.
Ceremonias famosas:

- Zcash Powers of Tau (2018): 87 participantes
- Ethereum KZG Ceremony (2023): 141,000+ participantes

El Algoritmo KZG

Definición 1.153 (KZG Commitment Scheme - Completo)**. Commit**$(p(x))$:
Dado polinomio $p(x) = \sum_{i=0}^{d} a_i x^i$:

$$C = g_1^{p(\tau)} = \prod_{i=0}^{d} (g_1^{\tau^i})^{a_i}$$

Nota: Calculamos $g_1^{p(\tau)}$ sin conocer τ, usando el SRS.
CreateProof$(p(x), z)$ para probar $p(z) = y$:

1. Calcular $y = p(z)$

2. Calcular polinomio cociente: $q(x) = \frac{p(x)-y}{x-z}$

   (Esto es exactamente divisible porque $p(z) = y$ implica $(x - z)|(p(x) - y)$)

3. Computar prueba: $\pi = g_1^{q(\tau)}$

Verify(C, z, y, π):
Verificar la ecuación de pairing:

$$e(C - g_1^y, g_2) = e(\pi, g_2^\tau - g_2^z)$$

O equivalentemente:

$$e(C - g_1^y, g_2) = e(\pi, g_2^\tau \cdot g_2^{-z})$$

¿Por Qué Funciona?

Matemáticas

Verificación de correctitud:
Queremos verificar que $p(z) = y$ usando el pairing.
Lado izquierdo:

$$e(C - g_1^y, g_2) = e(g_1^{p(\tau)} \cdot g_1^{-y}, g_2) = e(g_1^{p(\tau)-y}, g_2) = e(g_1, g_2)^{p(\tau)-y}$$

Lado derecho:

$$e(\pi, g_2^\tau - g_2^z) = e(g_1^{q(\tau)}, g_2^{\tau-z}) \tag{1.202}$$
$$= e(g_1, g_2)^{q(\tau)\cdot(\tau-z)} \tag{1.203}$$

La clave: Si $p(z) = y$, entonces:

$$p(x) - y = q(x) \cdot (x - z)$$

Evaluando en $x = \tau$:

$$p(\tau) - y = q(\tau) \cdot (\tau - z)$$

Por lo tanto, ambos lados del pairing son iguales:

$$e(g_1, g_2)^{p(\tau)-y} = e(g_1, g_2)^{q(\tau)(\tau-z)} \quad \checkmark$$

Ejemplo Numérico

Ejemplo 1.79 (KZG simplificado)**. Polinomio:** $p(x) = 2x^2 + 3x + 1$
Queremos probar: $p(2) = 2(4) + 3(2) + 1 = 8 + 6 + 1 = 15$
Paso 1: Commitment
Usando SRS $\{g, g^\tau, g^{\tau^2}\}$:

$$C = g^{p(\tau)} = g^{2\tau^2+3\tau+1} = (g^{\tau^2})^2 \cdot (g^\tau)^3 \cdot g^1$$

Paso 2: Polinomio cociente

$$q(x) = \frac{p(x) - 15}{x - 2} \tag{1.204}$$

$$= \frac{2x^2 + 3x + 1 - 15}{x - 2} \tag{1.205}$$

$$= \frac{2x^2 + 3x - 14}{x - 2} \tag{1.206}$$

División polinómica:

$$2x^2 + 3x - 14 = (x - 2)(2x + 7)$$

Verificación: $(x - 2)(2x + 7) = 2x^2 + 7x - 4x - 14 = 2x^2 + 3x - 14$
Por lo tanto: $q(x) = 2x + 7$

Paso 3: Prueba

$$\pi = g^{q(\tau)} = g^{2\tau+7} = (g^\tau)^2 \cdot g^7$$

Paso 4: Verificación

El verificador comprueba:

$$e(C \cdot g^{-15}, g) \stackrel{?}{=} e(\pi, g^\tau \cdot g^{-2})$$

Ambos lados evalúan a $e(g, g)^{q(\tau)(\tau-2)} = e(g, g)^{p(\tau)-15}$.

Propiedades de KZG

Propiedad	KZG
Tamaño de commitment	1 elemento $\mathbb{G}_1$ (48 bytes)
Tamaño de prueba (1 evaluación)	1 elemento $\mathbb{G}_1$ (48 bytes)
Tiempo de verificación	2 pairings
Hiding	Computationally hiding
Binding	Computationally binding (bajo q-SDH)
Homomórfico	Sí (aditivo)
Trusted setup	Sí (universal, actualizable)

Idea Clave

Homomorfismo de KZG:
Si $C_1 = \text{Commit}(p_1)$ y $C_2 = \text{Commit}(p_2)$:

$$C_1 \cdot C_2 = g^{p_1(\tau)} \cdot g^{p_2(\tau)} = g^{(p_1+p_2)(\tau)} = \text{Commit}(p_1 + p_2)$$

Podemos combinar commitments a polinomios sin conocer los polinomios.

Batch Opening

Definición 1.154 (Batch Opening)**.** Para probar múltiples evaluaciones $(z_1, y_1), \ldots, (z_k, y_k)$ del mismo polinomio:

Opción 1: Pruebas individuales k pruebas de 48 bytes cada una = $48k$ bytes

Opción 2: Batch proof Una sola prueba de 48 bytes (con técnicas de agregación)

Observación 1.49. El batch opening es crucial para eficiencia en SNARKs como PLONK, donde se verifican muchas evaluaciones simultáneamente.

KZG vs Otros Polynomial Commitments

Propiedad	KZG	FRI	Bulletproofs
Tamaño prueba	$O(1)$	$O(\log^2 d)$	$O(\log d)$
Tiempo verificación	$O(1)$ pairings	$O(\log^2 d)$ hashes	$O(d)$
Trusted setup	Sí	No	No
Quantum-safe	No	Sí	No
Asunción	q-SDH	Hash collision	DLP

Idea Clave

Cuándo usar cada uno:

- **KZG**: Cuando se pueden tolerar trusted setups y se necesita máxima eficiencia (PLONK, Danksharding)
- **FRI**: Cuando se necesita transparencia (sin trusted setup) y post-quantum es importante (STARKs)
- **Bulletproofs**: Para range proofs y cuando el tamaño de prueba importa más que el tiempo de verificación

KZG en Ethereum (Danksharding)

Definición 1.155 (Proto-Danksharding (EIP-4844))**.** Ethereum usa KZG para "blob transactions":

1. Rollups publican datos como "blobs" (∼128 KB cada uno)
2. Cada blob se compromete con KZG
3. El commitment va en el header del bloque
4. Los datos completos se "podan" después de ∼2 semanas

Observación 1.50. **Data Availability Sampling (futuro):**
Con KZG, los light clients podrán verificar que los datos están disponibles sin descargarlos:

1. Solicitar evaluaciones aleatorias del polinomio blob
2. Verificar proofs KZG de esas evaluaciones
3. Con suficientes muestras válidas, hay alta confianza de que los datos existen

Esto permite escalar Ethereum sin que cada nodo descargue todos los datos.

Implementación

Ejemplo 1.80 (KZG con arkworks (Rust)). `use ark_bls12_381::{Bls12_381, Fr, G1Projective};`

```
use ark_poly::{univariate::DensePolynomial, UVPolynomial};
use ark_poly_commit::kzg10::{KZG10, Powers};

fn main() {
    // Setup (en producción, usar ceremonia)
    let rng = &mut ark_std::test_rng();
    let degree = 10;
    let params = KZG10::<Bls12_381, DensePolynomial<Fr>>::setup(
        degree, false, rng
    ).unwrap();

    // Polinomio: p(x) = 2x^2 + 3x + 1
    let poly = DensePolynomial::from_coefficients_vec(vec![
        Fr::from(1u64),  // constante
        Fr::from(3u64),  // coef de x
        Fr::from(2u64),  // coef de x^2
    ]);

    // Commit
    let (commitment, randomness) = KZG10::commit(&params, &poly, None, None)
        .unwrap();

    // Evaluar y probar en z = 2
    let z = Fr::from(2u64);
    let y = poly.evaluate(&z);  // p(2) = 15

    let proof = KZG10::open(&params, &poly, z, &randomness).unwrap();

    // Verificar
    let valid = KZG10::check(&params.vk, &commitment, z, y, &proof).unwrap();
    assert!(valid);

    println!("p(2) = {} verificado!", y);
}
```

1.7.4 Comparación de Commitment Schemes

Propiedad	Hash	Pedersen	KZG
Tamaño commitment	32 B	32-48 B	48 B
Hiding	Comput.	Perfect	Comput.
Binding	Comput.	Comput.	Comput.
Homomórfico	No	Sí (aditivo)	Sí (aditivo)
Trusted setup	No	No	Sí
Evaluación verificable	No	No	Sí
Velocidad commit	Muy rápido	Rápido	Medio

1.7.5 Aplicación en Banco ZK

En Banco ZK

Los commitment schemes son fundamentales en toda la arquitectura de Banco ZK:

1. Note Commitments (Pedersen)

Cada "nota" (output de transacción) se compromete:

$$C_{\text{note}} = \text{Pedersen}(\text{value}, \text{pubkey}, \text{salt})$$

```
struct Note {
    value: u64,      // Monto en wei
    pubkey: Point,   // Clave pública del dueño
    salt: Field,     // Aleatoriedad
}

commitment = value * G_value + pubkey.x * G_pk + salt * H
```

¿Por qué Pedersen?

- Perfect hiding: Nadie puede saber el monto viendo C
- Homomórfico: Verificar suma de inputs = suma de outputs
- Eficiente en circuitos ZK

2. Nullifier Derivation

Para gastar una nota sin revelar cuál:

$$\text{nullifier} = \text{Poseidon}(C_{\text{note}}, \text{spending_key})$$

El nullifier es público (para prevenir doble gasto), pero no revela qué nota se gastó.

3. KZG para PLONK Proofs

Nuestro sistema SNARK usa PLONK, que internamente usa KZG:

```
Circuito de transacción:
```

```
1. Verificar Merkle proof de nota
2. Verificar nullifier correcto
3. Verificar firma
4. Verificar conservación de valor

PLONK convierte esto en:
- Polinomios de constraints
- KZG commitments a esos polinomios
- Prueba final: ~700 bytes
```

4. Confidential Transactions

Los montos permanecen ocultos usando Pedersen:

```
Input 1:  C_1 = 50 * G + r_1 * H
Input 2:  C_2 = 30 * G + r_2 * H
Output 1: C_3 = 70 * G + r_3 * H
Output 2: C_4 = 10 * G + r_4 * H  (fee)

Verificación (homomorfismo):
C_1 + C_2 - C_3 - C_4
  = (50+30-70-10)*G + (r_1+r_2-r_3-r_4)*H
  = 0*G + r_diff*H

Prover revela r_diff para verificar suma = 0.
```

5. Range Proofs

Para evitar montos negativos (que romperían la conservación), incluimos range proofs:

```
Para cada output con commitment C:
1. Probar que value  [0, 2^64)
2. Usamos Bulletproofs sobre el Pedersen commitment
3. O verificación dentro del circuito SNARK
```

6. Data Blobs (futuro)

Con EIP-4844, publicaremos datos de batch como blobs:

```
Batch de 1000 transacciones:
1. Serializar transacciones → ~128 KB
2. Codificar como polinomio
3. KZG commitment publicado on-chain
4. Datos disponibles por 2 semanas para verificadores
5. Después se prunan (solo commitment permanece)
```

7. Resumen de uso

Componente	Commitment	Razón
Note values	Pedersen	Perfect hiding, homomórfico
Merkle tree nodes	Poseidon hash	ZK-friendly
SNARK proofs	KZG (PLONK)	Tamaño constante
Data availability	KZG (blobs)	DAS eficiente
Addresses	Hash commitment	Simple, binding

1.7.6 Resumen

Idea Clave

Commitment Schemes: Puntos clave

1. **Dos propiedades:** Hiding (oculta mensaje) y Binding (no se puede cambiar)
2. **Trade-off:** No pueden ser perfectamente ambas simultáneamente
3. **Hash commitments:** Simples, pero no homomórficos
4. **Pedersen:** Perfect hiding, homomórfico, ideal para valores confidenciales
5. **KZG:** Compromete polinomios, pruebas de evaluación en tamaño constante, requiere trusted setup
6. **Aplicaciones:** ZK proofs, confidential transactions, data availability, voting, auctions

1.7.7 Polynomial Commitments

Generalización de compromisos a polinomios:

Esquema	Prueba	Verificación	Setup
KZG	$O(1)$	$O(1)$ pairing	Trusted
IPA	$O(\log n)$	$O(n)$	Ninguno
FRI	$O(\log^2 n)$	$O(\log^2 n)$	Ninguno

¿Por Qué Polinomios?

Intuición

Los polinomios son el "lenguaje universal" de los SNARKs modernos. Casi todos los sistemas de pruebas convierten el problema a verificar en ecuaciones polinomiales.

¿Por qué?

1. **Expresividad**: Cualquier circuito se puede representar como restricciones polinomiales

2. **Schwartz-Zippel**: Si dos polinomios diferentes de grado d coinciden en un punto aleatorio, la probabilidad es $\leq d/|F|$ (negligible)
3. **Compresión**: Un polinomio de grado n se puede "resumir" evaluándolo en un punto
4. **Homomorfismo**: Muchos esquemas permiten operar sobre commitments

El polynomial commitment es la herramienta que permite verificar propiedades de polinomios sin revelarlos.

Idea Clave

El paradigma moderno de SNARKs:

1. Convertir el problema en restricciones polinomiales
2. El prover compromete a los polinomios (usando PCS)
3. El verificador pide evaluaciones en puntos aleatorios
4. El prover proporciona evaluaciones con pruebas
5. Por Schwartz-Zippel, si las evaluaciones son correctas, los polinomios probablemente son correctos

El **Polynomial Commitment Scheme (PCS)** es el componente que hace posible los pasos 2-4.

Definición Formal

Definición 1.156 (Polynomial Commitment Scheme)**.** Un PCS consiste en los siguientes algoritmos:

Setup$(1^\lambda, d) \to$ pp:

- Input: parámetro de seguridad λ, grado máximo d
- Output: parámetros públicos pp

Commit$(\text{pp}, p(x)) \to (C, \text{aux})$:

- Input: polinomio $p(x)$ de grado $\leq d$
- Output: commitment C y datos auxiliares (opcionales)

Open$(\text{pp}, p(x), z, \text{aux}) \to \pi$:

- Input: polinomio, punto de evaluación z
- Output: prueba π de que $p(z) = y$

Verify$(\text{pp}, C, z, y, \pi) \to \{0, 1\}$:

- Input: commitment, punto, valor declarado, prueba
- Output: aceptar o rechazar

Definición 1.157 (Propiedades de Seguridad)**. Correctitud:** Si $p(z) = y$, entonces Verify acepta.

Binding: Es computacionalmente difícil encontrar C, z, y, y', π, π' con $y \neq y'$ tales que Verify acepta ambos (C, z, y, π) y (C, z, y', π').

Hiding (opcional): El commitment C no revela información sobre $p(x)$ más allá de lo que revelan las evaluaciones abiertas.

KZG (Kate-Zaverucha-Goldberg)

Observación 1.51. KZG fue cubierto en detalle en la sección anterior. Aquí resumimos sus características clave como PCS.

Definición 1.158 (KZG como PCS)**. Setup:** Trusted setup genera $\{g^{\tau^i}\}_{i=0}^{d}$ (Powers of Tau)
Commit: $C = g^{p(\tau)} = \prod_i (g^{\tau^i})^{a_i}$ para $p(x) = \sum_i a_i x^i$
Open en z: Calcular $q(x) = \frac{p(x)-p(z)}{x-z}$, retornar $\pi = g^{q(\tau)}$
Verify: Comprobar $e(C - g^{p(z)}, g) = e(\pi, g^{\tau} - g^{z})$

Aspecto	**KZG**
Tamaño commitment	48 bytes (1 elem. $\mathbb{G}_1$)
Tamaño prueba	48 bytes
Tiempo commit	$O(n)$ multiplicaciones
Tiempo prove	$O(n)$ multiplicaciones
Tiempo verify	$O(1)$ (2 pairings)
Setup	Trusted (ceremony)
Asunción	q-SDH
Post-quantum	No

Idea Clave

Fortalezas de KZG:

- Pruebas de tamaño constante (¡48 bytes para cualquier grado!)
- Verificación en tiempo constante
- Batch opening eficiente
- Homomórfico (se pueden combinar commitments)

Debilidades:

- Requiere trusted setup
- No es post-quantum (vulnerable a computadores cuánticos)
- Setup es específico para grado máximo

IPA (Inner Product Argument)

Definición 1.159 (Inner Product Argument)**.** IPA es un polynomial commitment scheme basado en el "inner product argument" de Bulletproofs. No requiere trusted setup ni pairings.

Idea central: Reducir recursivamente un inner product de tamaño n a uno de tamaño $n/2$, hasta llegar a tamaño 1.

Intuición

Imagina que quieres probar que conoces vectores $\vec{a}, \vec{b}$ tales que $\langle \vec{a}, \vec{b} \rangle = c$.
En lugar de enviar los vectores completos ($O(n)$ datos), IPA permite:

1. Comprimir a vectores de tamaño $n/2$
2. Repetir recursivamente $\log n$ veces
3. Prueba final de tamaño $O(\log n)$

Para polynomial commitments: representamos $p(x)$ como inner product de coeficientes con potencias de x.

Matemáticas

El protocolo IPA (simplificado):
Setup: Generadores $\vec{G} = (G_1, \ldots, G_n)$, $\vec{H} = (H_1, \ldots, H_n)$, y Q (para binding).
Commitment a polinomio $p(x) = \sum_i a_i x^i$**:**

$$C = \langle \vec{a}, \vec{G} \rangle = \sum_i a_i G_i$$

Para probar evaluación $p(z) = y$**:**
Sea $\vec{b} = (1, z, z^2, \ldots, z^{n-1})$. Entonces $p(z) = \langle \vec{a}, \vec{b} \rangle = y$.
Reducción recursiva:
Dividir $\vec{a} = (\vec{a}_L, \vec{a}_R)$ y $\vec{b} = (\vec{b}_L, \vec{b}_R)$ en mitades.
Calcular cross-terms:

$$L = \langle \vec{a}_L, \vec{G}_R \rangle + \langle \vec{a}_L, \vec{b}_R \rangle \cdot Q \tag{1.207}$$
$$R = \langle \vec{a}_R, \vec{G}_L \rangle + \langle \vec{a}_R, \vec{b}_L \rangle \cdot Q \tag{1.208}$$

Verificador envía challenge aleatorio x.
Nuevos vectores:

$$\vec{a}' = \vec{a}_L + x \cdot \vec{a}_R \tag{1.209}$$
$$\vec{b}' = \vec{b}_L + x^{-1} \cdot \vec{b}_R \tag{1.210}$$
$$\vec{G}' = \vec{G}_L + x^{-1} \cdot \vec{G}_R \tag{1.211}$$

Repetir hasta $n = 1$.

Prueba final: $(L_1, R_1, L_2, R_2, \ldots, L_{\log n}, R_{\log n}, a_{\text{final}})$

Aspecto	IPA
Tamaño commitment	32 bytes (1 elem. grupo)
Tamaño prueba	$O(\log n)$ elementos
Tiempo commit	$O(n)$
Tiempo prove	$O(n)$
Tiempo verify	$O(n)$ (!!)
Setup	Ninguno (transparent)
Asunción	DLP
Post-quantum	No

Advertencia

El problema del tiempo de verificación:
IPA tiene verificación en tiempo $O(n)$, no $O(\log n)$. Aunque la prueba es logarítmica, el verificador debe reconstruir los generadores $\vec{G}'$ en cada ronda.
Soluciones:

1. Usar "inner product argument" para verificar múltiples evaluaciones en batch
2. Halo-style accumulation: diferir la verificación costosa
3. Precomputar tablas de generadores

Idea Clave

Fortalezas de IPA:

- Sin trusted setup (completamente transparent)
- Pruebas razonablemente pequeñas ($O(\log n)$)
- Simple de implementar
- Base para Bulletproofs y Halo

Debilidades:

- Verificación $O(n)$ (costosa)
- No es post-quantum
- Menos eficiente que KZG para verificación individual

Ejemplo 1.81 (IPA en Halo 2)**.** Halo 2 (usado por Zcash y protocolos como Scroll) usa IPA con una técnica de "accumulation":

1. En lugar de verificar completamente cada prueba, se "acumula" el trabajo

2. Múltiples pruebas se combinan en un "acumulador"
3. Solo al final se hace la verificación costosa una vez

Esto amortiza el costo $O(n)$ sobre muchas pruebas.

FRI (Fast Reed-Solomon IOP)

Definición 1.160 (FRI)**.** Fast Reed-Solomon Interactive Oracle Proof (FRI) es un protocolo para probar que un commitment corresponde a la evaluación de un polinomio de bajo grado.

Diferencia clave: FRI trabaja con evaluaciones del polinomio sobre un dominio, no con coeficientes.

Intuición

FRI opera de forma diferente a KZG e IPA:
KZG/IPA: "He aquí un commitment a los *coeficientes* de mi polinomio"
FRI: "He aquí las *evaluaciones* de mi polinomio en muchos puntos. Puedo probar que son consistentes con un polinomio de grado $< d$"
FRI usa la propiedad de que los códigos Reed-Solomon tienen máxima distancia: un polinomio de grado d evaluado en $n \gg d$ puntos tiene una "firma" única.

Matemáticas

El protocolo FRI (simplificado):
Setup: Campo $\mathbb{F}$, dominio de evaluación D de tamaño $n = 2^k$, grado objetivo $d < n$.
Commit a polinomio $p(x)$ de grado $< d$:

1. Evaluar p en todo D: $(p(\omega^0), p(\omega^1), \ldots, p(\omega^{n-1}))$ donde ω es raíz de unidad
2. Construir Merkle tree de las evaluaciones
3. Commitment = raíz del Merkle tree

Protocolo de reducción (query phase):
Ronda 1:

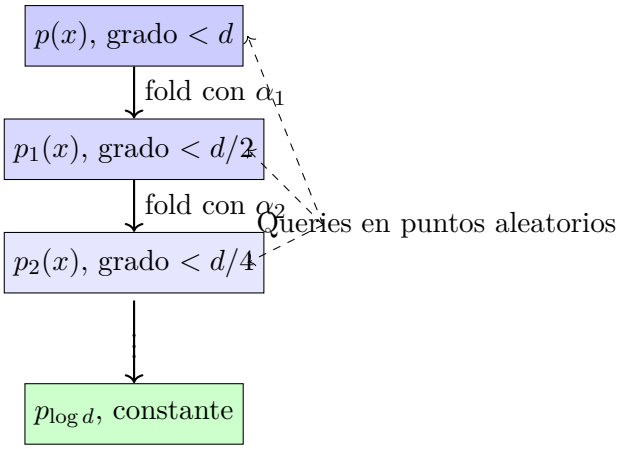
1. Verificador envía challenge aleatorio α_1
2. Prover construye $p_1(x)$ de grado $< d/2$ tal que:

$$p(x) = p_1(x^2) + x \cdot p_1^{\text{odd}}(x^2)$$

   donde "fold" combina coeficientes pares e impares usando α_1
3. Prover envía commitment a evaluaciones de p_1

Rondas 2 a $\log(d)$: Repetir, reduciendo grado a la mitad cada vez.
Ronda final: El polinomio tiene grado constante; prover lo envía directamente.
Verificación (query phase):

1. Verificador elige puntos aleatorios $z_1, \ldots, z_\lambda$
2. Para cada z_i, pide evaluaciones y Merkle proofs
3. Verifica consistencia de las reducciones



Aspecto	**FRI**
Tamaño commitment	$O(1)$ (raíz Merkle)
Tamaño prueba	$O(\lambda \cdot \log^2 n)$
Tiempo commit	$O(n \log n)$ FFT
Tiempo prove	$O(n \log n)$
Tiempo verify	$O(\lambda \cdot \log^2 n)$
Setup	Ninguno (transparent)
Asunción	Hash collision resistance
Post-quantum	Sí (!)

Idea Clave

Fortalezas de FRI:

- **Post-quantum**: Solo usa hashes, sobrevive a computadores cuánticos
- Sin trusted setup (completamente transparent)
- Verificación polylogarítmica
- Base para STARKs

Debilidades:

- Pruebas más grandes que KZG (∼100 KB vs 48 bytes)
- Más costoso computacionalmente
- Complejidad de implementación

Observación 1.52. **STARKs = FRI + AIR:**

Los STARKs (Scalable Transparent ARguments of Knowledge) combinan:

- **AIR** (Algebraic Intermediate Representation): El sistema de constraints
- **FRI**: El polynomial commitment scheme

Sistemas como StarkWare, Cairo, y Polygon Miden usan STARKs basados en FRI.

Comparación Detallada

Propiedad	KZG	IPA	FRI
Tamaño prueba	$O(1)$	$O(\log n)$	$O(\log^2 n)$
Verificación	$O(1)$	$O(n)$	$O(\log^2 n)$
Trusted setup	Sí	No	No
Post-quantum	No	No	Sí
Basado en	Pairings	DLP	Hashes
Homomórfico	Sí	Sí	No
Batch friendly	Muy sí	Sí	Sí

Matemáticas

Tamaños concretos para $n = 2^{20}$ (1 millón de grado):

	KZG	IPA	FRI
Commitment	48 B	32 B	32 B
Prueba (1 eval)	48 B	640 B	∼50 KB
Tiempo verify	∼2 ms	∼500 ms	∼50 ms

Nota: Los tiempos son aproximados y dependen de la implementación y hardware.

¿Cuál Elegir?

Idea Clave

Guía de selección:

Usar KZG cuando:

- El tamaño de prueba es crítico (ej: verificación on-chain)
- Se puede tolerar trusted setup (ceremonias disponibles)
- No se necesita seguridad post-quantum
- Ejemplo: PLONK, Ethereum data availability

Usar IPA cuando:

- No se puede tolerar trusted setup
- Se usará accumulation/recursión (ej: Halo 2)

- Verificación individual no es el cuello de botella
- Ejemplo: Zcash Orchard, Halo 2

Usar FRI cuando:

- Se necesita seguridad post-quantum
- No hay trusted setup disponible
- El tamaño de prueba no es crítico (∼100 KB aceptable)
- Ejemplo: STARKs, zkVMs como Cairo

Otros Polynomial Commitment Schemes

Observación 1.53. Además de KZG, IPA, y FRI, existen otros PCS:

DARK (2019):

- Basado en grupos de clase (class groups)
- Sin trusted setup
- Teóricamente interesante, poco usado en práctica

Dory (2020):

- Combina IPA con pairings
- Verificación $O(\log n)$ (mejor que IPA puro)
- Sin trusted setup

Brakedown (2021):

- Basado en códigos de corrección de errores
- Sin trusted setup, eficiente para el prover
- Usado en algunos zkVMs

Gemini (2022):

- Split accumulation scheme
- Combina ventajas de varios enfoques

Multilinear Polynomial Commitments

Definición 1.161 (Polinomios Multilineales)**.** Un polinomio multilineal en n variables tiene la forma:

$$f(x_1, \ldots, x_n) = \sum_{b \in \{0,1\}^n} c_b \cdot x_1^{b_1} \cdots x_n^{b_n}$$

donde cada variable aparece con grado ≤ 1.

Intuición

Los polinomios multilineales son naturales para circuitos:

- Un circuito de n wires se representa con polinomio de $\log n$ variables
- Cada "hypercube evaluation" corresponde a un wire
- La estructura permite verificación eficiente vía sumcheck

Observación 1.54. **PCS para multilineales:**

- **Zeromorph**: Adapta KZG para multilineales
- **Binius**: Basado en campos binarios, muy eficiente
- **HyperKZG**: KZG para hypercube

Sistemas como Lasso y Jolt usan multilinear PCS para lookup arguments eficientes.

Aplicación en Banco ZK

En Banco ZK

El PCS es un componente fundamental de nuestro sistema de pruebas:

1. Arquitectura actual: PLONK con KZG

Usamos PLONK con KZG como PCS porque:

- Pruebas pequeñas (∼700 bytes totales)
- Verificación on-chain eficiente (∼200K gas)
- Ceremony de Ethereum disponible

```
Circuito de transacción → PLONK → Polinomios → KZG commits
                                                    ↓
                                        Prueba final: ~700 bytes
                                        - 7 elementos G1 (KZG commits)
                                        - 2 elementos G1 (KZG opening proofs)
                                        - Evaluaciones de campo
```

2. Trade-offs considerados

Opción	Prueba	Gas	Setup
PLONK + KZG	∼700 B	∼200K	Trusted
Halo 2 + IPA	∼5 KB	∼2M	None
STARK + FRI	∼100 KB	∼5M	None

Para L2 con verificación on-chain, el gas domina. KZG gana claramente.

3. Recursión con diferentes PCS

Nuestro sistema de recursión combina:

```
Nivel 0: Pruebas individuales (KZG, rápido de generar)
    ↓
Nivel 1: Agregación (IPA/Halo style, sin trusted setup interno)
    ↓
Nivel 2: Prueba final (KZG para verificación on-chain)
```

4. Consideraciones de seguridad

Trusted setup:

- Usamos el SRS de la Ethereum KZG Ceremony (141,000+ participantes)
- Si 1 participante fue honesto, el setup es seguro
- Verificable públicamente

Post-quantum:

- KZG NO es post-quantum seguro
- Plan de migración: cuando amenaza cuántica sea realista, migrar a FRI
- Las pruebas tienen "temporal validity": solo importa que sean seguras hoy

5. Optimizaciones específicas

```
// Batch opening en KZG para múltiples evaluaciones
// En lugar de k proofs separados, uno combinado

fn batch_open(
    commitments: [C_1, ..., C_k],
    points: [z_1, ..., z_k],
    values: [y_1, ..., y_k]
) -> BatchProof {
    // 1. Combinar polinomios con challenges aleatorios
    // 2. Single KZG opening proof
    // Tamaño: O(1) en lugar de O(k)
}
```

6. Métricas de producción

Métrica	Valor
Grado del polinomio (circuito típico)	$\sim 2^{18}$
KZG commits por prueba	7
Tiempo de commit	∼500 ms
Tiempo de prove total	∼3 s
Tamaño de prueba	704 bytes
Gas de verificación	213,000

7. Roadmap futuro

- **Corto plazo:** Optimizar MSM para commits más rápidos
- **Medio plazo:** Explorar Gemini/Nova para mejor recursión
- **Largo plazo:** Preparar migración a FRI cuando post-quantum sea necesario

Resumen

Idea Clave

Polynomial Commitments: Puntos clave

1. **El por qué:** SNARKs modernos reducen todo a polinomios. PCS permite verificar sin revelar.
2. **KZG:** Pruebas $O(1)$, verificación $O(1)$, pero trusted setup. Ideal para on-chain.
3. **IPA:** Sin trusted setup, pruebas $O(\log n)$, pero verificación $O(n)$. Bueno para recursión.
4. **FRI:** Post-quantum, sin setup, pero pruebas grandes. Base para STARKs.
5. **Trade-off fundamental:** Tamaño de prueba vs trusted setup vs post-quantum.
6. **En la práctica:** Muchos sistemas usan KZG para eficiencia, con plan de migración a FRI.

Chapter 2

Blockchain Base (L1)

Bloque 2 de 20

2.1 Qué es L1

Definición 2.1 (Layer 1)**.** Blockchain base que proporciona:

- Consenso sobre el estado
- Disponibilidad de datos
- Ejecución de transacciones
- Settlement final

Ejemplos: Bitcoin, Ethereum, Solana, Avalanche, Polkadot.

2.1.1 Contexto Histórico

Intuición

Layer 1 es la "capa base" de una blockchain: el fundamento sobre el cual todo lo demás se construye. Es como el sistema operativo de una computadora—todo lo demás (aplicaciones, protocolos, tokens) depende de que L1 funcione correctamente.
Cuando hablamos de "heredar seguridad de L1", significa que un protocolo puede confiar en las garantías que L1 proporciona sin tener que reimplementarlas.

Observación 2.1. **La evolución de las blockchains L1:**

Generación	Años	Características
1ª Gen	2009-2013	Solo transferencias (Bitcoin)
2ª Gen	2015-2017	Smart contracts (Ethereum)
3ª Gen	2018-2020	Escalabilidad nativa (Solana, Avalanche)
4ª Gen	2021+	Modular, ZK-native (Celestia, Mina)

Cada generación intentó resolver las limitaciones de la anterior, pero ninguna ha logrado satisfacer completamente el "trilema" de escalabilidad, descentralización y seguridad.

2.1.2 Las Cuatro Funciones Fundamentales

Definición 2.2 (Las Cuatro Funciones de L1)**. 1. Consenso sobre el estado:** Todos los nodos acuerdan cuál es el estado actual del sistema. En Ethereum, esto incluye todos los balances, código de contratos, y storage.

2. Disponibilidad de datos (DA): Los datos necesarios para verificar transacciones están disponibles para cualquiera que los necesite. Sin DA, no se puede verificar que el estado es correcto.

3. Ejecución de transacciones: Procesar transacciones y calcular el nuevo estado. En Ethereum, esto significa ejecutar bytecode EVM.

4. Settlement final: Una vez que una transacción está "finalizada", no puede ser revertida. Diferentes L1s tienen diferentes tiempos y garantías de finalidad.

Matemáticas

Formalización del estado:
El estado de una blockchain en el bloque n se puede expresar como:

$$S_n = f(S_{n-1}, B_n)$$

donde:

- S_n es el estado después del bloque n
- S_{n-1} es el estado anterior
- B_n es el bloque n (con sus transacciones)
- f es la función de transición de estado (STF)

El estado génesis S_0 está definido en el protocolo.
En Ethereum:

$$\sigma : \text{Address} \to (\text{nonce}, \text{balance}, \text{storageRoot}, \text{codeHash})$$

La función de transición ejecuta cada transacción secuencialmente:

$$S_n = f(f(f(S_{n-1}, \text{tx}_1), \text{tx}_2), \ldots, \text{tx}_k)$$

donde $B_n = \{\text{tx}_1, \text{tx}_2, \ldots, \text{tx}_k\}$.

2.1.3 Comparación de L1s Principales

L1	Consenso	TPS	Finalidad	VM	Modelo
Bitcoin	PoW	∼7	∼60 min	Script	UTXO
Ethereum	PoS	∼15	∼15 min	EVM	Account
Solana	PoH+PoS	∼4000	∼13 s	SVM	Account
Avalanche	Avalanche	∼4500	∼2 s	EVM	Account
Polkadot	NPoS	∼1000	∼60 s	WASM	Account
Cardano	Ouroboros	∼250	∼20 min	Plutus	eUTXO
Near	Nightshade	∼1000	∼2 s	WASM	Account
Aptos	AptosBFT	∼10000	∼1 s	Move VM	Account

Advertencia

TPS (Transactions Per Second) es una métrica engañosa:

- Diferentes transacciones tienen diferentes complejidades
- Una transferencia simple ≠ un swap en DEX ≠ deploy de contrato
- Condiciones de prueba vs producción real difieren enormemente
- No captura la descentralización ni la seguridad

Métricas más útiles:

- Gas/compute units por segundo
- Costo por transacción
- Número de validadores independientes
- Nakamoto coefficient (cuántas entidades controlan 51%)
- Tiempo a finalidad económica

Ejemplo 2.1 (Nakamoto Coefficient)**.** El coeficiente de Nakamoto mide cuántas entidades independientes necesitan coludirse para atacar la red:

Red	Nakamoto Coef.
Bitcoin	∼4 (mining pools)
Ethereum	∼2-3 (Lido + exchanges)
Solana	∼19
Avalanche	∼26
Cosmos Hub	∼7

Un coeficiente más alto indica mayor descentralización.

2.2 Consenso

Intuición

El consenso es el mecanismo por el cual miles de computadoras independientes, que no confían entre sí, acuerdan una única verdad. Es el corazón de cualquier blockchain.
Sin consenso, cada nodo tendría su propia versión de la "verdad" y no podríamos tener un sistema coherente.
El problema fundamental: en un sistema distribuido abierto, ¿cómo decidimos qué transacciones son válidas y en qué orden ocurrieron?

Matemáticas

El Problema de los Generales Bizantinos (1982):
Lamport, Shostak y Pease formalizaron el problema:

- n generales deben acordar atacar o retirarse
- Algunos generales pueden ser traidores
- Comunicación solo por mensajeros (pueden ser interceptados)

Resultado: Con f traidores, se necesitan $n \geq 3f + 1$ generales para consenso.
En blockchain:

- Generales = nodos validadores
- Traidores = nodos maliciosos o caídos
- Mensaje = bloques y votos

2.2.1 Proof of Work (PoW)

Definición 2.3 (PoW)**.** Para crear bloque, encontrar nonce tal que:

$$H(\text{header}\|\text{nonce}) < \text{target}$$

- Dificultad ajustada para tiempo promedio constante
- Seguridad: requiere 51% del hashpower para atacar
- Consumo energético proporcional a seguridad
- Usado en: Bitcoin, Litecoin, (Ethereum pre-merge)

Historia y Orígenes

Observación 2.2. **La genealogía de Proof of Work:**

Año	Autor	Contribución
1992	Dwork & Naor	PoW para combatir spam de email
1997	Adam Back	Hashcash: PoW práctico
2004	Hal Finney	RPOW: Reusable Proofs of Work
2005	Nick Szabo	Bit Gold: proto-Bitcoin
2008	Satoshi Nakamoto	Bitcoin: PoW + incentivos + blockchain
2009	—	Bitcoin mainnet launch
2011	—	Primeras pools de minería
2013	—	Primeros ASICs para Bitcoin
2015	Ethereum	PoW con Ethash (memory-hard)
2022	Ethereum	The Merge: fin de PoW en Ethereum

La innovación de Satoshi no fue PoW en sí, sino combinarlo con:

1. Incentivos económicos (block rewards)
2. Una cadena de bloques con referencia al anterior
3. Ajuste dinámico de dificultad
4. Un mecanismo para resolver forks (cadena más larga)

El Algoritmo de Minería

Definición 2.4 (Algoritmo de Minería PoW)**.** `def mine_block(prev_hash, transactions, difficulty`

```
    """
    Minar un bloque válido.

    Args:
        prev_hash: Hash del bloque anterior
        transactions: Lista de transacciones a incluir
        difficulty_target: Número máximo que el hash puede ser

    Returns:
        (block_header, nonce, block_hash)
    """
    # Construir header
    merkle_root = compute_merkle_root(transactions)
    timestamp = current_time()

    header = {
        'version': CURRENT_VERSION,
        'prev_hash': prev_hash,
        'merkle_root': merkle_root,
        'timestamp': timestamp,
```

```
    'bits': encode_target(difficulty_target),
    'nonce': 0
}

# Buscar nonce válido
nonce = 0
extra_nonce = 0  # En coinbase transaction

while True:
    # Doble SHA-256 (Bitcoin)
    header['nonce'] = nonce
    header_bytes = serialize(header)
    block_hash = sha256(sha256(header_bytes))

    # ¿Es válido?
    if int(block_hash, 16) < difficulty_target:
        return header, nonce, block_hash

    nonce += 1

    # Nonce es solo 32 bits, puede agotarse
    if nonce > 0xFFFFFFFF:
        # Modificar extra_nonce en coinbase y recalcular merkle
        extra_nonce += 1
        transactions[0] = update_coinbase(transactions[0], extra_nonce)
        merkle_root = compute_merkle_root(transactions)
        header['merkle_root'] = merkle_root
        nonce = 0
```

Matemáticas

Probabilidad y Dificultad:

El espacio de hashes SHA-256 es $\{0, 1, \ldots, 2^{256} - 1\}$.

El target determina cuántos hashes son "válidos":

$$P(\text{hash válido}) = \frac{\text{target}}{2^{256}}$$

Ejemplo para Bitcoin (2024):

Con dificultad $D \approx 80 \times 10^{12}$:

$$\text{target} = \frac{\text{target}_{\max}}{D} \approx \frac{2^{224}}{80 \times 10^{12}} \approx 2^{177}$$

$$P(\text{válido}) = \frac{2^{177}}{2^{256}} = 2^{-79} \approx 1.6 \times 10^{-24}$$

Número esperado de hashes para encontrar bloque:

$$E[\text{hashes}] = \frac{1}{P} = 2^{79} \approx 6 \times 10^{23}$$

Con hashrate global de $\sim$500 EH/s $= 5 \times 10^{20}$ hashes/segundo:

$$\text{Tiempo esperado} = \frac{6 \times 10^{23}}{5 \times 10^{20}} = 1200 \text{ segundos} \approx 20 \text{ minutos}$$

Pero como hay muchos mineros compitiendo, en promedio se encuentra un bloque cada $\sim$10 minutos.

Ajuste de Dificultad

Definición 2.5 (Ajuste de Dificultad de Bitcoin)**.** Cada 2016 bloques ($\sim$2 semanas si cada bloque toma 10 min):

$$\text{nuevo_target} = \text{viejo_target} \times \frac{\text{tiempo_real}}{2016 \times 10 \text{ min}}$$

Con límites:

- Máximo incremento: $\times 4$ (si tiempo fue muy corto)
- Máximo decremento: $\div 4$ (si tiempo fue muy largo)

Ejemplo 2.2 (Cálculo de ajuste)**.** Supongamos que los últimos 2016 bloques tomaron 12 días en lugar de 14:

$$\text{factor} = \frac{12 \text{ días}}{14 \text{ días}} = \frac{12}{14} \approx 0.857$$

El nuevo target será:

$$\text{nuevo_target} = \text{viejo_target} \times 0.857$$

Como el target es más pequeño, la dificultad aumenta $\sim$17%. Los bloques estaban llegando muy rápido porque el hashrate aumentó.

Seguridad de PoW

Idea Clave

El Ataque del 51%:
Un atacante con más del 50% del hashpower puede:

1. Minar una cadena privada en secreto
2. Gastar monedas en la cadena pública
3. Cuando la cadena privada es más larga, publicarla
4. La red adopta la cadena más larga (Nakamoto consensus)
5. Las transacciones en la cadena abandonada desaparecen
6. El atacante tiene sus monedas de vuelta (doble gasto)

Matemáticas

Probabilidad de éxito del ataque:
Sea q la fracción de hashpower del atacante y $p = 1 - q$ la fracción honesta.
Si el atacante está z bloques detrás, la probabilidad de alcanzar a la cadena honesta es:

$$P(\text{catch up}) = \begin{cases} 1 & \text{si } q > 0.5 \\ \left(\frac{q}{p}\right)^z & \text{si } q \leq 0.5 \end{cases}$$

Ejemplo: Con $q = 0.3$ (30% del hashpower) y $z = 6$ confirmaciones:

$$P = \left(\frac{0.3}{0.7}\right)^6 = 0.43^6 \approx 0.006 = 0.6\%$$

Por eso Bitcoin recomienda esperar 6 confirmaciones para transacciones grandes.

Advertencia

Costo de un ataque del 51% a Bitcoin (2024):

Componente	Costo estimado
Hardware (>50% hashrate)	> \$10 mil millones
Electricidad por hora	~\$2 millones
Infraestructura	> \$100 millones
Tiempo de adquisición	Meses a años

Además, un ataque exitoso destruiría la confianza en Bitcoin, colapsando el precio y haciendo que el atacante pierda el valor de su inversión en hardware.
Ataques exitosos históricos:

Moneda	Fecha	Pérdida	Confirmaciones
Ethereum Classic	Ene 2019	$1.1M	100+
Bitcoin Gold	May 2018	$18M	22
Verge	Abr 2018	$1.7M	–
Bitcoin SV	Ago 2021	–	14

Las monedas con bajo hashrate son vulnerables. Se puede "alquilar" hashpower en NiceHash.

Selfish Mining

Definición 2.6 (Selfish Mining)**.** Estrategia donde un minero con >25-33% del hashpower puede obtener más recompensas que su proporción justa:

1. Minero encuentra bloque pero NO lo publica
2. Sigue minando en secreto
3. Cuando la red pública está cerca de alcanzarlo, publica su cadena
4. Red adopta cadena más larga, desperdiciando trabajo de otros

Matemáticas

El paper de Eyal & Sirer (2014) demostró que con α = fracción de hashpower y γ = fracción de la red que el atacante alcanza primero con su bloque:
El selfish mining es rentable cuando:

$$\alpha > \frac{1-\gamma}{3-2\gamma}$$

Con $\gamma = 0$: rentable si $\alpha > 0.333$ (33%) Con $\gamma = 0.5$: rentable si $\alpha > 0.25$ (25%)

Algoritmos de Hash para PoW

Algoritmo	Usado en	Tipo	Hardware dominante
SHA-256d	Bitcoin, BCH	CPU-friendly	ASICs
Scrypt	Litecoin, Doge	Memory-hard	ASICs (ahora)
Ethash	Ethereum (era)	Memory-hard DAG	GPUs
RandomX	Monero	CPU-optimizado	CPUs
Equihash	Zcash	Memory-hard	ASICs (moderado)
X11	Dash	11 algoritmos	ASICs
CryptoNight	Monero (old)	Memory-hard	CPUs/GPUs

Observación 2.3. **La carrera armamentista del hardware de minería:**

1. **2009-2010:** Minería con CPU (cualquier computadora)

2. **2010-2012:** GPUs (10-100x más eficientes)
3. **2013+:** ASICs (Application-Specific Integrated Circuits)
4. ASICs son 100,000x+ más eficientes que CPUs para SHA-256

Trade-off:

- ASICs: máxima eficiencia, pero centraliza minería
- Memory-hard: resiste ASICs, más descentralizado
- CPU-only: máxima descentralización, pero botnets son problema

Pools de Minería

Definición 2.7 (Mining Pool)**.** Grupo de mineros que combinan su hashpower y comparten recompensas proporcionalmente.

Intuición

Un minero pequeño con 0.001% del hashrate encontraría un bloque cada ∼1000 días en promedio. Con alta varianza, podría ser 5 años o 1 mes.
En un pool, recibe pequeños pagos constantes proporcionales a su contribución. Menos varianza, mismo valor esperado (menos fee del pool).

Métodos de pago:

Método	Descripción
PPS (Pay Per Share)	Pago fijo por cada share válido
PPLNS	Pago proporcional a shares en últimos N shares
FPPS	PPS + estimación de fees de transacción
SOLO	Solo ayuda a encontrar bloque, toda la recompensa

Advertencia

Distribución de hashpower Bitcoin (2024):

Pool	% Hashrate
Foundry USA	∼27%
AntPool	∼16%
F2Pool	∼13%
ViaBTC	∼12%
Binance Pool	∼8%
Otros	∼24%

Los top 4 pools controlan >65% del hashrate. Esto es preocupante para la descentralización, aunque los mineros individuales dentro de los pools pueden cambiar fácilmente.

2.2.2 Proof of Stake (PoS)

Definición 2.8 (PoS)**.** Validadores depositan stake como colateral. Seleccionados para proponer bloques según su stake.

Mecanismos:

- **Selección:** Aleatoria ponderada por stake
- **Slashing:** Perder stake por mal comportamiento
- **Rewards:** Ganar por participación honesta

Variantes:

- **Delegated PoS:** Usuarios delegan a validadores
- **Nominated PoS:** Polkadot, nominadores eligen
- **Liquid staking:** stETH, stake tokenizado

Historia y Motivación

Observación 2.4. **¿Por qué PoS?**

1. **Eficiencia energética:** PoW consume tanta electricidad como países enteros
2. **Descentralización de hardware:** No requiere ASICs especializados
3. **Seguridad económica:** Atacante pierde su propio stake
4. **Finalidad más rápida:** BFT puede dar finalidad en segundos
5. **Menor barrera de entrada:** Cualquiera con stake puede participar

Timeline de PoS:

Año	Evento
2012	Peercoin: primer PoS híbrido
2014	Primeras propuestas formales de PoS
2017	Cardano Ouroboros (primer PoS con prueba de seguridad)
2019	Cosmos Hub launch (Tendermint BFT)
2020	Ethereum Beacon Chain launch (PoS)
2020	Solana mainnet (PoH + PoS)
2022	Ethereum “The Merge”: transición completa a PoS

El Problema “Nothing at Stake”

Advertencia

Nothing at Stake:
En PoW, minar en múltiples forks divide tu hashpower (costo real).
En PoS naive, votar en múltiples forks no tiene costo—puedes votar en todos "por si acaso".
Problema: Si todos votan en todos los forks, nunca se alcanza consenso.
Solución: Slashing

- Si se detecta que votaste en múltiples forks conflictivos
- Pierdes parte o todo tu stake
- Ahora hay un costo real por comportamiento deshonesto

Ethereum PoS (Gasper)

Definición 2.9 (Gasper)**.** El protocolo de consenso de Ethereum combina:

- **Casper FFG:** Mecanismo de finalidad
- **LMD-GHOST:** Regla de elección de fork

Matemáticas

Estructura de Ethereum PoS:
Slots y Epochs:

- 1 slot = 12 segundos
- 1 epoch = 32 slots = 6.4 minutos
- Cada slot tiene un proposer designado
- Validadores divididos en comités por slot

Roles:

- **Proposer:** Propone bloque en su slot
- **Attester:** Vota por el bloque que considera correcto
- **Sync Committee:** 512 validadores que firman headers (para light clients)

Requisitos:

- Mínimo stake: 32 ETH
- Tiempo de activación: $\sim$1 día (cola de entrada)
- Tiempo de salida: $\sim$1 día + período de retiro

Definición 2.10 (Slashing Conditions en Ethereum)**.** Un validador es slashed si:

1. **Double voting:** Votar por dos bloques diferentes en el mismo slot
2. **Surround voting:** Hacer attestation que "rodea" otra attestation propia

Penalidades:

- Mínimo: 1/32 del stake (∼1 ETH)
- Correlación: Si muchos son slashed juntos, penalidad aumenta hasta 100%
- Exclusión forzosa de la red

Ejemplo 2.3 (Cálculo de recompensas de validador)**. Parámetros (circa 2024):**

- Total staked: ∼30 millones ETH
- Base reward por epoch: ∼0.000003 ETH
- Epochs por año: ∼82,125

APR aproximado:

$$\text{APR} \approx \frac{C}{\sqrt{\text{total_staked}}}$$

Con 30M ETH staked: APR ≈ 4-5%
Para un validador con 32 ETH:

$$\text{Ganancia anual} \approx 32 \times 0.045 = 1.44 \text{ ETH}$$

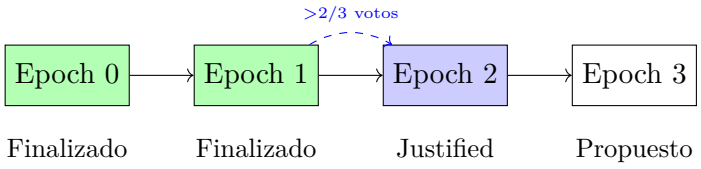
Más fees de transacción y MEV (variable).

Casper FFG: Finalidad

Definición 2.11 (Finalidad en Casper FFG)**.** Un bloque está **finalizado** cuando:

1. Es **justified**: >2/3 del stake votó por él
2. El siguiente checkpoint también es justified, con él como source

Un bloque finalizado NUNCA puede ser revertido (salvo que >1/3 del stake sea slashed).



LMD-GHOST: Regla de Fork Choice

Definición 2.12 (LMD-GHOST)**.** Latest Message Driven Greedy Heaviest Observed SubTree.
Para elegir la cadena canónica:

1. Empezar desde el último bloque finalizado
2. En cada fork, elegir la rama con más votos recientes
3. "Reciente" = último voto de cada validador

Intuición

LMD-GHOST es como una votación continua donde cada validador tiene un voto, y en cada bifurcación, la rama con más votos gana. Pero solo cuenta el voto más reciente de cada validador.
Esto permite:

- Adaptarse rápido a nuevos bloques
- Resistir censura (un bloque censurado puede ser votado después)
- Mantener liveness incluso con particiones de red temporales

Liquid Staking

Definición 2.13 (Liquid Staking)**.** Mecanismo donde el stake bloqueado se representa con un token líquido.

Ejemplo 2.4 (stETH de Lido)**.**

1. Usuario deposita ETH en Lido
2. Lido stakea el ETH con validadores
3. Usuario recibe stETH (1:1 inicialmente)
4. stETH acumula recompensas (rebasing)
5. Usuario puede usar stETH en DeFi
6. Cuando quiere salir: intercambia stETH por ETH

Ventajas:

- Liquidez: no hay lock-up
- Composabilidad: usar como colateral
- Sin mínimo de 32 ETH

Riesgos:

- Centralización: Lido tiene ∼30% del stake
- Smart contract risk
- Depeg en mercados secundarios

Advertencia

Riesgo de centralización en Ethereum PoS:

Entidad	% del stake
Lido	∼32%
Coinbase	∼13%
Kraken	∼7%
Binance	∼5%
Rocket Pool	∼3%
Otros	∼40%

Si Lido + Coinbase + Kraken cooperaran, tendrían >50% del stake. Esto es un riesgo de censura, aunque hay mecanismos sociales y técnicos para mitigarlo.

Comparación PoW vs PoS

Aspecto	PoW	PoS
Recurso escaso	Energía + hardware	Capital
Costo de ataque	Comprar/alquilar hashpower	Comprar stake
Después de ataque	Hardware sigue valiendo	Stake es slashed
Barrera de entrada	Hardware especializado	Capital + técnico
Consumo energético	Muy alto	Bajo
Finalidad	Probabilística	Determinística (con BFT)
Tiempo a finalidad	∼60 min (6 conf)	∼15 min
Descentralización	Por hashpower	Por stake

2.2.3 Proof of Authority (PoA)

Definición 2.14 (PoA)**.** Validadores son entidades conocidas y autorizadas.

- Muy rápido (no hay competencia)
- Menos descentralizado
- Usado en: testnets, redes empresariales, sidechains

Cuándo Usar PoA

Idea Clave

PoA es apropiado cuando:

- Los validadores son conocidos y tienen reputación que perder
- La descentralización completa no es el objetivo principal
- Se necesita alta velocidad y bajo costo
- Hay una entidad central que puede autorizar validadores

Casos de uso:

- **Testnets:** Ethereum Goerli, Sepolia
- **Sidechains:** Polygon PoS (originalmente), Ronin
- **Enterprise:** Hyperledger Besu, Quorum
- **Gaming:** Ronin (Axie Infinity)

Ejemplo 2.5 (Ronin Network)**.** Ronin es una sidechain para Axie Infinity que usaba PoA:

- 9 validadores autorizados por Sky Mavis
- 5/9 requeridos para firmar transacciones
- Muy rápido y barato para el juego

El problema: En marzo 2022, un atacante comprometió 5 de los 9 validadores (incluyendo 4 de Sky Mavis) y robó $625M del bridge.

Lección: PoA con pocos validadores es un punto único de fallo.

Algoritmos de PoA

Algoritmo	Usado en	Características
Clique	Ethereum testnets	Simple, round-robin
Aura	Parity/OpenEthereum	Round-robin con slots
IBFT 2.0	Hyperledger Besu	BFT, finalidad inmediata
QBFT	ConsenSys Quorum	BFT optimizado

2.2.4 Otros Mecanismos de Consenso

Observación 2.5. **Variantes y mecanismos alternativos:**

Proof of History (PoH) - Solana:

- No es consenso, es un reloj criptográfico
- SHA-256 secuencial crea timestamps verificables
- Combinado con PoS para consenso real
- Permite ordenar transacciones sin comunicación

Avalanche Consensus:

- Muestreo aleatorio repetido
- Pregunta a k nodos aleatorios
- Si $> \alpha k$ coinciden, adopta su opinión
- Repite hasta convergencia

- Finalidad sub-segundo

Tendermint BFT (Cosmos):

- BFT clásico con proposer rotativo
- Tolera hasta 1/3 maliciosos
- Finalidad inmediata (1 bloque)
- Usado en Cosmos Hub, Binance Chain

Ouroboros (Cardano):

- PoS con prueba formal de seguridad
- Divide tiempo en slots y epochs
- Líder elegido por VRF ponderado por stake

Nominated PoS (Polkadot):

- Nominadores eligen validadores
- Algoritmo Phragmén para distribución equitativa
- Slashing compartido entre validador y nominadores

En Banco ZK

Consenso en Banco ZK:
Banco ZK como L2 no necesita su propio consenso para seguridad—hereda la seguridad de Ethereum L1.
Sin embargo, necesitamos ordenar transacciones:

- **Fase actual:** Sequencer centralizado (eficiente pero requiere confianza)
- **Futuro:** Sequencer descentralizado o based sequencing (L1 ordena)

La privacidad de Banco ZK ayuda con consenso:

- Transacciones encriptadas previenen MEV del sequencer
- El sequencer no puede hacer frontrunning si no ve el contenido

2.3 Bloques y Transacciones

Intuición

Un bloque es como una página en un libro de contabilidad: contiene un conjunto de transacciones que ocurrieron en un período de tiempo. Cada página hace referencia a la anterior, creando una cadena ininterrumpida desde el inicio.
La estructura exacta del bloque varía entre blockchains, pero todas comparten el

concepto de:

- Header: metadatos sobre el bloque
- Body: las transacciones en sí

2.3.1 Estructura de Bloque

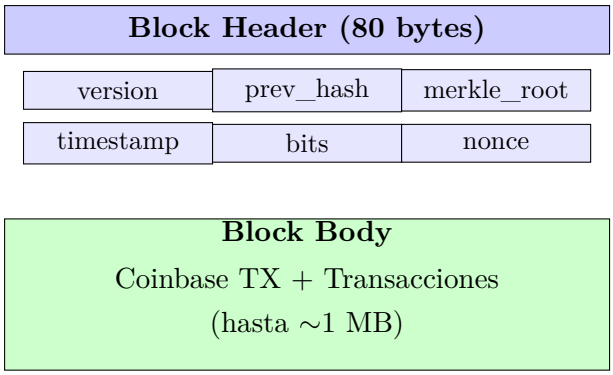
Bloque de Bitcoin

Definición 2.15 (Bloque de Bitcoin)**. Header (80 bytes):**

- `version` (4 bytes): Versión del protocolo
- `prev_block_hash` (32 bytes): Hash del bloque anterior
- `merkle_root` (32 bytes): Raíz del árbol de transacciones
- `timestamp` (4 bytes): Tiempo Unix
- `bits` (4 bytes): Target de dificultad codificado
- `nonce` (4 bytes): Valor para PoW

Body:

- `tx_count`: Número de transacciones (varint)
- `transactions`: Lista de transacciones



Ejemplo 2.6 (Bloque de Bitcoin real)**. Bloque #800000 (Julio 2023):**

```
Header:
  version: 0x27ffe000
  prev_block: 00000000000000000002a7c4c1e48d76...
  merkle_root: b2f7c58b5c8f2e7d4a1b6c9d8e2f3a4b...
  timestamp: 1690243200 (2023-07-24 16:00:00 UTC)
  bits: 0x17053894
  nonce: 3127834567
```

```
Block hash: 0000000000000000002a7c4c1e48d76c9...
  (empieza con muchos ceros por PoW)

Body:
  tx_count: 3721
  transactions: [coinbase, tx1, tx2, ..., tx3720]
  size: 1,563,789 bytes
```

Bloque de Ethereum (Post-Merge)

Definición 2.16 (Bloque de Ethereum)**. Header:**

- `parentHash`: Hash del bloque padre
- `stateRoot`: Raíz del Merkle Patricia Trie de estado
- `transactionsRoot`: Raíz del trie de transacciones
- `receiptsRoot`: Raíz del trie de receipts
- `logsBloom`: Bloom filter de logs (256 bytes)
- `number`: Número de bloque
- `gasLimit`: Límite de gas del bloque
- `gasUsed`: Gas usado en el bloque
- `timestamp`: Tiempo Unix
- `baseFeePerGas`: Base fee (EIP-1559)
- `withdrawalsRoot`: Raíz de withdrawals (post-Shanghai)
- `blobGasUsed`: Gas de blobs usado (EIP-4844)
- `excessBlobGas`: Exceso de blob gas

Body:

- `transactions`: Lista de transacciones
- `withdrawals`: Retiros de stake

Matemáticas

Los tres tries de Ethereum:
Cada bloque tiene raíces de tres Merkle Patricia Tries:
1. State Trie (`stateRoot`):

$$\text{Address} \to (\text{nonce}, \text{balance}, \text{storageRoot}, \text{codeHash})$$

Representa TODO el estado de Ethereum.

2. Transaction Trie (`transactionsRoot`):

$$\text{index} \rightarrow \text{RLP}(\text{transaction})$$

Solo las transacciones de este bloque.

3. Receipt Trie (`receiptsRoot`):

$$\text{index} \rightarrow \text{RLP}(\text{receipt})$$

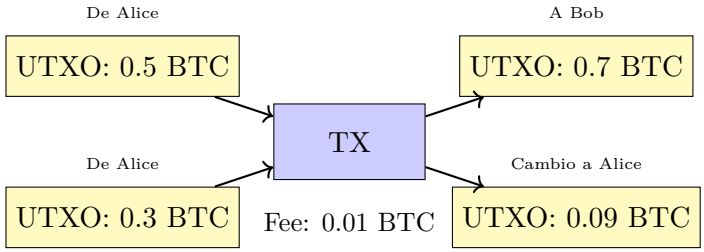
Resultado de cada transacción (status, gas usado, logs).

2.3.2 Transacciones

Modelo UTXO (Bitcoin)

Definición 2.17 (UTXO - Unspent Transaction Output)**.** En Bitcoin no hay "cuentas" con balances. En su lugar:

- Cada transacción consume UTXOs existentes (inputs)
- Y crea nuevos UTXOs (outputs)
- Tu "balance" es la suma de UTXOs que puedes gastar



Definición 2.18 (Estructura de Transacción Bitcoin)**.** `Transaction`:

```
version: 4 bytes

inputs: [
  {
    prev_txid: 32 bytes    # Hash de tx anterior
    prev_vout: 4 bytes     # Índice del output
    scriptSig: variable    # Script de desbloqueo
    sequence: 4 bytes      # Para timelocks
  },
  ...
]

outputs: [
```

```
  {
    value: 8 bytes          # Cantidad en satoshis
    scriptPubKey: variable # Script de bloqueo
  },
  ...
]

locktime: 4 bytes           # Tiempo mínimo de validez
```

Idea Clave

Scripts de Bitcoin:
Cada UTXO está "bloqueado" por un script que especifica las condiciones para gastarlo.
P2PKH (Pay to Public Key Hash):

```
scriptPubKey: OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
scriptSig: <signature> <publicKey>
```

P2SH (Pay to Script Hash):

```
scriptPubKey: OP_HASH160 <scriptHash> OP_EQUAL
scriptSig: <signatures...> <redeemScript>
```

P2WPKH (Native SegWit):

```
scriptPubKey: OP_0 <20-byte-pubKeyHash>
witness: [<signature>, <publicKey>]
```

Observación 2.6. **Ventajas del modelo UTXO:**

- Paralelismo natural: transacciones independientes no conflictan
- Privacidad: cada UTXO es distinto, dificulta tracking
- Validación simple: solo verificar que inputs existen y no gastados
- Sin estado global: no necesitas conocer todo el estado

Desventajas:

- Smart contracts complejos son difíciles
- Gestión de UTXOs puede ser engorrosa
- "Dust" (UTXOs muy pequeños) es problema

Modelo Account (Ethereum)

Definición 2.19 (Transacción de Ethereum)**. Campos:**

- `type`: Tipo de transacción (0, 1, 2, 3)

- `chainId`: ID de la red (1 = mainnet)
- `nonce`: Contador de transacciones del sender
- `to`: Dirección destino (null para deploy)
- `value`: Wei a transferir
- `data`: Calldata (para contratos)
- `gasLimit`: Máximo gas a usar
- `maxFeePerGas`: Máximo fee por gas (EIP-1559)
- `maxPriorityFeePerGas`: Máximo tip (EIP-1559)
- `accessList`: Direcciones/slots a pre-calentar (EIP-2930)
- `v, r, s`: Firma ECDSA

Ejemplo 2.7 (Transacción Ethereum decodificada)**.**

```
{
  "type": 2,                    // EIP-1559
  "chainId": 1,                 // Mainnet
  "nonce": 42,                  // 43ª tx de esta cuenta
  "to": "0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D",  // Uniswap
  "value": "0",                 // No envía ETH
  "data": "0x38ed1739...",      // swap() function call
  "gasLimit": 250000,
  "maxFeePerGas": "30000000000",      // 30 gwei
  "maxPriorityFeePerGas": "1000000000", // 1 gwei tip
  "accessList": [],

  // Firma
  "v": 1,
  "r": "0x3a4b5c...",
  "s": "0x8d9e0f..."
}

// Decodificando data (ABI):
// Function: swapExactTokensForTokens(
//   uint256 amountIn: 1000000000,        // 1000 USDC
//   uint256 amountOutMin: 999000000000,  // Min 0.999 ETH
//   address[] path: [USDC, WETH],
//   address to: 0xMyAddress,
//   uint256 deadline: 1699999999
// )
```

Matemáticas

Tipos de transacción en Ethereum:

Type	Nombre	EIP	Características
0	Legacy	Pre-EIP-2718	gasPrice fijo
1	Access List	EIP-2930	+ accessList
2	EIP-1559	EIP-1559	maxFee + priorityFee
3	Blob	EIP-4844	+ blobVersionedHashes

El Nonce

Definición 2.20 (Nonce de Cuenta)**.** Contador que indica cuántas transacciones ha enviado una cuenta.

Reglas:

- Transacción con nonce n solo válida si la cuenta tiene nonce n
- Después de ejecutar, nonce incrementa a $n + 1$
- No se pueden saltar nonces
- Se pueden enviar múltiples txs con nonces consecutivos

Advertencia

Problemas comunes con nonces:

1. **Nonce gap:**
   - Enviaste tx con nonce 5, pero nonce 4 falló
   - La tx 5 queda pendiente para siempre
   - Solución: reenviar tx 4 o cancelarla
2. **Nonce reuse:**
   - Enviar dos txs con mismo nonce
   - Solo una puede ser incluida
   - Útil para "cancelar": enviar tx vacía con mismo nonce y más gas
3. **Pending nonce:**
   - Si tx está pending, siguiente debe usar nonce+1
   - Algunos RPCs dan "pending nonce", otros no

Comparación UTXO vs Account

Aspecto	UTXO (Bitcoin)	Account (Ethereum)
Estado	Set de UTXOs	Mapping address → account
Balance	Suma de UTXOs	Campo en account
Transacción	Consume UTXOs, crea nuevos	Modifica estado de cuentas
Paralelismo	Natural (UTXOs independientes)	Requiere cuidado (mismo nonce)
Privacidad	Mejor (direcciones únicas)	Peor (misma dirección)
Smart contracts	Limitados (Script)	Completos (EVM)
Complejidad	Gestión de UTXOs	Gestión de nonces
Espacio	Crece con UTXOs	Crece con cuentas

2.4 Gas y Fees

Definición 2.21 (Gas)**.** Unidad de medida del costo computacional en Ethereum.

Intuición

El gas es como la gasolina de un auto:

- Cada operación consume cierta cantidad de gas
- Más compleja la operación = más gas
- Pagas por el gas que usas, no más
- Si te quedas sin gas, la transacción falla (pero pagas igual)

¿Por qué gas y no ETH directamente?
Separar la unidad de computación (gas) del precio (gwei) permite:

- El costo de operaciones es estable en gas
- El precio en ETH fluctúa según demanda
- Protege contra ataques de spam

2.4.1 Costos de Gas por Operación

Definición 2.22 (Costos de Gas (EVM))**.**

Categoría	Operación	Gas
4*Aritmética	ADD, SUB	3
	MUL, DIV	5
	MOD	5
	EXP	$10 + 50 \times$ bytes
3*Comparación	LT, GT, EQ	3
	ISZERO	3
	AND, OR, XOR	3
4*Storage	SLOAD	2,100 (cold), 100 (warm)
	SSTORE ($0 \rightarrow$ non-0)	20,000
	SSTORE (non-0 $\rightarrow$ non-0)	2,900
	SSTORE (non-0 $\rightarrow$ 0)	2,900 + refund
3*Memoria	MLOAD, MSTORE	3
	Memory expansion	$3 \times \text{words} + \text{words}^2/512$
	MSTORE8	3
3*Control	JUMP	8
	JUMPI	10
	CALL (cold)	2,600
	CALL (warm)	100
3*Crypto	SHA3/KECCAK256	$30 + 6 \times$ words
	ECRECOVER (precompile)	3,000
	ECADD (precompile)	150
3*Transacción	Base cost	21,000
	Zero byte in data	4
	Non-zero byte in data	16

Ejemplo 2.8 (Cálculo de gas para transferencia simple)**. Transferencia de ETH (sin contrato):**

```
Base transaction cost:     21,000 gas
Data (vacío):                   0 gas
---------------------------------------
Total:                     21,000 gas
```

Con gas price de 30 gwei:

$$\text{Costo} = 21,000 \times 30 \text{ gwei} = 630,000 \text{ gwei} = 0.00063 \text{ ETH}$$

A \$2500/ETH: \$1.58

Ejemplo 2.9 (Cálculo de gas para swap en Uniswap)**. Swap típico:**

```
Base transaction:          21,000
```

```
Calldata (228 bytes):       ~3,200  (mix de zeros y non-zeros)
Contract execution:
  - SLOAD (prices): 2x      4,200  (cold)
  - SLOAD (balances): 4x    8,400  (cold)
  - SSTORE (balances): 4x  11,600  (warm, modify)
  - Math operations:        ~1,000
  - Memory ops:               ~500
  - Event emission:         ~2,000
  - Other:                  ~5,000
--------------------------------------
Total:                    ~150,000 gas
```

Con gas price de 30 gwei:

$$\text{Costo} = 150,000 \times 30 \text{ gwei} = 0.0045 \text{ ETH} \approx \$11$$

2.4.2 EIP-1559: El Nuevo Modelo de Fees

Definición 2.23 (EIP-1559)**.** Modelo de fees introducido en el London hard fork (Agosto 2021):

Componentes:

- **Base fee:** Fee mínimo por gas, ajustado algorítmicamente, **quemado**
- **Priority fee (tip):** Propina opcional al validador
- **Max fee:** Máximo que el usuario está dispuesto a pagar

Fee efectivo por gas:

$$\text{effectiveFee} = \text{baseFee} + \min(\text{priorityFee}, \text{maxFee} - \text{baseFee})$$

Matemáticas

Ajuste del Base Fee:
El base fee se ajusta para mantener bloques al 50% de capacidad:

- Target: 15M gas (50% de 30M max)
- Si bloque $>$ 15M gas: base fee sube (hasta +12.5%)
- Si bloque $<$ 15M gas: base fee baja (hasta -12.5%)

Fórmula:

$$\text{baseFee}_{n+1} = \text{baseFee}_n \times \left(1 + \frac{1}{8} \times \frac{\text{gasUsed} - \text{target}}{\text{target}}\right)$$

Ejemplo:

- Base fee actual: 20 gwei
- Bloque usó: 25M gas (target: 15M)

- Exceso: $(25M - 15M)/15M = 0.667$
- Cambio: $0.667/8 = 0.0833 = 8.33\%$
- Nuevo base fee: $20 \times 1.0833 = 21.67$ gwei

Idea Clave

¿Por qué quemar el base fee?
Antes de EIP-1559:

- Todo el fee iba al minero
- Mineros tenían incentivo de incluir txs propias con fee alto para manipular precio
- Users no sabían cuánto pagar

Después de EIP-1559:

- Base fee se quema: nadie lo recibe
- Solo el tip va al validador
- Validadores no pueden manipular el base fee
- ETH se vuelve potencialmente deflacionario

Impacto deflacionario:
Si se quema más ETH del que se emite como rewards:

$$\text{ETH quemado} > \text{ETH emitido} \implies \text{Supply decrece}$$

En períodos de alta actividad, Ethereum se ha vuelto deflacionario.

Ejemplo 2.10 (Transacción EIP-1559)**.**

```
Usuario configura:
  maxFeePerGas: 50 gwei
  maxPriorityFeePerGas: 2 gwei

Estado actual:
  baseFee: 30 gwei

Cálculo:
  effectivePriorityFee = min(2, 50 - 30) = min(2, 20) = 2 gwei
  effectiveFee = 30 + 2 = 32 gwei

Distribución por gas:
  - Quemado: 30 gwei
```

```
  - Al validador: 2 gwei
  - Refund al usuario: 50 - 32 = 18 gwei (por unidad de gas limit)

Para 100,000 gas usado:
  - Total pagado: 3,200,000 gwei = 0.0032 ETH
  - Quemado: 3,000,000 gwei = 0.003 ETH
  - Validador: 200,000 gwei = 0.0002 ETH
```

2.4.3 Gas Limit y Gas Used

Definición 2.24 (Gas Limit)**. Por transacción:**

- Máximo gas que la transacción puede consumir
- Usuario lo especifica
- Si se excede, transacción falla pero gas no se devuelve

Por bloque:

- Máximo gas total en un bloque
- Actualmente: 30M gas (target: 15M)
- Validadores pueden votar para ajustarlo

Advertencia

Out of Gas:
Si tu transacción se queda sin gas:

1. La transacción se revierte completamente
2. PERO pagas por todo el gas que especificaste
3. El estado no cambia (excepto tu balance)

Estrategia: Siempre poner un poco más de gas limit del necesario. El gas no usado se devuelve.

2.4.4 EIP-4844: Blob Gas

Definición 2.25 (Blob Gas)**.** EIP-4844 introduce un mercado de gas separado para blobs:

- **Blob base fee:** Independiente del base fee normal
- **Target:** 3 blobs por bloque
- **Max:** 6 blobs por bloque
- Mismo mecanismo EIP-1559 para ajuste

Matemáticas

Fórmula del blob base fee:

$$\text{blobBaseFee}_{n+1} = \text{blobBaseFee}_n \times e^{\frac{\text{excessBlobGas}}{3338477}}$$

Donde:

- Cada blob consume 131,072 gas de blob
- Target por bloque: 393,216 blob gas (3 blobs)
- Max por bloque: 786,432 blob gas (6 blobs)

El blob base fee puede variar mucho más que el base fee normal (sin límite de 12.5%).

2.4.5 Optimización de Gas

Idea Clave

Técnicas para reducir gas en smart contracts:

1. Storage es caro, memoria es barata:

```
// Malo: múltiples SLOADs
function bad() external {
    total += values[0];  // SLOAD
    total += values[1];  // SLOAD
    total += values[2];  // SLOAD
}

// Bueno: cargar una vez a memoria
function good() external {
    uint256 _total = total;      // SLOAD una vez
    uint256[] memory _values = values; // SLOAD una vez
    for (uint i = 0; i < 3; i++) {
        _total += _values[i];    // MLOAD (barato)
    }
    total = _total;              // SSTORE una vez
}
```

2. Packing de variables:

```
// Malo: 3 slots de storage
struct Bad {
    uint256 a;  // slot 0
    uint8 b;    // slot 1 (desperdicia 31 bytes)
    uint256 c;  // slot 2
}
```

```
// Bueno: 2 slots
struct Good {
    uint256 a;  // slot 0
    uint256 c;  // slot 1
    uint8 b;    // slot 1 (comparte con c)
}
```

3. Usar eventos en vez de storage para datos históricos:

```
// Malo: guardar historial en storage
mapping(uint => Trade) public tradeHistory;

// Bueno: emitir eventos (logs son más baratos)
event TradeExecuted(address indexed user, uint amount, uint price);
```

4. Short-circuit evaluation:

```
// Poner condición más probable de fallar primero
require(amount > 0 && balances[msg.sender] >= amount);
```

5. Usar calldata en vez de memory para inputs:

```
// Malo
function process(uint256[] memory data) external { }

// Bueno (data es solo lectura)
function process(uint256[] calldata data) external { }
```

En Banco ZK

Gas en Banco ZK:
Como L2, Banco ZK tiene sus propios costos de gas internos, más el costo de publicar datos en L1.
Costos L2 (dentro de Banco ZK):

- Transferencia privada: ∼500 gas interno
- Verificación de prueba: incluida en batch
- Mucho más barato que L1

Costos L1 (data availability):

- Publicar batch: ∼100K-500K gas L1
- Dividido entre todas las txs del batch
- Con 1000 txs: ∼100-500 gas L1 por tx

Optimizaciones específicas:

- Comprimir datos al máximo antes de publicar
- Usar blobs (EIP-4844) cuando disponible
- Batching agresivo para amortizar costos
- Pruebas ZK permiten no publicar datos completos

2.5 EVM (Ethereum Virtual Machine)

Definición 2.26 (EVM)**.** Máquina virtual stack-based que ejecuta bytecode de smart contracts.

Intuición

La EVM es como una computadora virtual que existe en todos los nodos de Ethereum simultáneamente. Cuando ejecutas un smart contract:

1. Tu transacción llega a los validadores
2. Cada validador ejecuta el mismo código en su EVM
3. Todos llegan al mismo resultado (determinismo)
4. El nuevo estado se guarda

Es "la computadora mundial": una máquina que nadie controla pero todos pueden usar.

2.5.1 Arquitectura de la EVM

Definición 2.27 (Componentes de la EVM)**. 1. Stack:**

- Máximo 1024 elementos
- Cada elemento: 256 bits (32 bytes)
- Operaciones: PUSH, POP, DUP, SWAP
- LIFO (Last In, First Out)

2. Memory:

- Array de bytes, expandible
- Direccionada por byte (no por word)
- Volátil: se borra después de la ejecución
- Costo: 3 gas base + expansión cuadrática

3. Storage:

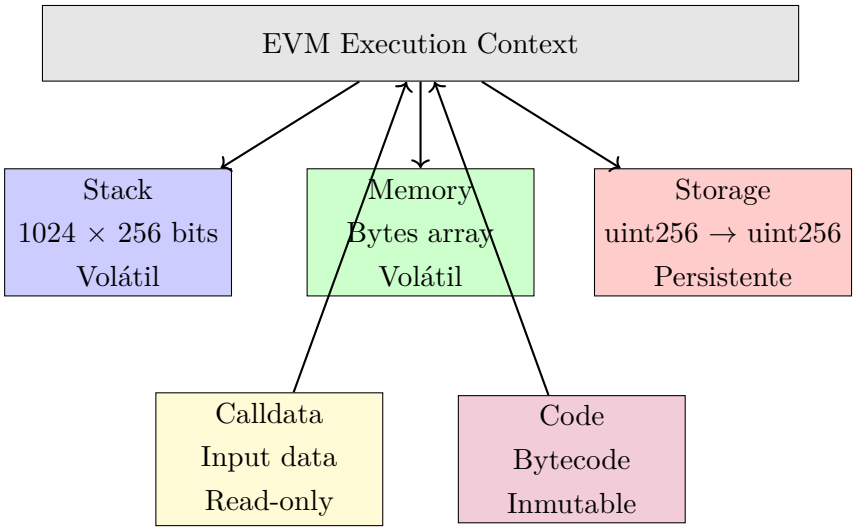
- Mapping persistente: uint256 → uint256
- Cada contrato tiene su propio storage
- Persistente entre transacciones
- MUY caro: 20,000 gas para escribir nuevo valor

4. Calldata:

- Datos de entrada de la transacción
- Read-only
- Más barato que memory

5. Code:

- Bytecode del contrato
- Inmutable después del deploy
- Accesible con CODECOPY



2.5.2 Opcodes de la EVM

Definición 2.28 (Categorías de Opcodes)**.** La EVM tiene ∼140 opcodes organizados en categorías:

Operaciones Aritméticas

Opcode	Hex	Gas	Descripción
ADD	0x01	3	$a + b$
MUL	0x02	5	$a \times b$
SUB	0x03	3	$a - b$
DIV	0x04	5	$a \div b$ (entero)
SDIV	0x05	5	División con signo
MOD	0x06	5	$a \mod b$
SMOD	0x07	5	Módulo con signo
ADDMOD	0x08	8	$(a + b) \mod N$
MULMOD	0x09	8	$(a \times b) \mod N$
EXP	0x0a	$10 + 50 \times$ bytes	a^b
SIGNEXTEND	0x0b	5	Extensión de signo

Operaciones de Comparación y Lógica

Opcode	Hex	Gas	Descripción
LT	0x10	3	$a < b$
GT	0x11	3	$a > b$
SLT	0x12	3	$<$ con signo
SGT	0x13	3	$>$ con signo
EQ	0x14	3	$a = b$
ISZERO	0x15	3	$a = 0$
AND	0x16	3	Bitwise AND
OR	0x17	3	Bitwise OR
XOR	0x18	3	Bitwise XOR
NOT	0x19	3	Bitwise NOT
SHL	0x1b	3	Shift left
SHR	0x1c	3	Shift right (logical)
SAR	0x1d	3	Shift right (arithmetic)

Operaciones de Storage y Memory

Opcode	Hex	Gas	Descripción
SLOAD	0x54	2100/100	Leer de storage
SSTORE	0x55	20000/2900	Escribir a storage
MLOAD	0x51	3	Leer 32 bytes de memory
MSTORE	0x52	3	Escribir 32 bytes a memory
MSTORE8	0x53	3	Escribir 1 byte a memory
CALLDATALOAD	0x35	3	Leer de calldata
CALLDATACOPY	0x37	3+	Copiar calldata a memory
CODECOPY	0x39	3+	Copiar code a memory

Operaciones de Control de Flujo

Opcode	Hex	Gas	Descripción
STOP	0x00	0	Terminar ejecución
JUMP	0x56	8	Salto incondicional
JUMPI	0x57	10	Salto condicional
JUMPDEST	0x5b	1	Marca destino válido
PC	0x58	2	Program counter
RETURN	0xf3	0	Retornar datos
REVERT	0xfd	0	Revertir con datos
INVALID	0xfe	all	Instrucción inválida
SELFDESTRUCT	0xff	5000+	Destruir contrato

Operaciones de Llamada

Opcode	Hex	Gas	Descripción
CALL	0xf1	100-2600	Llamar otro contrato
CALLCODE	0xf2	100-2600	Llamar con contexto propio
DELEGATECALL	0xf4	100-2600	Llamar preservando context
STATICCALL	0xfa	100-2600	Llamada read-only
CREATE	0xf0	32000	Crear nuevo contrato
CREATE2	0xf5	32000	Crear con salt determinístico

Matemáticas

Diferencia entre CALL, DELEGATECALL, STATICCALL:
CALL:

- Ejecuta código del contrato destino
- msg.sender = contrato que hace CALL
- Storage del contrato destino

DELEGATECALL:

- Ejecuta código del contrato destino
- msg.sender = sender original
- Storage del contrato que hace DELEGATECALL
- Usado para proxies y librerías

STATICCALL:

- Como CALL pero read-only
- Revierte si intenta modificar estado
- Usado para views y pure functions

Precompilados (Precompiled Contracts)

Definición 2.29 (Precompilados)**.** Contratos especiales con direcciones fijas que implementan operaciones costosas de forma nativa.

Address	Nombre	Gas	Función
0x01	ECRECOVER	3000	Recuperar pubkey de firma
0x02	SHA256	60+	Hash SHA-256
0x03	RIPEMD160	600+	Hash RIPEMD-160
0x04	IDENTITY	15+	Copiar datos (usado por DATACOPY)
0x05	MODEXP	variable	Exponenciación modular
0x06	ECADD	150	Suma en BN254
0x07	ECMUL	6000	Multiplicación escalar BN254
0x08	ECPAIRING	34000+	Pairing check BN254
0x09	BLAKE2F	1+	Compresión BLAKE2
0x0a	KZG_POINT_EVALUATION	50000	Verificar evaluación KZG

Idea Clave

¿Por qué precompilados?
Algunas operaciones son demasiado costosas para implementar en bytecode EVM:

- Verificación de firmas ECDSA
- Operaciones de curvas elípticas para ZK
- Hashes criptográficos

Los precompilados las implementan en código nativo, haciendo prácticas operaciones que serían prohibitivamente caras.
Ejemplo: Verificar una prueba Groth16 on-chain usa el precompilado de pairing (0x08).

2.5.3 Ejemplo de Ejecución EVM

Ejemplo 2.11 (Ejecución paso a paso)**. Código Solidity:**

```
function add(uint256 a, uint256 b) public pure returns (uint256) {
    return a + b;
}
```

Bytecode (simplificado):

```
PUSH1 0x00      // Posición en calldata para a
CALLDATALOAD    // Cargar a
PUSH1 0x20      // Posición para b
CALLDATALOAD    // Cargar b
```

```
ADD             // a + b
PUSH1 0x00      // Posición en memory
MSTORE          // Guardar resultado
PUSH1 0x20      // Tamaño del resultado
PUSH1 0x00      // Posición del resultado
RETURN          // Retornar
```

Ejecución con a=5, b=3:

Step	Opcode	Stack (top first)	Gas
0	PUSH1 0x00	[0x00]	3
1	CALLDATALOAD	[0x05]	3
2	PUSH1 0x20	[0x20, 0x05]	3
3	CALLDATALOAD	[0x03, 0x05]	3
4	ADD	[0x08]	3
5	PUSH1 0x00	[0x00, 0x08]	3
6	MSTORE	[]	3
7	PUSH1 0x20	[0x20]	3
8	PUSH1 0x00	[0x00, 0x20]	3
9	RETURN	–	0
	Total		27

Memory después de MSTORE:

```
0x00: 0x0000000000000000000000000000000000000000000000000000000000000008
```

RETURN devuelve los bytes [0x00, 0x20), que es el número 8.

2.5.4 Contexto de Ejecución

Definición 2.30 (Contexto de Ejecución)**.** Variables disponibles durante la ejecución:

Información del bloque:

- `BLOCKHASH`: Hash de bloque reciente
- `COINBASE`: Address del validador
- `TIMESTAMP`: Tiempo del bloque
- `NUMBER`: Número de bloque
- `DIFFICULTY/PREVRANDAO`: Aleatoriedad (post-merge)
- `GASLIMIT`: Gas limit del bloque
- `CHAINID`: ID de la cadena
- `BASEFEE`: Base fee actual

Información de la transacción:

- `ORIGIN`: EOA que inició la transacción
- `GASPRICE`: Precio del gas
- `BLOBHASH`: Hashes de blobs (EIP-4844)
- `BLOBBASEFEE`: Base fee de blobs

Información del call:

- `ADDRESS`: Dirección del contrato actual
- `CALLER`: Quien llamó (msg.sender)
- `CALLVALUE`: ETH enviado (msg.value)
- `CALLDATASIZE`: Tamaño de calldata

Advertencia

tx.origin vs msg.sender:

```
Usuario (EOA) -> Contrato A -> Contrato B

En Contrato B:
  tx.origin = Usuario (EOA)     // ORIGIN opcode
  msg.sender = Contrato A       // CALLER opcode
```

Nunca usar tx.origin para autenticación:

- Un contrato malicioso puede engañar al usuario para llamarlo
- Luego llamar a tu contrato
- tx.origin será el usuario, pero msg.sender será el contrato malicioso

2.6 Estado Global (State)

Definición 2.31 (Estado)**.** Mapeo de direcciones a cuentas:

$$\sigma : \text{Address} \to (\text{nonce}, \text{balance}, \text{storageRoot}, \text{codeHash})$$

- **EOA:** Externally Owned Account (usuario)
- **Contract:** Tiene código y storage
- Estado almacenado en Merkle Patricia Trie
- State root en cada block header

2.6.1 Tipos de Cuentas

Definición 2.32 (Externally Owned Account (EOA))**.** Cuenta controlada por una clave privada.

Características:

- Creada generando un par de claves
- Address = últimos 20 bytes de Keccak256(pubkey)
- Puede iniciar transacciones
- No tiene código
- storageRoot y codeHash son valores especiales vacíos

Campos de estado:

- `nonce`: Número de transacciones enviadas
- `balance`: ETH poseído (en wei)
- `storageRoot`: EMPTY_ROOT
- `codeHash`: KECCAK256_NULL

Definición 2.33 (Contract Account)**.** Cuenta con código ejecutable.

Características:

- Creada por una transacción de deploy
- Address determinada por creator + nonce (CREATE) o salt (CREATE2)
- No puede iniciar transacciones (solo responder)
- Tiene código inmutable
- Tiene storage persistente

Campos de estado:

- `nonce`: Número de contratos creados por este contrato
- `balance`: ETH poseído
- `storageRoot`: Raíz del MPT de storage
- `codeHash`: Keccak256 del bytecode

Matemáticas

Cálculo de direcciones:

Para EOA:

$$\text{address} = \text{Keccak256}(\text{pubkey})[12:32]$$

Para CREATE (nonce-based):

$$\text{address} = \text{Keccak256}(\text{RLP}([\text{sender}, \text{nonce}]))[12:32]$$

Para CREATE2 (determinístico):

$$\text{address} = \text{Keccak256}(\texttt{0xff} \| \text{sender} \| \text{salt} \| \text{Keccak256}(\text{initCode}))[12:32]$$

CREATE2 permite predecir la dirección antes del deploy, útil para:

- Counterfactual instantiation
- Factory patterns
- Upgradeable proxies

2.6.2 Storage Layout

Definición 2.34 (Storage Layout)**.** El storage de un contrato es un mapping de 256-bit a 256-bit:

$$\text{storage} : \mathbb{Z}_{2^{256}} \to \mathbb{Z}_{2^{256}}$$

Inicialmente, todos los slots son 0.

Idea Clave

Layout de Solidity:
Variables estáticas:

```
contract Example {
    uint256 a;      // slot 0
    uint256 b;      // slot 1
    uint128 c;      // slot 2 (primeros 16 bytes)
    uint128 d;      // slot 2 (siguientes 16 bytes, packed)
    uint256 e;      // slot 3
}
```

Arrays dinámicos:

```
uint256[] arr;  // slot n guarda length
// arr[i] está en: keccak256(n) + i
```

Mappings:

```
mapping(address => uint256) balances;  // slot m
// balances[addr] está en: keccak256(addr . m)
// donde . es concatenación
```

Mappings anidados:

```
mapping(address => mapping(address => uint256)) allowances;  // slot k
// allowances[a][b] está en: keccak256(b . keccak256(a . k))
```

Ejemplo 2.12 (Encontrar slot de storage). `contract Token {`

```
    string public name;           // slot 0
    string public symbol;         // slot 1
    uint8 public decimals;        // slot 2
    uint256 public totalSupply;   // slot 3
    mapping(address => uint256) public balanceOf;  // slot 4
    mapping(address => mapping(address => uint256)) public allowance; // slot 5
}

// Para encontrar balanceOf[0x123...]:
slot = keccak256(abi.encode(0x123..., 4))

// Para encontrar allowance[0x123...][0x456...]:
inner_slot = keccak256(abi.encode(0x123..., 5))
slot = keccak256(abi.encode(0x456..., inner_slot))
```

En Python:

```
from web3 import Web3

def get_balance_slot(address, mapping_slot=4):
    # Pad address to 32 bytes
    address_padded = address.lower()[2:].zfill(64)
    slot_padded = hex(mapping_slot)[2:].zfill(64)

    # Concatenate and hash
    data = address_padded + slot_padded
    slot = Web3.keccak(hexstr=data).hex()
    return slot

# Ejemplo
slot = get_balance_slot("0x123...")
# Ahora puedes usar eth_getStorageAt(contract, slot)
```

2.6.3 State Trie

Matemáticas

Estructura del State Trie:
El estado completo de Ethereum se almacena en un Merkle Patricia Trie:

- **Key**: Keccak256(address) - 32 bytes

- **Value**: RLP([nonce, balance, storageRoot, codeHash])

Cada contrato tiene su propio Storage Trie:

- **Key**: Keccak256(slot) - 32 bytes
- **Value**: RLP(valor) - hasta 32 bytes

Tamaño del estado (2024):

- ∼250 millones de cuentas
- ∼100+ GB de datos de estado
- Creciendo constantemente

2.7 Nodos

Intuición

Un nodo es una computadora que participa en la red de Ethereum. Diferentes tipos de nodos tienen diferentes responsabilidades y requisitos de recursos.
La descentralización de la red depende de que mucha gente pueda correr nodos.

2.7.1 Full Node

Definición 2.35 (Full Node)**.**

- Almacena estado actual completo
- Valida todas las transacciones y bloques
- Puede no tener historial completo (pruned)
- Participa en la red P2P

Matemáticas

Requisitos de Full Node (2024):

Recurso	Requisito
CPU	4+ cores
RAM	16+ GB
Storage (pruned)	1+ TB SSD
Bandwidth	25+ Mbps
Tiempo de sync	1-7 días

Clientes populares:

- **Geth**: Go, el más usado
- **Nethermind**: C#, rápido sync

- **Besu**: Java, enterprise-friendly
- **Erigon**: Go, optimizado para storage
- **Reth**: Rust, nuevo y rápido

2.7.2 Light Node

Definición 2.36 (Light Node)**.**

- Solo block headers
- Verifica con Merkle proofs
- Confía en full nodes para datos
- Usado en wallets móviles

Idea Clave

Cómo funciona un Light Node:

1. Descarga solo headers (∼500 bytes cada uno)
2. Para verificar un dato (ej: balance), pide Merkle proof a un full node
3. Verifica que el proof es consistente con el state root en el header
4. Confía en que el header es válido (verificado por consenso)

Trade-off:

- Pro: Muy ligero (∼100 MB vs ∼1 TB)
- Con: Depende de full nodes para datos
- Con: Puede ser engañado si todos los peers son maliciosos

2.7.3 Archive Node

Definición 2.37 (Archive Node)**.**

- Todo el historial de estados
- Puede consultar estado en cualquier bloque
- Requiere varios TB de almacenamiento
- Usado por exploradores, analytics

Matemáticas

Requisitos de Archive Node (2024):

Recurso	Requisito
Storage	15+ TB SSD
RAM	32+ GB
Tiempo de sync	Semanas a meses
Costo de hardware	$5,000+

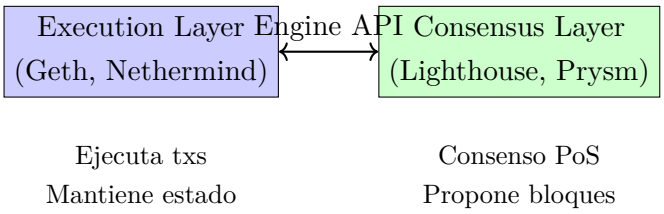
Uso:

- Etherscan y otros exploradores
- Servicios de analytics (Dune, Nansen)
- Debugging de transacciones históricas
- Investigación

2.7.4 Validator Node (Post-Merge)

Definición 2.38 (Validator Node)**.** Full node que además:

- Tiene 32 ETH depositados
- Corre Consensus Layer client (Lighthouse, Prysm, etc.)
- Propone y attesta bloques
- Gana recompensas por participación



2.7.5 RPC Nodes y Providers

Definición 2.39 (RPC Node)**.** Nodo que expone una API JSON-RPC para que aplicaciones interactúen con Ethereum.

Métodos RPC comunes:

```
eth_blockNumber          // Último bloque
eth_getBalance           // Balance de cuenta
eth_getTransactionCount  // Nonce de cuenta
eth_call                 // Simular llamada (view)
eth_sendRawTransaction   // Enviar tx firmada
eth_getTransactionReceipt // Resultado de tx
eth_getLogs              // Eventos filtrados
eth_getStorageAt         // Leer storage slot
```

Providers populares:

Provider	Tier gratuito	Notas
Infura	100k req/día	El más antiguo
Alchemy	300M CU/mes	Buen dashboard
QuickNode	Limitado	Multi-chain
Ankr	1M req/mes	Descentralizado

Advertencia

Dependencia de providers centralizados:
La mayoría de dApps usan Infura o Alchemy en lugar de nodos propios.
Riesgos:

- Censura: provider puede bloquear ciertas txs
- Privacidad: provider ve todas tus queries
- Single point of failure: si cae, tu app no funciona

Solución: Correr tu propio nodo o usar providers descentralizados (Pocket Network).

En Banco ZK

Nodos en Banco ZK:
L1 (Ethereum):

- Necesitamos nodo completo para verificar estado
- El sequencer monitorea eventos del contrato puente
- Archive node útil para debugging

L2 (Banco ZK):

- Nuestros propios nodos procesan txs privadas
- Prover nodes generan las pruebas ZK
- Users pueden correr light nodes para verificar

Descentralización futura:

- Múltiples sequencers en rotación
- Red de provers descentralizada
- Light clients verifican proofs sin confiar en nadie

2.8 Mempool

Definición 2.40 (Mempool)**.** Pool de transacciones pendientes esperando inclusión en un bloque.

- Transacciones broadcast a la red P2P
- Cada nodo mantiene su propia mempool
- Validadores/mineros seleccionan de su mempool
- Típicamente ordenado por fee (mayor fee primero)
- Público por defecto: cualquiera puede ver txs pendientes

2.8.1 Cómo Funciona la Mempool

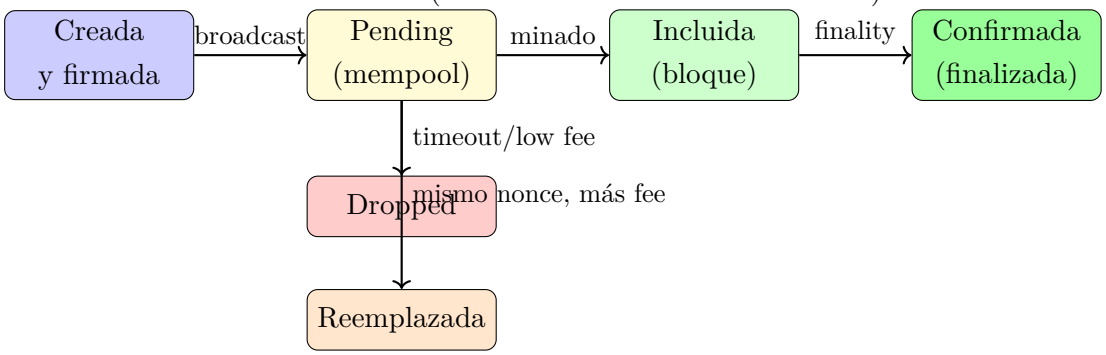
Intuición

La mempool es como una sala de espera para transacciones. Cuando envías una tx:

1. Tu wallet la firma y la envía a un nodo (vía RPC)
2. El nodo la valida (firma correcta, balance suficiente, etc.)
3. Si es válida, la añade a su mempool
4. La propaga a otros nodos via P2P (gossip)
5. Un validador la incluye en su próximo bloque
6. La tx sale de la mempool de todos los nodos

Como la mempool es pública, cualquiera puede ver qué transacciones están esperando. Esto tiene implicaciones importantes para MEV.

Definición 2.41 (Ciclo de Vida de una Transacción)**.**



2.8.2 Validación en Mempool

Definición 2.42 (Checks de Validación)**.** Antes de aceptar una tx en mempool, el nodo verifica:

Checks estáticos:

- Firma ECDSA válida
- Formato de tx correcto
- Gas limit $\geq$ gas intrínseco (21000 + data)
- Gas limit $\leq$ block gas limit

Checks de estado:

- Nonce = nonce actual de la cuenta
- Balance $\geq$ value + gasLimit $\times$ gasPrice
- Sender no está en blacklist (si aplica)

Checks de mempool:

- No duplicada (mismo hash)
- Si mismo nonce existe: nuevo fee debe ser $> 10\%$ mayor

2.8.3 Políticas de Mempool

Matemáticas

Límites y políticas típicas:

Parámetro	Valor típico (Geth)
Tamaño máximo	5,000 txs
Memoria máxima	1 GB
Tiempo máximo	3 horas
Gap máximo de nonce	64
Fee mínimo	1 gwei
Reemplazo mínimo	+10% fee

Cuando la mempool está llena:

- Txs con menor fee son evictadas primero
- Txs muy viejas son descartadas
- Nonce gaps excesivos son rechazados

2.8.4 Propagación P2P

Definición 2.43 (Gossip Protocol)**.** Las transacciones se propagan por la red usando gossip:

1. Nodo A recibe tx nueva
2. A envía announcement a sus peers

3. Peers que no tienen la tx la solicitan
4. A envía la tx completa
5. Peers repiten el proceso

Matemáticas

Optimizaciones de propagación:
Transaction announcements (EIP-2464):

- En lugar de enviar tx completa, enviar solo hash
- Receptor pide txs que no tiene
- Reduce bandwidth en ∼50%

Transaction pooling:

- Agrupar múltiples announcements
- Reducir overhead de mensajes

Latencia típica de propagación:

- A mayoría de nodos: < 1 segundo
- A 95% de la red: < 3 segundos
- Depende de la conectividad

2.8.5 Mempool Privadas y PBS

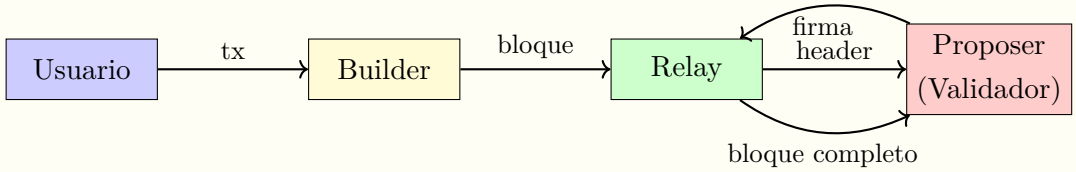
Definición 2.44 (Private Mempool)**.** Mempool no pública donde las transacciones no se propagan por gossip.

Ejemplos:

- Flashbots Protect: txs van directo a builders
- MEV Blocker: coalition de builders éticos
- Private RPC endpoints de builders

Idea Clave

Proposer-Builder Separation (PBS):
En Ethereum post-merge con MEV-Boost:



Flujo:

1. Users envían txs a builders (privado o público)
2. Builders construyen bloques optimizados
3. Builders envían bloques a relays
4. Relays muestran solo headers a proposers
5. Proposer firma el header más rentable
6. Relay revela el bloque completo
7. Proposer lo propone a la red

Beneficios:

- Proposers no necesitan construir bloques
- Builders compiten por eficiencia
- MEV se redistribuye más equitativamente

2.9 MEV (Maximal Extractable Value)

Definición 2.45 (MEV)**.** Valor que productores de bloques pueden extraer reordenando, incluyendo o excluyendo transacciones.

Intuición

Imagina que eres el dueño de un restaurante y puedes decidir el orden en que atiendes a los clientes. Si un cliente quiere comprar el último pastel, y ves que otro cliente también lo quiere, podrías:

1. Comprarlo tú primero
2. Vendérselo al que pague más
3. O simplemente cobrar una "cuota de prioridad"

En Ethereum, el "dueño del restaurante" es el builder/validator, y las "órdenes" son transacciones. MEV es el valor que pueden extraer por controlar el orden.

2.9.1 Tipos de MEV

Definición 2.46 (Tipos de MEV)**. 1. Frontrunning:** Ejecutar ANTES de una tx víctima.

```
Víctima: comprar 100 ETH de TOKEN a precio X
Attacker: compra TOKEN antes → sube precio
Víctima: compra a precio X +  (peor precio)
```

```
Attacker: vende TOKEN con profit
```

2. Backrunning: Ejecutar INMEDIATAMENTE DESPUÉS de una tx.

```
Tx legítima: gran compra que mueve precio
Backrunner: arbitraje entre DEXs aprovechando el movimiento
```

3. Sandwich Attack: Frontrun + Backrun combinados.

```
1. Attacker: compra TOKEN (frontrun)
2. Víctima: compra TOKEN (precio más alto)
3. Attacker: vende TOKEN (backrun, profit)
```

4. Liquidations: Competir por liquidar posiciones en lending protocols.

```
Posición underwater detectada
Múltiples bots compiten por liquidarla
El más rápido/mejor posicionado gana la recompensa
```

5. Arbitraje: Explotar diferencias de precio entre mercados.

```
ETH/USDC en Uniswap: 2000
ETH/USDC en Sushiswap: 2010
Arbitrageur: compra en Uni, vende en Sushi
Profit: $10 por ETH (menos fees)
```

2.9.2 Anatomía de un Sandwich Attack

Ejemplo 2.13 (Sandwich Attack detallado)**. Escenario:**

- Alice quiere comprar TOKEN con 10 ETH en Uniswap
- Slippage tolerance: 1%
- Pool: 1000 ETH / 1,000,000 TOKEN (precio: 1000 TOKEN/ETH)

Sin ataque:

```
Alice compra con 10 ETH
Pool después: 1010 ETH / 990,099 TOKEN
Alice recibe: ~9,901 TOKEN
Precio efectivo: ~990 TOKEN/ETH (0.99% slippage)
```

Con sandwich:

```
1. Frontrun - Attacker compra con 5 ETH:
   Pool: 1005 ETH / 995,025 TOKEN
   Attacker recibe: ~4,975 TOKEN

2. Víctima (Alice) compra con 10 ETH:
   Pool: 1015 ETH / 985,197 TOKEN
```

```
   Alice recibe: ~9,828 TOKEN (peor que sin ataque)

3. Backrun - Attacker vende ~4,975 TOKEN:
   Pool: 1009.95 ETH / 990,172 TOKEN
   Attacker recibe: ~5.05 ETH

Resultado:
- Alice: recibió 9,828 en vez de 9,901 TOKEN (73 TOKEN menos)
- Attacker: gastó 5 ETH, recibió 5.05 ETH = 0.05 ETH profit
- Attacker también pagó gas, pero con bundles eficientes es rentable
```

2.9.3 Ecosistema MEV

Definición 2.47 (Actores del Ecosistema MEV)**. 1. Searchers:**

- Encuentran oportunidades de MEV
- Escriben bots y algoritmos
- Compiten enviando bundles a builders
- Pagan "tips" a builders por inclusión

2. Builders:

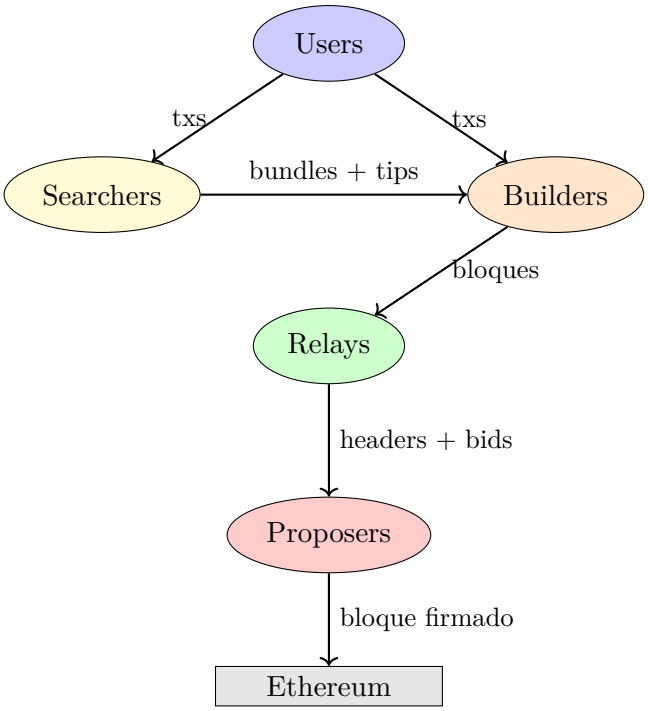
- Construyen bloques óptimos
- Reciben txs de searchers y mempool pública
- Ordenan para maximizar valor
- Compiten por que proposers elijan su bloque

3. Relays:

- Intermediarios entre builders y proposers
- Verifican bloques de builders
- Protegen a proposers de bloques inválidos
- Ejemplos: Flashbots, BloXroute, Ultrasound

4. Proposers (Validators):

- Proponen bloques a la red
- Eligen el bloque más rentable de los relays
- Reciben pago del builder
- No necesitan expertise en MEV



2.9.4 Flashbots

Definición 2.48 (Flashbots)**.** Organización e infraestructura para MEV "fair".

Productos:

- **Flashbots Auction**: Mercado de bundles
- **Flashbots Protect**: RPC que protege de sandwich
- **MEV-Boost**: Software para PBS
- **MEV-Share**: Redistribución de MEV a users
- **SUAVE**: Futuro sistema de encrypted mempools

Idea Clave

Flashbots Protect - Cómo funciona:

1. User envía tx a Flashbots RPC (no mempool pública)
2. Tx va directo a builders de Flashbots
3. Builders incluyen sin posibilidad de sandwich
4. Si tx no es incluida en X bloques, se puede enviar a mempool pública

Beneficios:

- No aparece en mempool pública = no puede ser sandwiched
- Si tx falla, no pagas gas (no hay revert público)

- Gratis para usuarios

Trade-offs:

- Confías en Flashbots para no hacer MEV ellos mismos
- Puede ser más lento (menos builders)
- Centralización hacia Flashbots

2.9.5 MEV por los Números

Matemáticas

Estadísticas de MEV (2023-2024):

Métrica	Valor
MEV total extraído (histórico)	$> \$700M$
MEV diario promedio	$\sim \$1 - 5M$
% bloques via MEV-Boost	$\sim 90\%$
Builders activos	$\sim 10 - 20$
Top builder market share	$\sim 40 - 60\%$
Sandwich attacks diarios	$\sim 1000 - 5000$

Distribución típica del MEV:

- Searchers: ~10-30% (compiten fuertemente)
- Builders: ~5-15%
- Proposers/Validators: ~60-80%
- La competencia ha comprimido márgenes

2.9.6 Impacto Negativo del MEV

Advertencia

Problemas causados por MEV:

1. **Peor ejecución para usuarios:**
   - Sandwich attacks roban valor de swaps
   - Usuarios pagan más slippage del necesario
   - Estimación: usuarios pierden $1-10 por swap afectado
2. **Centralización:**
   - Economías de escala favorecen grandes operadores

- Top 3 builders dominan >80% del mercado
- Proposers pequeños no pueden competir

3. **Congestión de red:**
   - Bots compiten con txs de spam
   - Gas prices artificialmente altos durante oportunidades
   - Failed txs desperdician blockspace
4. **Inestabilidad del consenso:**
   - Incentivo para reorgs si MEV > block reward
   - "Time-bandit attacks" teóricamente posibles
   - Reduce seguridad de finalidad

2.9.7 Soluciones y Mitigaciones

Idea Clave

Estrategias contra MEV:

1. **Private mempools:**
   - Flashbots Protect, MEV Blocker
   - Txs no visibles hasta inclusión
   - Elimina frontrunning
2. **Commit-reveal schemes:**
   - Usuario envía hash de tx primero
   - Revela contenido después de inclusión
   - Ejemplo: Submarine sends
3. **Batch auctions:**
   - Agrupar órdenes y ejecutar al mismo precio
   - CoW Protocol, 1inch Fusion
   - Elimina ventaja de orden
4. **Encrypted mempools:**
   - Txs encriptadas hasta threshold de validadores
   - Shutter Network, SUAVE (futuro)

- Nadie puede ver contenido antes de ordenar

5. **MEV redistribution:**
   - MEV-Share: devolver parte a usuarios
   - MEV taxes: protocolo captura MEV
   - Reducir daño aunque no eliminar

2.9.8 MEV-Share

Definición 2.49 (MEV-Share)**.** Protocolo de Flashbots para redistribuir MEV a usuarios.

Funcionamiento:

1. Usuario envía tx a Flashbots con hints
2. Hints revelan info parcial (ej: "swap en Uniswap")
3. Searchers pueden backrun basándose en hints
4. MEV generado se divide entre user y searcher

Ejemplo:

```
User swap genera $100 de oportunidad de arbitraje
Sin MEV-Share: Searcher toma $100, user recibe $0
Con MEV-Share: Searcher toma $20, user recibe $80
```

2.9.9 Aplicación en Banco ZK

En Banco ZK

La privacidad de Banco ZK mitiga MEV:

1. **Montos ocultos:**
   - Transacciones no revelan cantidades
   - No se puede calcular profit de sandwich
   - Frontrunning es inútil sin saber el impacto
2. **Intenciones ocultas:**
   - No se sabe qué quiere hacer el usuario
   - ¿Es un swap? ¿Una transferencia? ¿Un depósito?
   - Sin esta info, no hay MEV que extraer
3. **Encrypted mempools nativos:**
   - Las txs están encriptadas por diseño

- Sequencer no puede ver contenido
- Ordenamiento "fair" por defecto

4. Batch processing:

- Múltiples txs se procesan juntas
- Orden dentro del batch puede ser aleatorio
- Reduce ventaja de timing

Trade-offs:

- Privacidad tiene costo computacional (ZK proofs)
- Algunas operaciones no pueden ser privadas (bridges)
- MEV del sequencer aún posible en teoría

Diseño del sequencer:

```
// El sequencer de Banco ZK:
1. Recibe txs encriptadas de usuarios
2. No puede ver contenido (solo metadata mínima)
3. Ordena por timestamp de llegada (FIFO)
4. Genera prueba ZK del batch
5. Publica batch + proof en L1

// Incluso si quisiera hacer MEV:
- No sabe qué hacen las txs
- No puede reordenar basándose en contenido
- Solo puede censurar (detectable y punishable)
```

Mitigación de censura:

- Múltiples sequencers en rotación (futuro)
- Forced inclusion via L1 si sequencer censura
- Incentivos alineados: sequencer gana fees, no MEV

2.10 Resumen del Bloque 2

Idea Clave

Puntos clave de Blockchain Base (L1):

1. **L1 proporciona cuatro funciones:**
   - Consenso sobre el estado
   - Disponibilidad de datos
   - Ejecución de transacciones
   - Settlement final
2. **Consenso:**
   - PoW: seguridad por energía gastada, 51% attack
   - PoS: seguridad por capital en stake, slashing
   - PoA: validadores autorizados, centralizado pero rápido
3. **Modelos de transacciones:**
   - UTXO (Bitcoin): outputs gastables, privacy natural
   - Account (Ethereum): balances persistentes, smart contracts
4. **Gas y fees:**
   - Gas mide costo computacional
   - EIP-1559: base fee (quemado) + priority fee
   - Storage es muy caro, optimizar es crucial
5. **EVM:**
   - Máquina virtual stack-based
   - ∼140 opcodes
   - Precompilados para operaciones costosas
   - Determinística: todos los nodos llegan al mismo resultado
6. **Estado:**
   - Merkle Patricia Trie
   - EOAs y Contract accounts
   - Storage layout importa para gas

7. **Nodos:**
   - Full: estado actual, valida todo
   - Light: solo headers, usa proofs
   - Archive: todo el historial
   - Validator: participa en consenso
8. **Mempool:**
   - Pool de txs pendientes
   - Pública por defecto (problema para MEV)
   - PBS separa construcción de proposición
9. **MEV:**
   - Valor extraíble por reordenamiento
   - Tipos: frontrunning, sandwich, arbitraje, liquidaciones
   - Soluciones: private mempools, batch auctions, encryption
   - Privacidad (como en Banco ZK) mitiga MEV naturalmente

Chapter 3

Escalabilidad (L2/L3)

Bloque 3 de 20

3.1 El Problema de Escalabilidad (Trilema)

Definición 3.1 (Trilema de Escalabilidad)**.** Es difícil optimizar simultáneamente:

- Escalabilidad (throughput)
- Descentralización (número de nodos)
- Seguridad (costo de ataque)

Throughput actual:

- Bitcoin: ∼7 TPS
- Ethereum: ∼15 TPS
- Visa: ∼65,000 TPS

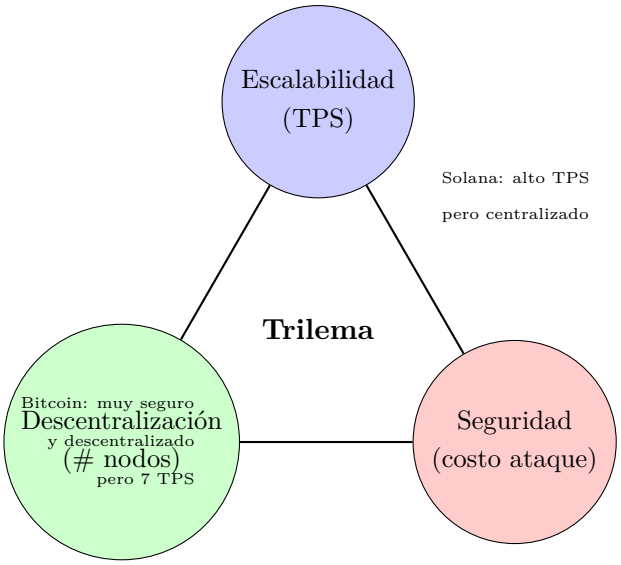
3.1.1 Historia del Trilema

Observación 3.1. El término "trilema de escalabilidad" fue popularizado por Vitalik Buterin alrededor de 2017, aunque el concepto existía antes en teoría de sistemas distribuidos.

La intuición:

- Para más throughput: procesar más txs por segundo
- Esto requiere: nodos más potentes O menos validación
- Nodos más potentes = menos gente puede participar = centralización
- Menos validación = más fácil de atacar = menos seguridad

No es un teorema matemático estricto, pero captura un trade-off fundamental en el diseño de blockchains.



3.1.2 Análisis de las Tres Propiedades

Definición 3.2 (Escalabilidad)**.** Capacidad de procesar un número creciente de transacciones.

Métricas:

- TPS (Transactions Per Second): transacciones por segundo
- Throughput: datos procesados por segundo
- Latency: tiempo desde envío hasta confirmación
- Finality: tiempo hasta que tx es irreversible

Matemáticas

¿Por qué Ethereum solo hace ∼15 TPS?

Límites actuales:

- Block gas limit: 30M gas
- Block time: 12 segundos
- Tx simple (transfer): 21,000 gas

Cálculo:

$$\text{TPS}_{\text{max}} = \frac{30,000,000 \text{ gas}}{12 \text{ s} \times 21,000 \text{ gas/tx}} \approx 119 \text{ TPS}$$

Pero en la práctica:

- Muchas txs son contratos complejos (>100K gas)
- El promedio real es ∼15-30 TPS
- Con más gas por tx = menos TPS

¿Por qué no simplemente aumentar el gas limit?

- Bloques más grandes = más tiempo para propagar
- Más validación = nodos más caros
- Estado crece más rápido = storage más caro
- Centralización hacia operadores grandes

Definición 3.3 (Descentralización)**.** Distribución del poder de decisión y validación entre múltiples entidades independientes.

Métricas:

- Número de nodos full
- Nakamoto coefficient (mínimo para 51%)
- Distribución geográfica
- Diversidad de clientes (software)
- Requisitos de hardware para participar

Ejemplo 3.1 (Requisitos de hardware y descentralización)**.**

Red	RAM	Storage	Bandwidth	Nodos
Bitcoin	2 GB	500 GB	5 Mbps	∼15,000
Ethereum	16 GB	1 TB	25 Mbps	∼6,000
Solana	128 GB	2 TB	1 Gbps	∼1,900
BSC	32 GB	2 TB	100 Mbps	∼21 (validators)

Solana tiene alto TPS pero pocos pueden correr nodos. Bitcoin tiene bajo TPS pero cualquiera puede validar con hardware modesto.

Definición 3.4 (Seguridad)**.** Resistencia a ataques que intenten modificar el estado o censurar transacciones.

Métricas:

- Costo de ataque 51%
- Tiempo para finalidad
- Resistencia a censura
- Historial de ataques exitosos

3.1.3 Enfoques para Escalar

Idea Clave

Dos filosofías de escalabilidad:

1. Escalar L1 (on-chain):

- Aumentar block size/gas limit
- Reducir block time
- Optimizar ejecución
- Sharding del estado
- Trade-off: sacrificar algo del trilema

2. Escalar off-chain (L2):

- Procesar txs fuera de L1
- Solo publicar resultados/proofs en L1
- Heredar seguridad de L1
- Trade-off: complejidad adicional

Ethereum eligió el camino L2:
"The Rollup-Centric Roadmap" (2020):

- L1 optimizado para seguridad y DA
- L2s manejan la ejecución masiva
- Mejor de ambos mundos

Observación 3.2. **Historia de los intentos de escalar Ethereum:**

Año	Propuesta	Estado
2017	State channels (Raiden)	Abandonado mayormente
2017	Plasma	Abandonado (data availability)
2018	Sharding original	Reemplazado por rollup-centric
2019	Optimistic Rollups	Activo (Arbitrum, Optimism)
2019	ZK Rollups	Activo (zkSync, StarkNet)
2020	Rollup-centric roadmap	Adoptado oficialmente
2024	EIP-4844 (blobs)	Implementado
202X	Full Danksharding	En desarrollo

3.1.4 El Throughput en Perspectiva

Matemáticas

¿Cuánto necesitamos realmente?
Demanda actual:

- Visa procesa ∼65,000 TPS pico, ∼1,700 TPS promedio
- Ethereum: ∼1.2M txs/día ≈ 14 TPS promedio
- En picos de demanda: fees de $50-500 por tx

Demanda futura (si crypto escala):

- 8 mil millones de personas
- Si cada una hace 10 txs/día
- = 80 mil millones de txs/día
- = 925,000 TPS

¿Pueden los L2s lograrlo?

Escenario	TPS L2	# L2s	TPS Total
Hoy	100-1000	10	∼10,000
2025	1000-5000	100	∼500,000
2030 (proyección)	10,000+	1000+	∼10,000,000

Con múltiples L2s, L3s, y optimizaciones, el throughput agregado puede escalar dramáticamente.

3.2 Qué es L2

Definición 3.5 (Layer 2)**.** Protocolo que procesa transacciones off-chain pero hereda seguridad de L1.

Tipos:

- State channels
- Plasma
- Rollups (Optimistic y ZK)

3.2.1 La Definición Precisa de L2

Intuición

Un L2 es como una sucursal de un banco:

- Puedes hacer transacciones en la sucursal (rápido, barato)
- La sucursal reporta al banco central (L1)
- Si la sucursal hace trampa, el banco central puede corregirlo
- Tu dinero está seguro porque está respaldado por el banco central

La clave es que **heredas la seguridad de L1**. No necesitas confiar en la sucursal si puedes verificar con el banco central.

Definición 3.6 (Criterios para ser "verdadero L2")**.** Según la comunidad de Ethereum, un L2 debe:

1. **Heredar seguridad de L1:**

- Un usuario puede forzar retiro solo interactuando con L1
- No requiere cooperación del operador L2
- L1 puede verificar correctitud del estado L2

2. **Publicar datos en L1:**

- Datos necesarios para reconstruir estado
- O prueba de que datos están disponibles
- Permite validación independiente

3. **No tener custodio confiable:**

- Fondos no controlados por multisig
- O escape hatch si multisig es malicioso

Advertencia

¿Es realmente un L2?
Muchos proyectos se llaman "L2" pero no cumplen todos los criterios:

Proyecto	Hereda seg.	DA en L1	Sin custodio
Arbitrum	Sí	Sí	Parcial*
Optimism	Sí	Sí	Parcial*
zkSync Era	Sí	Sí	Parcial*
Polygon PoS	No (propia)	No	No
Ronin	No (propia)	No	No
BSC	No (L1 separada)	N/A	N/A

*Los rollups actuales aún tienen "training wheels": multisigs que pueden pausar o actualizar. Ver L2Beat para estado actual de descentralización.

3.2.2 Tipos de Soluciones L2

State Channels

Definición 3.7 (State Channel)**.** Canal bilateral o multilateral donde partes hacen transacciones off-chain, solo tocando L1 para abrir y cerrar el canal.

Ejemplo 3.2 (Lightning Network (Bitcoin))**.**

```
1. Alice y Bob abren canal depositando 1 BTC cada u
   Estado inicial: Alice=1, Bob=1

2. Off-chain, intercambian estados firmados:
   Estado 1: Alice=0.9, Bob=1.1 (Alice paga 0.1 a Bob)
   Estado 2: Alice=0.8, Bob=1.2 (Alice paga 0.1 más)
   ...
   Estado 100: Alice=0.3, Bob=1.7

3. Pueden hacer miles de txs sin tocar L1

4. Para cerrar: publican último estado en L1 (1 tx)
   Alice recibe 0.3 BTC, Bob recibe 1.7 BTC

5. Si alguien intenta publicar estado viejo:
   - Periodo de disputa permite al otro probar fraude
   - El fraudulento pierde todo su balance
```

Idea Clave

Ventajas de State Channels:

- Latencia instantánea (sin esperar bloques)
- Costo por tx: prácticamente cero
- Privacidad: txs intermedias no se publican

- Finalidad: inmediata entre las partes

Desventajas:

- Requiere capital bloqueado
- Solo funciona entre partes del canal
- Routing complejo para pagos multi-hop
- Requiere estar online para disputar
- No soporta smart contracts generales

Plasma

Definición 3.8 (Plasma)**.** Cadena hija que publica merkle roots de su estado en L1, con mecanismo de exits en caso de fraude.

Observación 3.3. Plasma fue propuesto en 2017 por Vitalik Buterin y Joseph Poon (co-autor de Lightning).

El problema fatal: Data Availability

En Plasma, los datos completos NO van a L1 (solo merkle roots). Esto crea el problema de "data withholding":

1. Operador publica root válido
2. Pero NO publica los datos
3. Usuarios no pueden probar qué poseen
4. No pueden hacer exit correctamente

Solución intentada: Mass exits

Si operador no da datos, todos hacen exit simultáneo. Pero esto:

- Congestiona L1
- Es costosísimo
- Crea pánico

Por estas razones, Plasma fue mayormente abandonado en favor de Rollups.

Rollups

Definición 3.9 (Rollup)**.** L2 que "enrolla" múltiples transacciones en un batch, publica datos en L1, y usa pruebas para verificar correctitud.

Idea Clave

¿Por qué Rollups funcionan mejor que Plasma?
Rollups publican datos en L1:

- Cualquiera puede reconstruir el estado
- No hay problema de data withholding
- Exits siempre son posibles

Compresión hace que sea económico:

- Una tx Ethereum: ∼110 bytes
- Una tx en rollup (compressed): ∼12 bytes
- 10x menos espacio = 10x más txs por bloque

Verificación es barata:

- Optimistic: solo verificar si hay disputa
- ZK: verificar proof es $O(1)$

3.2.3 Comparación de Tipos de L2

Propiedad	Channels	Plasma	Opt. Rollup	ZK Rollup
DA	Off-chain	Off-chain	On-chain	On-chain
Finalidad L1	Inmediata	∼7 días	∼7 días	Minutos
Smart contracts	Limitados	Limitados	Completos	Completos*
Capital locked	Sí	No	No	No
Online required	Sí	Sí	No	No
Complejidad	Media	Alta	Media	Alta
Madurez	Alta	Abandonado	Alta	Media

*ZK Rollups para smart contracts generales (zkEVM) son relativamente nuevos.

3.3 Sidechains vs Rollups

Aspecto	Sidechain	Rollup
Seguridad	Propia (validadores)	Hereda de L1
Datos	En la sidechain	En L1
Trust	Confiar en validadores	Trustless (cripto)
Bridge	Requiere confianza	Native, seguro

Sidechains: Polygon PoS, Ronin, Gnosis Chain.

3.3.1 Qué es una Sidechain

Definición 3.10 (Sidechain)**.** Blockchain independiente con su propio consenso, conectada a otra blockchain mediante un bridge.

Intuición

Una sidechain es como un país extranjero:

- Tiene sus propias leyes (consenso)
- Su propia policía (validadores)
- Su propia moneda puede estar "pegged" a otra
- Puedes ir y venir (bridge), pero si el país colapsa, pierdes todo

Un rollup es más como una provincia autónoma:

- Tiene cierta autonomía (procesa sus txs)
- Pero está bajo la jurisdicción del país central (L1)
- Si hay problemas, el gobierno central interviene
- Tu propiedad está protegida por ley nacional (L1)

3.3.2 Análisis de Seguridad

Matemáticas

Modelo de seguridad de Sidechain:
Sea n el número de validadores y f los maliciosos.
Para BFT típico (Tendermint, etc.):

- Requiere $f < n/3$ para safety
- Si $f \geq n/3$: pueden detener la cadena
- Si $f \geq 2n/3$: pueden robar fondos

Ejemplo: Ronin

- 9 validadores, threshold 5/9
- 4 controlados por Sky Mavis
- 1 más comprometido por phishing
- Resultado: $625M robados

Modelo de seguridad de Rollup:

- 1 participante honesto puede forzar correctitud

- Operador malicioso: usuarios hacen exit via L1
- Seguridad = seguridad de L1 (miles de validadores)

Advertencia

El problema de los bridges de sidechains:

1. Usuario deposita ETH en bridge de sidechain
2. Bridge mintea "wrapped ETH" en sidechain
3. Usuario usa wrapped ETH normalmente
4. Para salir: quemar wrapped, bridge libera ETH real

¿Quién controla el ETH real?
Típicamente un multisig de validadores de la sidechain.
Si el multisig es comprometido:

- Pueden robar todo el ETH del bridge
- Los wrapped tokens en sidechain se vuelven sin respaldo
- Usuarios pierden todo

Hacks de bridges de sidechains:

Bridge	Causa	Pérdida
Ronin	Multisig comprometido	$625M
Harmony	2/5 multisig comprometido	$100M

3.3.3 Bridges de Rollups vs Sidechains

Idea Clave

Bridge nativo de Rollup (ejemplo: Arbitrum)
Depósito (L1 → L2):

1. Usuario envía ETH a contrato bridge en L1
2. Contrato bloquea ETH y emite evento
3. Sequencer ve evento y acredita ETH en L2
4. Si sequencer no lo hace: después del challenge period, usuario puede reclamar en L2 forzadamente

Retiro (L2 → L1):

1. Usuario inicia retiro en L2
2. Transacción incluida en batch publicado en L1
3. Esperar challenge period (∼7 días para Optimistic)
4. Usuario ejecuta retiro en L1
5. Contrato libera ETH

Seguridad:

- No hay multisig que pueda robar fondos
- Contrato en L1 solo libera con proof válido
- Usuario siempre puede hacer exit si tiene los datos

3.3.4 ¿Cuándo Usar Cada Uno?

Idea Clave

Sidechains son apropiadas cuando:

- Los validadores son confiables (empresa, consorcio)
- El valor en juego es limitado
- La velocidad es más importante que la seguridad
- Se necesita compatibilidad EVM inmediata
- Ejemplo: testnet, juegos con items de bajo valor

Rollups son apropiados cuando:

- Se maneja valor significativo
- La seguridad es prioritaria
- Se acepta algo de latencia para finalidad
- Se quiere heredar seguridad de Ethereum
- Ejemplo: DeFi, pagos, cualquier aplicación seria

Observación 3.4. **La evolución de Polygon:**

Producto	**Tipo**	**Seguridad**
Polygon PoS	Sidechain	Validadores propios
Polygon zkEVM	ZK Rollup	Hereda de Ethereum
Polygon CDK	Framework	Configurable

Polygon comenzó como sidechain pero está migrando hacia ZK rollups para mejor seguridad. Planean "upgrade" Polygon PoS a un validium/rollup.

En Banco ZK

Banco ZK como Rollup:
Banco ZK es un **ZK Rollup**, no una sidechain. Esto significa:

- **Seguridad de Ethereum:** Aunque el sequencer sea malicioso, tus fondos están seguros
- **Datos en L1:** Publicamos datos comprimidos en Ethereum
- **Bridge trustless:** El bridge está asegurado por proofs ZK
- **Forced exits:** Usuarios pueden salir sin cooperación del sequencer

La privacidad es ortogonal a esto:

- Podríamos ser sidechain privada (menos seguro)
- O rollup privado (lo que somos, más seguro)
- La privacidad viene de ZK proofs sobre estado encriptado
- La seguridad viene de ser rollup con DA en L1

3.4 Optimistic Rollups

Definición 3.11 (Optimistic Rollup)**.** Asume validez de transacciones, permite disputas.

Intuición

"Optimistic" porque asume que todo es correcto hasta que alguien demuestre lo contrario. Es como un sistema legal:

- Las personas son inocentes hasta que se pruebe su culpabilidad
- Cualquiera puede acusar (disputar)
- El acusador debe probar el fraude
- Si hay fraude probado, se castiga al culpable

Esto es eficiente porque la mayoría del tiempo no hay fraude, y solo pagamos el costo de verificación cuando es necesario.

3.4.1 Cómo Funcionan

1. Sequencer ejecuta txs off-chain
2. Publica batch + state root en L1
3. Período de disputa (∼7 días)
4. Cualquiera puede disputar con fraud proof
5. Si no hay disputa, estado es final

Flujo Detallado

Definición 3.12 (Flujo de Optimistic Rollup)**. 1. Recepción de transacciones:**

```
Usuario envía tx al sequencer (via RPC)
    ↓
Sequencer valida tx (firma, balance, nonce)
    ↓
Sequencer añade tx a su mempool local
    ↓
Sequencer ordena txs y las ejecuta
```

2. Ejecución y batching:

```
Sequencer ejecuta txs secuencialmente
    ↓
Calcula nuevo state root después de cada tx
    ↓
Acumula txs en un batch (ej: cada 1-10 minutos)
    ↓
Comprime datos del batch
```

3. Publicación en L1:

```
Sequencer envía tx a L1 con:
  - Compressed batch data (calldata o blobs)
  - Previous state root
  - New state root (claimed)
  - Batch metadata
    ↓
Contrato en L1 registra el batch
```

4. Challenge period:

```
Durante ~7 días:
  - Cualquiera puede verificar el batch
  - Si encuentra error: inicia disputa
  - Si no hay disputa: batch se considera final
    ↓
Después del periodo:
  - State root se marca como finalizado
  - Retiros de ese batch pueden ejecutarse
```

3.4.2 Fraud Proofs

Definición 3.13 (Fraud Proof)**.** Prueba de que una transacción fue mal ejecutada.

Tipos:

- **Non-interactive:** Prueba completa en una tx
- **Interactive:** Bisection game para encontrar paso inválido

Non-Interactive Fraud Proofs

Definición 3.14 (Non-Interactive Fraud Proof)**.** Una sola transacción en L1 que demuestra el fraude completo.

Contenido:

- Estado antes de la tx disputada
- La transacción en cuestión
- Merkle proofs de todos los datos accedidos

- Ejecución que muestra resultado diferente al claimed

Advertencia

Problema con non-interactive proofs:
Una transacción puede acceder mucho estado:

- Loops que leen muchos storage slots
- Llamadas a múltiples contratos
- Cada acceso requiere Merkle proof

En el peor caso, el fraud proof podría ser más grande que un bloque L1. Por esto, la mayoría de optimistic rollups usan proofs interactivos.

Interactive Fraud Proofs (Bisection Game)

Definición 3.15 (Bisection Game)**.** Protocolo interactivo para encontrar el primer paso de ejecución donde el operador diverge del resultado correcto.

Participantes:

- Asserter: quien publicó el state root (defiende)
- Challenger: quien disputa (ataca)

Matemáticas

Cómo funciona el bisection:
Sea N el número de pasos de ejecución en el batch disputado.
Ronda 1:

- Asserter divide ejecución en 2 mitades
- Publica hash del estado en el punto medio ($N/2$)
- Challenger indica en qué mitad está el desacuerdo

Ronda 2:

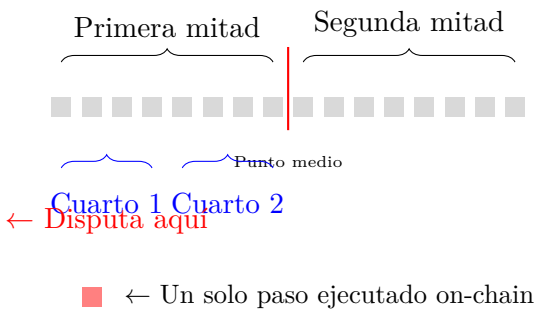
- Asserter divide la mitad disputada en 2
- Publica hash del estado en nuevo punto medio ($N/4$ o $3N/4$)
- Challenger indica en qué cuarto está el desacuerdo

Continúa hasta:

- Llegar a un solo paso de ejecución
- Este único paso se ejecuta on-chain
- L1 determina quién tiene razón

Complejidad:

- Número de rondas: $O(\log N)$
- Si $N = 2^{30}$ pasos: solo 30 rondas
- Cada ronda es una tx en L1
- Final: ejecutar 1 paso on-chain (factible)



Ejemplo 3.3 (Bisection en Arbitrum)**.**

```
Batch con 1 billón de pasos de ejecución (N = 10^9)

Rondas necesarias: log2(10^9)  30 rondas

Cada ronda:
  - Asserter: publica commitment (~100K gas)
  - Challenger: responde con elección (~50K gas)

Costo total (sin contar bond):
  ~30 × 150K gas = 4.5M gas  $20-100 (depende de gas price)

Tiempo total:
  - Cada ronda permite ~1 día para responder
  - En la práctica: 1-2 semanas para resolver

Final:
  - Un solo paso de WASM/EVM se ejecuta en L1
  - Contrato determina quién tenía razón
  - Perdedor pierde su bond (~ETH)
```

3.4.3 Arquitectura de Arbitrum

Definición 3.16 (Arbitrum One)**.** El optimistic rollup más grande por TVL, desarrollado por Offchain Labs.

Idea Clave

Componentes de Arbitrum:

1. **Sequencer:**
   - Recibe txs de usuarios
   - Ordena y ejecuta off-chain
   - Publica batches en L1
   - Actualmente centralizado (Offchain Labs)
2. **Validators/Asserters:**
   - Publican state roots (assertions)
   - Deben depositar bond
   - Cualquiera puede ser validator (permissionless)
3. **Challengers:**
   - Monitorean assertions
   - Disputan si encuentran error
   - También necesitan bond
4. **Contracts en L1:**
   - Rollup contract: gestiona assertions y challenges
   - Bridge: maneja depósitos y retiros
   - Inbox: recibe batches del sequencer
   - Outbox: procesa retiros

Observación 3.5. **Arbitrum vs Optimism (diferencias técnicas):**

Aspecto	Arbitrum	Optimism
VM	AVM (custom) / WASM	EVM equivalente
Fraud proof	Interactivo (bisection)	Single-round*
Challenge period	∼7 días	∼7 días
EVM compat	Nitro (muy alta)	Muy alta
Governance	ARB token	OP token

*Optimism está migrando a fraud proofs basados en Cannon (MIPS).

3.4.4 El Problema de los 7 Días

Advertencia

El delay de retiro:
Para retirar de un optimistic rollup a L1:

1. Iniciar retiro en L2
2. Esperar que batch sea publicado en L1
3. Esperar challenge period completo (∼7 días)
4. Ejecutar retiro en L1

¿Por qué 7 días?

- Tiempo suficiente para que alguien detecte fraude
- Tiempo para ejecutar el bisection game completo
- Balance entre seguridad y UX

Soluciones para usuarios impacientes:

1. **Liquidity providers:** Servicios como Hop, Across
   - LP te da fondos en L1 inmediatamente
   - LP espera los 7 días y cobra fee
   - Fee típico: 0.1-0.5%
2. **CEX:** Algunos exchanges aceptan depósitos directos desde L2
3. **Cross-L2:** Bridges entre L2s sin pasar por L1

3.4.5 Compresión de Datos

Matemáticas

Cómo Arbitrum comprime transacciones:
Tx original Ethereum:

```
{
  nonce: 4 bytes
  gasPrice: 8 bytes
  gasLimit: 4 bytes
  to: 20 bytes
  value: 8 bytes
  data: variable
  v, r, s: 65 bytes
```

```
  ---
  Total: ~110+ bytes (sin data)
}
```

Tx comprimida en Arbitrum batch:

```
{
  to: 4 bytes (índice, no dirección completa)
  value: 4 bytes (comprimido)
  nonce: 0-4 bytes (delta encoding)
  signature: 64 bytes (sin recovery byte)
  data: comprimido
  ---
  Total: ~12-20 bytes (sin data)
}
```

Técnicas de compresión:

- Direcciones indexadas: mapa de direcciones frecuentes
- Delta encoding: solo diferencia del nonce anterior
- Valores pequeños: encoding variable-length
- Firmas batch: BLS aggregation (futuro)
- Compresión general: zlib sobre todo el batch

Resultado:

- Compresión 5-10x típica
- Una transferencia: ∼12 bytes
- Costo L1 por tx: ∼$0.01-0.10

3.4.6 Seguridad y Trust Assumptions

Idea Clave

¿En qué confías al usar un optimistic rollup?
Asunciones de seguridad:

1. **Al menos 1 validator honesto:**
   - Alguien debe disputar si hay fraude
   - Si todos los validators son maliciosos, fraude pasa
   - En la práctica: múltiples validators independientes
2. **L1 está disponible:**

- Disputas ocurren en L1
- Si L1 está censurado, no se puede disputar

3. **Datos disponibles:**
   - Batch data publicado en L1 (calldata/blobs)
   - Necesario para verificar y disputar

Riesgos actuales (training wheels):
La mayoría de optimistic rollups tienen:

- Upgrade keys: multisig puede actualizar contratos
- Pausability: pueden pausar el rollup
- Sequencer centralizado: puede censurar
- Whitelist de validators: no cualquiera puede disputar

Roadmap a trustlessness:

- Remover upgrade keys (time delays primero)
- Validators permissionless
- Sequencer descentralizado
- Emergency exit mechanisms

3.4.7 Ejemplos: Arbitrum, Optimism, Base

Observación 3.6. **Estado del ecosistema Optimistic Rollup (2024):**

Rollup	TVL	TPS	Fraud Proofs	Sequencer
Arbitrum One	∼$10B	∼40	Live (permissioned)	Centralizado
Optimism	∼$5B	∼30	En desarrollo	Centralizado
Base	∼$3B	∼25	En desarrollo	Centralizado
Mantle	∼$1B	∼20	En desarrollo	Centralizado

OP Stack (Superchain):
Optimism creó el OP Stack, un framework para lanzar optimistic rollups:

- Base (Coinbase)
- Zora
- Mode
- Worldcoin

- Muchos más

La visión "Superchain" es que todos estos L2s compartan:

- Sequencing (eventualmente)
- Bridge nativo entre ellos
- Standards comunes

3.4.8 Comparación con ZK Rollups

Aspecto	Optimistic	ZK
Validez	Asumida, disputable	Probada matemáticamente
Finalidad L1	∼7 días	Minutos-horas
Costo L2 tx	Muy bajo	Bajo (más overhead)
Costo L1 (proof)	0 si no hay disputa	Siempre (verify)
EVM compat	Nativa	Requiere zkEVM
Madurez	Alta	Media
Complejidad	Media	Alta

Idea Clave

¿Cuándo elegir Optimistic vs ZK?
Optimistic es mejor para:

- Máxima compatibilidad EVM hoy
- Costos más bajos (especialmente con poco volumen)
- Aplicaciones donde 7 días de delay es aceptable
- Casos donde ya hay liquidez en Optimistic L2s

ZK es mejor para:

- Finalidad rápida es crítica
- Privacidad (como Banco ZK)
- Bridges seguros (verificación instantánea)
- Aplicaciones financieras de alto valor

Tendencia: A largo plazo, muchos creen que ZK dominará porque:

- Verificación instantánea > esperar 7 días
- Hardware ZK mejora rápidamente
- zkEVMs están madurando

3.5 ZK Rollups

Definición 3.17 (ZK Rollup)**.** Genera prueba criptográfica de ejecución correcta.

Intuición

Un ZK Rollup es como un examen con respuestas verificadas:

- El estudiante (prover) resuelve muchos problemas (ejecuta txs)
- En lugar de mostrar todo su trabajo, genera un "certificado de corrección"
- El profesor (L1) verifica el certificado en segundos
- Si el certificado es válido, las respuestas son correctas garantizado
- No hay necesidad de esperar quejas de otros estudiantes

La magia es que verificar el certificado (proof) es muchísimo más barato que re-ejecutar todos los cálculos.

3.5.1 Cómo Funcionan

1. Sequencer ejecuta batch off-chain
2. Prover genera ZK proof de validez
3. Publica datos + proof en L1
4. L1 verifica proof (barato) y acepta estado
5. Finalidad inmediata (sin período de disputa)

Flujo Detallado

Definición 3.18 (Flujo de ZK Rollup)**. 1. Recepción y ejecución:**

```
Usuario envía tx al sequencer
    ↓
Sequencer valida y ejecuta tx
    ↓
Actualiza estado interno (off-chain)
    ↓
Acumula txs en batch
```

2. Generación de prueba:

```
Prover recibe batch de txs
    ↓
Construye circuito ZK que representa:
  - Verificación de firmas
```

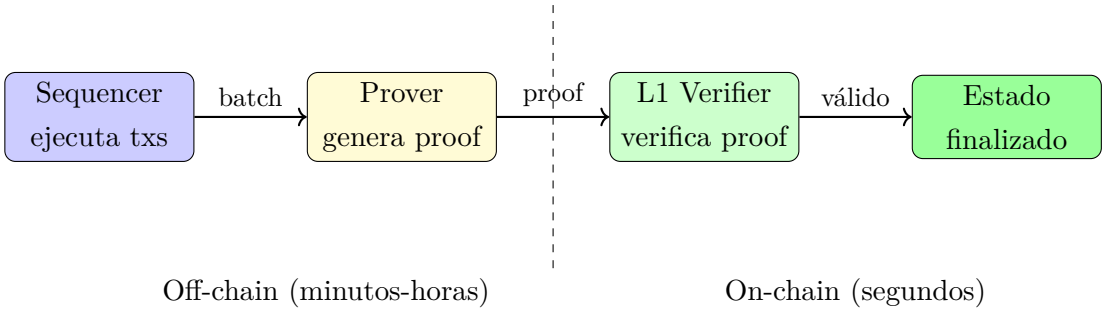
```
  - Transiciones de estado
  - Actualización de Merkle trees
  - Cualquier lógica del protocolo
    ↓
Ejecuta el circuito con witnesses (datos privados)
    ↓
Genera proof (puede tomar minutos a horas)
```

3. Publicación en L1:

```
Sequencer/Prover envía a L1:
  - Compressed batch data (para DA)
  - Previous state root
  - New state root
  - ZK proof
    ↓
Contrato en L1:
  - Verifica proof (precompilado)
  - Si válido: actualiza state root
  - Emite evento
```

4. Finalización:

```
Inmediatamente después de verificación:
  - Nuevo estado es final
  - Retiros pueden procesarse
  - No hay challenge period
```



3.5.2 Validity Proofs

La prueba ZK garantiza:

- Todas las firmas son válidas
- Transiciones de estado son correctas
- Balances no son negativos
- Reglas del protocolo se cumplen

Matemáticas

¿Qué exactamente prueba el ZK proof?

El circuito ZK codifica la función de transición de estado:

$$\text{proof} : \pi \text{ tal que } \text{Verify}(\pi, S_{\text{old}}, S_{\text{new}}, \text{batch}) = 1$$

La prueba garantiza:

1. Para cada tx en el batch:
   - La firma ECDSA/EdDSA es válida
   - El nonce es correcto
   - El sender tiene balance suficiente
   - La ejecución sigue las reglas
2. El state root S_{new} es el resultado correcto de aplicar todas las txs a S_{old}
3. Los datos publicados coinciden con las txs ejecutadas

Propiedades:

- **Soundness:** Es computacionalmente imposible crear un proof falso
- **Zero-knowledge (opcional):** El proof no revela información adicional
- **Succinctness:** El proof es pequeño y rápido de verificar

3.5.3 Tipos de ZK Rollups

Definición 3.19 (Tipos según sistema de pruebas)**. 1. SNARK-based:**

- zkSync Era, Scroll, Polygon zkEVM, Linea
- Pruebas pequeñas (∼200-500 bytes)
- Verificación barata (200-500K gas)
- Típicamente requieren trusted setup

2. STARK-based:

- StarkNet, StarkEx
- Pruebas grandes (∼50-200 KB)
- No trusted setup (transparent)
- Post-quantum seguro

Definición 3.20 (Tipos según compatibilidad EVM)**. Tipo 1 - Ethereum equivalent:**

- Idéntico a Ethereum, incluyendo bugs
- Máxima compatibilidad
- Más difícil de probar en ZK
- Ejemplo: Taiko (en desarrollo)

Tipo 2 - EVM equivalent:

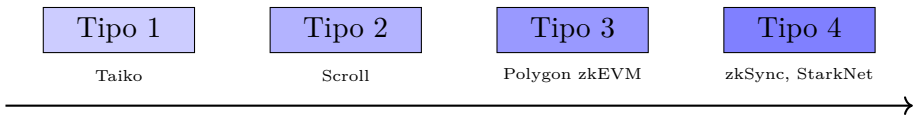
- Compatible a nivel EVM
- Pequeñas diferencias en edge cases
- Ejemplo: Scroll, Polygon zkEVM

Tipo 3 - EVM compatible:

- La mayoría de código funciona
- Algunas modificaciones necesarias
- Ejemplo: Scroll (en transición a tipo 2)

Tipo 4 - Language compatible:

- Lenguaje propio que compila similar a EVM
- Solidity/Vyper compilan a VM diferente
- Ejemplo: zkSync Era, StarkNet (Cairo)



Más compatible Más eficiente

3.5.4 zkEVM: Probando la EVM

Idea Clave

El desafío del zkEVM:
La EVM no fue diseñada para ser probada en ZK:

- Usa Keccak256 (no ZK-friendly)
- Aritmética de 256 bits (muy grande para circuitos)
- Stack machine (muchas operaciones por paso)
- Storage como Merkle Patricia Trie (complejo)

Estrategias de los zkEVMs:
1. Probar EVM directamente (tipo 1-2):

- Circuito que simula cada opcode
- Muy grande pero compatible
- Scroll, Polygon zkEVM

2. Compilar a VM ZK-friendly (tipo 4):

- Transformar Solidity a otra representación
- Más eficiente pero menos compatible
- zkSync Era, StarkNet

Matemáticas

Costos de opcodes en zkEVM:

Operación	Gas EVM	Constraints ZK	Ratio
ADD	3	∼100	33x
MUL	5	∼500	100x
SHA3/Keccak	30+6/word	∼150,000	5000x
ECDSA verify	3000	∼500,000	167x
SLOAD	2100	∼10,000	5x
Merkle update	N/A	∼50,000	N/A

Nota: Keccak es especialmente caro en ZK. Por eso algunos zkEVMs usan Poseidon para hashes internos.

3.5.5 Ejemplos: zkSync, StarkNet, Scroll, Polygon zkEVM

Observación 3.7. **Comparación de ZK Rollups principales (2024):**

	zkSync Era	StarkNet	Scroll	Polygon zkEVM
Tipo zkEVM	4	4 (Cairo)	2	2-3
Proof system	PLONK	STARK	PLONK	PLONK
DA	L1 (blobs)	L1 + validium	L1	L1
TVL	∼$800M	∼$300M	∼$200M	∼$100M
Mainnet	Mar 2023	Nov 2022	Oct 2023	Mar 2023

zkSync Era:

- Desarrollado por Matter Labs
- Usa su propia VM (zkEVM, tipo 4)
- Soporta account abstraction nativo

- Compilador llvm-based para Solidity

StarkNet:

- Desarrollado por StarkWare
- Usa Cairo (lenguaje propio, no EVM)
- STARKs (no trusted setup, post-quantum)
- Pruebas más grandes pero más seguras a largo plazo

Scroll:

- Colaboración con Ethereum Foundation
- zkEVM tipo 2 (muy compatible)
- Usa Halo2 (sin trusted setup)
- Enfocado en equivalencia bytecode

Polygon zkEVM:

- Parte del ecosistema Polygon
- zkEVM tipo 2-3
- Proof recursion para eficiencia
- Integración con Polygon ecosystem

3.5.6 Prover: El Cuello de Botella

Advertencia

El costo del proving:
Generar una prueba ZK es computacionalmente intensivo:

Batch size	Tiempo	Hardware
100 txs simples	∼1 min	GPU potente
1000 txs	∼10 min	Cluster GPUs
10000 txs	∼1 hora	Datacenter

Por qué es caro:

- Operaciones de campo grande (FFT, MSM)
- Circuitos con millones de constraints
- Mucha memoria (100+ GB para batches grandes)

Soluciones:

- Hardware especializado (FPGAs, ASICs)
- Paralelización masiva (GPUs)
- Proof recursion (probar proofs)
- Optimización de circuitos
- Mercados de proving descentralizados

Matemáticas

Proof Recursion:

En lugar de una prueba gigante para todo el batch:

```
Batch de 10,000 txs:
    ↓
Dividir en 100 mini-batches de 100 txs
    ↓
Generar 100 pruebas pequeñas (paralelo)
    ↓
Generar prueba que verifica las 100 pruebas
    ↓
Una sola prueba final para L1
```

Beneficios:

- Paralelización: 100 provers trabajan simultáneamente
- Latencia reducida: no esperar por mega-batch
- Flexibilidad: agregar más provers linealmente

Overhead:

- Prueba recursiva tiene overhead
- Pero la paralelización lo compensa

3.5.7 Data Availability en ZK Rollups

Idea Clave

¿Por qué publicar datos si ya tenemos proof?

El proof garantiza que la transición de estado es correcta, pero:

- Usuarios necesitan conocer su estado para interactuar
- Sin datos, no pueden construir txs futuras

- Sin datos, no pueden hacer exit si operador desaparece

Opciones de DA para ZK Rollups:

1. ZK Rollup (datos on-chain):

- Publica batch data en L1 (calldata/blobs)
- Máxima seguridad
- Más caro

2. Validium (datos off-chain):

- Solo proof en L1
- Datos en comité (DAC) o DA layer
- Más barato pero menos seguro

3. Volition (híbrido):

- Usuario elige por tx
- Txs importantes: on-chain
- Txs menos críticas: off-chain

3.5.8 Ventajas y Desventajas de ZK Rollups

Ventajas	Desventajas
Finalidad rápida (minutos)	Proving es caro/lento
Sin challenge period	Compatibilidad EVM limitada*
Seguridad matemática	Complejidad técnica alta
Bridges más seguros	Circuitos pueden tener bugs
Privacidad posible	Menos maduros que Optimistic

*Mejorando rápidamente con zkEVMs.

3.5.9 ZK Rollups con Privacidad

En Banco ZK

Banco ZK: ZK Rollup + Privacidad

Banco ZK combina las ventajas de ZK Rollups con privacidad transaccional:

Como ZK Rollup estándar:

- Publica datos en L1 (para DA)
- Pruebas de validez verificadas en L1
- Hereda seguridad de Ethereum

- Finalidad rápida

Capa de privacidad adicional:

- Txs son commitments encriptados
- Montos ocultos con Pedersen
- Identidades ocultas con nullifiers
- Prueba ZK incluye prueba de privacidad

Arquitectura:

```
Usuario genera tx privada:
  - Note commitment (oculta monto y owner)
  - Nullifier (prueba gasto sin revelar cuál note)
  - ZK proof de validez
    ↓
Sequencer incluye en batch:
  - No puede ver contenido
  - Solo verifica que proof es válido
    ↓
Prover genera proof de batch:
  - Incluye verificación de cada tx
  - Transiciones de estado privadas
    ↓
L1 verifica:
  - Proof de todo el batch
  - Nullifiers no duplicados
  - State root correcto
```

Diferencia con zkSync/StarkNet/Scroll:

- Ellos: txs públicas, solo escalabilidad
- Banco ZK: txs privadas, escalabilidad + privacidad
- El costo de privacidad: circuitos más grandes

Similar a:

- Aztec Protocol (ZK rollup privado)
- Aleo (L1 ZK nativo)
- Zcash (pero como rollup sobre Ethereum)

3.6 Sequencers

Definición 3.21 (Sequencer)**.** Entidad que ordena y ejecuta transacciones en un rollup.

Estado actual:

- Mayoría de rollups tienen sequencer centralizado
- Más eficiente pero punto único de falla
- Riesgos: censura, MEV extraction, downtime

3.6.1 Rol del Sequencer

Intuición

El sequencer es como el director de orquesta del rollup:

- Recibe todas las "notas" (transacciones) de los músicos (usuarios)
- Decide el orden en que se tocan
- Coordina la ejecución
- Presenta el resultado final

El problema es que un director corrupto podría:

- Ignorar a ciertos músicos (censura)
- Reordenar para su beneficio (MEV)
- Simplemente no presentarse (downtime)

Definición 3.22 (Funciones del Sequencer)**. 1. Recepción:**

- Acepta txs de usuarios vía RPC
- Valida formato y firmas
- Mantiene mempool local

2. Ordenamiento:

- Decide el orden de las txs
- Puede ser FIFO, por fee, o arbitrario
- Este orden es crítico para MEV

3. Ejecución:

- Ejecuta txs contra el estado
- Calcula nuevos state roots

- Proporciona “soft confirmations” a usuarios

4. Batching:

- Agrupa txs en batches

- Comprime datos

- Publica en L1

3.6.2 Centralización vs Descentralización

Advertencia

Riesgos de sequencer centralizado:

1. **Censura:**
   - Sequencer puede ignorar ciertas txs
   - Usuarios de ciertos países/wallets bloqueados
   - Mitigación: forced inclusion via L1
2. **MEV extraction:**
   - Sequencer ve todas las txs pendientes
   - Puede hacer frontrunning/sandwiching
   - Captura valor que debería ir a usuarios
3. **Downtime:**
   - Si sequencer cae, L2 se detiene
   - Usuarios no pueden hacer txs
   - Dependen de que vuelva a funcionar
4. **Jurisdicción:**
   - Sequencer es una empresa en algún país
   - Puede ser obligado a censurar
   - Single point of regulatory attack

Estado actual (2024):

Rollup	Sequencer	Forced inclusion
Arbitrum	Offchain Labs	Sí (delayed)
Optimism	Optimism Foundation	Sí (delayed)
zkSync	Matter Labs	En desarrollo
StarkNet	StarkWare	Sí
Base	Coinbase	Sí (delayed)

3.6.3 Soluciones para Descentralización

Idea Clave

Enfoques para descentralizar sequencers:

1. **Rotación de sequencers:**
   - Múltiples sequencers que rotan

- Selección por stake, lottery, o round-robin
- Si uno censura, el siguiente incluye
- Ejemplo: Diseño futuro de Arbitrum

2. **Based rollups (L1 sequencing):**
   - L1 validators ordenan txs del rollup
   - Hereda descentralización de L1
   - No hay sequencer L2 separado
   - Trade-off: latencia de L1 (12s)
   - Ejemplo: Taiko
3. **Shared sequencing:**
   - Múltiples rollups comparten un sequencer descentralizado
   - Beneficios de atomic cross-rollup txs
   - Economías de escala
   - Ejemplos: Espresso, Astria
4. **Encrypted mempools:**
   - Txs encriptadas hasta inclusión
   - Sequencer ordena sin ver contenido
   - Elimina MEV del sequencer
   - Ejemplo: Shutter Network, SUAVE

Matemáticas

Based Rollups - Cómo funcionan:

```
Flujo tradicional:
  User → Sequencer L2 → L1 (datos)
                ↑
          Ordering aquí (centralizado)

Flujo based rollup:
  User → L1 Mempool → L1 Block → L2 deriva estado
                ↑
          Ordering por L1 validators (descentralizado)
```

Ventajas:

- Descentralización heredada de L1
- Sin operador L2 que pueda censurar
- Atomicidad con L1 (composability)

Desventajas:

- Latencia mínima = block time L1 (12s)
- No hay "soft confirmations" instantáneas
- Menos optimización posible

Observación 3.8. **Shared Sequencing - Espresso Systems:**
Espresso propone un sequencer compartido entre múltiples rollups:

Beneficios:

- Atomic bundles cross-rollup
- Compartir infraestructura
- Interoperabilidad mejorada

3.7 Data Availability (DA)

Definición 3.23 (Data Availability)**.** Garantía de que datos para reconstruir estado están disponibles.

3.7.1 Por Qué Importa

Sin DA, operador podría:

- Publicar estado válido pero ocultar txs
- Usuarios no pueden calcular sus balances
- No pueden generar proofs para retiros

Intuición

Imagina que el banco te dice "tu balance es $1000" pero no te muestra los movimientos:

- No puedes verificar si es correcto
- No sabes de dónde vino el dinero
- Si el banco desaparece, no puedes probar nada

Data Availability es tener acceso a los "movimientos" para poder verificar y actuar independientemente.

Matemáticas

El problema de Data Withholding:
Escenario malicioso:

1. Operador ejecuta batch correctamente
2. Publica state root válido en L1
3. PERO no publica los datos del batch
4. Estado en L1 es "correcto" pero nadie sabe qué contiene

Consecuencias:

- Usuarios no saben sus balances actuales
- No pueden construir txs (necesitan Merkle proofs)
- No pueden hacer exit (necesitan probar ownership)
- Operador puede robar fondos en el siguiente batch

Por qué no solo pedir los datos?

- Operador malicioso simplemente no responde
- ¿Cómo probar que NO te dieron los datos?
- Problema fundamentalmente asimétrico

3.7.2 Opciones de DA

Opción	Seguridad	Costo
Calldata L1	Máxima	Alto
Blobs (EIP-4844)	Alta	Medio
DAC (comité)	Media	Bajo
DA layers	Media-Alta	Bajo

DA layers: Celestia, EigenDA, Avail, NearDA.

Calldata en L1

Definición 3.24 (Calldata)**.** Datos incluidos en transacciones L1 que son permanentemente almacenados por todos los nodos.

- **Seguridad:** Máxima (misma que L1)
- **Costo:** 16 gas por byte non-zero, 4 por zero
- **Disponibilidad:** Para siempre
- **Acceso:** No accesible desde EVM, pero almacenado

Matemáticas

Cálculo de costo de calldata:
Para un batch de 1000 txs comprimidas (∼15 KB):

```
15,000 bytes × ~12 gas/byte (promedio) = 180,000 gas
A 30 gwei: 180,000 × 30 = 5.4M gwei = 0.0054 ETH
A $2500/ETH: ~$13.50

Por tx: $13.50 / 1000 = $0.0135
```

Este era el costo antes de EIP-4844. Aún usado como fallback.

Data Availability Committees (DAC)

Definición 3.25 (DAC)**.** Grupo de entidades que prometen mantener datos disponibles.

- Rollup envía datos a comité (no a L1)
- Comité firma attestation de que tienen los datos
- Solo la attestation va a L1 (muy pequeña)
- Si usuario pide datos, comité debe proporcionar

Advertencia

Riesgos de DAC:

- Si comité colude: pueden withhold datos
- Trust assumption: mayoría honesta del comité
- Típicamente N-of-M: si N miembros honestos, datos disponibles

Ejemplos:

- StarkEx (validium mode): DAC de entidades conocidas

- Immutable X: DAC para gaming

DA Layers Externos

Definición 3.26 (DA Layer)**.** Blockchain especializada en almacenar datos con garantías criptoeconómicas.

Observación 3.9. **Principales DA Layers:**

Celestia:

- Primera DA layer dedicada (mainnet Oct 2023)
- Usa Data Availability Sampling (DAS)
- Light nodes pueden verificar DA sin descargar todo
- Costo: $∼0.001 por KB

EigenDA:

- DA sobre EigenLayer (restaking)
- Seguridad heredada de ETH restaked
- Integración nativa con Ethereum validators

Avail:

- Fork de Polygon, enfocado en DA
- KZG commitments
- Validity proofs para DA

NearDA:

- DA usando Near Protocol
- Sharding nativo de Near
- Bajo costo

Matemáticas

Data Availability Sampling (DAS):
DAS permite verificar que datos están disponibles sin descargarlos completos:

1. Datos se codifican con Reed-Solomon (redundancia)
2. Se divide en muchos chunks pequeños
3. Light node solicita chunks aleatorios
4. Si recibe suficientes chunks, alta confianza de que todo está disponible

Matemática:

- Con 50% de datos disponibles, se puede reconstruir 100%
- Si un atacante oculta >50%, será detectado
- Probabilidad de no detectar con k samples: $(0.5)^k$
- Con $k = 30$ samples: probabilidad de error $< 10^{-9}$

Esto permite que nodes con poco bandwidth verifiquen DA.

3.8 Blobs (EIP-4844)

Definición 3.27 (Blob)**.** Nuevo tipo de dato en Ethereum optimizado para rollups.

Características:

- 128 KB por blob, hasta 6 por bloque
- No accesible desde EVM (solo por hash)
- Se elimina después de ∼18 días
- Mercado de fees separado
- Compromiso KZG para verificación

3.8.1 Contexto y Motivación

Observación 3.10. **El camino a EIP-4844:**

Problema:

- Rollups publicaban datos como calldata
- Calldata es permanente y cara
- Rollups competían con txs normales por blockspace
- Fees de L2 dependían de congestión L1

Solución - Danksharding (visión completa):

- Nuevos "data shards" dedicados para rollups
- Separar datos de ejecución
- Mucho más espacio para datos

Proto-Danksharding (EIP-4844):

- Paso intermedio hacia full Danksharding

- Introduce blobs sin sharding completo
- Mercado de fees separado
- Implementado en Dencun (Marzo 2024)

3.8.2 Anatomía de un Blob

Definición 3.28 (Estructura de Blob)**. Tamaño:** 128 KB (131,072 bytes) = 4096 field elements de 32 bytes

Representación:

- Polinomio de grado 4095 sobre BLS12-381
- Cada "field element" es un coeficiente
- KZG commitment: un punto de curva (48 bytes)

En el bloque:

- Header contiene hashes de blobs
- Blobs se propagan separadamente
- Solo el hash va al execution payload

Matemáticas

KZG Commitment para Blobs:
Cada blob tiene un KZG commitment que permite:

1. Verificar que ciertos datos están en el blob
2. Sin descargar el blob completo
3. Proofs de evaluación en puntos específicos

Proceso:

```
Blob data (128 KB)
    ↓
Interpretar como polinomio p(x)
    ↓
Commitment: C = [p()]_1 (punto en G1)
    ↓
Para probar p(z) = y:
    - Quotient: q(x) = (p(x) - y) / (x - z)
    - Proof:  = [q()]_1
    - Verify: e(C - [y]_1, [1]_2) = e(, [ - z]_2)
```

Esto es la base para DAS en Full Danksharding.

3.8.3 Blob Transactions

Definición 3.29 (Blob Transaction (Type 3))**.** Nueva transacción que incluye blobs:

```
{
  type: 3,
  chainId: ...,
  nonce: ...,
  maxFeePerGas: ...,
  maxPriorityFeePerGas: ...,
  gasLimit: ...,
  to: ...,
  value: ...,
  data: ...,
  accessList: [...],

  // Nuevos campos para blobs:
  maxFeePerBlobGas: ...,
  blobVersionedHashes: [hash1, hash2, ...],

  // Blobs van en "sidecar" separado
}
```

Idea Clave

Mercado de fees separado para blobs:
Execution gas (existente):

- Paga por computación y calldata
- Base fee + priority fee
- Target: 15M gas/bloque

Blob gas (nuevo):

- Paga por espacio de blob
- Su propio base fee (empieza muy bajo)
- Target: 3 blobs/bloque (384 KB)
- Max: 6 blobs/bloque (768 KB)

Ajuste del blob base fee:

- Mismo mecanismo EIP-1559
- Pero con parámetros diferentes
- Puede subir/bajar más rápido que el normal

3.8.4 Proto-Danksharding vs Full Danksharding

- **EIP-4844 (actual):** 6 blobs/bloque, ∼1 MB/bloque
- **Full Danksharding:** 64+ blobs, Data Availability Sampling
- DAS permite verificar DA sin descargar todo

Matemáticas

Comparación:

Aspecto	Proto-Dank (4844)	Full Danksharding
Blobs/bloque	6	64-256
DA por bloque	∼750 KB	∼8-32 MB
DAS	No	Sí
Verificación	Full nodes descargan todo	Light nodes muestrean
Timeline	Live (Mar 2024)	2025-2026+

Por qué DAS es necesario para más blobs:

- Con 6 blobs: full nodes pueden descargar todo
- Con 256 blobs: 32 MB/bloque = 160 Mbps solo para DA
- DAS permite verificar sin descargar todo
- Mantiene descentralización con más datos

3.8.5 Impacto de EIP-4844

Observación 3.11. **Reducción de costos post-Dencun:**

Rollup	Pre-4844	Post-4844	Reducción
Arbitrum	$0.10-0.50	$0.001-0.01	90-99%
Optimism	$0.10-0.50	$0.001-0.01	90-99%
Base	$0.05-0.20	$0.001-0.005	95-99%
zkSync Era	$0.20-1.00	$0.01-0.05	90-95%

Nota: Los costos varían según congestión. El blob base fee puede subir si la demanda excede el target.

Impacto en adopción:

- Txs de $0.01 hacen viable micropagos
- Gaming y social apps ahora factibles
- Más usuarios pueden permitirse usar L2s

En Banco ZK

Data Availability en Banco ZK:
Banco ZK usa una estrategia híbrida de DA:
Datos on-chain (blobs):

- Commitments de notas (encriptados)
- Nullifiers (públicos, para prevenir doble gasto)
- State root updates

¿Por qué no validium?
Aunque más barato, validium requiere confiar en el comité:

- Si comité colude con operador, pueden robar fondos
- Para un sistema privado, la confianza es especialmente problemática
- Preferimos seguridad de L1 aunque cueste más

Optimizaciones de DA:

- Solo publicamos commitments, no datos completos
- Compresión agresiva de nullifiers
- Batch aggregation para amortizar costos

Costo estimado por tx:

- Pre-4844: $0.10-0.30
- Post-4844: $0.01-0.05
- Target futuro: <$0.01

3.9 L3 y App-chains

Definición 3.30 (L3)**.** Layer 3: rollup sobre un L2.

Motivaciones:

- **Escalabilidad:** Más compresión
- **Personalización:** Parámetros específicos
- **Privacidad:** Features especializadas
- **Aislamiento:** Separar aplicaciones

3.9.1 ¿Por Qué L3?

Intuición

Si L2 escala L1, ¿por qué no L3 escala L2?
La respuesta no es tan simple:

- Cada capa añade latencia
- Cada capa tiene su propio sequencer/prover
- Los beneficios de escalabilidad tienen rendimientos decrecientes

Entonces, ¿cuál es el punto de L3?
No es solo escalabilidad, sino **especialización**:

- L2 general-purpose (Arbitrum, zkSync)
- L3 específico para una app o caso de uso
- Parámetros optimizados para ese caso

Idea Clave

Casos de uso para L3:

1. **Privacidad:**
   - L2 público para la mayoría de txs
   - L3 privado para txs confidenciales
   - Banco ZK podría ser L3 privado sobre un L2
2. **Gaming:**
   - Alto volumen de txs pequeñas
   - Latencia ultra-baja necesaria
   - Puede permitirse DA más barata (validium)
   - Items de bajo valor = menor necesidad de seguridad
3. **Enterprise:**
   - Permisos y compliance específicos
   - Aislamiento de otras apps
   - Gobernanza controlada
4. **Experimentación:**
   - Probar nuevos mecanismos
   - Fallos no afectan al L2 principal
   - Iteración rápida

Matemáticas

Vitalik sobre L2 vs L3:
En su post "What kind of layer 3s make sense?" (2022):
L3 para escalabilidad pura: No tiene mucho sentido

- Si L2 da 100x, L3 daría 10000x
- Pero L2 puede simplemente usar más DA
- No necesitas otra capa

L3 para customización: Tiene sentido

- L2: plataforma general (EVM)
- L3: app-specific (gaming VM, privacy VM)
- Diferentes trade-offs para diferentes usos

Arquitectura recomendada:

```
L1 (Ethereum): Seguridad + DA
    ↑
L2 (Arbitrum/zkSync): Ejecución general
    ↑
L3 (App-chain): Customización específica
```

3.9.2 App-chains

Definición 3.31 (App-chain)**.** Blockchain/rollup dedicado a una sola aplicación.

Frameworks: Arbitrum Orbit, OP Stack, Polygon CDK, zkSync Hyperchains.

Observación 3.12. **Frameworks para App-chains:**
Arbitrum Orbit:

- Fork de Arbitrum One/Nova
- Puede ser L2 o L3
- Customizable: DA, gas token, permisos
- Licencia específica de Arbitrum

OP Stack:

- Framework modular de Optimism
- Base, Zora, Mode lo usan
- Open source (MIT license)

- "Superchain" vision de interoperabilidad

Polygon CDK:

- Chain Development Kit de Polygon
- ZK-based (zkEVM)
- Puede conectar al "AggLayer"
- Validium o rollup configurable

zkSync Hyperchains:

- Chains conectadas a zkSync Era
- Comparten prover infrastructure
- Interoperabilidad nativa
- ZK-based

Ejemplo 3.4 (App-chains en producción)**.**

App-chain	Tipo	Framework	Caso de uso
Xai	L3	Arbitrum Orbit	Gaming (Arbitrum)
Degen Chain	L3	Arbitrum Orbit	Social/Memes
Zora	L2	OP Stack	NFTs
Worldcoin	L2	OP Stack	Identity
Immutable X	L2	StarkEx	Gaming NFTs
dYdX v4	L1	Cosmos SDK	Derivatives

Nota: dYdX migró de StarkEx L2 a su propia L1 en Cosmos, mostrando que la elección óptima depende del caso de uso.

3.9.3 Interoperabilidad entre L2s/L3s

Advertencia

El problema de fragmentación:
Con docenas de L2s y cientos de L3s:

- Liquidez fragmentada entre chains
- UX complicada (bridging constante)
- Composability perdida (no puedes llamar contratos cross-chain atómicamente)

Soluciones en desarrollo:

1. Shared sequencing:

- Mismo sequencer ordena txs de múltiples chains
- Permite atomic bundles cross-chain

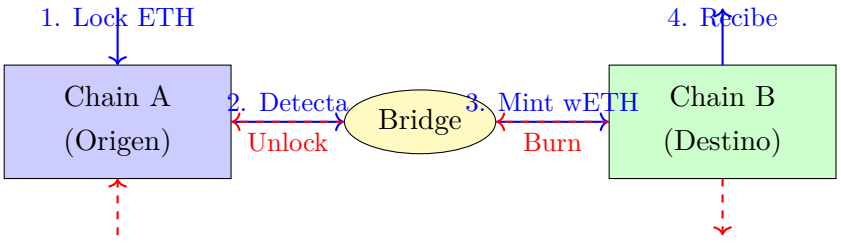
2. **Proof aggregation:**
   - Múltiples L3s comparten un proof en L2/L1
   - Reduce costo de verificación
3. **Messaging protocols:**
   - LayerZero, Hyperlane, Axelar
   - Mensajes verificables cross-chain
   - No atomic, pero funciona
4. **Superchain/AggLayer:**
   - Visión de chains interconectadas
   - Standards comunes
   - Interoperabilidad nativa

3.10 Bridges

Definición 3.32 (Bridge)**.** Protocolo para transferir activos entre blockchains.

3.10.1 Cómo Funcionan

1. Usuario deposita en contrato de origen
2. Bridge detecta depósito
3. Emite tokens equivalentes en destino
4. Para retirar, proceso inverso



3.10.2 Tipos de Bridges

Definición 3.33 (Tipos de Bridges)**. 1. Native (Canonical):**

- Integrado en el rollup
- Usa el mecanismo de seguridad del rollup

- El más seguro para ese rollup
- Ejemplo: Bridge nativo de Arbitrum/Optimism

2. Lock-and-mint:

- Bloquear token en chain A
- Mintear wrapped token en chain B
- Wrapped token representa el original
- Ejemplo: WBTC (Bitcoin wrapped en Ethereum)

3. Burn-and-mint:

- Quemar token en chain A
- Mintear token "nativo" en chain B
- Token no es wrapped, es nuevo
- Requiere coordinación entre chains
- Ejemplo: USDC (Circle mint/burn)

4. Liquidity Networks:

- Pools de liquidez en ambas chains
- No hay lock/mint, solo swaps
- Más rápido pero limitado por liquidez
- Ejemplo: Hop, Across, Stargate

Matemáticas

Comparación de tipos:

Tipo	Velocidad	Liquidez	Seguridad	Costo
Native	Lento (7d opt)	Ilimitada	Máxima	Medio
Lock-mint	Medio	Ilimitada	Variable	Medio
Burn-mint	Medio	Ilimitada	Alta	Medio
Liquidity	Rápido	Limitada	Alta	Alto (slippage)

3.10.3 Mecanismos de Verificación

Idea Clave

¿Cómo sabe Chain B que el depósito en Chain A es real?

1. Multisig/MPC:

- Grupo de firmantes atestiguan el depósito
- Si M-de-N firman, se considera válido
- Trust assumption: mayoría honesta
- Problema: punto único de fallo

2. **Optimistic (con challenge):**
   - Cualquiera puede atestiguar
   - Período de disputa para cuestionar
   - Fraud proof si attestation es falsa
   - Lento pero trustless
3. **Light client:**
   - Verificar headers de la otra chain
   - Verificar Merkle proof de la tx
   - Trustless si light client es correcto
   - Requiere implementar consenso de la otra chain
4. **ZK:**
   - Generar proof de que tx existe en chain A
   - Verificar proof en chain B
   - Trustless y rápido
   - Costoso de generar proofs

3.10.4 Riesgos de Bridges

Hacks históricos:

- Ronin: $625M (multisig comprometido)
- Wormhole: $320M (bug en verificación)
- Nomad: $190M (bug permite mint arbitrario)

Advertencia

Anatomía de los hacks de bridges:
Ronin (Marzo 2022):

- Bridge controlado por 9 validadores, threshold 5

- 4 de Sky Mavis + 1 de Axie DAO comprometidos
- Atacante obtuvo 5 claves privadas
- Firmó retiros fraudulentos
- $625M robados, no detectado por 6 días

Wormhole (Febrero 2022):

- Bug en verificación de firmas
- Atacante bypassed signature check
- Minteó 120,000 wETH sin depósito real
- Jump Crypto (backer) cubrió las pérdidas

Nomad (Agosto 2022):

- Bug en inicialización de contrato
- Cualquier mensaje se consideraba válido
- Cientos de atacantes copiaron el exploit
- "Decentralized robbery"

Lecciones:

- Multisigs son puntos únicos de fallo
- Bugs en verificación son catastróficos
- Bridges son los targets más valiosos
- ZK bridges son más seguros pero más complejos

3.10.5 ZK Bridges

Soluciones:

- ZK bridges: verificar consenso en ZK
- Light client bridges: verificar headers
- Optimistic bridges: período de disputa

Definición 3.34 (ZK Bridge)**.** Bridge que usa pruebas ZK para verificar eventos en la chain de origen.

Matemáticas

Cómo funciona un ZK Bridge:

```
Chain A (origen):
1. Usuario deposita en contrato
2. Tx incluida en bloque
3. Bloque tiene header firmado por validators

Prover (off-chain):
4. Obtiene header y signatures de Chain A
5. Genera ZK proof de:
   - Header es válido (cumple reglas de consenso)
   - Signatures son válidas (>2/3 de validators)
   - Tx de depósito está en el bloque (Merkle proof)
6. Proof es pequeño (~200 bytes)

Chain B (destino):
7. Verifica ZK proof (barato, ~200K gas)
8. Si válido: mintea tokens
9. No necesita confiar en nadie
```

Ventajas:

- Trustless: solo confías en matemáticas
- Rápido: verificación instantánea
- Barato: proof pequeño

Desafíos:

- Probar consenso de otra chain es complejo
- Cada chain tiene reglas diferentes
- Generar proofs toma tiempo

Observación 3.13. **Proyectos de ZK Bridges:**

Proyecto	Enfoque	Estado
Succinct	ZK light clients genéricos	Producción
Polyhedra (zkBridge)	Cross-chain messaging	Producción
Lagrange	State proofs	Desarrollo
Electron Labs	Cosmos $\leftrightarrow$ Ethereum	Desarrollo

En Banco ZK

Bridge de Banco ZK:

El bridge de Banco ZK conecta L1 (Ethereum) con nuestro L2 privado:

Depósitos (L1 → L2):

1. Usuario deposita ETH/tokens en contrato L1
2. Genera commitment secreto off-chain
3. Contrato registra el depósito
4. Sequencer detecta evento
5. Crea nota privada equivalente en L2
6. Usuario tiene acceso con su secreto

Retiros (L2 → L1):

1. Usuario genera nullifier y ZK proof
2. Proof demuestra: tiene nota válida sin revelar cuál
3. Sequencer incluye en batch
4. Batch publicado en L1 con ZK proof agregado
5. Contrato L1 verifica proof
6. Fondos liberados a dirección de retiro

Seguridad:

- Bridge nativo: misma seguridad que el rollup
- No hay multisig que pueda robar fondos
- Si sequencer es malicioso: forced exit via L1
- Privacidad: nadie puede vincular depósito con retiro

Diferencia con bridges externos:

- Bridge nativo: trustless, lento (esperar proof)
- LP bridges (Hop/Across): rápido, pero no privados
- Para Banco ZK usamos principalmente el nativo

3.11 Resumen del Bloque 3

Idea Clave

Puntos clave de Escalabilidad (L2/L3):

1. **El Trilema:**
   - Escalabilidad vs Descentralización vs Seguridad
   - No es un teorema, pero captura trade-off real
   - L2s intentan "escapar" el trilema
2. **L2 = Off-chain execution + L1 security:**
   - Procesar txs fuera de L1
   - Publicar resultados/proofs en L1
   - Heredar seguridad de L1
3. **Sidechains $\neq$ Rollups:**
   - Sidechains: seguridad propia (validadores)
   - Rollups: seguridad de L1 (proofs)
   - Rollups son "verdaderos L2s"
4. **Optimistic Rollups:**
   - Asume validez, permite disputas
   - Fraud proofs (interactive bisection)
   - 7 días de challenge period
   - Ejemplos: Arbitrum, Optimism, Base
5. **ZK Rollups:**
   - Prueba criptográfica de validez
   - Finalidad rápida (minutos)
   - zkEVM para compatibilidad
   - Ejemplos: zkSync, StarkNet, Scroll
6. **Sequencers:**
   - Actualmente centralizados
   - Riesgos: censura, MEV, downtime

- Futuro: descentralización, shared sequencing, based rollups

7. **Data Availability:**
   - Crítico para seguridad
   - Opciones: calldata, blobs, DAC, DA layers
   - EIP-4844 (blobs) redujo costos 90%+
8. **L3 y App-chains:**
   - L3: rollup sobre L2
   - Motivación: especialización, no solo escalabilidad
   - Frameworks: Orbit, OP Stack, Polygon CDK
9. **Bridges:**
   - Conectan diferentes chains
   - Tipos: native, lock-mint, liquidity
   - Riesgos: hacks multimillonarios históricos
   - ZK bridges: futuro trustless
10. **Visión general:**
    - L1 (Ethereum): seguridad y DA
    - L2s: ejecución escalable
    - L3s/App-chains: especialización
    - Bridges: interoperabilidad
    - El ecosistema está convergiendo hacia esta arquitectura modular

Chapter 4

Zero-Knowledge Proofs (Core)

Bloque 4 de 20

4.1 Qué es una ZK Proof

Definición 4.1 (Prueba de Conocimiento Cero)**.** Protocolo donde un Prover convence a un Verifier de que una afirmación es verdadera, sin revelar información adicional.

Intuición

Imagina que quieres probar que conoces la contraseña de una caja fuerte sin revelar la contraseña. Una ZK proof permite exactamente esto: convencer a alguien de que *sabes algo* sin revelar *qué es* ese algo.
Esto parece mágico, pero es matemáticamente riguroso y tiene aplicaciones profundas en criptografía, privacidad, y escalabilidad de blockchains.

4.1.1 Historia y Contexto

Observación 4.1. **La historia de las Zero-Knowledge Proofs:**

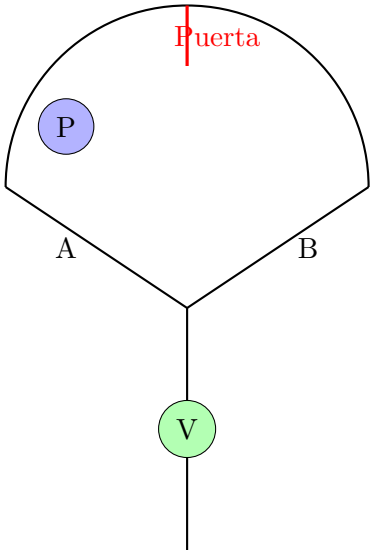
Año	Autores	Contribución
1985	Goldwasser, Micali, Rackoff	Definición formal de ZK
1986	Fiat, Shamir	Transformación a no-interactivo
1988	Goldreich, Micali, Wigderson	ZK para todo NP
1991	Schnorr	Protocolo eficiente de identificación
2010	Groth	Pruebas succinctas eficientes
2012	Bitansky et al.	SNARKs
2013	Gennaro et al.	Pinocchio (primer SNARK práctico)
2016	Groth	Groth16 (el más eficiente)
2018	Ben-Sasson et al.	STARKs (sin trusted setup)
2019	Gabizon et al.	PLONK (universal setup)
2019	Bünz et al.	Bulletproofs recursivos
2020	Zcash team	Halo (recursión sin trusted setup)

El paper original de 1985 "The Knowledge Complexity of Interactive Proof Systems" ganó el premio Gödel en 1993 y sentó las bases de toda la criptografía moderna basada en ZK.

4.1.2 La Intuición: Cueva de Ali Baba

Ejemplo 4.1 (Cueva de Ali Baba)**.** Probar que conoces la palabra secreta de una puerta sin revelarla:

1. Victor espera afuera, Peggy entra y elige camino A o B
2. Victor entra y pide "sal por A" o "sal por B"
3. Peggy sale por el camino pedido (atravesando puerta si necesario)
4. Repetir 20 veces: probabilidad de engaño $< 0.0001\%$



Protocolo:

1. Peggy entra, elige A o B
2. Victor entra, dice "sal por X"
3. Peggy sale por X
4. Si conoce la palabra: siempre puede
5. Si no conoce: 50% de fallar
6. Después de 20 rondas: 2^{-20} de engaño

Matemáticas

Análisis del protocolo:
Sea p la probabilidad de que Peggy NO conozca la palabra secreta pero convenza a Victor.
Una ronda:

- Peggy elige camino (digamos A) sin saber qué pedirá Victor
- Victor pide A o B con probabilidad 1/2 cada uno
- Si pide A: Peggy sale (éxito sin importar si conoce la palabra)
- Si pide B: Peggy necesita la palabra para cruzar

$P(\text{éxito sin palabra}) = 1/2$
n **rondas:** $P(\text{éxito en todas sin palabra}) = (1/2)^n$
Con $n = 20$: $P = 2^{-20} \approx 10^{-6}$ Con $n = 40$: $P = 2^{-40} \approx 10^{-12}$
¿Por qué es Zero-Knowledge?

Victor no aprende nada sobre la palabra porque:

- Solo ve a Peggy salir por el camino que pidió
- Esto ocurriría 100% si conoce la palabra
- No puede distinguir si cruzó la puerta o no
- La interacción no revela la palabra

4.1.3 Definición Formal

Definición 4.2 (Sistema de Prueba Interactivo)**.** Un sistema de prueba interactivo para un lenguaje L consiste en:

- **Prover $\mathcal{P}$:** Algoritmo (posiblemente sin límite de tiempo)
- **Verifier $\mathcal{V}$:** Algoritmo de tiempo polinomial
- **Interacción:** Secuencia de mensajes $\langle \mathcal{P}, \mathcal{V} \rangle(x)$
- **Resultado:** $\mathcal{V}$ acepta o rechaza

Definición 4.3 (Prueba de Conocimiento Cero - Formal)**.** Un sistema de prueba interactivo $(\mathcal{P}, \mathcal{V})$ para L es **zero-knowledge** si satisface:

1. **Completeness:**

$$\forall x \in L, \forall w \text{ válido} : \Pr[\langle \mathcal{P}(w), \mathcal{V} \rangle(x) = 1] = 1$$

2. **Soundness:**

$$\forall x \notin L, \forall \mathcal{P}^* : \Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(x) = 1] \leq \epsilon$$

3. **Zero-Knowledge:**

$$\exists \text{ Simulador } \mathcal{S} : \forall \mathcal{V}^*, \text{View}_{\mathcal{V}^*}[\langle \mathcal{P}, \mathcal{V}^* \rangle(x)] \approx \mathcal{S}(x)$$

Idea Clave

Lo que significa cada propiedad intuitivamente:
Completeness: "Si digo la verdad, siempre me creerán"

- Un prover honesto con un witness válido siempre convence
- No hay falsos negativos

Soundness: "Si miento, me descubrirán (casi seguro)"

- Un prover malicioso no puede convencer si la afirmación es falsa
- Excepto con probabilidad negligible ϵ

Zero-Knowledge: "No revelo más de lo necesario"

- El verificador solo aprende que $x \in L$
- Nada sobre el witness w
- Nada que no pudiera calcular solo

4.2 Propiedades: Completeness, Soundness, Zero-Knowledge

4.2.1 Completeness (Completitud)

Si la afirmación es verdadera, un prover honesto siempre convence al verificador.

$$x \in L \implies \Pr[\text{Verifier acepta}] = 1$$

Definición 4.4 (Completeness - Tipos)**. Perfect Completeness:**

$$\Pr[\langle \mathcal{P}(w), \mathcal{V} \rangle(x) = 1] = 1$$

El verificador acepta con probabilidad exactamente 1.
Statistical Completeness:

$$\Pr[\langle \mathcal{P}(w), \mathcal{V} \rangle(x) = 1] \geq 1 - \text{negl}(\lambda)$$

El verificador acepta con probabilidad abrumadora.

Ejemplo 4.2 (Completeness en protocolo Schnorr)**.** Para probar conocimiento de x tal que $Y = g^x$:

```
Prover (conoce x):
    1. Elige r aleatorio
    2. Envía R = g^r

Verifier:
    3. Envía challenge c aleatorio

Prover:
    4. Calcula z = r + c*x
    5. Envía z

Verifier:
    6. Acepta si g^z = R * Y^c
```

Verificación de completeness:

$$g^z = g^{r+cx} = g^r \cdot g^{cx} = R \cdot (g^x)^c = R \cdot Y^c \checkmark$$

Si el prover es honesto, la verificación siempre pasa.

4.2.2 Soundness (Solidez)

Si la afirmación es falsa, ningún prover puede convencer al verificador (excepto con probabilidad negligible).

$$x \notin L \implies \Pr[\text{Verifier acepta}] \leq \epsilon$$

Definición 4.5 (Tipos de Soundness)**. Statistical Soundness:**

$$\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(x) = 1] \leq 2^{-\lambda}$$

Seguro contra provers computacionalmente ilimitados.

Computational Soundness (Argument):

$$\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(x) = 1] \leq \text{negl}(\lambda)$$

Solo seguro contra provers de tiempo polinomial.

Matemáticas

Proof vs Argument:

Proof (prueba): Soundness estadístico

- Seguro incluso contra adversarios con poder computacional infinito
- Basado en teoría de la información
- Típicamente pruebas más grandes

Argument (argumento): Soundness computacional

- Seguro solo contra adversarios eficientes
- Basado en asunciones criptográficas
- Permite pruebas más pequeñas (SNARKs)

SNARKs son *arguments*, no *proofs* en el sentido técnico.

Knowledge soundness: Además, si el verifier acepta, existe un "extractor" que puede obtener el witness del prover.

Definición 4.6 (Knowledge Soundness)**.** Un sistema tiene knowledge soundness si existe un extractor $\mathcal{E}$ tal que:

Para todo prover $\mathcal{P}^*$ que hace que el verificador acepte con probabilidad ϵ, $\mathcal{E}$ puede extraer un witness válido w con probabilidad $\epsilon - \text{negl}(\lambda)$.

Formalmente:

$$\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(x) = 1] = \epsilon \implies \Pr[\mathcal{E}^{\mathcal{P}^*}(x) = w : R(x, w) = 1] \geq \epsilon - \text{negl}(\lambda)$$

Idea Clave

¿Por qué Knowledge Soundness importa?
Soundness normal solo dice: "no puedes convencer si $x \notin L$"
Knowledge soundness dice: "si convences, realmente CONOCES un witness"
Esto es crucial para aplicaciones como:

- Probar que conoces una clave privada
- Probar que conoces la preimagen de un hash
- Probar que ejecutaste un cálculo correctamente

Sin knowledge soundness, podrías "adivinar" la prueba correcta sin realmente conocer el secreto (aunque con probabilidad exponencialmente pequeña).

Ejemplo 4.3 (Extracción en Schnorr)**.** Para el protocolo Schnorr, el extractor funciona así:

```
Extractor E:
    1. Ejecuta P* hasta que envía R
    2. Envía challenge c1, obtiene z1
    3. "Rebobina" P* al estado después de R
    4. Envía challenge c2  c1, obtiene z2

    5. Si ambos verifican:
       g^z1 = R * Y^c1
       g^z2 = R * Y^c2

    6. Entonces:
       g^(z1-z2) = Y^(c1-c2)
       g^((z1-z2)/(c1-c2)) = Y
       x = (z1-z2)/(c1-c2)  ← ¡Witness extraído!
```

Esto demuestra que si $\mathcal{P}^*$ puede responder a dos challenges diferentes para el mismo R, el extractor puede calcular x.

4.2.3 Zero-Knowledge

El verificador no aprende nada más allá de "$x \in L$".

Definición 4.7 (Simulador)**.** Existe un algoritmo (simulador) que puede generar transcripciones indistinguibles de interacciones reales, sin conocer el witness.

Matemáticas

La definición formal de Zero-Knowledge:
Sea $\text{View}_{\mathcal{V}^*}[\langle \mathcal{P}(w), \mathcal{V}^* \rangle(x)]$ la "vista" del verificador:

- Todos los mensajes que recibió

- Su aleatoriedad interna
- Su estado final

Un protocolo es ZK si existe un simulador $\mathcal{S}$ que, dado solo x (sin w), puede producir una vista indistinguible:

$$\text{View}_{\mathcal{V}^*}[\langle \mathcal{P}(w), \mathcal{V}^* \rangle(x)] \approx \mathcal{S}(x)$$

Intuición: Si el simulador puede producir la misma "experiencia" sin el witness, entonces el verificador no pudo haber aprendido nada del witness durante la interacción real.

Tipos:

- **Perfect ZK:** Distribuciones idénticas
- **Statistical ZK:** Distribuciones estadísticamente cercanas
- **Computational ZK:** Indistinguibles para adversarios eficientes

Definición 4.8 (Tipos de Zero-Knowledge)**. Perfect ZK:**

$$\text{View[real]} \equiv \mathcal{S}(x)$$

Distribuciones exactamente iguales.

Statistical ZK:

$$\Delta(\text{View[real]}, \mathcal{S}(x)) \leq \text{negl}(\lambda)$$

Distancia estadística negligible.

Computational ZK:

$$\text{View[real]} \approx_c \mathcal{S}(x)$$

Computacionalmente indistinguibles.

Ejemplo 4.4 (Simulador para Schnorr)**. Protocolo real:**

```
P: R = g^r (r aleatorio)
V: c aleatorio
P: z = r + c*x
Transcript: (R, c, z)
Verifica: g^z = R * Y^c
```

Simulador (sin conocer x):

```
S:
    1. Elige z y c aleatorios
    2. Calcula R = g^z / Y^c
    3. Output: (R, c, z)
```

Verificación de que simula correctamente:

$$g^z = R \cdot Y^c \iff R = g^z / Y^c \checkmark$$

El transcript simulado tiene la misma distribución que el real:

- En ambos casos, z es uniforme
- c es uniforme en ambos
- R está determinado por z y c

¡El simulador produce transcripts indistinguibles sin conocer x!

Idea Clave

¿Por qué el simulador demuestra zero-knowledge?
El argumento es por contradicción:

1. Supón que el verificador $\mathcal{V}^*$ aprende algo sobre w
2. Entonces podría distinguir entre transcripts reales y simulados
3. Pero el simulador produce transcripts indistinguibles
4. Por lo tanto, $\mathcal{V}^*$ no puede haber aprendido nada

Cualquier cosa que $\mathcal{V}^*$ puede computar del transcript real, también la puede computar del transcript simulado (que no usó w).

4.2.4 Trade-offs entre Propiedades

Observación 4.2. **No puedes tener todo:**

Sistema	Completeness	Soundness	ZK
Statistical Proof	Perfect	Statistical	Computational
SNARK	Perfect	Computational	Computational
STARK	Perfect	Statistical	Computational

Teorema (informal): No puede existir un sistema con soundness perfecto, ZK perfecto, y completeness perfecto para lenguajes no triviales.

Por eso la mayoría de sistemas prácticos tienen:

- Perfect completeness (lo más fácil)
- Computational soundness (para pruebas cortas)
- Computational ZK (suficiente en práctica)

En Banco ZK

Propiedades ZK en Banco ZK:
Las pruebas de Banco ZK garantizan:
Completeness:

- Si tu transacción es válida, siempre se genera una prueba válida
- Nunca se rechaza una tx honesta

Soundness:

- No puedes crear pruebas para txs inválidas (doble gasto, monto negativo)
- Basado en la dureza del problema DLP/asunciones de pairings

Zero-Knowledge:

- La prueba no revela: quién envía, quién recibe, cuánto
- Solo revela: "la tx es válida según las reglas del protocolo"
- El verificador (L1) no aprende nada sobre los participantes

4.3 Interactive vs Non-interactive Proofs

4.3.1 Pruebas Interactivas

Múltiples rondas de comunicación:

$$\mathcal{P} \leftrightarrow \mathcal{V}: \quad \text{commit} \rightarrow \text{challenge} \rightarrow \text{response}$$

Intuición

En una prueba interactiva, prover y verifier tienen un "diálogo":

1. Prover hace un commitment (se "compromete" a algo)
2. Verifier envía un challenge aleatorio
3. Prover responde al challenge
4. Verifier decide si acepta

La interacción es crucial porque el challenge debe ser:

- Impredecible para el prover (generado después del commitment)
- Aleatorio (para que no pueda prepararse)

Si el prover pudiera predecir el challenge, podría hacer trampa.

Protocolo Sigma (Σ-protocol)

Protocolo Σ (3 rondas):

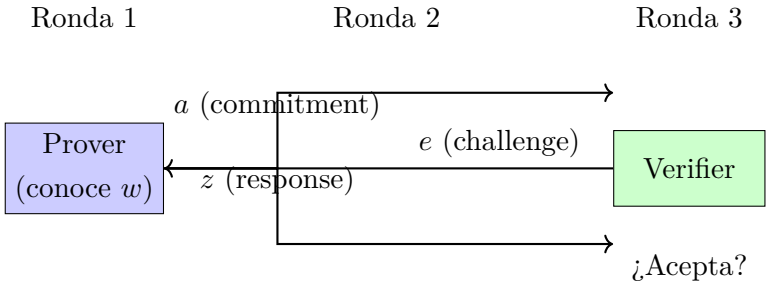
1. Prover envía commitment a
2. Verifier envía challenge aleatorio e
3. Prover envía response z
4. Verifier comprueba relación

Definición 4.9 (Σ-Protocol)**.** Un protocolo de 3 rondas (a, e, z) es un Σ-protocol si satisface:

1. Completeness: Prover honesto siempre convence.

2. Special Soundness: Dados dos transcripts aceptados (a, e, z) y (a, e', z') con $e \neq e'$, se puede extraer el witness.

3. Special Honest-Verifier ZK (SHVZK): Existe simulador para verificador honesto.



Ejemplo 4.5 (Schnorr como Σ-protocol)**.** Para relación $R = \{(Y, x) : Y = g^x\}$:

Prover (conoce x)		**Verifier**
$r \xleftarrow{\$} \mathbb{Z}_q$		
$a = g^r$		
	$\xleftarrow{a}$	
		$e \xleftarrow{\$} \mathbb{Z}_q$
$\xrightarrow{e}$		
$z = r + e \cdot x \mod q$		
	$\xleftarrow{z}$	
		Verificar: $g^z \stackrel{?}{=} a \cdot Y^e$

Special Soundness:

Si tenemos (a, e, z) y (a, e', z') con $e \neq e'$:

$$\begin{aligned} g^z &= a \cdot Y^e \\ g^{z'} &= a \cdot Y^{e'} \\ \implies g^{z-z'} &= Y^{e-e'} \\ \implies Y &= g^{(z-z')/(e-e')} \end{aligned}$$

Por lo tanto $x = (z - z')/(e - e')$.

Múltiples Rondas

Matemáticas

¿Por qué 3 rondas son comunes?

1 ronda: El prover envía todo de una vez

- No puede ser ZK (verifier no contribuye aleatoriedad)
- Excepto en el modelo de random oracle (NIZK)

3 rondas (Σ-protocol):

- Mínimo para ZK con verifier honesto
- Commit → Challenge → Response
- Muy eficiente, bien estudiado

Más rondas:

- Necesario para ZK contra verifier malicioso
- O para soundness más fuerte
- Generalmente se evita por eficiencia

Teorema (Goldreich-Kahan): 4 rondas son suficientes para ZK contra cualquier verifier.

4.3.2 NIZK (Non-Interactive ZK)

Un solo mensaje del prover al verificador.

Requiere:

- **CRS:** Common Reference String
- **Random Oracle:** Hash modelado como función aleatoria

Definición 4.10 (NIZK)**.** Un NIZK para lenguaje L consiste en:

Setup: $\text{crs} \leftarrow \text{Setup}(1^\lambda)$

Prove: $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$

Verify: $b \leftarrow \text{Verify}(\text{crs}, x, \pi)$

Con las propiedades usuales de completeness, soundness, y zero-knowledge.

Idea Clave

¿Por qué necesitamos CRS o Random Oracle?

En pruebas interactivas, la aleatoriedad del verifier es crucial:

- El prover no puede predecirla
- Por eso no puede hacer trampa

En NIZK, no hay verifier enviando challenges. Necesitamos otra fuente de aleatoriedad impredecible:

Opción 1 - CRS (Common Reference String):

- String aleatorio generado una vez
- Todos confían en que es aleatorio
- El prover no lo eligió, así que no puede explotarlo

Opción 2 - Random Oracle (Fiat-Shamir):

- Hash modelado como función aleatoria
- Challenge = Hash(commitment)
- El prover no puede predecir el output del hash

4.3.3 Transformación Fiat-Shamir

Convierte prueba interactiva en no interactiva:

1. En protocolo original, verifier envía challenge e
2. En Fiat-Shamir: $e = H(\text{transcript})$
3. Hash "simula" aleatoriedad del verifier

Definición 4.11 (Transformación Fiat-Shamir)**.** Dado un Σ-protocol (a, e, z), la versión Fiat-Shamir es:

Prove:

1. Prover genera commitment a
2. Calcula challenge: $e = H(x \| a)$
3. Calcula response z
4. Output: $\pi = (a, z)$ (nota: e no se incluye, es derivable)

Verify:

1. Recalcula $e = H(x \| a)$
2. Verifica la relación con (a, e, z)

Ejemplo 4.6 (Schnorr con Fiat-Shamir)**. Interactivo:**

```
P: a = g^r
V: e ← random
P: z = r + e*x
V: check g^z = a * Y^e
```

Non-interactivo (Fiat-Shamir):

```
P:
    a = g^r
    e = H(Y || a)           // Challenge derivado del hash
    z = r + e*x
    output  = (a, z)

V:
    e = H(Y || a)           // Recalcula el mismo challenge
    check g^z = a * Y^e
```

Matemáticas

Seguridad de Fiat-Shamir:

En el Random Oracle Model (ROM):

Si el hash H es un "random oracle" (función verdaderamente aleatoria), entonces Fiat-Shamir es seguro:

- El prover no puede predecir $H(x\|a)$ antes de elegir a
- Es como si el verifier eligiera e después de ver a

En la práctica (Standard Model):

Hashes reales (SHA-256, Keccak) no son random oracles:

- Hay ataques teóricos en algunos casos
- Pero en la práctica, funciona bien para esquemas diseñados cuidadosamente
- Se debe incluir suficiente contexto en el hash: $H(\text{dominio}\|x\|a)$

Errores comunes:

- No incluir x en el hash $\rightarrow$ maleable
- No incluir información de dominio $\rightarrow$ ataques cross-protocol

Advertencia

Weak Fiat-Shamir:

Un error común es calcular $e = H(a)$ en lugar de $e = H(x\|a)$.

Ataque:

1. Atacante quiere probar x' (statement diferente)
2. Reutiliza la misma prueba (a, z)
3. Challenge $e = H(a)$ es el mismo
4. Si la verificación solo usa e y no x directamente, acepta

Regla: Siempre incluir el statement completo en el hash.

```
// INCORRECTO
e = H(a)

// CORRECTO
e = H(context || statement || a || public_inputs)
```

4.3.4 Comparación Interactive vs NIZK

Aspecto	Interactive	NIZK
Comunicación	Múltiples rondas	1 mensaje
Verificador online	Sí	No
Reusable	No (fresh challenges)	Sí
Publicable	No	Sí (on-chain)
Asunción	Ninguna adicional	ROM o CRS
Tamaño prueba	Similar	Similar

Idea Clave

¿Por qué blockchains usan NIZK?
Las pruebas en blockchain deben ser:

- **No interactivas:** No hay "verifier" online
- **Verificables por cualquiera:** Cualquier nodo debe poder verificar
- **Publicables:** Van en la cadena
- **Reusables:** La misma prueba vale para todos

Esto hace que NIZK sea esencial para ZK rollups, Zcash, etc.

4.4 Witness (Testigo)

Definición 4.12 (Witness)**.** Información secreta que satisface la relación y permite generar la prueba.

Para lenguaje NP con relación R:

$$x \in L \iff \exists w : R(x, w) = 1$$

- x: statement (público)
- w: witness (privado)

4.4.1 Lenguajes NP y Relaciones

Matemáticas

Conexión con complejidad computacional:
Clase NP: Un lenguaje L está en NP si existe:

- Relación R verificable en tiempo polinomial
- Para cada $x \in L$, existe witness w de tamaño polinomial
- Tal que $R(x, w) = 1$

ZK Proofs para NP:
El teorema fundamental (Goldreich, Micali, Wigderson, 1991):

> Todo lenguaje en NP tiene una prueba zero-knowledge.

Esto significa que cualquier problema cuya solución se pueda verificar eficientemente, también se puede probar en zero-knowledge.

Ejemplo 4.7 (Witness para diferentes problemas)**. 1. Conocimiento de preimagen:**

- Statement: y (el hash)
- Witness: x tal que $H(x) = y$
- Relación: verificar $H(x) = y$

2. Conocimiento de clave privada:

- Statement: Y (clave pública)
- Witness: x tal que $Y = g^x$
- Relación: verificar $g^x = Y$

3. Transacción válida (simplificado):

- Statement: (old_root, new_root, nullifier)
- Witness: (secret_key, amount, merkle_path, randomness)
- Relación: verificar Merkle proof, firma, balance ≥ 0

4. Solución de Sudoku:

- Statement: puzzle con algunas celdas llenas
- Witness: valores de todas las celdas
- Relación: verificar reglas de Sudoku

5. Grafo 3-coloreable:

- Statement: grafo $G = (V, E)$
- Witness: coloración $c : V \to \{1, 2, 3\}$
- Relación: $\forall (u, v) \in E : c(u) \neq c(v)$

4.4.2 Witness en Circuitos ZK

Definición 4.13 (Witness en Circuitos Aritméticos)**.** En sistemas como SNARKs, el witness consiste en los valores de todos los "wires" internos del circuito.

Public inputs: Valores conocidos por el verifier **Private inputs (witness):** Valores conocidos solo por el prover **Intermediate wires:** Valores calculados durante la ejecución

Ejemplo 4.8 (Witness para $x^3 + x + 5 = y$)**.** Circuito:

```
    w1 = x         // Input privado
    w2 = w1 * w1   // x^2
    w3 = w2 * w1   // x^3
    w4 = w3 + w1   // x^3 + x
    w5 = w4 + 5    // x^3 + x + 5

    assert(w5 == y)  // y es público

Para x = 3, y = 35:
    Witness completo: (w1=3, w2=9, w3=27, w4=30, w5=35)

Public: y = 35
Private: x = 3
Intermediate: w2, w3, w4
```

4.4.3 Witness Generation

Idea Clave

El proceso de generar el witness:

```
Entrada:
    - Circuito C
    - Inputs públicos (statement)
    - Inputs privados (secretos del prover)

Proceso:
    1. Asignar inputs a los wires correspondientes
    2. Evaluar el circuito gate por gate
    3. Calcular valor de cada wire intermedio
    4. Verificar que constraints se satisfacen

Salida:
```

```
    - Witness completo (todos los valores de wires)
    - O error si los inputs no satisfacen el circuito
```

Complejidad: $O(|C|)$ donde $|C|$ es el tamaño del circuito.
El witness generation es generalmente la parte más rápida. El proving (generar la prueba dado el witness) es lo costoso.

Ejemplo 4.9 (Witness generation en Circom)**.**

```
// Circuito Circom
template Multiplier() {
    signal input a;
    signal input b;
    signal output c;

    c <== a * b;
}

// Witness generation (JavaScript)
const input = {
    a: "3",
    b: "5"
};

// snarkjs calcula:
// c = 3 * 5 = 15
// witness = [1, 15, 3, 5]  // [one, c, a, b]
```

4.5 Prover vs Verifier

Aspecto	Prover	Verifier
Input	Statement + Witness	Statement + Proof
Cómputo	Costoso ($O(n)$ a $O(n \log n)$)	Barato ($O(1)$ a $O(\log n)$)
Memoria	Alta	Baja
Rol	Convencer	Verificar

4.5.1 El Rol del Prover

Definición 4.14 (Prover)**.** El prover tiene acceso a:

- El statement x (público)
- El witness w (secreto)
- Los parámetros del sistema (CRS si aplica)

Su objetivo: generar una prueba π que convenza al verifier de que $x \in L$.

Matemáticas

Complejidad del Prover:

Para diferentes sistemas:

Sistema	Tiempo	Memoria
Groth16	$O(n \log n)$	$O(n)$
PLONK	$O(n \log n)$	$O(n)$
STARK	$O(n \log^2 n)$	$O(n)$
Bulletproofs	$O(n)$	$O(n)$

Donde n es el tamaño del circuito/statement.

Operaciones dominantes:

- FFT (Fast Fourier Transform): $O(n \log n)$
- MSM (Multi-Scalar Multiplication): $O(n)$
- Evaluación de polinomios: $O(n)$

El proving es inherentemente costoso porque "comprime" un cálculo de tamaño n en una prueba pequeña.

4.5.2 El Rol del Verifier

Definición 4.15 (Verifier)**.** El verifier tiene acceso a:

- El statement x (público)
- La prueba π
- Los parámetros del sistema

Su objetivo: decidir si acepta o rechaza, idealmente en tiempo sub-lineal en el tamaño del statement original.

Matemáticas

Complejidad del Verifier:

Sistema	Tiempo	Tamaño prueba
Groth16	$O(1)$ (3 pairings)	128 bytes
PLONK	$O(1)$ (1-2 pairings)	∼400 bytes
STARK	$O(\log^2 n)$	∼50-200 KB
Bulletproofs	$O(n)$	$O(\log n)$

La magia de la succinctness:

Para un circuito de 1 millón de gates:

- Ejecutar el circuito directamente: ∼1M operaciones
- Verificar Groth16: ∼3 operaciones de pairing
- Ratio: ∼300,000x más rápido

Esta asimetría es lo que hace útiles los ZK proofs para blockchain.

4.5.3 Asimetría Prover-Verifier

Idea Clave

¿Por qué la asimetría es buena?
En blockchain:

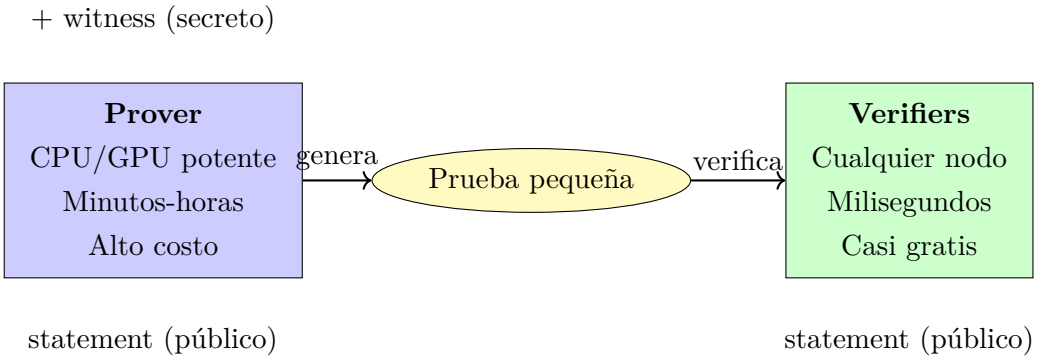
- **Provers:** Pocos, tienen recursos (servidores, GPUs)
- **Verifiers:** Muchos (todos los nodos), deben ser eficientes

Ejemplo ZK Rollup:

- 1 prover genera prueba para 10,000 txs: 5 minutos, $10 de compute
- 10,000 nodos verifican: 10ms cada uno, prácticamente gratis

Si la verificación fuera $O(n)$, los nodos tendrían que re-ejecutar todo, perdiendo el beneficio de escalabilidad.
Regla general:

- Está bien que proving sea caro (pocos provers)
- Verificación DEBE ser barata (muchos verificadores)
- El costo de proving se amortiza entre todos los usuarios



En Banco ZK

Prover y Verifier en Banco ZK:
Arquitectura de proving:

```
Usuario genera tx privada
    ↓
Usuario genera witness local
(clave, monto, Merkle path, randomness)
    ↓
Prover (puede ser usuario o servidor dedicado)
genera ZK proof de la tx individual
    ↓
Sequencer agrega txs en batch
    ↓
Prover central genera proof del batch completo
(puede usar recursión para combinar proofs)
    ↓
Proof publicado en L1
    ↓
Contrato L1 verifica (3 pairings, ~200K gas)
```

Trade-offs:

- Proving en cliente: máxima privacidad, puede ser lento
- Proving en servidor: más rápido, pero servidor ve el witness
- Híbrido: proof parcial en cliente, agregación en servidor

4.6 Trusted Setup vs Transparent Setup

4.6.1 Trusted Setup

Definición 4.16 (Trusted Setup)**.** Ceremonia para generar parámetros públicos. Se crea "toxic waste" que debe destruirse.

- Si alguien tiene toxic waste: puede crear pruebas falsas
- Mitigación: MPC ceremony (basta 1 participante honesto)
- Powers of Tau: ceremonia reutilizable

Sistemas: Groth16, PLONK (KZG).

¿Qué es el Toxic Waste?

Intuición

El trusted setup genera parámetros públicos que "codifican" un secreto τ (tau). Este secreto es el "toxic waste".
Imagina que los parámetros son una caja fuerte:

- La caja fuerte está diseñada para que solo quien conoce un código específico pueda abrirla
- El código se destruye después de crear la caja
- Si alguien conservó el código, puede abrir cualquier caja de ese tipo

En ZK proofs:

- Los parámetros permiten crear pruebas válidas
- Pero quien conoce τ puede crear pruebas de statements FALSOS
- Por eso τ debe ser destruido irrecuperablemente

Matemáticas

El toxic waste en KZG/Groth16:
Setup:
Se elige $\tau \xleftarrow{\$} \mathbb{Z}_p$ aleatorio (el toxic waste).
Se generan los parámetros públicos (SRS - Structured Reference String):

$$\text{SRS} = \{g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d}\}$$

Donde g es generador del grupo.
Uso legítimo:
Para commitir a un polinomio $p(x) = \sum_{i=0}^{d} a_i x^i$:

$$C = \prod_{i=0}^{d} (g^{\tau^i})^{a_i} = g^{\sum a_i \tau^i} = g^{p(\tau)}$$

El prover no conoce τ, solo puede evaluar $g^{p(\tau)}$ usando el SRS.
Ataque si conoces τ:
Si conoces τ, puedes:

1. Crear commitment a cualquier polinomio
2. Crear prueba de evaluación para cualquier punto
3. Incluso para relaciones que no son verdad

El sistema de prueba asume que nadie conoce τ.

MPC Ceremonies (Multi-Party Computation)

Definición 4.17 (Ceremonia MPC)**.** Protocolo donde múltiples participantes contribuyen aleatoriedad al setup, de forma que:

- Nadie conoce el secreto completo
- Basta que 1 participante sea honesto para que el setup sea seguro

Idea Clave

Cómo funciona una ceremonia MPC:
Idea básica:
En lugar de un τ único, se construye como producto de contribuciones:

$$\tau = \tau_1 \cdot \tau_2 \cdot \tau_3 \cdots \tau_n$$

Cada participante i:

1. Recibe los parámetros del participante anterior
2. Elige su τ_i aleatorio
3. "Multiplica" los parámetros por τ_i
4. Destruye τ_i
5. Pasa los nuevos parámetros al siguiente

Seguridad:
Para conocer τ completo, necesitas TODOS los τ_i.
Si al menos un participante:

- Generó su τ_i aleatoriamente
- Lo destruyó correctamente

Entonces τ es desconocido para todos, incluyendo atacantes.
Trust assumption: "1-of-n honest" (al menos 1 de n es honesto)

Ejemplo 4.10 (Ceremonia simplificada)**.** `Inicio: params_0 = (g, g^1) = (g, g)  // "tau = 1" inici`

```
Participante 1:
    tau_1 = random()
    params_1 = (g, g^tau_1)
    delete tau_1

Participante 2:
    tau_2 = random()
    params_2 = (g, (g^tau_1)^tau_2) = (g, g^(tau_1*tau_2))
```

```
    delete tau_2

Participante 3:
    tau_3 = random()
    params_3 = (g, g^(tau_1*tau_2*tau_3))
    delete tau_3

Final:
    tau_efectivo = tau_1 * tau_2 * tau_3

    Para conocer tau_efectivo necesitas tau_1 AND tau_2 AND tau_3
    Si cualquiera es desconocido, tau_efectivo es seguro
```

Ceremonias Históricas

Observación 4.3. **Ceremonias famosas:**

Zcash Sprout (2016):

- 6 participantes
- Protocolo muy seguro pero complejo
- Participantes destruyeron hardware físicamente
- Críticas: pocos participantes

Zcash Powers of Tau (2017-2018):

- 87 participantes públicos
- Cualquiera podía participar
- "Perpetual Powers of Tau" - sigue abierta

Ethereum KZG Ceremony (2023):

- Para EIP-4844 (blobs)
- 141,000+ participantes
- Récord de participación
- Interfaz web fácil de usar
- Probabilidad de que todos sean deshonestos: ≈ 0

Ceremonia	Participantes	Año	Propósito
Zcash Sprout	6	2016	Zcash shielded
Powers of Tau	87+	2017	Universal
Aztec Ignition	200+	2020	PLONK
Hermez	100+	2021	zkRollup
Ethereum KZG	141,000+	2023	EIP-4844

Per-Circuit vs Universal Setup

Definición 4.18 (Tipos de Trusted Setup)**. Per-circuit (Groth16):**

- Setup específico para cada circuito
- Si cambias el circuito, nueva ceremonia
- Pruebas más pequeñas

Universal (PLONK, Marlin):

- Setup para un tamaño máximo de circuito
- Cualquier circuito $\leq$ tamaño max funciona
- Una ceremonia para muchos circuitos

Updatable (PLONK):

- Cualquiera puede "actualizar" el setup
- Añade su propia aleatoriedad
- Setup se vuelve más seguro con el tiempo

4.6.2 Transparent Setup

Definición 4.19 (Transparent Setup)**.** Parámetros derivados de valores públicos, sin secretos.

- "Nothing up my sleeve": semillas públicas
- Sin confianza en ceremonia
- Pruebas típicamente más grandes

Sistemas: STARKs, Bulletproofs, Halo (IPA).

¿Cómo Funciona Transparent Setup?

Idea Clave

Generación de parámetros sin secretos:
Método 1: Hash-to-curve ("nothing up my sleeve"):

```
// Generar puntos de curva sin conocer logaritmo discreto
seed = "Banco ZK Genesis 2024"
for i in 0..n:
    G_i = hash_to_curve(seed || i)

// Nadie conoce x tal que G_i = x * G_0
// Los puntos son "aleatorios" pero determinísticos
```

Método 2: Usar constantes públicas:

- Dígitos de π: "3.14159..."
- Dígitos de e: "2.71828..."
- Hash de algo público (headline de periódico)

Verificabilidad:
Cualquiera puede regenerar los parámetros:

- Ejecutar el mismo algoritmo
- Verificar que coincide con los parámetros publicados
- No hay "ceremonia" de la que depender

Ejemplo 4.11 (Generadores en Bulletproofs)**.** `// Bulletproofs necesita generadores G_i, H_i`

```
def generate_generators(n):
    generators_g = []
    generators_h = []

    for i in range(n):
        // Hash to curve con dominio separado
        g_i = hash_to_curve(b"bulletproofs_g" + i.to_bytes())
        h_i = hash_to_curve(b"bulletproofs_h" + i.to_bytes())

        generators_g.append(g_i)
        generators_h.append(h_i)

    return generators_g, generators_h

// Propiedades:
// - Determinístico: siempre genera los mismos puntos
// - Verificable: cualquiera puede regenerar
// - Sin trapdoor: nadie conoce log_g(g_i) o log_g(h_i)
```

Trade-offs: Trusted vs Transparent

Aspecto	Trusted Setup	Transparent
Confianza	1-of-n honesto en ceremonia	Ninguna
Tamaño prueba	Pequeño (∼100-500 B)	Grande (∼10-200 KB)
Verificación	Muy rápida (pairings)	Más lenta
Post-quantum	No*	Puede ser sí
Complejidad setup	Ceremonia MPC	Determinístico
Actualizar circuito	Nueva ceremonia*	Sin cambios

*Universal setup mitiga estos problemas.

Advertencia

¿Es tan malo el trusted setup?
Argumentos en contra:

- Si la ceremonia falla, todo el sistema es inseguro
- Difícil de auditar
- "Confianza" parece contrario al ethos de crypto

Argumentos a favor:

- Con 141,000+ participantes, la probabilidad de fallo es ≈ 0
- Las pruebas son mucho más pequeñas y baratas de verificar
- En la práctica, el riesgo es mínimo
- El setup se hace una vez; los beneficios son permanentes

Tendencia actual:
La comunidad se mueve hacia transparent setups cuando es práctico, pero trusted setup con muchos participantes sigue siendo aceptable para producción.

4.7 CRS (Common Reference String)

Definición 4.20 (CRS)**.** Parámetros públicos compartidos entre prover y verifier.

Tipos:

- **Structured:** Codifica información sobre τ (KZG)
- **Uniform:** Aleatorio puro
- **Per-circuit:** Específico para cada circuito (Groth16)
- **Universal:** Funciona para cualquier circuito hasta cierto tamaño (PLONK)
- **Updatable:** Cualquiera puede contribuir

4.7.1 Estructura del CRS

Matemáticas

CRS para diferentes sistemas:

Groth16 (per-circuit):

```
CRS = (
    // Proving key (para prover)
    pk = {
        []_1, []_1, []_2, []_1, []_2,
        [^i]_1 para i = 0..n,
        [·A_j() + ·B_j()]_1 para cada wire j,
        ...
    },

    // Verification key (para verifier)
    vk = {
        []_1, []_2, []_2, []_2,
        [^(-1)·(·A_i() + ·B_i() + C_i())]_1
    }
)
```

El CRS codifica el circuito específico. Cambiar el circuito = nuevo CRS.

PLONK (universal):

```
CRS = {
    // Powers of tau en G1
    [1]_1, []_1, [^2]_1, ..., [^n]_1,

    // Powers of tau en G2 (menos)
    [1]_2, []_2
}
```

El CRS solo depende del tamaño máximo n. Cualquier circuito de tamaño $\leq n$ puede usar este CRS.

STARKs (transparent):

```
// No hay CRS estructurado
// Solo parámetros públicos:
params = {
    field: F_p,
    hash: Poseidon/Keccak,
    expansion_factor: 8,
    num_queries: 30
}
```

Cualquiera puede verificar que estos son los parámetros correctos.

4.7.2 Proving Key vs Verification Key

Definición 4.21 (Keys en ZK Systems)**. Proving Key (pk):**

- Usado por el prover
- Típicamente grande (MB a GB)
- Contiene información para generar pruebas
- Secreto del prover? No, es público

Verification Key (vk):

- Usado por el verifier
- Pequeño (cientos de bytes a KB)
- Contiene información para verificar pruebas
- Público, puede ir on-chain

Ejemplo 4.12 (Tamaños típicos)**.** Para un circuito de 1 millón de constraints:

Sistema	Proving Key	Verification Key
Groth16	∼1 GB	∼1 KB
PLONK	∼500 MB	∼2 KB
STARK	N/A (transparent)	∼100 bytes

La proving key grande es un trade-off por pruebas pequeñas.

4.7.3 Derivación de CRS Específico de Circuito

Idea Clave

De CRS universal a CRS de circuito:
Con PLONK universal:

```
// CRS universal (de la ceremonia)
srs = {[^i]_1 para i = 0..N_max}

// Circuito específico
circuit = compile("mi_programa.circom")

// Derivar proving/verification keys
(pk, vk) = derive_keys(srs, circuit)

// pk y vk son específicos del circuito
// pero srs es reutilizable
```

Proceso de derivación:

1. Tomar el SRS universal
2. Computar polinomios que describen el circuito
3. "Evaluar" esos polinomios en el SRS
4. Resultado: keys específicas del circuito

No se necesita nueva ceremonia, solo cómputo local.

En Banco ZK

Setup en Banco ZK:
Nuestra elección: PLONK con KZG
Razones:

- Setup universal: un setup para todos nuestros circuitos
- Pruebas pequeñas (∼500 bytes)
- Verificación barata (∼200K gas en Ethereum)
- Podemos actualizar circuitos sin nueva ceremonia

SRS utilizado:
Usamos el SRS de la Ethereum KZG Ceremony:

- 141,000+ participantes
- Públicamente verificable
- Soporta circuitos hasta 2^{28} constraints

Keys derivadas:

```
// Circuitos de Banco ZK
circuits = {
    "transfer": circuito para transferencias privadas,
    "deposit": circuito para depósitos desde L1,
    "withdraw": circuito para retiros a L1,
    "batch": circuito para agregar múltiples txs
}

// Derivar keys para cada circuito
for circuit in circuits:
    (pk, vk) = derive_keys(ethereum_srs, circuit)

// vk va al contrato verificador en L1
// pk se distribuye a los provers
```

Consideración post-quantum:
KZG/PLONK no es post-quantum. Nuestro roadmap incluye:

- Corto plazo: PLONK+KZG (actual)
- Medio plazo: Evaluar Plonky3/STARK híbrido
- Largo plazo: Migrar a STARKs si amenaza cuántica se materializa

4.8 Sistemas de Prueba

Intuición

Un "sistema de prueba" es un conjunto de algoritmos que implementan las primitivas ZK:

- **Setup:** Generar parámetros
- **Prove:** Crear prueba dado witness
- **Verify:** Verificar prueba

Diferentes sistemas hacen diferentes trade-offs entre:

- Tamaño de prueba
- Tiempo de proving
- Tiempo de verificación
- Necesidad de trusted setup
- Seguridad post-quantum
- Asunciones criptográficas

4.8.1 SNARKs

Definición 4.22 (SNARK)**.** Succinct Non-interactive ARgument of Knowledge.

- **Succinct:** Prueba pequeña ($O(1)$), verificación rápida
- **Non-interactive:** Un mensaje
- **ARgument:** Seguridad computacional
- **of Knowledge:** Extractor existe

Matemáticas

Desglose del acrónimo:
S - Succinct (Sucinto):

- Tamaño de prueba: $O(1)$ o $O(\log n)$
- Tiempo de verificación: $O(1)$ o $O(\log n)$
- Independiente (o logarítmico) del tamaño del statement

N - Non-interactive (No interactivo):

- Una sola comunicación: prover $\rightarrow$ verifier
- No hay rondas de challenge-response
- Publicable, verificable offline

AR - ARgument (Argumento):

- Soundness computacional (no estadístico)
- Seguro contra adversarios de tiempo polinomial
- Permite pruebas más pequeñas que "proofs" estadísticos

K - of Knowledge (de Conocimiento):

- Si el prover convence, "conoce" un witness
- Existe extractor que puede obtener el witness
- Más fuerte que soundness básico

Observación 4.4. **Historia de los SNARKs:**

Año	Sistema	Contribución
2010	Groth	Primeros SNARKs eficientes
2013	Pinocchio	Primer SNARK práctico
2013	GGPR	Base teórica para muchos SNARKs
2016	Groth16	El más eficiente (aún usado)
2018	Sonic	Primer universal SNARK
2019	PLONK	Universal + custom gates
2019	Marlin	Universal con mejoras
2020	Halo	Sin trusted setup (IPA)
2022	Plonky2	PLONK + FRI, muy rápido

4.8.2 STARKs

Definición 4.23 (STARK)**.** Scalable Transparent ARgument of Knowledge.

- **Scalable:** Prover quasi-lineal, verifier logarítmico
- **Transparent:** Sin trusted setup
- Basado en hashes (post-quantum)
- Pruebas más grandes ($O(\log^2 n)$)

Matemáticas

Diferencias clave SNARK vs STARK:

Propiedad	SNARK	STARK
Basado en	Curvas elípticas	Hashes
Trusted setup	Generalmente sí	No
Post-quantum	No	Sí
Tamaño prueba	$O(1)$ ~100 B	$O(\log^2 n)$ ~100 KB
Verify time	$O(1)$ ms	$O(\log^2 n)$ ms
Prove time	$O(n \log n)$	$O(n \log^2 n)$
Asunciones	DLP, Pairings	Collision-resistant hash

Idea Clave

¿Cómo funcionan los STARKs?
Idea central: AIR (Algebraic Intermediate Representation)

1. Representar cómputo como tabla de "trace"
2. Cada fila es un estado
3. Constraints relacionan filas consecutivas
4. Interpolar trace como polinomios
5. Probar que polinomios satisfacen constraints
6. Usar FRI para probar bajo grado

Ejemplo simplificado:
Probar que calculé Fibonacci correctamente:

```
Trace (10 pasos):
| step | a  | b  |
|------|----|----|
| 0    | 1  | 1  |
| 1    | 1  | 2  |
| 2    | 2  | 3  |
| 3    | 3  | 5  |
| ...  | ...| ...|
```

```
| 9    | 34 | 55 |

Constraints:
- a_{i+1} = b_i
- b_{i+1} = a_i + b_i
- a_0 = 1, b_0 = 1

Estos constraints se convierten en ecuaciones polinomiales
que deben ser cero en todos los puntos del trace.
```

FRI (Fast Reed-Solomon IOP):

- Protocolo para probar que un polinomio tiene grado bajo
- Basado en códigos Reed-Solomon
- Solo usa hashes (no curvas)
- Pruebas logarítmicas

Observación 4.5. **¿Quién usa STARKs?**

- **StarkNet:** L2 de StarkWare
- **StarkEx:** Engine para dYdX, Immutable X
- **Polygon Miden:** VM basada en STARKs
- **Winterfell:** Librería de Facebook/Meta

Cairo es el lenguaje de StarkWare para escribir programas STARK-provable.

4.8.3 Bulletproofs

- Sin trusted setup
- Prueba logarítmica: $O(\log n)$
- Verificación lineal: $O(n)$
- Ideal para range proofs
- Usado en Monero

Definición 4.24 (Bulletproofs)**.** Sistema de prueba basado en Inner Product Arguments (IPA).

Componentes:

1. Pedersen commitments para comprometer valores
2. Inner product argument para probar relaciones

3. Range proofs como aplicación principal

Matemáticas

Inner Product Argument:
Probar que $\langle \mathbf{a}, \mathbf{b} \rangle = c$ dado:

- Commitment a $\mathbf{a}$: $A = \langle \mathbf{a}, \mathbf{G} \rangle$
- Commitment a $\mathbf{b}$: $B = \langle \mathbf{b}, \mathbf{H} \rangle$
- El producto interno c

Protocolo (simplificado):

1. Split $\mathbf{a}, \mathbf{b}$ en mitades: $\mathbf{a}_L, \mathbf{a}_R, \mathbf{b}_L, \mathbf{b}_R$
2. Calcular cross-terms L, R
3. Verifier envía challenge x
4. Fold: $\mathbf{a}' = \mathbf{a}_L + x \cdot \mathbf{a}_R$
5. Repetir hasta $n = 1$

Resultado:

- Prueba de tamaño $O(\log n)$: $2 \log n$ puntos de curva
- Pero verificación es $O(n)$: necesita recomputar los folds

Ejemplo 4.13 (Range Proof con Bulletproofs)**.** Probar que $v \in [0, 2^{64})$ sin revelar v:

```
// Commitment al valor
V = g^v * h^r  // Pedersen commitment

// Descomponer v en bits
v = sum_{i=0}^{63} v_i * 2^i

// Probar:
// 1. Cada v_i es 0 o 1 (constraints cuadráticos)
// 2. Los bits reconstruyen v
// 3. Todo en ZK

Tamaño de prueba: 64 bytes base + 2*log2(64) puntos
                 = 64 + 2*6*32 = ~448 bytes

(vs ~2KB para prueba naive bit por bit)
```

Observación 4.6. **Bulletproofs en Monero:**

Antes de Bulletproofs, Monero usaba range proofs que escalaban linealmente. Una transacción con múltiples outputs era muy grande.

Con Bulletproofs:

- Tamaño de tx reducido ∼80%
- Fees reducidos proporcionalmente
- Verificación sigue siendo $O(n)$ pero más eficiente

Bulletproofs+ (mejora): verificación batched más eficiente.

4.8.4 PLONK

- Setup universal (una ceremonia para todos los circuitos)
- Prueba: ∼400 bytes
- Custom gates y lookups
- Variantes: TurboPLONK, UltraPLONK, HyperPLONK

Definición 4.25 (PLONK)**.** Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge.

Innovaciones:

1. **Universal setup:** Un SRS para cualquier circuito
2. **Custom gates:** Más que solo multiplicación
3. **Permutation argument:** Para copy constraints
4. **Grand product:** Verificación eficiente

Matemáticas

Arithmetización PLONKish:
PLONK representa circuitos como sistema de ecuaciones sobre columnas:
Columnas básicas:

- a, b, c: valores de los wires
- q_L, q_R, q_O, q_M, q_C: selectores (constantes)

Gate equation:

$$q_L \cdot a + q_R \cdot b + q_O \cdot c + q_M \cdot a \cdot b + q_C = 0$$

Ejemplos:
Suma: $a + b = c$

$$q_L = 1, q_R = 1, q_O = -1, q_M = 0, q_C = 0$$

Multiplicación: $a \cdot b = c$

$$q_L = 0, q_R = 0, q_O = -1, q_M = 1, q_C = 0$$

Constante: $a = 5$

$$q_L = 1, q_R = 0, q_O = 0, q_M = 0, q_C = -5$$

Copy constraints:
Para conectar wires entre gates (ej: output de gate 1 = input de gate 2), PLONK usa un *permutation argument*.

Idea Clave

Variantes de PLONK:
Vanilla PLONK:

- Gate equation básica
- 5 selectores
- Suficiente para circuitos generales

TurboPLONK:

- Custom gates adicionales
- Más columnas de wires
- Más eficiente para operaciones específicas

UltraPLONK:

- + Lookup tables (Plookup)
- Muy útil para operaciones no-algebraicas (XOR, ranges)
- Usado en Aztec

HyperPLONK:

- Basado en polinomios multilineales
- Sumcheck protocol
- Potencialmente más eficiente

4.8.5 Groth16

- Prueba más pequeña: 128 bytes
- Verificación más rápida: 3 pairings

- Trusted setup por circuito
- Estándar en Ethereum (precompilados)

Definición 4.26 (Groth16)**.** El sistema de prueba más eficiente en términos de tamaño de prueba y verificación.

Estructura de la prueba:

$$\pi = ([A]_1, [B]_2, [C]_1)$$

Solo 3 elementos de grupo: 2 en $\mathbb{G}_1$, 1 en $\mathbb{G}_2$.

Verificación:

$$e(A, B) = e(\alpha, \beta) \cdot e(\sum_{i \in \text{pub}} x_i \cdot L_i, \gamma) \cdot e(C, \delta)$$

Solo 3 pairings (más uno precalculado).

Matemáticas

Tamaños exactos en Groth16:
Para curva BN254:

- $[A]_1$: 64 bytes (punto en $\mathbb{G}_1$)
- $[B]_2$: 128 bytes (punto en $\mathbb{G}_2$)
- $[C]_1$: 64 bytes (punto en $\mathbb{G}_1$)
- Total: 256 bytes sin comprimir
- Comprimido: 128 bytes (solo coordenada x + signo)

Para BLS12-381:

- $[A]_1$: 48 bytes comprimido
- $[B]_2$: 96 bytes comprimido
- $[C]_1$: 48 bytes comprimido
- Total: 192 bytes comprimido

Ejemplo 4.14 (Verificación Groth16 en Solidity)**.**

```
// Contrato verificador Groth16
contract Groth16Verifier {
    // Puntos del verification key
    uint256[2] alpha1;
    uint256[2][2] beta2;
    uint256[2][2] gamma2;
    uint256[2][2] delta2;
    uint256[2][] IC;  // Para public inputs
```

```
    function verify(
        uint256[2] memory a,    // A en G1
        uint256[2][2] memory b, // B en G2
        uint256[2] memory c,    // C en G1
        uint256[] memory input  // Public inputs
    ) public view returns (bool) {

        // Calcular vk_x = IC[0] + sum(input[i] * IC[i+1])
        uint256[2] memory vk_x = IC[0];
        for (uint i = 0; i < input.length; i++) {
            vk_x = addition(vk_x, scalar_mul(IC[i+1], input[i]));
        }

        // Verificar pairing equation
        // e(A, B) == e(alpha, beta) * e(vk_x, gamma) * e(C, delta)
        return pairing(
            negate(a), b,           // -A, B
            alpha1, beta2,          // alpha, beta
            vk_x, gamma2,           // vk_x, gamma
            c, delta2               // C, delta
        );
    }
}
```

Costo en gas: ∼200-250K gas.

4.8.6 Halo2

- PLONKish arithmetization
- Sin trusted setup (usando IPA)
- Custom gates, lookups
- Recursión eficiente
- Usado en Zcash Orchard, Scroll

Definición 4.27 (Halo2)**.** Sistema desarrollado por Electric Coin Company (Zcash) que combina:

1. Arithmetización PLONK
2. Polynomial commitment con IPA (sin trusted setup)
3. Técnica de “accumulation” para recursión

Idea Clave

Innovación principal: Recursión sin trusted setup

El problema:

Para recursión (probar una prueba dentro de otra), necesitas verificar un SNARK dentro de un circuito.

Verificar pairings en un circuito es muy caro (∼millones de constraints).

Solución Halo: Accumulation

En lugar de verificar completamente:

1. Acumula las verificaciones en un "acumulador"
2. Al final, una sola verificación para todas
3. Costo amortizado: casi gratis por prueba recursiva

Trade-off:

- Pro: recursión muy eficiente
- Pro: sin trusted setup
- Con: verificación final es $O(n)$ en lugar de $O(1)$
- Con: pruebas más grandes que Groth16

Observación 4.7. **Halo2 en producción:**

Zcash Orchard:

- Nuevo pool de shielded transactions
- Reemplaza a Sprout (Groth16)
- Sin trusted setup = más confianza

Scroll:

- zkEVM usando Halo2
- Custom gates para opcodes EVM
- Lookups para tablas de operaciones

4.8.7 Plonky2/Plonky3

- PLONK + FRI (de STARKs)
- Campos pequeños (Goldilocks: 64 bits)
- Muy rápido para recursión
- Usado en Polygon zkEVM

Definición 4.28 (Plonky2)**.** Sistema que combina lo mejor de SNARKs y STARKs:

- Arithmetización PLONKish (flexible)
- Polynomial commitment con FRI (transparent)
- Campo pequeño (64 bits) para eficiencia

Matemáticas

¿Por qué campos pequeños?
Campo Goldilocks:

$$p = 2^{64} - 2^{32} + 1$$

Ventajas:

- Operaciones en 64 bits: nativas en CPU
- Multiplicación: una instrucción
- NTT (FFT en campo finito): muy rápido

Comparación de velocidad:

Campo	Bits	Velocidad relativa
BN254	254	1x
BLS12-381	381	0.7x
Goldilocks	64	10-20x
Baby Bear	31	30-50x

Plonky2 usa Goldilocks, Plonky3 explora campos aún más pequeños.

Idea Clave

FRI en lugar de KZG:
KZG (usado en PLONK tradicional):

- Pruebas pequeñas (48 bytes)
- Verificación rápida (pairings)
- Requiere trusted setup
- No post-quantum

FRI (usado en Plonky2):

- Pruebas más grandes (∼KB)
- Verificación hash-based
- Sin trusted setup (transparent)

- Post-quantum seguro

Plonky2 = PLONK + FRI:
Obtiene:

- Flexibilidad de PLONK (custom gates, lookups)
- Transparencia de STARKs (no trusted setup)
- Velocidad de campos pequeños

4.8.8 Tabla Comparativa de Sistemas

Sistema	Prueba	Verify	Setup	Post-Q
Groth16	128 B	3 pairings	Trusted/circ	No
PLONK	400 B	1-2 pairings	Universal	No
STARK	50-200 KB	$O(\log^2 n)$	Ninguno	Sí
Bulletproof	$O(\log n)$	$O(n)$	Ninguno	No
Halo2	∼5 KB	$O(n)$	Ninguno	No
Plonky2	∼50 KB	$O(\log^2 n)$	Ninguno	Sí

Idea Clave

Guía para elegir sistema:
Elige Groth16 si:

- Necesitas pruebas mínimas
- Verificación on-chain es crítica
- Trusted setup es aceptable
- El circuito es estable (no cambia)

Elige PLONK si:

- Quieres flexibilidad (custom gates)
- Universal setup es importante
- Balancear tamaño y flexibilidad

Elige STARK si:

- No quieres trusted setup
- Necesitas post-quantum security
- El tamaño de prueba no es crítico
- Tienes mucho cómputo (escalabilidad)

Elige Bulletproofs si:

- Solo necesitas range proofs
- Verificación off-chain
- Sin trusted setup es prioritario

Elige Halo2/Plonky2 si:

- Necesitas recursión eficiente
- Sin trusted setup
- Dispuesto a trade-off en tamaño

En Banco ZK

Elección de sistema en Banco ZK:
Sistema principal: PLONK con KZG
Razones:

- Universal setup: podemos iterar circuitos rápido
- Custom gates: optimizar operaciones de curva elíptica
- Lookups: eficiente para range checks
- Pruebas razonablemente pequeñas (∼500 bytes)
- Verificación on-chain barata (∼200K gas)

Para recursión: Considerando Plonky2

- Agregar múltiples proofs de usuario
- Proof final KZG-based para L1
- Trade-off: prover más complejo

Futuro: Migración a STARKs/Plonky3

- Cuando amenaza cuántica sea realista
- Hardware STARK-proving mejorando
- L1 agregando precompilados STARK

4.9 Resumen del Bloque 4

Idea Clave

Puntos clave de Zero-Knowledge Proofs:

1. **Definición:**
   - Probar conocimiento sin revelar el conocimiento
   - Prover convence a Verifier de que $x \in L$
   - Sin revelar el witness w
2. **Tres propiedades fundamentales:**
   - Completeness: honesto siempre convence
   - Soundness: deshonesto no puede convencer
   - Zero-Knowledge: verifier no aprende nada extra
3. **Interactivo vs NIZK:**
   - Interactivo: múltiples rondas de comunicación
   - NIZK: un solo mensaje, requiere CRS o Random Oracle
   - Fiat-Shamir: transforma interactivo a no-interactivo
4. **Witness:**
   - Información secreta que satisface la relación
   - Lo que el prover conoce pero no revela
   - Witness generation: calcular todos los valores del circuito
5. **Prover vs Verifier:**
   - Prover: costoso (tiempo y memoria)
   - Verifier: barato (idealmente $O(1)$)
   - Asimetría es característica clave de SNARKs
6. **Setup:**
   - Trusted: ceremonia MPC, toxic waste
   - Transparent: parámetros públicos derivables
   - Trade-off: tamaño vs confianza
7. **Sistemas de prueba:**

- SNARKs: sucintos, trusted setup (Groth16, PLONK)
- STARKs: transparentes, post-quantum, pruebas grandes
- Bulletproofs: logarítmicos, verificación lineal
- Halo2/Plonky2: recursión eficiente, híbridos

8. **Tendencias:**
   - Hacia setups universales y transparentes
   - Recursión para escalabilidad
   - Hardware acelerado (GPUs, FPGAs, ASICs)
   - Campos pequeños para velocidad

Chapter 5

Circuitos ZK

Bloque 5 de 20

5.1 Qué es un Circuito Aritmético

Definición 5.1 (Circuito Aritmético)**.** DAG (grafo dirigido acíclico) donde:

- Nodos de entrada: variables o constantes
- Nodos internos: compuertas de suma o multiplicación
- Nodos de salida: resultados a verificar

Operaciones sobre campo finito $\mathbb{F}_p$.

5.1.1 Contexto e Intuición

Intuición

Un circuito aritmético es como una receta de cocina matemática:

- **Ingredientes** (inputs): los valores que entran
- **Pasos** (gates): operaciones de suma y multiplicación
- **Resultado** (output): lo que sale al final

La diferencia con un programa normal es que todo se expresa como sumas y multiplicaciones sobre un campo finito. No hay if/else, loops, ni operaciones de bits directas—todo debe traducirse a aritmética.

¿Por qué campos finitos?

Los sistemas ZK (SNARKs, STARKs) están diseñados para trabajar sobre campos finitos porque:

- Los polinomios sobre campos finitos tienen propiedades algebraicas útiles
- Permiten codificar constraints de forma eficiente
- Los compromisos criptográficos (KZG, Pedersen) trabajan sobre grupos finitos

Definición 5.2 (Campo Finito $\mathbb{F}_p$)**.** Un campo finito de orden primo p es el conjunto $\{0, 1, 2, \ldots, p-1\}$ con:

- Suma: $(a + b) \mod p$
- Multiplicación: $(a \cdot b) \mod p$
- Inverso aditivo: $-a = p - a$
- Inverso multiplicativo: a^{-1} tal que $a \cdot a^{-1} \equiv 1 \pmod{p}$

Campos comunes en ZK:

Curva	Campo escalar	Bits
BN254	$p = 21888...$	254
BLS12-381	$p = 52435...$	255
Pasta (Pallas)	$p = 28948...$	255
Goldilocks	$p = 2^{64} - 2^{32} + 1$	64

5.1.2 Estructura de un Circuito

Matemáticas

Componentes formales:

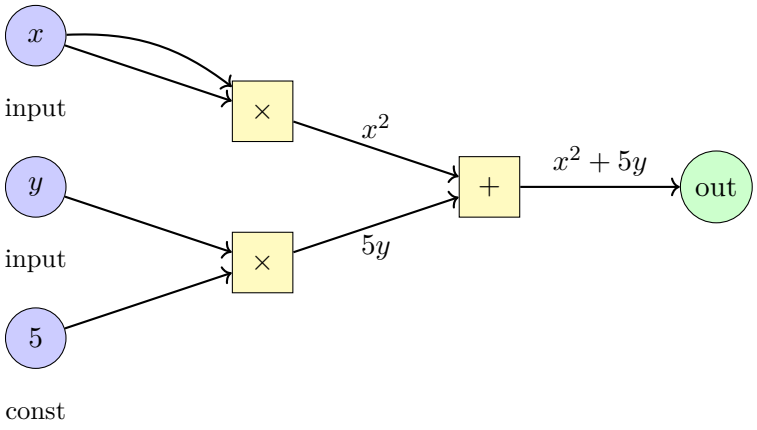
Un circuito aritmético C sobre campo $\mathbb{F}$ consiste en:

1. **Nodos de entrada:** $x_1, \ldots, x_n$ (inputs públicos y privados)
2. **Constantes:** $c_1, \ldots, c_k \in \mathbb{F}$

3. **Gates:** Cada gate g tiene:
   - Tipo: ADD o MUL
   - Dos inputs: wires de entrada
   - Un output: wire de salida
4. **Wires:** Conexiones entre gates
5. **Output:** Wire(s) designado(s) como salida

Evaluación:
El circuito define una función $f : \mathbb{F}^n \to \mathbb{F}^m$:

- Propagar valores desde inputs hacia outputs
- En cada gate, aplicar la operación correspondiente
- El output es el resultado de los wires de salida



Ejemplo 5.1 (Circuito para $f(x, y) = x^2 + 5y$)**. Inputs:**

- x: variable de entrada
- y: variable de entrada
- 5: constante

Gates:

1. $w_1 = x \times x$ (multiplicación: x^2)
2. $w_2 = 5 \times y$ (multiplicación: $5y$)
3. $w_3 = w_1 + w_2$ (suma: $x^2 + 5y$)

Output: w_3
Número de gates: 2 multiplicaciones + 1 suma = 3 gates
En R1CS, solo las multiplicaciones generan constraints, así que este circuito tiene 2 constraints.

5.1.3 De Programas a Circuitos

Idea Clave

El desafío de la compilación a circuitos:
Los programas normales tienen:

- Condicionales (if/else)
- Loops (for, while)
- Operaciones de bits (AND, OR, XOR, shifts)
- Comparaciones ($<$, $>$, $=$)
- División y módulo

Pero los circuitos aritméticos solo tienen:

- Suma
- Multiplicación

Solución: Expresar todo como aritmética de campo.

Ejemplo 5.2 (Transformaciones comunes)**. 1. Booleano (0 o 1):**

Forzar que $b \in \{0, 1\}$:

$$b \times (1 - b) = 0$$

Si $b = 0$: $0 \times 1 = 0$ Si $b = 1$: $1 \times 0 = 0$ Si $b = 2$: $2 \times (-1) \neq 0$

2. Selección (if/else):

out $= b?x : y$ donde $b \in \{0, 1\}$:

$$\text{out} = b \cdot x + (1 - b) \cdot y$$

Si $b = 1$: out $= 1 \cdot x + 0 \cdot y = x$ Si $b = 0$: out $= 0 \cdot x + 1 \cdot y = y$

3. Igualdad ($x = y$):

Probar que $x = y$:

$$x - y = 0$$

4. No-igualdad ($x \neq y$):

Probar que $x \neq y$ requiere un witness auxiliar inv:

$$inv = (x - y)^{-1}$$

$$(x - y) \times inv = 1$$

Si $x = y$, no existe inv que satisfaga esto.

5. Descomposición en bits:

Para probar que x tiene representación binaria $b_0, b_1, \ldots, b_{n-1}$:

$$x = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

$$\forall i : b_i \times (1 - b_i) = 0$$

Esto cuesta n constraints (uno por cada bit).

Advertencia

Operaciones costosas en circuitos:

Operación	Constraints	Razón
Suma	0	Combinación lineal
Multiplicación	1	Gate básico
División	1	Verificar $a \cdot b^{-1} = c$
Comparación ($<$)	$\sim n$	Descomposición en bits
Hash SHA-256	$\sim 25,000$	Muchas ops de bits
Hash Poseidon	~ 250	Diseñado para circuitos
ECDSA verify	$\sim 500,000$	Aritmética de curva

Los hashes como SHA-256 son muy caros porque usan XOR, rotaciones, y operaciones de bits que no son nativas en aritmética de campo.

5.2 R1CS (Rank-1 Constraint System)

Definición 5.3 (R1CS)**.** Sistema de m ecuaciones sobre n variables:

$$(A_i \cdot \vec{w}) \times (B_i \cdot \vec{w}) = C_i \cdot \vec{w} \quad \forall i \in [m]$$

donde A_i, B_i, C_i son vectores de coeficientes y $\vec{w}$ es el witness.

Forma matricial:

$$(A \cdot \vec{w}) \circ (B \cdot \vec{w}) = C \cdot \vec{w}$$

donde $\circ$ es producto Hadamard (elemento a elemento).

5.2.1 Entendiendo R1CS

Intuición

R1CS es como un sistema de ecuaciones especial:

- Cada ecuación tiene la forma: (algo) $\times$ (algo) = (algo)
- Los "algos" son combinaciones lineales de las variables
- Una multiplicación por ecuación (por eso "Rank-1")

Es la representación intermedia más común entre un programa y un SNARK.
Flujo típico:

```
Programa → Circuito → R1CS → QAP → SNARK
     (Circom)        (compilador)   (Groth16/PLONK)
```

Matemáticas

Estructura detallada:
Witness vector:

$$\vec{w} = (1, x_1, x_2, \ldots, x_n, w_1, w_2, \ldots, w_k)$$

Donde:

- 1: constante (siempre el primer elemento)
- x_i: inputs públicos y privados
- w_j: variables intermedias (wires internos)

Matrices A, B, C:
Cada matriz tiene m filas (una por constraint) y $n + k + 1$ columnas (una por variable).
Constraint i:

$$\underbrace{(a_{i,0} + a_{i,1}x_1 + \ldots + a_{i,n}w_k)}_{\text{lado izquierdo}} \times \underbrace{(b_{i,0} + b_{i,1}x_1 + \ldots)}_{\text{lado derecho}} = \underbrace{(c_{i,0} + c_{i,1}x_1 + \ldots)}_{\text{resultado}}$$

5.2.2 Ejemplo Completo de R1CS

Ejemplo 5.3 (R1CS para $x^3 + x + 5 = 35$)**. Paso 1: Descomponer en multiplicaciones**

La expresión $x^3 + x + 5$ requiere calcular x^3:

$$\begin{aligned} w_1 &= x \times x \quad (\text{compute } x^2) \\ w_2 &= w_1 \times x \quad (\text{compute } x^3) \\ w_3 &= w_2 + x + 5 \quad (\text{compute } x^3 + x + 5) \end{aligned}$$

Y verificar que $w_3 = 35$.

Paso 2: Convertir a constraints R1CS

Witness vector: $\vec{w} = (1, x, w_1, w_2, w_3, \text{out})$

Con índices: $w_0 = 1$, $w_1 = x$, $w_2 = x^2$, $w_3 = x^3$, $w_4 = x^3 + x + 5$, $w_5 = 35$

Constraint 1: $x \times x = w_1$

$$A_1 = (0, 1, 0, 0, 0, 0) \quad \text{(selecciona } x\text{)}$$
$$B_1 = (0, 1, 0, 0, 0, 0) \quad \text{(selecciona } x\text{)}$$
$$C_1 = (0, 0, 1, 0, 0, 0) \quad \text{(selecciona } w_1\text{)}$$

Verificación: $(0 + 1 \cdot x) \times (0 + 1 \cdot x) = (0 + 0 + 1 \cdot w_1) \Rightarrow x \times x = w_1$

Constraint 2: $w_1 \times x = w_2$

$$A_2 = (0, 0, 1, 0, 0, 0)$$
$$B_2 = (0, 1, 0, 0, 0, 0)$$
$$C_2 = (0, 0, 0, 1, 0, 0)$$

Constraint 3: $1 \times (w_2 + x + 5) = w_3$

$$A_3 = (1, 0, 0, 0, 0, 0) \quad \text{(selecciona 1)}$$
$$B_3 = (5, 1, 0, 1, 0, 0) \quad \text{(selecciona } 5 + x + w_2\text{)}$$
$$C_3 = (0, 0, 0, 0, 1, 0)$$

Constraint 4: $1 \times w_3 = 35$

$$A_4 = (1, 0, 0, 0, 0, 0)$$
$$B_4 = (0, 0, 0, 0, 1, 0)$$
$$C_4 = (35, 0, 0, 0, 0, 0)$$

Paso 3: Verificar con $x = 3$

$\vec{w} = (1, 3, 9, 27, 35, 35)$

- Constraint 1: $3 \times 3 = 9$
- Constraint 2: $9 \times 3 = 27$
- Constraint 3: $1 \times (27 + 3 + 5) = 35$
- Constraint 4: $1 \times 35 = 35$

5.2.3 Matrices R1CS

Matemáticas

Visualización matricial del ejemplo:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad B = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 5 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \quad C = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 35 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Verificación:

$$(A \cdot \vec{w}) \circ (B \cdot \vec{w}) = \begin{pmatrix} 3 \\ 9 \\ 1 \\ 1 \end{pmatrix} \circ \begin{pmatrix} 3 \\ 3 \\ 35 \\ 35 \end{pmatrix} = \begin{pmatrix} 9 \\ 27 \\ 35 \\ 35 \end{pmatrix} = C \cdot \vec{w}$$

5.3 Constraints

Definición 5.4 (Constraint)**.** Ecuación que el witness debe satisfacer.

Forma general R1CS:

$$(\sum_j a_j w_j) \times (\sum_k b_k w_k) = \sum_l c_l w_l$$

Idea Clave

En R1CS:

- Sumas son "gratis" (combinaciones lineales)
- Solo multiplicaciones generan constraints
- Número de constraints = tamaño del circuito

5.3.1 Costos de Operaciones

Operación	Constraints
Suma/resta	0
Multiplicación	1
División	1 (verificar $a \times b^{-1} = c$)
Comparación ($<$)	$\sim n$ bits
Binario check	1 ($b(1-b) = 0$)
Hash Poseidon	~ 250
Hash SHA-256	$\sim 25,000$
ECDSA verify	$\sim 500,000$

5.3.2 Constraints Comunes en Detalle

Ejemplo 5.4 (IsZero: verificar si $x = 0$)**.** Queremos $out = 1$ si $x = 0$, y $out = 0$ si $x \neq 0$.

Estrategia: Usar inverso como witness auxiliar.

Constraints:

$$\text{inv} \leftarrow \begin{cases} 0 & \text{si } x = 0 \\ x^{-1} & \text{si } x \neq 0 \end{cases}$$

$$\text{out} = 1 - x \cdot \text{inv}$$

$$x \cdot \text{out} = 0$$

Análisis:

- Si $x = 0$: $\text{out} = 1 - 0 = 1$, y $0 \cdot 1 = 0$
- Si $x \neq 0$: $\text{inv} = x^{-1}$, $\text{out} = 1 - 1 = 0$, y $x \cdot 0 = 0$
- Si $x \neq 0$ pero intenta $\text{out} = 1$: $x \cdot 1 = x \neq 0$

Costo: 2 constraints.

Ejemplo 5.5 (LessThan: verificar $a < b$)**.** Para comparar en un campo finito, necesitamos descomponer en bits.

Estrategia:

1. Calcular $\text{diff} = b - a + 2^n$ (asegurar positivo si $a < b$)
2. Descomponer diff en $n + 1$ bits
3. El bit más significativo indica si $a < b$

Para n-bit numbers:

```
// Pseudo-código
diff = b - a + 2^n
bits = decompose(diff, n+1)
// Si a < b: diff está en [2^n, 2^(n+1)), bit[n] = 1
// Si a >= b: diff está en [0, 2^n), bit[n] = 0

// Constraints:
for i in 0..n+1:
    bits[i] * (1 - bits[i]) = 0  // Es binario

sum(bits[i] * 2^i) = diff        // Reconstrucción

out = bits[n]                     // MSB indica resultado
```

Costo: $n + 2$ constraints (uno por bit + reconstrucción).

Ejemplo 5.6 (Range check: $x \in [0, 2^n)$)**.** Verificar que x está en un rango.

Método 1: Descomposición en bits

```
bits = decompose(x, n)
for i in 0..n:
    bits[i] * (1 - bits[i]) = 0  // Cada bit es 0 o 1
sum(bits[i] * 2^i) = x           // Reconstrucción
```

Costo: n constraints.

Método 2: Lookup tables (PLONKish)

En sistemas como PLONK con Plookup:

- Tabla precalculada: $\{0, 1, 2, \ldots, 2^{16} - 1\}$
- Verificar: x está en la tabla
- Costo: $\sim$1 constraint (amortizado)

Mucho más eficiente para rangos grandes.

5.3.3 Optimización de Constraints

Idea Clave

Técnicas para reducir constraints:

1. Agrupar operaciones lineales:

```
// Malo: 2 multiplicaciones
w1 = a * b
w2 = w1 * c

// Equivalente pero unavoidable
// Solo podemos multiplicar 2 cosas a la vez
```

2. Usar constraints de "igualdad gratuita":

```
// Malo: copiar valor con constraint
w2 = w1 * 1  // 1 constraint

// Bueno: usar el mismo wire
// No hay constraint, solo referencia
```

3. Reutilizar cálculos:

```
// Malo: calcular x^2 dos veces
y = x * x
z = x * x + 1

// Bueno: calcular una vez
w = x * x    // 1 constraint
y = w
z = w + 1   // 0 constraints (suma)
```

4. Elegir representaciones eficientes:

```
// Para booleanos, usar 0/1 en lugar de true/false
// Para enums, usar índices numéricos
// Para hashes, usar Poseidon en lugar de SHA-256
```

En Banco ZK

Constraints en Banco ZK:
Circuito de transferencia privada:

Componente	Constraints aprox.
Verificar Merkle path (depth 20)	5,000
Hash del nullifier (Poseidon)	250
Hash del nuevo commitment (Poseidon)	250
Verificar firma (EdDSA)	3,000
Range check del monto	64
Verificar balance suficiente	100
Total aproximado	∼9,000

Con un prover optimizado, generar prueba para 9,000 constraints toma ∼1-2 segundos.
Por qué usamos Poseidon:

- SHA-256: 25,000 constraints por hash
- Poseidon: 250 constraints por hash
- Ahorro: 100x menos constraints
- Un Merkle tree de depth 20 con SHA-256 sería ∼500,000 constraints

5.4 Señales (Signals)

Definición 5.5 (Signal)**.** Variable en un circuito, contiene elemento de $\mathbb{F}_p$.

Tipos:

- **input public:** Visible al verifier
- **input private:** Solo conocido por prover
- **output:** Resultado del circuito
- **intermediate:** Variables internas

5.4.1 Anatomía de las Señales

Intuición

Las señales son los "cables" del circuito:

- Transportan valores entre gates
- Cada señal tiene un valor en $\mathbb{F}_p$
- Las señales conectan inputs, gates, y outputs

En el contexto de ZK:

- **Públicas:** El verifier las conoce (parte del statement)
- **Privadas:** Solo el prover las conoce (parte del witness)

Matemáticas

Clasificación formal de señales:

1. **Public Inputs (x):**
   - Conocidos por prover Y verifier
   - Parte del "statement" que se prueba
   - Ejemplo: el hash resultante, el root del Merkle tree
2. **Private Inputs (witness w):**
   - Conocidos SOLO por el prover
   - Lo que queremos mantener secreto
   - Ejemplo: la preimagen del hash, la clave privada
3. **Intermediate Signals:**
   - Calculados durante la ejecución
   - Parte del witness extendido
   - Ejemplo: resultados de multiplicaciones intermedias
4. **Output Signals:**
   - Resultados finales del circuito
   - Pueden ser públicos o privados según el diseño

Witness completo:

$$\vec{w} = (1, x_1, \ldots, x_l, w_1, \ldots, w_m)$$

donde x_i son public inputs y w_j son todas las señales privadas + intermedias.

Ejemplo 5.7 (Señales en un circuito de hash)**.** `Circuito: Probar conocimiento de preimagen`

```
Public inputs:
    - hash_output: H(x) = y (conocido públicamente)

Private inputs:
    - preimage: x (secreto del prover)

Intermediate signals:
    - Todos los estados internos del hash
    - Variables temporales de las rondas

Verificación:
    - El circuito calcula H(preimage)
    - Constraint: H(preimage) === hash_output
```

El verifier solo ve `hash_output`. No aprende nada sobre `preimage`.

5.4.2 Señales en Diferentes Frameworks

Concepto	Circom	Halo2	Noir
Input público	`signal input`	`Instance`	`pub param`
Input privado	`signal private input`	`Advice`	`param`
Output	`signal output`	`Advice`	return value
Intermedio	`signal`	`Advice`	`let`
Constante	literal/`var`	`Fixed`	`const`

5.5 Witness Generation

Definición 5.6 (Witness)**.** Asignación de valores a todas las señales que satisface todos los constraints.

Proceso:

1. Input: valores de entrada (JSON)
2. Ejecutar lógica del circuito
3. Calcular todas las señales intermedias
4. Output: archivo .wtns con todos los valores

5.5.1 El Proceso de Witness Generation

Idea Clave

Witness generation es "ejecutar" el circuito:

```
Input (JSON):
{
    "a": "3",
    "b": "5"
}

Circuito:
    signal input a;
    signal input b;
    signal c;
    signal output d;

    c <== a * b;      // c = 15
    d <== c + a;      // d = 18

Output (witness):
    w = [1, 3, 5, 15, 18]
    //   1  a  b  c   d
```

El witness generator:

1. Lee los inputs
2. Propaga valores a través del circuito
3. Evalúa cada constraint/asignación
4. Produce el vector completo de señales

5.5.2 Witness vs Constraints

Matemáticas

Dos fases separadas:

1. Witness Generation (ejecución):

- Calcula valores de todas las señales
- Usa código imperativo (asignaciones `<-`)
- Puede usar if/else, loops, funciones auxiliares
- Output: vector de valores

2. Constraint Checking (verificación):

- Verifica que el witness satisface todos los constraints
- Solo usa ecuaciones algebraicas (`===`)
- Sin control de flujo
- Output: válido/inválido

Importante: El prover debe generar un witness que satisfaga TODOS los constraints. El verifier no ejecuta el witness generation—solo verifica la prueba.

Advertencia

Error común: witness generation vs constraints

```
// INCORRECTO: constraint sin asignación
signal x;
x === a * b;  // Error: x no tiene valor asignado

// CORRECTO: asignación + constraint
signal x;
x <== a * b;  // Asigna Y crea constraint

// TAMBIÉN CORRECTO: separado
signal x;
x <-- a * b;  // Solo asignación
x === a * b;  // Solo constraint
```

El operador `<==` es azúcar sintáctica para `<-` seguido de `===`.

5.5.3 Formato del Witness

Ejemplo 5.8 (Archivo witness (.wtns))**.** El formato .wtns es binario, pero conceptualmente:

```
// Header
version: 2
sections: [
    {type: "header", field: "bn128", n_vars: 5},
    {type: "data", values: [...]}
]

// Data section (little-endian field elements)
w[0] = 1                    // Siempre 1
w[1] = 3                    // Primera variable
```

```
w[2] = 5                    // Segunda variable
w[3] = 15                   // Señal intermedia
w[4] = 18                   // Output

// Cada valor es un elemento de F_p (32 bytes para BN254)
```

5.6 Circom

Definición 5.7 (Circom)**.** Lenguaje de dominio específico (DSL) para escribir circuitos ZK.

5.6.1 Historia y Contexto

Observación 5.1. **Historia de Circom:**

Año	Versión	Desarrollo
2018	Circom 1.0	Creado por iden3/Jordi Baylina
2020	Circom 1.x	Mejoras de sintaxis
2021	Circom 2.0	Reescrito en Rust, más rápido
2022	Circom 2.1	Custom templates, mejoras
2023	Actual	Estándar de facto para ZK

Circom es el lenguaje más usado para circuitos ZK en el ecosistema Ethereum. Proyectos que usan Circom:

- Tornado Cash (mixer privado)
- Semaphore (identidad anónima)
- Hermez/Polygon zkEVM
- Dark Forest (juego ZK)
- WorldCoin

5.6.2 Sintaxis Básica

```
pragma circom 2.0.0;

template Multiplicador() {
    signal input a;
    signal input b;
    signal output c;

    c <== a * b;
}

component main = Multiplicador();
```

Matemáticas

Estructura de un archivo Circom:

```
// 1. Pragma (versión del compilador)
pragma circom 2.0.0;

// 2. Includes (bibliotecas)
include "circomlib/poseidon.circom";

// 3. Templates (definiciones de circuitos)
template MiCircuito(param1, param2) {
    // Declaraciones de señales
    signal input x;
    signal output y;

    // Lógica y constraints
    y <== x * x;
}

// 4. Componente main
component main {public [x]} = MiCircuito(10, 20);
```

El main especifica:

- Qué template es el punto de entrada
- Qué inputs son públicos (`public [...]`)
- Parámetros del template

5.6.3 Operadores

- `<-`: Asignación sin constraint
- `->`: Asignación inversa sin constraint
- `<==`: Asignación CON constraint (lo más usado)
- `==>`: Asignación inversa con constraint
- `===`: Solo constraint, sin asignación

Idea Clave

Entendiendo los operadores de Circom:
`<-` (asignación sin constraint):

```
// Solo calcula el valor, NO verifica nada
```

```
x <-- y / z;  // División (no es operación de campo nativa)
x <-- cond ? a : b;  // Condicional
```

Útil cuando la operación no es directamente expresable como constraint.

<== (asignación + constraint):

```
// Calcula Y verifica
x <== a * b;  // Asigna x = a*b, añade constraint x === a*b
x <== a + b;  // Asigna x = a+b, añade constraint x === a+b
```

Usar siempre que sea posible—es más seguro.

=== (solo constraint):

```
// Solo verifica, no asigna
a * b === c;  // Verifica que a*b = c
x === 5;      // Verifica que x = 5
```

Usar después de <- para completar la verificación.

Patrón común:

```
// Para división: z = x / y
z <-- x / y;      // Calcular (no es constraint válido)
z * y === x;      // Verificar que z * y = x
```

Advertencia

Peligro: <- sin constraint

```
// VULNERABLE: el prover puede poner cualquier valor
signal output result;
result <-- computeSecretValue();  // Sin constraint!

// Un prover malicioso puede:
result <-- 999999;  // Valor arbitrario, nadie lo verifica

// CORRECTO: siempre añadir constraint
result <-- computeSecretValue();
result === expectedValue;  // O algún constraint que lo ate
```

Regla: Todo <- debe tener un === correspondiente que lo verifique.

5.6.4 Templates

Definición 5.8 (Template)**.** Función que genera porción de circuito, reutilizable y parametrizable.

```
template IsZero() {
```

```
    signal input in;
    signal output out;

    signal inv;
    inv <-- in != 0 ? 1/in : 0;
    out <== -in * inv + 1;
    in * out === 0;
}
```

Matemáticas

Análisis del template IsZero:
Objetivo: $out = 1$ si $in = 0$, $out = 0$ si $in \neq 0$.
Línea por línea:

`inv <-- in != 0 ? 1/in : 0;`

Calcula el inverso si existe. Si $in = 0$, pone $inv = 0$.

`out <== -in * inv + 1;`

Si $in \neq 0$: $out = -in \cdot (1/in) + 1 = -1 + 1 = 0$ Si $in = 0$: $out = 0 \cdot 0 + 1 = 1$

`in * out === 0;`

Garantiza que no puedes tener $in \neq 0$ Y $out \neq 0$.
Constraints: 2 (uno de `<==`, uno de `===`)

Ejemplo 5.9 (Templates con parámetros)**.**

```
// Template parametrizado para N bits
template Bits2Num(n) {
    signal input in[n];
    signal output out;

    var lc = 0;
    var e2 = 1;

    for (var i = 0; i < n; i++) {
        lc += in[i] * e2;
        e2 = e2 * 2;
    }

    out <== lc;
}

// Uso: convertir 8 bits a número
component b2n = Bits2Num(8);
```

```
b2n.in <== [1, 0, 1, 0, 0, 0, 0, 0];  // 5 en binario
// b2n.out será 5
```

5.6.5 Components

Definición 5.9 (Component)**.** Instancia de un template dentro de otro circuito.

```
template Main() {
    signal input x;
    signal output isZ;

    component iz = IsZero();
    iz.in <== x;
    isZ <== iz.out;
}
```

Idea Clave

Composición de circuitos:

Los components permiten construir circuitos complejos a partir de bloques simples:

```
template ComplexCircuit() {
    signal input a, b, c;
    signal output result;

    // Usar múltiples components
    component mul = Multiplier();
    component add = Adder();
    component isz = IsZero();

    // Conectar
    mul.a <== a;
    mul.b <== b;

    add.a <== mul.out;
    add.b <== c;

    isz.in <== add.out;

    result <== isz.out;
}
```

Cada component añade sus propios constraints al circuito total.

5.6.6 Arrays y Loops

Ejemplo 5.10 (Trabajando con arrays)**.**

```
template SumArray(n) {
    signal input in[n];
    signal output out;

    // Variables auxiliares (no son señales)
    var sum = 0;

    // Loops se desenrollan en compile-time
    for (var i = 0; i < n; i++) {
        sum += in[i];
    }

    out <== sum;
}

template DotProduct(n) {
    signal input a[n];
    signal input b[n];
    signal output out;

    signal products[n];
    var sum = 0;

    for (var i = 0; i < n; i++) {
        products[i] <== a[i] * b[i];  // n constraints
        sum += products[i];
    }

    out <== sum;  // 1 constraint
}
```

5.6.7 Circomlib: Biblioteca Estándar

Observación 5.2. **Circomlib es la biblioteca estándar de circuitos:**

Categoría	Templates
Comparadores	IsZero, IsEqual, LessThan, GreaterThan
Aritmética	Multiplier, Sum, Division
Bits	Num2Bits, Bits2Num, AND, OR, XOR
Hashes	Poseidon, MiMC, Pedersen
Merkle trees	MerkleTreeChecker, MerkleTreeInclusionProof
Firmas	EdDSA, ECDSA
Otros	Mux, Decoder, Switcher

Instalación:

```
npm install circomlib
```

Uso:

```
include "circomlib/circuits/poseidon.circom";
include "circomlib/circuits/comparators.circom";
```

Ejemplo 5.11 (Verificación de Merkle proof con Circomlib). `pragma circom 2.0.0;`

```
include "circomlib/circuits/poseidon.circom";
include "circomlib/circuits/mux1.circom";

template MerkleProof(levels) {
    signal input leaf;
    signal input pathElements[levels];
    signal input pathIndices[levels];  // 0 = izq, 1 = der
    signal output root;

    signal hashes[levels + 1];
    hashes[0] <== leaf;

    component hashers[levels];
    component mux[levels];

    for (var i = 0; i < levels; i++) {
        // Seleccionar orden según pathIndex
        mux[i] = MultiMux1(2);
        mux[i].c[0][0] <== hashes[i];
        mux[i].c[0][1] <== pathElements[i];
        mux[i].c[1][0] <== pathElements[i];
        mux[i].c[1][1] <== hashes[i];
        mux[i].s <== pathIndices[i];
```

```
        // Hash de los dos hijos
        hashers[i] = Poseidon(2);
        hashers[i].inputs[0] <== mux[i].out[0];
        hashers[i].inputs[1] <== mux[i].out[1];

        hashes[i + 1] <== hashers[i].out;
    }

    root <== hashes[levels];
}
```

En Banco ZK

Circom en Banco ZK:

Estructura de nuestros circuitos:

```
banco-zk-circuits/
 src/
    transfer.circom      // Transferencia privada
    deposit.circom       // Depósito desde L1
    withdraw.circom      // Retiro a L1
    batch.circom         // Agregación de proofs
 lib/
    merkle.circom        // Verificación Merkle
    nullifier.circom     // Generación de nullifiers
    commitment.circom    // Creación de commitments
 test/
    *.test.js
 circomlib/               // Dependencia
```

Template principal de transferencia:

```
template Transfer(levels) {
    // Inputs públicos
    signal input root;              // Merkle root actual
    signal input nullifierHash;     // Nullifier de input
    signal input newCommitment;     // Commitment de output

    // Inputs privados (witness)
    signal private input secret;    // Clave del usuario
    signal private input amount;    // Monto
    signal private input pathElements[levels];
    signal private input pathIndices[levels];

    // Verificaciones
```

```
    component merkle = MerkleProof(levels);
    component nullGen = NullifierGenerator();
    component commGen = CommitmentGenerator();

    // ... constraints ...
}
```

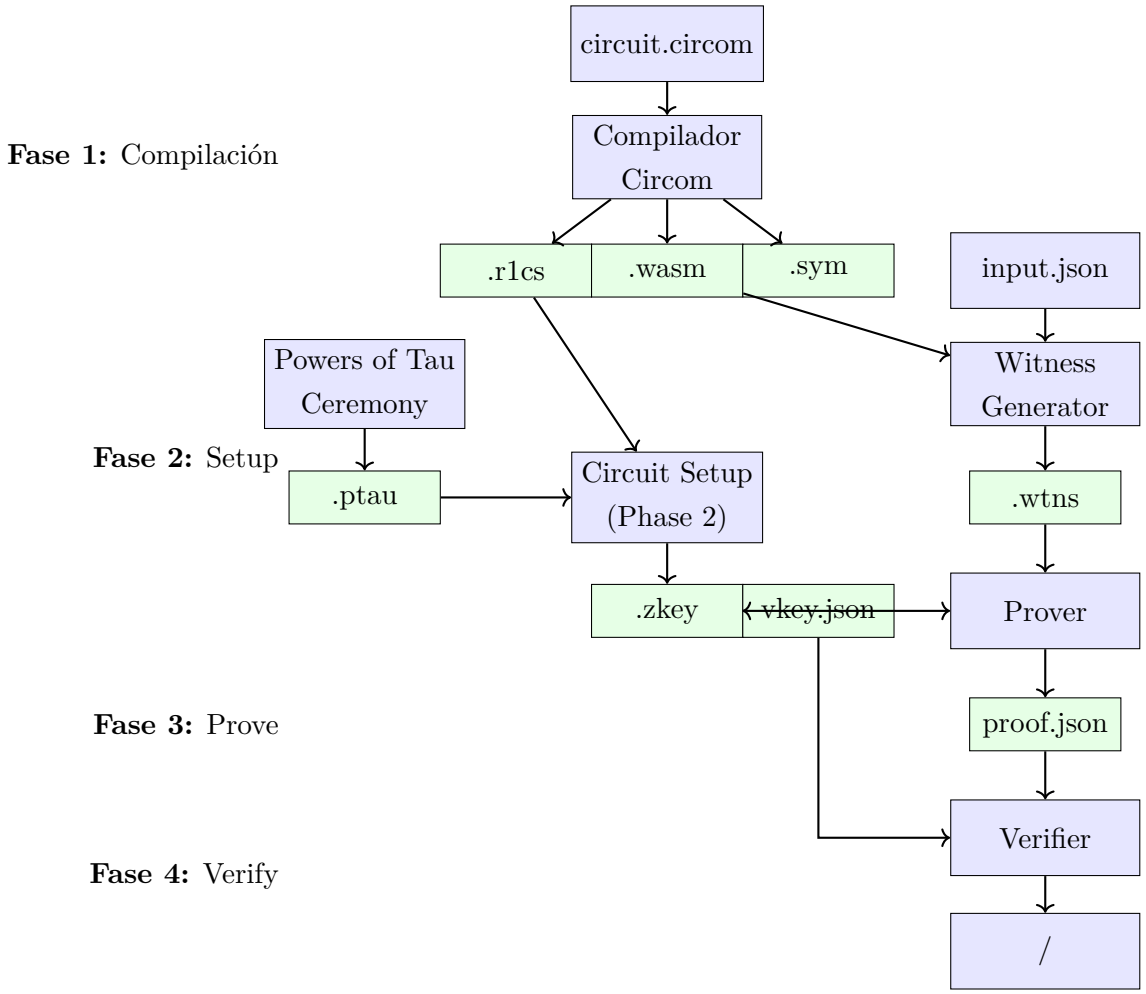
5.7 snarkjs

Herramienta JavaScript para el flujo completo:

Definición 5.10 (snarkjs)**.** Biblioteca JavaScript/WASM que implementa el flujo completo de SNARKs:

- Setup de ceremonias (Powers of Tau)
- Generación de claves (proving key, verification key)
- Generación de pruebas
- Verificación de pruebas
- Exportación a Solidity

5.7.1 Visión General del Flujo



5.7.2 Setup

```
# Compilar circuito
circom circuit.circom --r1cs --wasm --sym

# Powers of Tau (fase 1)
snarkjs powersoftau new bn128 12 pot12_0.ptau
snarkjs powersoftau contribute pot12_0.ptau pot12_1.ptau
snarkjs powersoftau prepare phase2 pot12_1.ptau pot12_final.ptau

# Setup del circuito (fase 2)
snarkjs groth16 setup circuit.r1cs pot12_final.ptau circuit.zkey
```

Paso 1: Compilar el Circuito

Matemáticas

Comando de compilación:

```
circom circuit.circom --r1cs --wasm --sym --c --O2
```

Flags:

- `-r1cs`: Genera el sistema R1CS (.r1cs)
- `-wasm`: Genera el witness generator en WebAssembly
- `-sym`: Genera archivo de símbolos (debug)
- `-c`: También genera código C++ (más rápido)
- `-O0/O1/O2`: Nivel de optimización

Archivos generados:

```
circuit.r1cs          # Sistema de constraints
circuit_js/
 circuit.wasm      # Witness generator (WASM)
 generate_witness.js
 witness_calculator.js
circuit.sym           # Mapeo de símbolos
circuit_cpp/          # (si usaste --c)
 ...
```

Ejemplo 5.12 (Información del R1CS)**.** `$ snarkjs r1cs info circuit.r1cs`

```
[INFO]  snarkjs: Curve: bn-128
[INFO]  snarkjs: # of Wires: 1234
[INFO]  snarkjs: # of Constraints: 1000
[INFO]  snarkjs: # of Private Inputs: 5
[INFO]  snarkjs: # of Public Inputs: 3
[INFO]  snarkjs: # of Labels: 1234
[INFO]  snarkjs: # of Outputs: 2
```

Interpretación:

- **Wires:** Total de señales en el circuito
- **Constraints:** Número de ecuaciones R1CS (determina tiempo de proving)
- **Private Inputs:** Señales del witness
- **Public Inputs:** Señales conocidas por el verifier

Paso 2: Powers of Tau (Fase 1)

Idea Clave

¿Qué es Powers of Tau?

Es la ceremonia de trusted setup universal:

- Genera parámetros reutilizables para cualquier circuito
- Múltiples participantes contribuyen aleatoriedad
- El archivo .ptau es el resultado

Tamaño del ceremony (2^n):

Power	Max constraints	Tamaño .ptau
12	4,096	~4 MB
14	16,384	~16 MB
16	65,536	~64 MB
18	262,144	~256 MB
20	1,048,576	~1 GB
22	4,194,304	~4 GB

Usar el power mínimo necesario para tu circuito.

```
# Crear ceremonia nueva (solo para testing)
snarkjs powersoftau new bn128 14 pot14_0.ptau -v

# Contribuir aleatoriedad
snarkjs powersoftau contribute pot14_0.ptau pot14_1.ptau \
    --name="First contribution" -v

# Más contribuciones (de diferentes personas)
snarkjs powersoftau contribute pot14_1.ptau pot14_2.ptau \
    --name="Second contribution" -v

# Aplicar random beacon (opcional, para finalizar)
snarkjs powersoftau beacon pot14_2.ptau pot14_beacon.ptau \
    0102030405060708 10 -n="Final Beacon"

# Preparar para fase 2
snarkjs powersoftau prepare phase2 pot14_beacon.ptau pot14_final.ptau -v
```

Observación 5.3. **En producción, usa ceremonias existentes:**

Para no hacer tu propia ceremonia (que sería menos segura):

```
# Descargar .ptau de Hermez (trusted)
```

```
wget https://hermez.s3-eu-west-1.amazonaws.com/\
    powersOfTau28_hez_final_14.ptau
```

Hermez realizó una ceremonia con 1000+ participantes.

Paso 3: Setup del Circuito (Fase 2)

```
# Groth16 setup (per-circuit)
snarkjs groth16 setup circuit.r1cs pot14_final.ptau circuit_0.zkey

# Contribuir a la ceremonia de fase 2 (opcional pero recomendado)
snarkjs zkey contribute circuit_0.zkey circuit_1.zkey \
    --name="First phase2 contribution" -v

# Exportar verification key
snarkjs zkey export verificationkey circuit_1.zkey verification_key.json
```

Matemáticas

Fase 1 vs Fase 2:
Fase 1 (Powers of Tau):

- Universal para todos los circuitos
- Solo depende del tamaño máximo
- Se hace una vez, se reutiliza

Fase 2 (Circuit-specific):

- Específica para cada circuito
- Incorpora la estructura del R1CS
- Genera las proving/verification keys finales

Archivos generados:

- `circuit.zkey`: Proving key (grande, para el prover)
- `verification_key.json`: Verification key (pequeña, para el verifier)

5.7.3 Prove

```
# Generar witness
node circuit_js/generate_witness.js circuit.wasm input.json witness.wtns

# Generar prueba
snarkjs groth16 prove circuit.zkey witness.wtns proof.json public.json
```

Paso 1: Generar el Witness

Ejemplo 5.13 (Input y witness generation)**. Archivo input.json:**

```
{
    "a": "3",
    "b": "5",
    "c": "15"
}
```

Generar witness:

```
# Usando el WASM generator
node circuit_js/generate_witness.js \
    circuit_js/circuit.wasm \
    input.json \
    witness.wtns

# Alternativamente, usando C++ (más rápido)
./circuit_cpp/circuit input.json witness.wtns
```

Ver el witness:

```
$ snarkjs wtns export json witness.wtns witness.json
$ cat witness.json
[
 "1",
 "15",   // output c (público)
 "3",    // input a
 "5"     // input b
]
```

Paso 2: Generar la Prueba

```
snarkjs groth16 prove circuit.zkey witness.wtns proof.json public.json
```

Ejemplo 5.14 (Archivos de salida)**. proof.json:**

```
{
 "pi_a": [
  "12345678901234567890...",
  "98765432109876543210...",
  "1"
 ],
 "pi_b": [
  ["12345...", "67890..."],
  ["11111...", "22222..."],
```

```
  ["1", "0"]
 ],
 "pi_c": [
  "55555555555555555555...",
  "44444444444444444444...",
  "1"
 ],
 "protocol": "groth16",
 "curve": "bn128"
}
```

public.json: (public inputs + outputs)

```
[
 "15"  // El valor público c
]
```

Matemáticas

Estructura de la prueba Groth16:

- `pi_a`: Punto en $\mathbb{G}_1$ (2 coordenadas + 1)
- `pi_b`: Punto en $\mathbb{G}_2$ (2x2 coordenadas + 1)
- `pi_c`: Punto en $\mathbb{G}_1$ (2 coordenadas + 1)

Tamaño total: ∼256 bytes sin comprimir, ∼128 bytes comprimido.

Tiempo de proving:

- 1,000 constraints: ∼1 segundo
- 10,000 constraints: ∼5 segundos
- 100,000 constraints: ∼30 segundos
- 1,000,000 constraints: ∼5 minutos

(Tiempos aproximados en laptop moderna)

5.7.4 Verify

```
# Exportar verification key
snarkjs zkey export verificationkey circuit.zkey vkey.json

# Verificar
snarkjs groth16 verify vkey.json public.json proof.json
```

Ejemplo 5.15 (Verificación)**.** `$ snarkjs groth16 verify verification_key.json public.json proof.j`

```
[INFO]  snarkjs: OK!

# Si la prueba es inválida:
$ snarkjs groth16 verify verification_key.json public.json bad_proof.json
[ERROR] snarkjs: Invalid proof
```

Tiempo de verificación: ∼10-50 ms (independiente del tamaño del circuito).

5.7.5 Exportar a Solidity

```
snarkjs zkey export solidityverifier circuit.zkey Verifier.sol
```

Ejemplo 5.16 (Contrato verificador generado)**.**

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Groth16Verifier {
    // Parámetros de la curva BN254
    uint256 constant PRIME_Q =
        21888242871839275222246405745257275088696311157297823662689037894645226208583;

    // Verification key (hardcoded)
    uint256 constant alphax = ...;
    uint256 constant alphay = ...;
    // ... más constantes ...

    // IC (para public inputs)
    uint256[2][2] IC;

    function verifyProof(
        uint[2] calldata _pA,
        uint[2][2] calldata _pB,
        uint[2] calldata _pC,
        uint[1] calldata _pubSignals
    ) public view returns (bool) {
        // ... verificación con pairings ...
        return pairing(...);
    }
}
```

Idea Clave

Usando el verificador en Solidity:

```
// En tu contrato
contract MyApp {
    Groth16Verifier verifier;
```

```
    constructor(address _verifier) {
        verifier = Groth16Verifier(_verifier);
    }

    function doSomethingWithProof(
        uint[2] calldata a,
        uint[2][2] calldata b,
        uint[2] calldata c,
        uint[1] calldata publicInputs
    ) external {
        require(verifier.verifyProof(a, b, c, publicInputs),
                "Invalid proof");

        // La prueba es válida, proceder con la lógica
        uint result = publicInputs[0];
        // ...
    }
}
```

Gas cost: ~200,000-300,000 gas por verificación.

5.7.6 Generar Calldata

```
# Generar calldata para llamar al contrato
snarkjs zkey export soliditycalldata public.json proof.json
```

Ejemplo 5.17 (Calldata generado). `$ snarkjs zkey export soliditycalldata public.json proof.jso`

```
["0x12345...", "0x67890..."],
[["0x11111...", "0x22222..."], ["0x33333...", "0x44444..."]],
["0x55555...", "0x66666..."],
["0x0000000000000000000000000000000000000000000000000000000000000001"]
```

Este output se puede usar directamente en Etherscan o en scripts de deployment.

5.7.7 Script Completo de Ejemplo

Ejemplo 5.18 (Script de desarrollo end-to-end). `#!/bin/bash`

```
set -e

CIRCUIT=multiplier

echo "=== 1. Compilar circuito ==="
circom ${CIRCUIT}.circom --r1cs --wasm --sym --c
```

```
echo "=== 2. Información del circuito ==="
snarkjs r1cs info ${CIRCUIT}.r1cs

echo "=== 3. Descargar Powers of Tau ==="
if [ ! -f pot14_final.ptau ]; then
    wget https://hermez.s3-eu-west-1.amazonaws.com/\
        powersOfTau28_hez_final_14.ptau \
        -O pot14_final.ptau
fi

echo "=== 4. Setup (Groth16) ==="
snarkjs groth16 setup ${CIRCUIT}.r1cs pot14_final.ptau ${CIRCUIT}_0.zkey

echo "=== 5. Contribuir a fase 2 ==="
snarkjs zkey contribute ${CIRCUIT}_0.zkey ${CIRCUIT}_final.zkey \
    --name="Dev contribution" -v -e="random entropy"

echo "=== 6. Exportar verification key ==="
snarkjs zkey export verificationkey ${CIRCUIT}_final.zkey vkey.json

echo "=== 7. Generar witness ==="
node ${CIRCUIT}_js/generate_witness.js \
    ${CIRCUIT}_js/${CIRCUIT}.wasm \
    input.json \
    witness.wtns

echo "=== 8. Generar prueba ==="
snarkjs groth16 prove ${CIRCUIT}_final.zkey witness.wtns proof.json public.json

echo "=== 9. Verificar prueba ==="
snarkjs groth16 verify vkey.json public.json proof.json

echo "=== 10. Exportar verificador Solidity ==="
snarkjs zkey export solidityverifier ${CIRCUIT}_final.zkey Verifier.sol

echo "=== 11. Generar calldata ==="
snarkjs zkey export soliditycalldata public.json proof.json > calldata.txt

echo "¡Completado!"
```

5.7.8 Otros Backends: PLONK y FFLONK

Observación 5.4. **snarkjs soporta múltiples sistemas:**

Groth16:

```
snarkjs groth16 setup ...
snarkjs groth16 prove ...
snarkjs groth16 verify ...
```

PLONK:

```
snarkjs plonk setup circuit.r1cs pot_final.ptau circuit.zkey
snarkjs plonk prove circuit.zkey witness.wtns proof.json public.json
snarkjs plonk verify vkey.json public.json proof.json
```

FFLONK (más nuevo, más eficiente):

```
snarkjs fflonk setup circuit.r1cs pot_final.ptau circuit.zkey
snarkjs fflonk prove circuit.zkey witness.wtns proof.json public.json
snarkjs fflonk verify vkey.json public.json proof.json
```

Sistema	Setup	Prueba	Verify gas
Groth16	Per-circuit	128 B	∼230K
PLONK	Universal	768 B	∼300K
FFLONK	Universal	576 B	∼250K

En Banco ZK

Flujo de desarrollo en Banco ZK:

```
# Nuestro Makefile
.PHONY: build prove verify deploy

PTAU=powersOfTau28_hez_final_20.ptau

build:
    circom src/transfer.circom -o build/ --r1cs --wasm --sym
    snarkjs groth16 setup build/transfer.r1cs $(PTAU) \
        build/transfer.zkey
    snarkjs zkey export verificationkey build/transfer.zkey \
        build/vkey.json
    snarkjs zkey export solidityverifier build/transfer.zkey \
        contracts/TransferVerifier.sol

test:
    npm run test:circuits
```

```
prove:
    node scripts/prove.js $(INPUT)

deploy:
    forge script script/Deploy.s.sol --broadcast
```

Optimizaciones en producción:

- Usamos rapidsnark (C++) en lugar de snarkjs para proving
- Witness generation con código C++ compilado
- Verificador Solidity optimizado para gas
- Batch proving para múltiples transacciones

5.8 Halo2

Definición 5.11 (Halo2)**.** Framework ZK basado en PLONKish arithmetization.

5.8.1 Historia y Contexto

Observación 5.5. **Evolución de Halo a Halo2:**

Año	Versión	Innovación
2019	Halo	Recursión sin trusted setup (ECC)
2020	Halo 2	PLONKish + lookups + mejor API
2021	Orchard	Halo2 en Zcash mainnet
2022	Scroll	Halo2 para zkEVM
2023	PSE fork	Mejoras de la comunidad

Proyectos usando Halo2:

- Zcash (Orchard shielded pool)
- Scroll (zkEVM)
- Axiom (ZK coprocessor)
- Taiko (Type-1 zkEVM)
- zkEVM Community Edition

5.8.2 PLONKish Arithmetization

En lugar de R1CS, usa tablas con columnas tipadas:

- **Advice:** Witness privado

- **Instance:** Inputs públicos
- **Fixed:** Constantes del circuito
- **Selector:** Activan/desactivan gates

Intuición

De R1CS a PLONKish:
R1CS piensa en constraints individuales:

```
Constraint 1: a * b = c
Constraint 2: c * d = e
Constraint 3: e + f = g
```

PLONKish piensa en una tabla:

```
| row | a   | b   | c   | selector |
|-----|-----|-----|-----|----------|
| 0   | a   | b   | c   | mul      |
| 1   | c   | d   | e   | mul      |
| 2   | e   | f   | g   | add      |
```

La tabla tiene:

- **Columnas:** Variables del circuito (advice)
- **Filas:** Diferentes "pasos" o gates
- **Selectores:** Qué tipo de gate aplica en cada fila

Matemáticas

Estructura de una tabla Halo2:

row	a (advice)	b (advice)	c (advice)	s_mul	s_add	const (fixed)
0	3	4	12	1	0	0
1	12	5	60	1	0	0
2	60	7	67	0	1	0
3	67	0	67	0	0	0

Tipos de columnas:

- **Advice:** Valores del witness (privados)
- **Instance:** Inputs públicos (conocidos por verifier)
- **Fixed:** Constantes compiladas en el circuito
- **Selector:** Columnas fixed que activan gates

Gates:

- Row 0: $s_{mul} \cdot (a \cdot b - c) = 0 \Rightarrow 3 \cdot 4 = 12$
- Row 1: $s_{mul} \cdot (a \cdot b - c) = 0 \Rightarrow 12 \cdot 5 = 60$
- Row 2: $s_{add} \cdot (a + b - c) = 0 \Rightarrow 60 + 7 = 67$

5.8.3 Custom Gates

Definición 5.12 (Custom Gate)**.** Expresión polinomial arbitraria sobre columnas.

Ejemplo 5.19. Gate para Poseidon S-box (x^5):

$$s(X) \cdot (a(X)^5 - b(X)) = 0$$

Cuando selector s activo, verifica $a^5 = b$.

Idea Clave

Ventaja de custom gates:
En R1CS: Solo puedes hacer $a \cdot b = c$ (una multiplicación por constraint).
Para calcular x^5:

```
x2 = x * x      // constraint 1
x3 = x2 * x     // constraint 2
x4 = x3 * x     // constraint 3
x5 = x4 * x     // constraint 4
// Total: 4 constraints
```

En PLONKish: Puedes definir gate de grado 5:

```
s_pow5 * (a^5 - b) = 0  // 1 constraint
// Total: 1 constraint (grado más alto)
```

Trade-off:

- Custom gates reducen número de rows
- Pero aumentan el grado del polinomio
- Grado más alto = prover más lento
- Balance óptimo depende del circuito

Ejemplo 5.20 (Custom gates para operaciones comunes)**.**

```
// Gate de multiplicación estándar
s_mul * (a * b - c) = 0

// Gate de suma
s_add * (a + b - c) = 0
```

```
// Gate de multiplicación por constante
s_mul_const * (a * constant - b) = 0

// Gate bool (verifica que es 0 o 1)
s_bool * a * (1 - a) = 0

// Gate ternario (if-then-else)
// if cond then a else b = out
s_ternary * (cond * (a - b) + b - out) = 0

// Gate Poseidon full round
s_poseidon * (
    (state[0] + round_const)^5 - new_state[0] +
    matrix_multiply(...)
) = 0
```

5.8.4 Lookup Tables

Definición 5.13 (Lookup)**.** Verificar que valores están en una tabla predefinida.

Usos:

- Range checks (valor en $[0, 2^{16})$)
- Tablas XOR para operaciones de bits
- S-boxes para criptografía simétrica

Intuición

¿Por qué lookups?
Algunas operaciones son difíciles de expresar algebraicamente:

- XOR de bits
- Comparaciones
- Funciones no-algebraicas

Solución: Precalcular todos los resultados en una tabla y verificar que el valor está en la tabla.
Ejemplo XOR de 4 bits:

```
Tabla precalculada (256 entradas):
| a | b | a XOR b |
|---|---|---------|
| 0 | 0 |    0    |
| 0 | 1 |    1    |
```

```
| 1 | 0 |    1    |
| 1 | 1 |    0    |
| 0 | 2 |    2    |
| ...
```

En lugar de 4 constraints por XOR (descomponer en bits), un lookup es ∼1 constraint amortizado.

Matemáticas

Cómo funcionan los lookups (Plookup):
Setup:

- Tabla T con n filas de valores válidos
- La tabla es parte del circuito (fixed)

Claim:

- El prover dice "mis valores (a, b, c) están todos en T"

Verificación:
Usando grand product argument:

$$\prod_i \frac{(\gamma + f_i)}{(\gamma + t_i)} = 1$$

Donde f_i son los valores a verificar y t_i son la tabla más valores repetidos.
Costo:

- Cada lookup: ∼1 constraint adicional
- Overhead por tabla: polinomios adicionales
- Muy eficiente para muchos lookups a la misma tabla

Ejemplo 5.21 (Range check con lookup)**.**

```
// Sin lookup (para 16 bits):
// Descomponer en bits: 16 constraints
// Verificar cada bit es 0/1: 16 constraints
// Total: 32 constraints

// Con lookup:
// Tabla: {0, 1, 2, ..., 65535}
// Lookup: 1 constraint
// Total: ~1 constraint

// En Halo2:
meta.lookup(|meta| {
```

```
    let value = meta.query_advice(a, Rotation::cur());
    let table = meta.query_fixed(range_table, Rotation::cur());

    vec![(value, table)]
});
```

5.8.5 API de Halo2 (Rust)

Ejemplo 5.22 (Circuito simple en Halo2)**.**

```
use halo2_proofs::{
    circuit::{Layouter, SimpleFloorPlanner, Value},
    plonk::{Advice, Circuit, Column, ConstraintSystem, Error, Selector},
    poly::Rotation,
};

#[derive(Clone)]
struct MyConfig {
    advice: [Column<Advice>; 2],
    selector: Selector,
}

struct MyCircuit {
    a: Value<Fp>,
    b: Value<Fp>,
}

impl Circuit<Fp> for MyCircuit {
    type Config = MyConfig;
    type FloorPlanner = SimpleFloorPlanner;

    fn configure(meta: &mut ConstraintSystem<Fp>) -> Self::Config {
        let advice = [meta.advice_column(), meta.advice_column()];
        let selector = meta.selector();

        // Definir el gate de multiplicación
        meta.create_gate("mul", |meta| {
            let s = meta.query_selector(selector);
            let a = meta.query_advice(advice[0], Rotation::cur());
            let b = meta.query_advice(advice[1], Rotation::cur());
            let c = meta.query_advice(advice[0], Rotation::next());

            // s * (a * b - c) = 0
            vec![s * (a * b - c)]
        });
```

```
        MyConfig { advice, selector }
    }

    fn synthesize(
        &self,
        config: Self::Config,
        mut layouter: impl Layouter<Fp>,
    ) -> Result<(), Error> {
        layouter.assign_region(
            || "multiplication",
            |mut region| {
                config.selector.enable(&mut region, 0)?;

                region.assign_advice(
                    || "a",
                    config.advice[0],
                    0,
                    || self.a,
                )?;
                region.assign_advice(
                    || "b",
                    config.advice[1],
                    0,
                    || self.b,
                )?;
                region.assign_advice(
                    || "c = a * b",
                    config.advice[0],
                    1,
                    || self.a * self.b,
                )?;

                Ok(())
            },
        )
    }
}
```

Idea Clave

Componentes de un circuito Halo2:

1. **Config:** Define la estructura del circuito

- Columnas (advice, fixed, instance)
- Selectores
- Gates y lookups

2. **configure():** Crea los gates
   - Define expresiones polinomiales
   - Configura lookups
   - Se ejecuta una vez al compilar
3. **synthesize():** Asigna valores
   - Llena las columnas con valores concretos
   - Habilita selectores en filas específicas
   - Se ejecuta para cada prueba
4. **FloorPlanner:** Organiza el layout
   - SimpleFloorPlanner: básico, secuencial
   - V1: más optimizado

5.8.6 Chips y Regiones

Definición 5.14 (Chip)**.** Módulo reutilizable que implementa una funcionalidad específica.

Ejemplo 5.23 (Estructura de chips)**.**
```
// Chip para operaciones de campo
struct FieldChip<F: FieldExt> {
    config: FieldConfig,
    _marker: PhantomData<F>,
}

impl<F: FieldExt> FieldChip<F> {
    fn mul(
        &self,
        layouter: &mut impl Layouter<F>,
        a: AssignedCell<F, F>,
        b: AssignedCell<F, F>,
    ) -> Result<AssignedCell<F, F>, Error> {
        layouter.assign_region(
            || "mul",
            |mut region| {
                self.config.selector.enable(&mut region, 0)?;
```

```
                a.copy_advice(|| "a", &mut region, self.config.advice[0], 0)?;
                b.copy_advice(|| "b", &mut region, self.config.advice[1], 0)?;

                let c = a.value().copied() * b.value();
                region.assign_advice(
                    || "a * b",
                    self.config.advice[2],
                    0,
                    || c,
                )
            },
        )
    }
}

// Uso:
let field_chip = FieldChip::construct(config);
let a = field_chip.load_private(layouter, self.a)?;
let b = field_chip.load_private(layouter, self.b)?;
let c = field_chip.mul(layouter, a, b)?;
```

5.8.7 Halo2 vs Circom

Aspecto	Circom	Halo2
Lenguaje	DSL propio	Rust
Arithmetization	R1CS	PLONKish
Custom gates	No	Sí
Lookups	No nativo	Sí
Curva learning	Baja	Alta
Flexibilidad	Media	Alta
Tooling	Maduro	En desarrollo
Performance	Buena	Muy buena
Trusted setup	Sí (Groth16)	No (IPA)

Observación 5.6. **¿Cuándo usar cada uno?**

Usar Circom si:

- Prototipando rápido
- Circuito relativamente simple
- Quieres ecosistema maduro (snarkjs, circomlib)
- Team no muy experimentado en ZK

Usar Halo2 si:

- Circuito muy grande/complejo (zkEVM)
- Necesitas custom gates para eficiencia
- Necesitas lookups extensivamente
- No quieres trusted setup
- Team con experiencia en Rust/ZK

En Banco ZK

Elección en Banco ZK:
Actualmente: Circom + Groth16

- Desarrollo más rápido
- Circuitos de tamaño manejable (∼10K constraints)
- Verificación en Ethereum es crítica (Groth16 más barato)
- Team familiar con el stack

Roadmap: Migración gradual a Halo2

- Para circuitos nuevos más complejos
- Eliminar dependencia de trusted setup
- Mejor performance con custom gates para Poseidon
- Lookups para range checks eficientes

Híbrido potencial:

- Circuitos de usuario: Circom (simplicidad)
- Agregador/batch prover: Halo2 (performance)
- Wrapper final: Groth16 (verificación barata en L1)

5.9 Arkworks

Definición 5.15 (Arkworks)**.** Ecosistema Rust de bibliotecas para criptografía ZK.

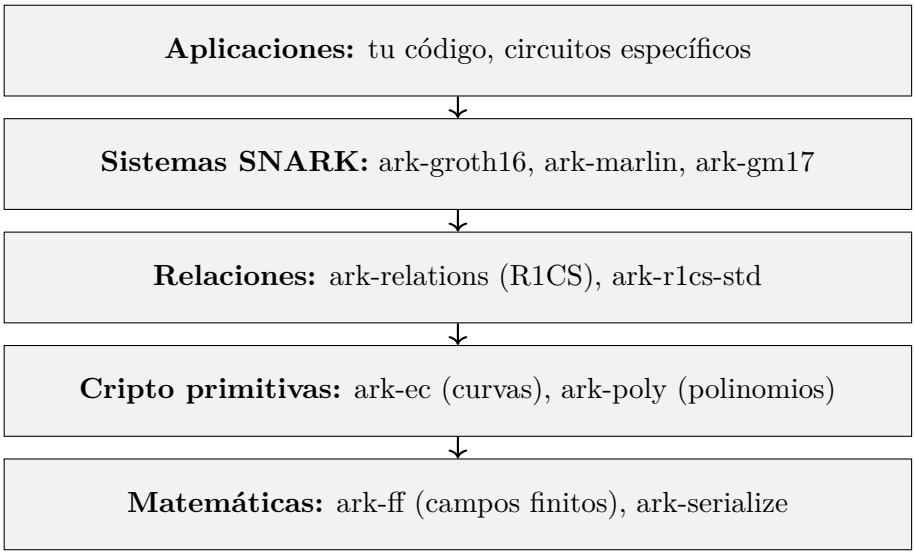
Componentes:

- **ark-ff:** Aritmética de campos finitos
- **ark-ec:** Curvas elípticas
- **ark-poly:** Polinomios
- **ark-relations:** R1CS

- **ark-snark:** Interfaces de sistemas
- **ark-groth16:** Implementación Groth16
- **ark-marlin:** Sistema universal

Uso: investigación, implementaciones personalizadas.

5.9.1 Arquitectura del Ecosistema



Observación 5.7. **Historia de Arkworks:**

- Originalmente "zexe" de la Universidad de Berkeley
- Renombrado a Arkworks en 2021
- Mantenido por comunidad + academia
- Base de muchos proyectos de investigación
- Usado por Aleo, Mina, Penumbra, y otros

Filosofía: Bibliotecas modulares y componibles, no un framework monolítico.

5.9.2 ark-ff: Campos Finitos

Ejemplo 5.24 (Definiendo y usando campos)**.**
```
use ark_ff::{Field, PrimeField, BigInteger};
use ark_bn254::Fr;  // Campo escalar de BN254

fn field_operations() {
    // Crear elementos
    let a = Fr::from(5u64);
    let b = Fr::from(7u64);
```

```
    // Operaciones básicas
    let sum = a + b;           // 12
    let product = a * b;       // 35
    let diff = a - b;          // p - 2 (wraps around)

    // Inverso
    let inv = a.inverse().unwrap();  // a^(-1) mod p
    assert_eq!(a * inv, Fr::one());

    // Exponenciación
    let pow = a.pow([3u64]);   // a^3 = 125

    // Raíz cuadrada (si existe)
    let sqrt = a.sqrt();

    // Serialización
    let bytes = a.into_repr().to_bytes_le();
}
```

5.9.3 ark-relations: R1CS

Ejemplo 5.25 (Definiendo un circuito R1CS). `use ark_relations::{`

```
    lc,
    r1cs::{ConstraintSynthesizer, ConstraintSystemRef, SynthesisError},
};
use ark_ff::Field;

// Circuito: probar que conocemos x tal que x^3 + x + 5 = y
pub struct CubicCircuit<F: Field> {
    pub x: Option<F>,  // Witness (privado)
    pub y: F,          // Público
}

impl<F: Field> ConstraintSynthesizer<F> for CubicCircuit<F> {
    fn generate_constraints(
        self,
        cs: ConstraintSystemRef<F>,
    ) -> Result<(), SynthesisError> {
        // Allocate x (private witness)
        let x = cs.new_witness_variable(|| {
            self.x.ok_or(SynthesisError::AssignmentMissing)
        })?;
```

```
        // Allocate y (public input)
        let y = cs.new_input_variable(|| Ok(self.y))?;

        // x^2
        let x_squared = cs.new_witness_variable(|| {
            let x_val = self.x.ok_or(SynthesisError::AssignmentMissing)?;
            Ok(x_val * x_val)
        })?;

        // Constraint: x * x = x_squared
        cs.enforce_constraint(lc!() + x, lc!() + x, lc!() + x_squared)?;

        // x^3
        let x_cubed = cs.new_witness_variable(|| {
            let x_val = self.x.ok_or(SynthesisError::AssignmentMissing)?;
            Ok(x_val * x_val * x_val)
        })?;

        // Constraint: x_squared * x = x_cubed
        cs.enforce_constraint(lc!() + x_squared, lc!() + x, lc!() + x_cubed)?;

        // Constraint: x_cubed + x + 5 = y
        let five = F::from(5u64);
        cs.enforce_constraint(
            lc!() + x_cubed + x + (five, CS::one()),
            lc!() + CS::one(),
            lc!() + y,
        )?;

        Ok(())
    }
}
```

5.9.4 ark-groth16: Generando Pruebas

Ejemplo 5.26 (Flujo completo con Groth16). `use ark_groth16::{Groth16, ProvingKey, VerifyingKe`

```
use ark_bn254::{Bn254, Fr};
use ark_snark::SNARK;
use ark_std::rand::thread_rng;

fn prove_and_verify() {
    let mut rng = thread_rng();
```

```
    // 1. Crear circuito de ejemplo para setup
    let circuit = CubicCircuit::<Fr> {
        x: None,  // No necesitamos valores para setup
        y: Fr::from(35u64),
    };

    // 2. Setup (trusted setup)
    let (pk, vk) = Groth16::<Bn254>::circuit_specific_setup(
        circuit.clone(),
        &mut rng,
    ).unwrap();

    // 3. Crear circuito con witness real
    let circuit_with_witness = CubicCircuit {
        x: Some(Fr::from(3u64)),  // x = 3
        y: Fr::from(35u64),       // y = 3^3 + 3 + 5 = 35
    };

    // 4. Generar prueba
    let proof = Groth16::<Bn254>::prove(
        &pk,
        circuit_with_witness,
        &mut rng,
    ).unwrap();

    // 5. Preparar public inputs
    let public_inputs = vec![Fr::from(35u64)];

    // 6. Verificar
    let valid = Groth16::<Bn254>::verify(
        &vk,
        &public_inputs,
        &proof,
    ).unwrap();

    assert!(valid);
    println!("Proof verified successfully!");
}
```

5.9.5 ark-r1cs-std: Gadgets

Matemáticas

Gadgets estándar disponibles:

```
use ark_r1cs_std::{
    // Tipos básicos
    fields::fp::FpVar,
    bits::boolean::Boolean,
    bits::uint8::UInt8,

    // Comparaciones
    eq::EqGadget,
    cmp::CmpGadget,

    // Curvas elípticas
    groups::CurveVar,

    // Hashes
    // (en ark-crypto-primitives)
};

// Ejemplo: comparar dos valores
let a = FpVar::new_witness(cs.clone(), || Ok(Fr::from(5)))?;
let b = FpVar::new_witness(cs.clone(), || Ok(Fr::from(3)))?;

// a == b ?
let is_eq = a.is_eq(&b)?;
is_eq.enforce_equal(&Boolean::FALSE)?;

// a > b ?
let is_greater = a.is_gt(&b)?;
is_greater.enforce_equal(&Boolean::TRUE)?;
```

5.9.6 Cuándo Usar Arkworks

Idea Clave

Arkworks es ideal para:

- **Investigación:** Implementar nuevos esquemas, papers
- **Customización:** Cuando necesitas control total
- **Performance:** Optimizaciones específicas

- **Nuevas curvas:** Experimentar con diferentes curvas
- **Integración:** Combinar múltiples primitivas

NO es ideal para:

- Prototipado rápido (usar Circom)
- Circuitos simples (overhead de Rust)
- Teams sin experiencia en Rust/crypto

5.10 Noir

Definición 5.16 (Noir)**.** Lenguaje de alto nivel de Aztec para circuitos ZK.

```
fn main(x: Field, y: pub Field) {
    assert(x * x == y);
}
```

Características:

- Sintaxis similar a Rust
- Compila a ACIR (Abstract Circuit IR)
- Backend-agnostic
- Ideal para private smart contracts

5.10.1 Historia y Filosofía

Observación 5.8. **Evolución de Noir:**

Año	Versión	Desarrollo
2021	Concepto	Aztec diseña lenguaje de alto nivel
2022	0.1-0.5	Primera versión pública
2023	0.6-0.19	Mejoras significativas
2024	0.20+	Producción-ready

Filosofía:

- "ZK para todos": sintaxis familiar
- Abstracción del backend: escribir una vez, compilar a cualquier sistema
- Seguridad: el compilador previene errores comunes
- Performance: optimizaciones automáticas

5.10.2 Sintaxis Básica

Ejemplo 5.27 (Programa Noir completo)**.** `// main.nr`

```
// Función principal - define el circuito
fn main(
    // Inputs privados (witness)
    secret: Field,
    password: [u8; 32],

    // Inputs públicos (prefijo 'pub')
    pub expected_hash: Field,
    pub min_value: Field,
) {
    // Computar hash del secreto
    let computed_hash = std::hash::pedersen([secret]);

    // Verificar igualdad (constraint)
    assert(computed_hash == expected_hash);

    // Verificar rango
    assert(secret >= min_value);

    // Operaciones con arrays
    let mut sum: Field = 0;
    for i in 0..32 {
        sum += password[i] as Field;
    }
    assert(sum > 0);
}

// Función auxiliar (no genera constraints directamente)
fn helper_function(x: Field) -> Field {
    x * x + 1
}

// Tests
#[test]
fn test_main() {
    main(5, [1; 32], pedersen([5]), 3);
}
```

5.10.3 Tipos de Datos

Matemáticas

Tipos disponibles en Noir:

```
// Campos finitos
let a: Field = 5;
let b: Field = -1;  // p - 1 en el campo

// Enteros
let u: u32 = 100;
let i: i8 = -50;
// Soporta: u1, u8, u16, u32, u64, i8, i16, i32, i64

// Booleanos
let flag: bool = true;

// Arrays (tamaño fijo)
let arr: [Field; 3] = [1, 2, 3];
let nested: [[u8; 2]; 3] = [[1, 2], [3, 4], [5, 6]];

// Tuplas
let tuple: (Field, bool) = (5, true);

// Structs
struct Point {
    x: Field,
    y: Field,
}

// Strings (tamaño fijo)
let s: str<5> = "hello";
```

Nota: No hay tipos dinámicos (no hay Vec, no hay String dinámico). Todo debe ser de tamaño conocido en tiempo de compilación.

5.10.4 Control de Flujo

Ejemplo 5.28 (Control de flujo en Noir)**.** `fn control_examples(x: Field, condition: bool) -> Field`

```
    // If-else (ambas ramas se computan, resultado se selecciona)
    let result = if condition {
        x * 2
    } else {
        x + 1
```

```
    };

    // For loops (bounds deben ser constantes)
    let mut sum: Field = 0;
    for i in 0..10 {
        sum += i as Field;
    }

    // While no existe - usar for con break
    // (en versiones recientes)

    // Match
    let value = match x {
        0 => 100,
        1 => 200,
        _ => 0,
    };

    result + sum + value
}

// IMPORTANTE: Los loops deben tener bounds conocidos en compile-time
// Esto NO compila:
// for i in 0..n { ... }  // Error: n no es constante
```

5.10.5 Biblioteca Estándar

Observación 5.9. **Funciones estándar de Noir:**

```
use dep::std;

// Hashes
let h1 = std::hash::pedersen([a, b, c]);
let h2 = std::hash::poseidon::bn254::hash_2([a, b]);
let h3 = std::hash::keccak256([...]);
let h4 = std::hash::sha256([...]);

// Merkle trees
let root = std::merkle::compute_merkle_root(leaf, index, path);
let valid = std::merkle::check_membership(root, leaf, index, path);

// ECDSA / Schnorr
let valid = std::ecdsa_secp256k1::verify_signature(
    pub_key_x, pub_key_y, signature, message_hash
```

```
);
let valid = std::schnorr::verify_signature(
    pub_key_x, pub_key_y, signature, message
);

// EdDSA (BabyJubJub)
let valid = std::eddsa::eddsa_poseidon_verify(
    pub_key_x, pub_key_y, signature_s, signature_r8_x, signature_r8_y, message
);
```

5.10.6 Toolchain de Noir

Ejemplo 5.29 (Flujo de desarrollo con Nargo)**.**

```
# Instalar Noir
curl -L https://noir-lang.org/install.sh | bash
noirup

# Crear proyecto
nargo new my_circuit
cd my_circuit

# Estructura del proyecto
my_circuit/
 Nargo.toml        # Configuración
 src/
    main.nr       # Código del circuito
 Prover.toml       # Inputs para prover
 Verifier.toml     # Inputs públicos

# Compilar
nargo compile

# Ejecutar (witness generation + check)
nargo execute

# Generar prueba
nargo prove

# Verificar
nargo verify

# Generar verificador Solidity
nargo codegen-verifier
```

Ejemplo 5.30 (Archivos de configuración)**. Nargo.toml:**

```
[package]
name = "my_circuit"
type = "bin"
authors = [""]
compiler_version = ">=0.25.0"

[dependencies]
# Dependencias de otras librerías Noir
```

Prover.toml:

```
# Inputs para generar la prueba
secret = "0x1234..."
password = [1, 2, 3, ...]
expected_hash = "0xabcd..."
min_value = "5"
```

5.10.7 ACIR: Representación Intermedia

Idea Clave

ACIR (Abstract Circuit Intermediate Representation):
Noir compila a ACIR, no directamente a R1CS o PLONKish.

```
Noir code → ACIR → Backend-specific format
                      → Barretenberg (default)
                      → Arkworks
                      → Otros backends
```

Ventajas:

- Un lenguaje, múltiples backends
- Optimizaciones a nivel de ACIR
- Portabilidad del código

Backends disponibles:

- **Barretenberg:** UltraPlonk, default de Aztec
- **arkworks:** Groth16, Marlin
- **halo2:** En desarrollo

5.10.8 Comparación de Lenguajes ZK

Aspecto	Circom	Halo2	Noir	Arkworks
Tipo	DSL	Framework	Lenguaje	Biblioteca
Lenguaje base	Propio	Rust	Rust-like	Rust
Curva learning	Baja	Alta	Media	Alta
Abstracción	Media	Baja	Alta	Baja
Flexibilidad	Media	Alta	Media	Muy alta
Backends	Groth16	IPA	Múltiples	Múltiples
Custom gates	No	Sí	Vía backend	Sí
Lookups	No*	Sí	Sí	Depende
Madurez	Alta	Media	Media	Alta

*Circom tiene extensiones con lookups en desarrollo.

5.11 Resumen del Bloque 5

Idea Clave

Puntos clave de Circuitos ZK:

1. **Circuitos Aritméticos:**
   - DAGs de operaciones sobre campos finitos
   - Solo suma y multiplicación como operaciones base
   - Todo debe traducirse a aritmética de campo
2. **R1CS:**
   - Sistema de constraints: $(A \cdot w) \times (B \cdot w) = C \cdot w$
   - Una multiplicación por constraint
   - Sumas son "gratis"
   - Número de constraints $\approx$ complejidad del circuito
3. **Señales y Witness:**
   - Public inputs: conocidos por verifier
   - Private inputs: solo conocidos por prover
   - Witness: asignación de valores que satisface constraints
4. **Circom:**
   - DSL más usado para ZK en Ethereum
   - Templates reutilizables, components

- Operadores: `<--`, `<==`, `===`
- Circomlib: biblioteca estándar

5. **snarkjs:**
   - Flujo: Compilar → Setup → Prove → Verify
   - Powers of Tau: ceremonia universal
   - Exporta verificadores a Solidity
6. **Halo2/PLONKish:**
   - Tablas en lugar de constraints individuales
   - Custom gates: expresiones polinomiales arbitrarias
   - Lookups: verificar pertenencia a tablas
   - Sin trusted setup (con IPA)
7. **Otros frameworks:**
   - Arkworks: bibliotecas Rust modulares
   - Noir: lenguaje de alto nivel, backend-agnostic
8. **Elección de herramientas:**
   - Prototipo rápido: Circom
   - Circuitos complejos: Halo2
   - Abstracción: Noir
   - Investigación: Arkworks

En Banco ZK

Stack de Banco ZK:

```
Capa de usuario:
 Wallet → genera witness local
 SDK → helpers para inputs

Capa de circuitos:
 Circom (actual)
    transfer.circom
    deposit.circom
    withdraw.circom
```

```
 Noir (futuro)
     Mismos circuitos, más legibles

Capa de proving:
 snarkjs (desarrollo)
 rapidsnark (producción)

Capa de verificación:
 Verifier.sol (L1)
 On-chain verification (~200K gas)
```

Métricas objetivo:

- Transfer circuit: <15K constraints
- Prove time: <5 segundos (cliente)
- Verify time: <10 ms (off-chain)
- Verify gas: <250K gas (on-chain)

Apéndice: Notación Matemática

Conjuntos y Estructuras

$\mathbb{Z}$	Números enteros
$\mathbb{Z}_n$	Enteros módulo n
$\mathbb{Z}_n^*$	Grupo multiplicativo de $\mathbb{Z}_n$
$\mathbb{F}_p$	Campo finito de orden primo p
$\mathbb{F}_{p^k}$	Extensión de grado k de $\mathbb{F}_p$
$\mathbb{G}$	Grupo genérico
$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$	Grupos de pairing
$E(\mathbb{F}_p)$	Puntos de curva elíptica sobre $\mathbb{F}_p$

Operaciones

$a \mod n$	Residuo de a módulo n
$a \equiv b \pmod n$	a congruente a b módulo n
g^x	Exponenciación en grupo (notación multiplicativa)
$[x]G$	Multiplicación escalar en curva (notación aditiva)
$e(P, Q)$	Pairing bilineal de P y Q
$H(m)$	Hash de mensaje m
$\|$	Concatenación

Criptografía

pk, sk	Clave pública, clave secreta
$\mathsf{Enc}_k(m)$	Cifrado de m con clave k
$\mathsf{Dec}_k(c)$	Descifrado de c con clave k
$\mathsf{Sign}_{\mathsf{sk}}(m)$	Firma de m con clave secreta
$\mathsf{Verify}_{\mathsf{pk}}(m, \sigma)$	Verificación de firma
$\mathsf{Commit}(v; r)$	Commitment de valor v con randomness r
$\mathsf{Open}(c, v, r)$	Apertura de commitment

Zero-Knowledge

π	Prueba (proof)
x	Statement público
w	Witness (testigo privado)
$\mathcal{R}$	Relación: $(x, w) \in \mathcal{R}$ si w es witness válido para x
$\mathsf{Setup}(\lambda) \to \mathsf{crs}$	Generación de parámetros
$\mathsf{Prove}(\mathsf{crs}, x, w) \to \pi$	Generación de prueba
$\mathsf{Verify}(\mathsf{crs}, x, \pi) \to \{0, 1\}$	Verificación de prueba

Identidad y Credenciales

$\mathsf{Issue}(\mathsf{sk}_I, \mathsf{attrs}) \to \mathsf{cred}$	Emisión de credencial
$\mathsf{Present}(\mathsf{cred}, \mathsf{disclosed}) \to \mathsf{pres}$	Presentación selectiva
$\mathsf{VerifyPres}(\mathsf{pk}_I, \mathsf{pres}) \to \{0, 1\}$	Verificación de presentación
DID	Decentralized Identifier
VC	Verifiable Credential

Complejidad

$\mathcal{O}(f(n))$	Cota superior asintótica
λ	Parámetro de seguridad (típicamente 128 o 256 bits)
$\mathsf{negl}(\lambda)$	Función negligible en λ
$\mathsf{poly}(\lambda)$	Función polinomial en λ
PPT	Probabilistic Polynomial Time

Fin del Volumen 1

Continúa en:

Banco ZK: Volumen 2

Aplicaciones Avanzadas y Ecosistema

Bloques 6–13

Volumen 2 incluye:

Compromisos • Infraestructura ZK • Privacidad Avanzada

DeFi • Anonimización • Privacy Coins

Unlinkability • Investigación Avanzada